

WEISS · DATA STRUCTURES & ALGORITHM ANALYSIS

数据结构与算法分析

Python 版教辅 · 逐章精讲

维斯《数据结构与算法分析——Python 语言描述》12 讲全程配套
面向会 C 语言、数据结构零基础的读者 · 知识导图 + 详细讲解 + 例题精讲 + 章末实验

第1章 绪论 · 第2章 算法分析 · 第3章 表、栈和队列 · 第4章 树
第5章 散列 · 第6章 优先队列(堆) · 第7章 排序 · 第8章 不相交集
第9章 图论算法 · 第10章 算法设计技巧 · 第11章 摊还分析 · 第12章 高级数据结构

知识导图 × C 语言对照 × 复杂度速查 × 章末实验

第1章 绪论

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 复习全书的地基数学:指数、对数、级数、模运算及其算法用途
- 理解"程序 = 数据结构 + 算法",建立正确性与效率维度,预告复杂度记号
- 掌握**递归的四法则**,能写出阶乘等经典递归
- 完成 C 到 Python 的第一波迁移:print/input、list、def、class
- 亲手实现斐波那契的朴素递归与记忆化两版,体会 $O(2^n)$ 与 $O(n)$ 之别
- 掌握 RecursionError 的成因与 sys.setrecursionlimit 的用法和危险
- 学完能独立实现汉诺塔与全排列(章末实验),验证 $2^n - 1$ 与 $n!$

💡 从 C 到 Python:本章主题如何迁移

全书一句话直觉:数据结构决定"数据怎么放",算法决定"问题怎么做",两者合起来决定"程序跑多快"。第1章是唯一的地基章,不打地基,后面 11 章都要还债。

语言迁移上,你的 C 技能大部分直接平移:循环、分支、函数——对应,差异在"形态":块用缩进、变量不声明类型、数组换成了自动扩容的 list。本章只补四样新工具:print/input、list(动态数组 + 切片)、def、class;先学会它们,第3章起才能专心学数据结构。

Python 有两个 C 没有的独特优势:整数任意精度(永不溢出)与**解释器自动管理内存**(不用 malloc/free)。代价是同样代码通常比 C++ 慢 30-100 倍,但增长阶是语言无关的(第2章):Python 的 $O(n)$ 照样碾压 C 的 $O(n^2)$ 。

算法直觉上,本章最重要的思想是**递归**:把大问题缩小成"同样结构的小问题"。同一个斐波那契,笨写是指数级,加一行"记忆化"就变成线性——全书第一次"复杂度决定生死"的现场演示;Python 的 functools.lru_cache 让记忆化一行实现。

📦 前置知识自查

数学前置:高中数学:幂运算与对数(§1.1 系统复习);求和记号 Σ 会读即可(§1.1 给出全部公式)。

C 语言前置:函数定义与调用、for/while 循环、数组与下标、printf/scanf、struct 与指针(会更好)。

依赖的前面章节:无,本章是全书第一章。

🚀 本章学习路线

1. **第一阶段(1 小时):**读 §1.1-§1.2,在 REPL 里验证指数/对数/级数/取模的例子,背下里程碑数值
2. **第二阶段(1.5 小时):**读 §1.3-§1.4,把 §1.4 的完整小程序存为 hello.py 并运行、改造
3. **第三阶段(1.5 小时):**读 §1.5-§1.6,亲手运行朴素与记忆化斐波那契,观察时间如何爆炸
4. **第四阶段(实验,1 小时):**完成章末实验汉诺塔与全排列,验证 $2^n - 1$ 与 $n!$
5. **验收标准:**能默写递归四法则,独立写出朴素与记忆化斐波那契,解释 RecursionError 与 setrecursionlimit 的危险

本章知识导图

- §1.1 数学复习:指数、对数与换底、级数三公式、负数取模(C/Python 差异)
- §1.2 算法概念:程序 = 数据结构 + 算法、正确性、效率、复杂度预告
- §1.3 递归简介:递归四法则、调用栈回顾、阶乘示例
- §1.4 从 C 到 Python 速成:print/input、list、def、class、完整小程序
- §1.5 完整示例:斐波那契——朴素 vs 记忆化、手写 vs lru_cache
- §1.6 递归的工程注意:RecursionError、setrecursionlimit 的用法与危险
- §1.7 全书路线图:12 章一句话导航

§1.1 数学复习

1.1.1 指数规则

指数是全书最频繁出现的运算,下列规则必须刻进直觉,后面推导复杂度时随时在用:

- $a^0 = 1 (a \neq 0); a^1 = a; a^{-n} = 1/a^n$
- **同底数相乘,指数相加:** $a^m \cdot a^n = a^{m+n}$
- **幂的幂,指数相乘:** $(a^m)^n = a^{m \cdot n}$
- **同底数相除,指数相减:** $a^m / a^n = a^{m-n}$

背几个里程碑数值: $2^{10} = 1024; 2^{20} \approx 10^6; 2^{30} \approx 10^9; 2^{40} \approx 10^{12}$ 。反过来记: $10^6 \approx 2^{20}$,意味着对 100 万规模做"每次减半"的查找最多约 20 步——这是第2章对数直觉的起点。可在 REPL 用 `2 ** 30` 验证。

记号说明(全书约定,首次出现)

本书数学记号统一如下:幂用上标,如 n^2 、 2^n ;对数的底数用下标,如 $\log_2 n$;递推式行内用等宽代码,如 $T(n) = T(n-1) + O(1)$;求和用希腊字母 Σ ,如 $\Sigma_{i=1}^n i$;模运算在正文写 $a \bmod b$,在代码里一律写语言语法 `a % b`。代码与数学记号严格分离:代码里只用 `%`、`**`。

1.1.2 对数

对数的定义: $\log_b X = a$ 当且仅当 $b^a = X (b > 0, b \neq 1, X > 0)$ 。一句话直觉:对数回答"几次方"——"2 的几次方等于 8",答 $\log_2 8 = 3$ 。对数与指数互为逆运算,是"减半""倍增"类算法(二分查找、快速幂、欧几里得算法)的天然度量。

换底公式(全书最重要的对数公式): $\log_b X = \log_a X / \log_a b$ 。推导:设 $\log_b X = c$,则 $b^c = X$,两边取对数得 $c \cdot \log_a b = \log_a X$ 。工程含义:换底只差常数倍,而常数倍会被大 O 记号吞掉(第2章),所以**底数在复杂度分析里不重要**。

记法约定:计算机领域 $\log$ 默认以 2 为底; $\ln$ 是以 $e \approx 2.718$ 为底; $\lg$ 有时指以 10 为底。三个里程碑: $\log_2 1024 = 10; \log_2 10^6 \approx 20; \log_2 10^9 \approx 30$ 。对数"几乎不增长":规模翻 1000 倍, $\log_2 n$ 只从 10 长到 20——对数级算法都值得信赖。

1.1.3 级数

三个级数公式是第2章复杂度计算的原料:

- **算术级数**: $\sum_{i=1}^n i = n(n+1)/2 \approx n^2/2$ ——双重循环的总迭代次数,量级 $O(n^2)$ 。例: $1+2+\dots+100 = 5050$
- **几何级数**: $\sum_{i=0}^n 2^i = 2^{n+1} - 1$ ——倍增翻倍的总量,量级 $O(2^n)$ 。例: $1+2+4+8+16+32 = 63$;推广: $\sum_{i=0}^n r^i = (r^{n+1} - 1)/(r-1)$
- **调和级数**: $\sum_{i=1}^n 1/i \approx \ln n$ ——量级 $O(\log n)$,出现在快排平均分析中

1.1.4 模运算

模运算: $a \bmod b$ 表示 a 除以 b 的余数,数学上写成 $a = q \cdot b + r$ (q 为商, r 为余数);"余数"取决于"商怎么取整",这正是各语言分歧的根源(★见下)。两条最重要的性质(散列、循环数组下标、快速幂的全部基础):

- $(a+b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$
- $(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m$

★ 重点:Python 与 C 的负数取模不一样

取模的关键分歧在**负数**:C 除法"向 0 截断",余数符号与被除数相同;Python 除法"向下取整",余数符号与除数相同。同一表达式,两种语言答案不同:

C 的写法(商向 0 截断)	Python 的写法(商向负无穷取整)
<code>-7 % 3</code> 结果是 -1 (<code>-7/3</code> 截断为 <code>-2</code> ,余数 <code>-7 - (-2)·3 = -1</code>)	<code>-7 % 3</code> 结果是 2 (<code>-7/3</code> 向下取整为 <code>-3</code> ,余数 <code>-7 - (-3)·3 = 2</code>)
<code>7 % -3</code> 结果是 1 (<code>-2</code> 为商,1 为余数)	<code>7 % -3</code> 结果是 -2 (<code>-3</code> 为商, <code>-2</code> 为余数)
<code>-7 / 3</code> 结果是 -2 (截断)	<code>-7 // 3</code> 结果是 -3 (向下取整)

两者在"同号"时一致,只在异号时分歧。判奇偶、判整除不受影响——受影响的是"余数的正负与大小"。本书代码都在"非负输入"语境中使用 `%`,但从 C 移植时会悄悄改变结果,务必知道这个坑。

⚠ 误区 1:把 C 的取模语义带进 Python

Python 的 `-7 % 3` 是 **2** 而不是 **-1**:商向负无穷取整,余数恒与除数同号(除数非负时恒非负)。从 C 移植含负数除法的代码,结果可能悄悄改变: `a % b == 0` 的整除判断不变,但 `a / b` 的符号行为完全不同(C 截断,Python `//` 向下取整),需要 C 语义用 `math.trunc(a / b)`。另一个专属坑: `5 / 2` 得到浮点 `2.5`——真除法与整除(`//`)是两回事。

⚠ 误区 2:int 溢出?Python 不存在

C 的 `int` 是 32 位, `INT_MAX + 1` 回绕成负数或是未定义行为;斐波那契 `F(47)` 就超界(§1.5)。Python 的整数是**任意精度**的: `21000`、`1000!` 这种 C 中早已溢出的数,Python 精确到每一位。代价:大整数运算比固定宽度慢,每个整数都是堆上对象,同样 $O(n)$ 循环通常比 C++ 慢 30–100 倍。

示例 1-1 Python 负数取模实测


```
# Python 与 C 不同:取模结果是"向下取整"除法的余数
# 公式:余数 = 被除数 - 商 × 除数,商向负无穷取整
print(-7 % 3)      # 2:-7/3 = -2.33,商向下取整为 -3,余数 -7 - (-3)*3 = 2
print(7 % -3)       # -2:商向下取整为 -3,余数 7 - (-3)*(-3) = -2
print(-7 // 3)      # -3:整除同样向下取整(与 C 的截断不同!)
print(-7 / 3)       # -2.3333333333333335:真除法,结果总是浮点数
print(7 % 3, -7 % -3) # 1 -1:同号时与 C 的行为一致
print((-7) % 3)     # 2:加括号防混淆,与 print(-7 % 3) 结果相同
```

§1.2 算法概念

1.2.1 程序 = 数据结构 + 算法

瑞士计算机科学家 Niklaus Wirth 的名言: **程序 = 数据结构 + 算法**。**数据结构**回答"数据如何组织",**算法**回答"问题如何求解",两者互相成就:同一份数据,组织方式不同,解法难度天差地别。经典例子——电话簿:按姓名排序可以二分查找(第2章)瞬间定位;打乱顺序只能逐个翻。

1.2.2 正确性与效率

一个解法要配得上"算法"二字,必须同时满足两条:**正确性**——对一切合法输入给出正确输出;**效率**——时间与空间在可接受范围内。正确但慢到跑不完等于不能用,快但错比慢更危险。

例:求 2^n ,连乘 n 次——正确,但 $n = 10^6$ 时要一百万次乘法;第2章的快速幂约 20 次搞定。本书的尺度:先保证正确,再追求高效;复杂度分析(第2章)给"效率"机器无关的精确度量,本章只需建立直觉:数据规模变大时,运行时间如何增长。

1.2.3 复杂度记号预告

全书用大 O 记号描述效率。先混个脸熟,第2章给严格定义:

- $O(1)$ 常数:步数与规模无关——按下标取列表元素
- $O(\log n)$ 对数:规模翻倍只多一步
- $O(n)$ 线性:遍历一趟
- $O(n \log n)$ 线性对数:归并排序
- $O(n^2)$ 平方:双重循环——插入排序
- $O(2^n)$ 指数:穷举、朴素斐波那契,稍大就不可用

增长阶排序(必须背诵):常数 < 对数 < 线性 < 线性对数 < 平方 < 立方 < 指数。本章预用两个结论(方法见第2章):**递推式 $T(n) = T(n-1) + O(1)$ 的解是 $O(n)$** (阶乘),**朴素斐波那契的递推式 $T(n) = T(n-1) + T(n-2) + O(1)$ 的解是 $O(2^n)$** 。

§1.3 递归简介

1.3.1 递归四法则

Weiss 教材把递归写成四条法则,每条都配一个"违反后果":

1. **基准情形(base case)**:存在不需要递归、直接得出答案的情形。没有基准情形 = 无限递归,程序永不停止。
2. **不断推进**:每次递归调用都必须让问题朝基准情形靠近。不推进 = 死循环;推进太快跳过基准情形 = 边界错误。

3. **设计法则**:假设所有递归调用都能正确工作,只设计"当前层如何利用子问题的答案"。不要手工跟踪每一层——人脑栈只有几层深,程序栈可以有上千层。
4. **合成效益法则**:不要在递归调用的不同分支中重复求解同一个问题实例。同一子问题被算很多遍,复杂度会爆炸成指数级——§1.5 的朴素斐波那契正是反面教材。

1.3.2 C 递归与调用栈回顾

你学 C 时写过递归,现在补上底层机制。程序运行时,系统维护一块**调用栈**(call stack):每次函数调用,系统把"返回地址 + 参数 + 局部变量"打包成一个**栈帧**(stack frame)压入调用栈;函数返回时弹出,继续执行调用点的下一条语句。递归调用也是普通调用,只是被调用的函数与自己同名。

由此得出两个推论:① **递归深度 = 同时存活(尚未返回)的栈帧数**;② 递归占用额外内存(每层一个栈帧)。C 的调用栈约 8MB,深度过大时进程静默崩溃;Python 用"递归深度上限 + RecursionError"显式保护——这是两版刻意不同的工程注意点(§1.6 详解)。

C 与 Python 的递归语法几乎一致:定义、调用、参数传递同构;差异在函数体靠缩进分层,且默认深度上限只有 1000(§1.6)。

1.3.3 阶乘示例

阶乘 $n! = n \times (n-1) \times \dots \times 1$,基准情形 $0! = 1$ 。递归定义: $n! = n \cdot (n-1)!$ ——线性递归的标准形态,递推式 $T(n) = T(n-1) + O(1)$,解为 $O(n)$ 。

示例 1-2 阶乘:递归版与迭代版

```
# 递归版:n! = n * (n-1)!, n <= 1 是基准情形
# 与 C 的写法几乎一致:递归是语言无关的思想
def fact_rec(n):
    if n <= 1:
        return 1
    return n * fact_rec(n - 1)

# 迭代版:循环乘过去,不占调用栈
def fact_iter(n):
    r = 1
    for i in range(2, n + 1):
        r *= i
    return r

# 验证:两种写法一致,且 100! 不溢出(任意精度)
print(fact_rec(10), fact_iter(10))
print(len(str(fact_iter(100))))
```

基准情形: $0! = 1! = 1$
不断推进: n 越来越小
3628800 3628800
158:100! 有 158 位数字

§1.4 从 C 到 Python 速成

1.4.1 输入输出:printf/scanf → print/input

Python 的 `print()` 替代 `printf`, `input()` 替代 `scanf`: `print(x, y)` 自动以空格分隔并换行; `input()` 读取**一整行字符串**并去掉末尾换行。最重要的差异: `input()` 返回字符串,读数字必须手动转换——`int(input())` 读整数、`float(input())` 读浮点数,忘记转换是入门第一坑(C 的 `%d` 自动解析,Python 必须显式)。

C 的写法	Python 的写法
<code>printf("n = %d\n", n);</code>	<code>print(f"n = {n}")</code> 或 <code>print("n =", n)</code>
<code>scanf("%d", &n);</code>	<code>n = int(input())</code>
<code>scanf("%s %d", name, &score);</code>	<code>name, score = input().split()</code>
格式符写错 = 静默读错/崩溃	类型不匹配直接抛异常,显式暴露

f 字符串(`f"平均分: {avg}"`)是格式化输出的标准姿势,省略了 C 的 `%d` 位置参数。

1.4.2 动态数组与字符串:malloc 数组/char* → list/str

C 里动态数组要自己管"长度、容量、扩容、释放"四件事;Python 的 `list` 全部内置:自动扩容、垃圾回收、自带 `len()`。字符串 `str` 是不可变字符序列,操作对应 C 的 `<string.h>` 函数。学习阶段要理解 list 的底层(第3章会手写一遍倍增扩容),工程阶段直接用。

C 的写法	Python 的写法
<code>int *a = malloc(n * sizeof(int));</code>	<code>a = [0] * n</code> 或 <code>a = []</code>
<code>a[len++] = x;</code> (自己管长度)	<code>a.append(x)</code> (自动扩容)
<code>strcpy(buf, s); strcat(buf, t);</code>	<code>s = "hello"; s += " world"</code>
<code>free(a);</code> (忘记即泄漏)	垃圾回收器自动释放
<code>a[i]</code> 越界 = 未定义行为(可能崩溃)	<code>a[i]</code> 越界抛 <code>IndexError</code> ,显式暴露

示例 1-3 list 与字符串速览

```

s = "hello"           # 字符串:不可变字符序列
s += " world"         # 拼接(产生新字符串)
print(len(s), s[0])   # 11 h
v = [3, 1, 4]         # 列表:自动扩容的动态数组
v.append(1)           # 尾部插入
v[0] = 7              # 按下标访问 0(1)
print(len(v), v[0])   # 4 7
for x in v:           # 遍历
    print(x, end=" ")
print(v[1:3])         # 切片:取下标 1..2 的视图
print(sorted(v))      # 排序返回新列表(第7章)

```

1.4.3 函数:C 的函数 → def

Python 函数用 `def 函数名(参数):` 定义,函数体靠缩进分层(C 靠花括号)。与 C 的三个差异:① 参数与返回值都不写类型;② 可有默认参数 `def f(a, b=1)`;③ 返回多个值用元组 `return a, b`,调用处 `x, y = f()` 解包。递归写法与 C 同构——定义里调用自己即可。函数是一等对象,可当参数传递(第10章,§1.5 实验会用到)。

1.4.4 类入门:struct+函数 → class

C 里"数据 + 操作"分离:struct 存数据,全局函数操作它,每次调用都要传指针。Python 的 `class` 把数据与操作封装成整体,方法(成员函数)的第一个参数固定是 `self` (指对象自己)。这是全书所有数据结构(栈、树、堆.....)的呈现形式,第3章还会从 ADT 角度深化。

示例 1-4 类入门:计数器

```
# C 的做法:struct + 全局函数 counter_inc(&c)
# Python 的做法:class 把数据与操作打包,self 指对象自己
class Counter:
    def __init__(self):          # 构造函数:创建对象时自动调用
        self.cnt = 0           # 成员变量

    def inc(self):              # 方法:操作自己的数据
        self.cnt += 1

    def get(self):
        return self.cnt

c = Counter()                  # C 里要 counter_init(&c),这里自动执行
c.inc()
c.inc()
print(c.get())                 # 2
```

1.4.5 运行与完整小程序

Python 不需要编译:保存为 `hello.py` 后运行 `python hello.py`,或输入 `python` 进入 REPL 逐行试验。程序入口习惯用 `if __name__ == "__main__":` 包裹主流程,既能直接运行、也能被 `import`(第10章用到)。

示例 1-5 完整小程序:学生成绩统计(综合本节全部知识点)

```
# hello.py — 第一个完整 Python 程序:学生成绩统计
# 输入:姓名与分数;输出:平均分与最高分
class Student:
    def __init__(self, name, score): # 构造函数
        self.name = name           # 成员变量
        self.score = score

def main():
    n = int(input("学生人数: "))    # input 返回字符串,必须转 int
    stu = []                        # 列表,存 Student 对象
    for _ in range(n):
        name, score = input().split() # "张三 90" -> ["张三", "90"]
        stu.append(Student(name, int(score)))
    total = 0
    best = 0
    for s in stu:                   # 遍历
        total += s.score
        if s.score > best:
            best = s.score
    print(f"平均分: {total // n},最高分: {best}")

if __name__ == "__main__":
    main()

# 运行命令:python hello.py
# 运行示例:
# 学生人数: 3
# 张三 90 / 李四 85 / 王五 95
# 平均分: 90,最高分: 95
```

§1.5 完整示例:斐波那契

1.5.1 朴素递归:指数级

斐波那契数列: $F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2)$ 。照定义直接写递归——这是教科书般的"正确但低效":

示例 1-6 朴素递归斐波那契, $O(2^n)$

```
# F(0)=0, F(1)=1, F(n) = F(n-1) + F(n-2)
# 朴素递归:同一个 F(k) 被重复计算无数遍(违反合成效益法则)
def fib_naive(n):
    if n <= 1:
        return n                # 基准情形
    return fib_naive(n - 1) + fib_naive(n - 2) # 不断推进,但一裂为二

# 调用次数 = 2*F(n+1) - 1(F(40) 约 3.3 亿次)
# 递推式为  $O(2^n)$ ; n 每加 10, 调用数约乘 120
```

⚠ 误区 3:"递归只是慢一点"——是数量级爆炸

同一输入,朴素版 $F(40)$ 约 3.3 亿次调用(本机实测 $F(34) \approx 0.43$ 秒,推算 $F(40) \approx 8$ 秒),记忆化版(下节)约 41 次调用,相差约八百万倍。指数级增长意味着:输入每加一点,时间乘一个常数—— n 从 40 到 50,调用数再乘约 120($\varphi^{10} \approx 123$),任何现实规模都不成立。另外,朴素版在 n 接近 1000 时还会触发 `RecursionError`(§1.6)——"指数级 + 深递归"是双重灾难。

1.5.2 记忆化:线性

观察调用树: $F(5)$ 要算 $F(4)$ 、 $F(3)$, $F(4)$ 又要算 $F(3)$ 、 $F(2)$ —— $F(3)$ 被重复计算了两次, $F(2)$ 更多次。解决办法就是**记忆化**(memoization):第一次算出 $F(k)$ 就存进表格,以后再要直接查表。每个 $F(k)$ 只算一次,共 $n+1$ 个不同子问题,每个 $O(1)$,总计 $O(n)$ 。

示例 1-7 手写记忆化 vs 标准库 `lru_cache`

```

from functools import lru_cache

# 手写记忆化:用字典记住已经算过的结果
memo = {}
def fib_hand(n):
    if n <= 1:
        return n
    if n not in memo:          # 没算过才算,算过直接取
        memo[n] = fib_hand(n - 1) + fib_hand(n - 2)
    return memo[n]

# 标准库写法:lru_cache 一行实现同样缓存
@lru_cache(maxsize=None)
def fib_cache(n):
    if n <= 1:
        return n
    return fib_cache(n - 1) + fib_cache(n - 2)

# 两种写法结果一致,F(100) 瞬间算完(C 的 long long 早已溢出)
print(fib_hand(100))          # 354224848179261915075
print(fib_cache(100))         # 354224848179261915075

```

1.5.3 手写 vs 标准库

上面并排展示了本章"手写 + 标准库"的范例:手写版用字典显式管理缓存,让你看清机制; `lru_cache` 一行实现——内部正是手写版的机制(带"最近最少使用"淘汰策略,C/C++ 没有直接对应物)。工程中:单个函数需要缓存直接用 `@lru_cache`;需要控制缓存生命周期(如按测试用例清空)时手写字典。两者都要会:看懂手写版才能排查 `lru_cache` 的问题。

1.5.4 迭代版:空间也省下来

斐波那契还能更省:只保留前两个值滚动前进,时间 $O(n)$ 、空间 $O(1)$: `a, b = 0, 1`, 循环 `a, b = b, a + b` (元组同时赋值,C 里要写三行临时变量)。对比三版:朴素 $O(2^n)$ 时间、记忆化 $O(n)$ 时间 $O(n)$ 空间、迭代 $O(n)$ 时间 $O(1)$ 空间。记忆化解决"重复计算",迭代解决"重复计算 + 栈空间"——能迭代时优先迭代。用迭代算 $F(10^6)$ 也只需几十毫秒。

§1.6 递归的工程注意

1.6.1 RecursionError 与递归深度上限

与 C/C++ 的"静默段错误"不同,Python 用显式保护代替崩溃:递归深度默认上限 `sys.getrecursionlimit()` = 1000, 超过即抛 `RecursionError`——这是**好事**:问题在错误堆栈里暴露得一清二楚,而不是进程无声消失。

示例 1-8 RecursionError 现场


```
# 危险代码!运行会抛 RecursionError
def count_down(n):
    print(n)
    count_down(n - 1)      # 永不停止,直到超过递归深度上限

# count_down(10**6)
# 打印到 996 附近时抛出异常:
# RecursionError: maximum recursion depth exceeded
# 到达上限才报错,不是立刻崩溃
```

⚠ 误区 4:递归深度 = 递归"层数",不是 n 的位数

常见误解:递归 $n = 10^6$ 只是"数字大一点"。事实:线性递归的深度就是 n —— $F(10^6)$ 需要 10^6 层栈帧,远超 1000 上限,必然 RecursionError。凡是深度可能超过约 1000 的递归,在 Python 里都写不了,必须改迭代或分治(log 深度永远安全,第2章)。

1.6.2 sys.setrecursionlimit:用法与危险

上限可以调大: `sys.setrecursionlimit(20000)` 提到 2 万层。合法场景:"深度合理但超过默认值"的递归——例如对 10^4 规模的链表递归遍历(第3、4章的树,深度 = 树高)。

示例 1-9 setrecursionlimit 的用法与边界

```
import sys
print(sys.getrecursionlimit())      # 默认 1000

def sum_rec(n):
    if n == 0:
        return 0
    return n + sum_rec(n - 1)

sys.setrecursionlimit(20000)        # 调大到 2 万层
print(sum_rec(10000))               # 50005000:深度 1 万,轻松完成
# 危险:若调到  $10^6$  并真的递归  $10^6$  层——
# ① 解释器自身的 C 栈先耗尽,进程直接段错误崩溃,
#   连 try/except 都拦不住;
# ② 深度  $10^6$  可能吃掉数百 MB 内存;
# ③ RecursionError 这道"安全网"失效
```

🚀 工程建议:setrecursionlimit 的三条纪律

① 只小幅调高(默认值 10–20 倍以内),调大不调小;② 调大后仍用迭代优先的算法设计,把递归当作"深度可控的例外";③ 宁可写显式栈(第3章)也不赌大上限——显式栈深度只受内存限制。记住: `setrecursionlimit` 是"最后一根稻草",不是常规工具。

1.6.3 如何避免深层递归

避免深层递归的三招:① **循环改写**——尾递归(递归调用在末尾)最容易改写,空间从 $O(n)$ 变 $O(1)$,§1.5.4 的迭代斐波那契就是例子;② **显式栈**——自己维护的列表模拟系统调用栈(第3章),不受深度限制;③ **控制深度**——"减半"式递归(分治,第2章)深度只有 $\log_2 n$, 10^6 规模也不到 20 层,永远安全。

§1.7 全书路线图

全书 12 章,一句话导航:

- **第2章 算法分析:**大 O 记号、运行时间计算
- **第3章 表、栈和队列:**三种最基础 ADT,两种实现
- **第4章 树:**遍历、表达式树、BST、AVL
- **第5章 散列:**散列函数、分离链接与开放定址
- **第6章 优先队列(堆):**二叉堆、堆排序
- **第7章 排序:**插入/希尔/堆/归并/快排/桶排
- **第8章 不相交集:**按秩合并与路径压缩的并查集
- **第9章 图论算法:**DFS/BFS、拓扑排序、最短路径、最小生成树
- **第10章 算法设计技巧:**贪心、分治、动态规划、回溯、随机化
- **第11章 摊还分析:**聚集、会计、势能三种方法
- **第12章 高级数据结构:**伸展树、B 树、跳表

依赖关系:第2章是第3章起所有复杂度结论的依赖,务必学透;第3–9章是"结构线",第10章是"算法线",两线在第10章汇合;第11章为摊还补上严格证明。每章结束时回看这张图。

本章复杂度速查表

操作 / 算法	数据结构 / 算法	平均	最坏
求 a^n	朴素连乘(手写循环)	$O(n)$	$O(n)$
求 a^n	快速幂(第2章)	$O(\log n)$	$O(\log n)$
求 $\log_b X$	换底公式 + 库函数 $\log$	$O(1)$	$O(1)$
级数求和	算术级数(公式直接算)	$O(1)$	$O(1)$
级数求和	几何级数(公式直接算)	$O(1)$	$O(1)$
级数求和	调和级数(逐项累加)	$O(n)$	$O(n)$
整数取模	C(商向 0 截断)	$O(1)$	$O(1)$
整数取模	Python(商向下取整)	$O(1)$	$O(1)$
阶乘 $n!$	递归(线性递归)	$O(n)$	$O(n)$
阶乘 $n!$	迭代循环	$O(n)$	$O(n)$
斐波那契 $F(n)$	朴素递归(违反合成效益法则)	$O(2^n)$	$O(2^n)$
斐波那契 $F(n)$	记忆化递归	$O(n)$	$O(n)$
斐波那契 $F(n)$	迭代滚动(两个变量)	$O(n)$	$O(n)$
递归调用	递推式 $T(n) = T(n-1) + O(1)$	$O(n)$	$O(n)$
递归调用	递推式 $T(n) = T(n-1) + T(n-2) + O(1)$	$O(2^n)$	$O(2^n)$
递归深度	Python 默认上限(RecursionError)	1000 层	setrecursionlimit 可调大,但过大有崩溃风险
调用栈空间	C/C++ 递归深度 n (默认栈约 8MB)	$O(n)$ 栈空间	深度过大即段错误

最小必会清单

- 能默写指数四条规则与里程碑数值
- 能说出对数定义、换底公式与 $\log/\log_2/\ln$ 约定,并解释"底数不重要"
- 能默写算术、几何、调和三个级数公式并说出各自量级
- 能说出 Python 与 C 负数取模的区别(Python 向下取整, $-7\%3=2$; C 截断, $-7\%3=-1$)与"整除 vs 真除法"
- 能说出 Python 整数任意精度与 C 的 int 溢出的对比
- 能说出"程序 = 数据结构 + 算法",举例说明数据结构如何影响算法难度
- 能默写递归四法则,并指出朴素斐波那契违反的是合成效益法则
- 能独立写出阶乘的递归版与迭代版,说出 $T(n) = T(n-1) + O(1)$ 的解
- 能独立写出朴素与记忆化斐波那契(手写字典版与 @lru_cache 版),说出各自复杂度
- 能解释 RecursionError 与 setrecursionlimit 的用法和危险
- 能用 python hello.py 运行一个含 input/print、list、def、class 的完整程序
- 能说出本书 12 章的大致内容与依赖关系

自测题

Q 1. 计算: 2^{10} 是多少? $\log_2 1024$ 是多少? $\sum_{i=1}^{100} i$ 是多少?

✓ 参考答案

$2^{10} = 1024$; $\log_2 1024 = 10$; $\sum_{i=1}^{100} i = 100 \times 101 / 2 = 5050$ 。对应指数、对数、算术级数三个公式。

Q 2. Python 中 $-7 \% 3$ 与 $7 \% -3$ 分别等于多少?与 C 有何不同?

✓ 参考答案

$-7 \% 3 = 2$, $7 \% -3 = -2$ 。Python 除法向下取整: $-7/3$ 取商 -3 , 余数 $-7 - (-3) \times 3 = 2$; $7/-3$ 取商 -3 , 余数 $7 - (-3) \times (-3) = -2$; C 向 0 截断, 分别为 -1 与 1 。同号一致, 异号不同; 判奇偶、判整除不受影响。

Q 3. 递归四法则是什么?朴素递归斐波那契违反了哪一条?

✓ 参考答案

① 基准情形: 有不需递归的直接答案; ② 不断推进: 每次调用都朝基准情形靠近; ③ 设计法则: 假设所有递归调用都正确, 只组装当前层; ④ 合成效益法则: 不在调用间重复求解同一问题实例。朴素斐波那契违反第④条, 复杂度爆炸到 $O(2^n)$ 。

Q 4. 递推式 $T(n) = T(n-1) + O(1)$ 的解是什么?给出直觉推导。

✓ 参考答案

解为 $O(n)$ 。展开: $T(n) = T(n-1) + c = T(n-2) + 2c = \dots = T(0) + n \cdot c$, 共 n 步、每步常数代价。阶乘与递归求和都是这个递推式。

Q 5. 朴素递归斐波那契计算 $F(40)$ 大约需要多少次递归调用?记忆化后呢?为什么 Python 里还可能先遇到 RecursionError?

✓ 参考答案

朴素版调用次数 $= 2 \times F(41) - 1 \approx 3.3$ 亿次(本机实测约 8 秒); 记忆化后约 41 次调用, 瞬间完成。但 $n = 40$ 时递归深度只有 40, 不会触发 1000 层上限, 只有 n 接近 1000 才会先撞 RecursionError。更现实的风险是: 指数级调用本身就慢到不可用。

Q 6. Python 默认递归深度上限是多少?把 `sys.setrecursionlimit` 调得过大有什么危险?

✓ 参考答案

默认 1000 层(`sys.getrecursionlimit()` 查看)。危险: ① 调得过大(如 10^6)并真递归到那个深度时, 解释器自身的 C 栈先耗尽, 进程段错误崩溃, 连 `try/except` 都拦不住; ② 每层帧内存开销可观; ③ RecursionError 这道安全网失效。正确姿势: 能改迭代就改迭代, `setrecursionlimit` 只小幅调高。

章末实验题:递归综合实验:汉诺塔与全排列

实验 1:汉诺塔与全排列(递归综合)

实验目标:用递归实现汉诺塔与全排列,验证 $2^n - 1$ 与 $n!$ 两个计数结论,并用真实计时感受朴素斐波那契的指数级增长。

实验要求:

- 任务 1:汉诺塔——递归打印移动步骤、统计总步数,验证 $n = 1..10$ 步数恒为 $2^n - 1$ (知识点:§1.3 递归四法则、基准情形)
- 任务 2:递归生成全排列——交换 + 回溯,验证 $n = 3、4$ 的排列总数为 $3! = 6、4! = 24$ (知识点:§1.3 树形递归、回溯思想)
- 任务 3:对比 n 增大时朴素递归斐波那契的时间增长——记录 $n = 28/31/34$ 的耗时,验证 n 加 3 时耗时约乘 4~5 (调用数比值恰为 $\varphi^3 \approx 4.2$; 知识点:§1.5 指数级复杂度、§1.2 效率直觉)

实验环境:Python 3.8+。运行: `python lab1.py`。

第一步 写 `hanoi`:基准情形 $n == 1$ (只剩一个盘子,直接搬);否则三步——上面 $n-1$ 个搬到中转柱、搬最大盘、再搬 $n-1$ 个。注意三次调用中参数轮换的顺序;用全局变量统计步数(先声明 `global`)

第二步 开关只对 $n = 3$ 打开, $n = 1..10$ 只输出"实际 vs 理论 $2^n - 1$ "对照行

第三步 `permute`:固定前 k 位;for 循环把 `a[k..]` 每个元素换到第 k 位;递归排列剩余部分;换回来还原现场(回溯三件套);基准情形 $k == n$ 时用 `a[:]` 拷贝当前排列

第四步 时用 `time.perf_counter()` (§2.1 系统讲),对 $n = 28/31/34$ 各测一次,算"每加 3 乘几"的比值

✅ 参考答案(完整可运行程序,分两段)

lab1.py(第一段:任务 1 汉诺塔与任务 2 全排列)

```
# lab1.py — 递归综合实验(第一段:任务1与任务2)
import time

move_cnt = 0                                # 汉诺塔移动总步数

# 任务 1:汉诺塔。print_move=True 时打印每步
def hanoi(n, src, via, dst, print_move):
    global move_cnt
    if n == 1:                                # 基准情形:只剩一个盘子,直接搬
        if print_move:
            print(f" {src} -> {dst}")
        move_cnt += 1
        return
    hanoi(n - 1, src, dst, via, print_move)    # 1. 上面 n-1 个搬到中转柱
    if print_move:
        print(f" {src} -> {dst}")
    move_cnt += 1                                # 2. 最大盘子搬到目标柱
    hanoi(n - 1, via, src, dst, print_move)    # 3. n-1 个从中转柱搬到目标

# 任务 2:全排列(回溯)。固定前 k 位,把 a[k..] 逐个换到 a[k]
def permute(a, k, result):
    if k == len(a):                            # 基准情形:最后一位也固定了
        result.append(a[:])                  # 拷贝当前排列(避免后续回溯改掉它)
        return
    for i in range(k, len(a)):
        a[k], a[i] = a[i], a[k]              # 把 a[i] 换到第 k 位
        permute(a, k + 1, result)            # 递归排列剩余部分
        a[k], a[i] = a[i], a[k]              # 回溯:换回来,还原现场
```

lab1.py(第二段:任务 3 计时与主程序)

```

# 任务 3:朴素递归斐波那契(用于计时)
def fib_naive(n):
    if n <= 1:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)

def main():
    global move_cnt
    print("==== 任务 1:汉诺塔(n = 3)====")
    hanoi(3, "A", "B", "C", True)
    print(f"移动步数: {move_cnt}(理论 2^3 - 1 = 7)")
    move_cnt = 0 # 清零,再进入批量验证
    for n in range(1, 11): # 验证 2^n - 1
        hanoi(n, "A", "B", "C", False)
        expect = 2 ** n - 1
        print(f"n = {n}: 实际 {move_cnt},理论 {expect}",
              "一致" if move_cnt == expect else "不一致!")
        move_cnt = 0

    print("\n==== 任务 2:全排列(n = 3)====")
    result = []
    permute([1, 2, 3], 0, result)
    for p in result:
        print(" ".join(map(str, p)))
    print(f"排列总数: {len(result)}(理论 3! = 6)")
    result = []
    permute([1, 2, 3, 4], 0, result) # 再验证 4! = 24
    print(f"n = 4 排列总数: {len(result)}(理论 4! = 24)")

    print("\n==== 任务 3:朴素斐波那契计时 ====")
    for n in (28, 31, 34): # n 每次 +3,调用数约 x4.2
        t0 = time.perf_counter()
        f = fib_naive(n)
        ms = (time.perf_counter() - t0) * 1000
        print(f"fib({n}) = {f},耗时 {ms:.1f} ms")
    print("观察: n 每加 3,耗时约乘 4~5(调用数 2*F(n+1) 的比值约 phi^3 = 4.2)")

if __name__ == "__main__":
    main()

```

运行命令: python lab1.py 。示例输出(第一段,n = 3 的移动过程):

===== 任务 1:汉诺塔(n = 3)=====

A -> C
A -> B
C -> B
A -> C
B -> A
B -> C
A -> C

移动步数: 7(理论 $2^3 - 1 = 7$)

n = 1: 实际 1,理论 1 一致
n = 2: 实际 3,理论 3 一致
n = 3: 实际 7,理论 7 一致
n = 4: 实际 15,理论 15 一致
n = 5: 实际 31,理论 31 一致
n = 6: 实际 63,理论 63 一致
n = 7: 实际 127,理论 127 一致
n = 8: 实际 255,理论 255 一致
n = 9: 实际 511,理论 511 一致
n = 10: 实际 1023,理论 1023 一致

示例输出(第二段;耗时因机器而异,比值规律一致):

===== 任务 2:全排列(n = 3)=====

1 2 3
1 3 2
2 1 3
2 3 1
3 2 1
3 1 2

排列总数: 6(理论 $3! = 6$)

n = 4 排列总数: 24(理论 $4! = 24$)

===== 任务 3:朴素斐波那契计时 =====

fib(28) = 317811,耗时 23.0 ms

fib(31) = 1346269,耗时 103.8 ms

fib(34) = 5702887,耗时 433.7 ms

观察: n 每加 3,耗时约乘 4~5(调用数 $2 * F(n+1)$ 的比值约 $\phi^3 = 4.2$)

设计说明:① 三个任务共用"基准情形 + 不断推进"的递归骨架:汉诺塔的基准是 $n == 1$,全排列是"最后一位也固定",斐波那契是 $n \leq 1$ ——同一个思想的三次演练;② 汉诺塔的两次递归调用展示了"参数轮换"技巧,调用次数 $2^n - 1$ 正是第2章递推式 $T(n) = 2T(n-1) + O(1)$ 的实物原型;③ 全排列的"交换 → 递归 → 换回"是回溯法(第10章)的最小模板;④ 任务 3 用真实计时把 $O(2^n)$ 变成肉眼可见:本机 $n = 28/31/34$ 耗时 23/104/434 ms,每加 3 约乘 4~5(比值恰为 $\phi^3 \approx 4.2$);⑤ 打印与计数分离(print_move 开关)让"演示过程"与"批量验证"互不干扰;⑥ `time.perf_counter()` 是第2章所有实验的标准计时器,本实验提前练手。

下一章预告:第2章 算法分析——我们将为大 O 记号建立严格定义,学会对循环与递归做复杂度计算,并把最大子序列和问题从 $O(n^3)$ 一路优化到 $O(n)$ 。本章的递推式、对数与级数公式,正是第2章一切推导的数学地基。

第2章 算法分析

建议学时:4-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解为什么"程序能跑"远远不够——**复杂度**决定程序在数据变大时能不能跑完
- 掌握 O 、 Ω 、 Θ 、 o 四种记号的准确定义与直觉,能说出它们的区别
- 能背诵增长阶排序:常数 < 对数 < 线性 < 线性对数 < 平方 < 立方 < 指数
- 会计算顺序/循环/递归代码的运行时间,能把循环翻译成求和记号 Σ
- 亲手实现**最大子序列和**问题的四种算法,体会 $O(n^3)$ 到 $O(n)$ 的天壤之别
- 能说明二分查找、快速幂、欧几里得算法为什么都是 $O(\log n)$
- 学完能独立用 `time.perf_counter` (C 中对应 `clock()`) 实测验证自己的复杂度分析

💡 从 C 到 Python:本章主题如何迁移

本章是全书唯一"几乎不写新数据结构"的一章,但它是**全书的通用语言**:第 3 章起,每章都以"这个操作 $O(\text{多少})$ "作为结论。算法分析的直觉一句话:**复杂度是数据规模变大时,运行时间的增长方式**——不是秒数,不是毫秒,而是"规模翻倍,时间大约乘以几"。

在 C 里,你也许用过 `clock()` 计时:包含 `time.h`,前后各取一次,除以 `CLOCKS_PER_SEC`。Python 的 `time.perf_counter()` 思想相同但更省事:单调时钟、纳秒级分辨率、一行调用,不受系统修改时间的影响。本章全部计时实验都用它。

Python 还有一个 C 没有的独特优势:**整数任意精度,永不溢出**。做 §2.7 的实测实验时,你可以放心把数组元素取得很大、把 n 推到 10^7 ,而 C/C++ 的 `int` 在累加中可能悄悄溢出变负数。代价是 Python 每条语句都是解释执行、每个整数都是堆上对象——同样 $O(n)$ 的循环,Python 通常比 C++ 慢 30-100 倍。但请记住本章的核心结论:**增长阶是语言无关的**,Python 的 $O(n \log n)$ 在大数据上照样碾压 C++ 的 $O(n^2)$ 。

本章路线:先建立记号体系(§2.1-§2.3),再学计算规则(§2.4),然后在一个真实问题上亲手把 $O(n^3)$ 一路优化到 $O(n)$ (§2.5),最后用计时实验验证理论(§2.6-§2.7)。

📦 前置知识自查

数学前置:指数与对数的基本性质(第1章)、求和记号 Σ 、递归四法则与递推式入门(第1章)。

C 语言前置:单层/嵌套 `for` 循环、函数定义与调用、数组下标遍历; `clock()` 计时(可选,会更好)。

依赖的前面章节:第1章绪论(递归思想、 $\log$ 与指数的运算性质)。

🚀 本章学习路线

1. **第一阶段(1 小时)**:读 §2.1–§2.3, 背下增长阶排序与常见函数值表
2. **第二阶段(1.5 小时)**:读 §2.4–§2.5, 亲手推导最大子序列和四种算法的复杂度, 并在 REPL 里运行验证输出正确
3. **第三阶段(1.5 小时)**:读 §2.6–§2.7, 运行计时程序, 观察“n 翻倍、时间乘几”与理论预测是否吻合
4. **第四阶段(实验, 1 小时)**:完成章末实验: 四种算法的增长阶实测报告
5. **验收标准**:能随口说出“二分查找 $O(\log n)$ ”“快排平均 $O(n \log n)$ ”并解释理由

本章知识导图

- §2.1 为什么要分析算法:从“能跑”到“跑得完”(规模、计时工具、C 的 `clock()` 与 `perf_counter` 对照)
- §2.2 大 O 、 Ω 、 Θ 、小 o 定义、直觉、常见误用
- §2.3 增长阶排序与常见函数:七个等级、数值表
- §2.4 运行时间计算规则:顺序/循环/递归、求和记号
- §2.5 最大子序列和:四种算法, $O(n^3) \rightarrow O(n)$ 的递进
- §2.6 对数级复杂度的三种来源:二分查找、快速幂、欧几里得算法
- §2.7 实体验证:计时方法、n 翻倍实验、常数因子的意义

§2.1 为什么要分析算法

2.1.1 规模决定生死

同一个问题, 算法不同, 命运不同。假设处理一个元素耗时 1 微秒, 那么:

问题规模 n	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
10^3	≈0.01 秒	1 秒	16.7 分钟	约 10^{286} 年
10^4	≈0.13 秒	100 秒	11.6 天	超出宇宙年龄
10^5	≈1.7 秒	2.8 小时	31.7 年	—
10^6	≈20 秒	11.6 天	3.2 万年	—

结论: 指数算法在任何现实规模上都不存在; 平方算法在十万级数据上就要等几小时。这就是为什么必须先学会“估计算法代价”——否则优化方向可能完全错误(去优化一个 $O(n)$ 的循环, 却对 $O(n^2)$ 的嵌套循环视而不见)。

2.1.2 从 C 的 `clock()` 到 Python 的 `perf_counter`

C 的计时套路你也许写过:

C 的写法	Python 的写法
<code>#include <time.h></code>	<code>import time</code>
<code>clock_t t0 = clock();</code>	<code>t0 = time.perf_counter()</code>
<code>double s = (double)(clock() - t0)/CLOCKS_PER_SEC;</code>	<code>elapsed = time.perf_counter() - t0</code>
精度毫秒级,测 CPU 时间	纳秒级分辨率,测墙上时间,单调不受系统改时间影响

示例 2-1 Python 计时框架(本章所有实验都用它)

```
import time

t0 = time.perf_counter()
# ---- 被测代码 ----
s = 0
for i in range(1_000_000):
    s += i
# -----
elapsed = time.perf_counter() - t0
print(f"耗时: {elapsed * 1000:.3f} ms, 结果 = {s}")
```

★ 重点:计时不是分析的全部

计时是验证分析的工具,不是分析本身。同一程序换个机器、换个负载,时间可能差几倍——但“n 翻倍、时间约乘 4”这个比值规律几乎不变。复杂度分析研究的正是这个机器无关的规律。

§2.2 大 O、Ω、Θ、小 o

2.2.1 大 O:上界

定义(实用版): $T(n) = O(f(n))$ 表示存在常数 c 与 n_0 ,使得对所有 $n \geq n_0$,都有 $T(n) \leq c \cdot f(n)$ 。直白地说:当 n 足够大时, $T(n)$ 的增长速度不超过 $f(n)$ 的某个常数倍。例如 $T(n) = 3n^2 + 10n + 7$,取 $c = 4$ 、 n_0 足够大即可证明 $T(n) = O(n^2)$ ——因为 $10n + 7$ 相对 $3n^2$ 微不足道。

所以大 O 会“吞掉”低阶项与常数: $O(3n^2 + 10n + 7) = O(n^2)$, $O(2n) = O(n)$ 。这也是它的局限:两个同为 $O(n)$ 的算法可能实际相差 10 倍。

2.2.2 Ω 与 Θ:下界与紧界

$\Omega(f(n))$ 是对偶记号: $T(n) \geq c \cdot f(n)$ 对所有足够大的 n 成立,表示“增长不低于 $f(n)$ ”。而 $\Theta(f(n))$ 表示“恰好同阶”:既 $O(f(n))$ 又 $\Omega(f(n))$ 。工程上最常用的是大 O(上界保证“最坏不会更差”)与 Θ(精确刻画);当说“这个算法是 $\Theta(n \log n)$ ”时,意思是上下界都压紧了。

2.2.3 小 o 与三种常见误用

小 $o(f(n))$ 表示“增长速度严格低于 $f(n)$ ”(大 O 允许相等,小 o 不允许): $n = o(n^2)$ 成立,但 $n \neq o(n)$ 。它常用于比较两个上界谁更“松”。

🚀 记号说明(本章约定,全书沿用)

正文中的 $O(n)$ 读作"大 O n"; n^2 表示 n 的平方; $\log_2 n$ 表示以 2 为底的对数, 不写底数时 $\log n$ 默认以 2 为底(常数倍差异被大 O 吞掉, 所以底数不重要, 但推导过程要写清楚); $\ln$ 表示自然对数。求和写成 $\sum_{i=1}^n i = n(n+1)/2$ 。代码里一律使用语言语法(`%`、`>>`、`pow`), 不使用数学记号。

⚠ 误区 1: 把常数当成增长阶

写 $O(2n)$ 、 $O(100n \log n)$ 、 $O(n+100)$ 都是错误的: 大 O 定义明确吞掉常数, 应写 $O(n)$ 、 $O(n \log n)$ 、 $O(n)$ 。"常数不重要"仅在分析增长阶时成立; 实际工程选型时, 两个 $O(n)$ 算法的常数差距可能决定生死。

⚠ 误区 2: 混淆最坏情形与记号本身

"快排是 $O(n^2)$ 的"——这句话必须补全: 快排最坏情形是 $O(n^2)$, 平均情形是 $O(n \log n)$ 。大 O 描述的是某个约定情形(最坏/平均/最好)下的增长阶, 不是算法本身的属性。今后每章的复杂度速查表都会区分"平均"与"最坏"两列。

§2.3 增长阶排序与常见函数

2.3.1 七个等级

- $O(1)$ 常数: 按下标访问列表元素、dict 查找(平均)
- $O(\log n)$ 对数: 二分查找—— n 翻倍只多一步
- $O(n)$ 线性: 一趟遍历、线性搜索
- $O(n \log n)$ 线性对数: 归并排序、快排(平均)、堆排序
- $O(n^2)$ 平方: 二重循环、插入/冒泡排序、朴素矩阵乘法
- $O(n^3)$ 立方: 三重循环、Floyd 全源最短路
- $O(2^n)$ 指数: 穷举、朴素递归斐波那契——现实规模下不可用

2.3.2 数值表: 让直觉落地

$f(n)$	$n=10$	$n=100$	$n=10^3$	$n=10^4$	$n=10^5$	$n=10^6$
$\log_2 n$	3.3	6.6	10	13	17	20
n	10	100	10^3	10^4	10^5	10^6
$n \log_2 n$	33	664	10^4	1.3×10^5	1.7×10^6	2×10^7
n^2	100	10^4	10^6	10^8	10^{10}	10^{12}
n^3	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}
2^n	1024	约 10^{30}	约 10^{301}	约 10^{3010}	约 10^{30103}	约 10^{301030}

读这张表的要领: 从 $n=10^3$ 到 10^6 , 规模扩大 1000 倍—— n 增长 1000 倍, n^2 增长 100 万倍, 2^n 则直接冲出宇宙。同时注意 $\log_2 n$ 一行: 规模增长 1000 倍, 它只从 10 长到 20。对数函数是"几乎不增长"的函数, 凡是对数级的算法都值得信赖。Python 的大整数能精确算出这张表里 2^{100} 的每一位数字——不妨在 REPL 里亲自验证一下。

⚠ 误区 3:把 $2n$ 与 n^2 混为一谈

2^n 与 n^2 只差一个“位置”,命运完全不同: $n = 64$ 时, $n^2 = 4096$, 而 $2^n \approx 1.8 \times 10^{19}$ 。判断口诀:底数是 n 的多项式,指数是 n 的才是指数级。 1.1^n 也是指数级(底数略大于 1 仍是灾难), n^{100} 再大也只是多项式。

§2.4 运行时间计算规则

2.4.1 三条基本规则

- **顺序相加**:两个语句依次执行,总时间取两者之和,大 O 取最大者: $O(n) + O(n^2) = O(n^2)$
- **嵌套相乘**:循环体内嵌套循环,总时间相乘
- **分支取最大**:if/else 取两个分支中较慢的一个

2.4.2 循环与求和记号

循环本质上是一个求和:第 i 次迭代的工作量 $w(i)$,总工作量 $= \sum_i w(i)$ 。例:

示例 2-2 嵌套循环的求和推导

```
# 推导:内层循环执行  $n - i$  次
# 总次数  $= \sum_{i=0}^{n-1} (n - i) = n + (n-1) + \dots + 1 = n(n+1)/2$ 
# 所以是  $O(n^2)$ , 而不是“感觉上的  $n^2/2$ ”—常数  $1/2$  被大  $O$  吞掉
# 对照:C 的写法 for (i = 0; i < n; i++) 完全一致,结论不变
cnt = 0
for i in range(n):
    for j in range(i, n):
        cnt += 1
```

三个最常用的求和公式(本章及全书反复使用):

- $\sum_{i=1}^n i = n(n+1)/2$ (算术级数, $O(n^2)$)
- $\sum_{i=0}^n 2^i = 2^{n+1} - 1$ (几何级数, $O(2^n)$)
- $\sum_{i=1}^n 1/i \approx \ln n$ (调和级数, $O(\log n)$)

2.4.3 递归:递推式入门

递归代码的复杂度由**递推式**描述: $T(n) = 2T(n/2) + n$ 表示“把规模 n 的问题分成两个 $n/2$ 的子问题,再花 $O(n)$ 合并”。逐层展开:第 1 层代价 n ,第 2 层 $2 \cdot (n/2) = n$,第 3 层 $4 \cdot (n/4) = n \dots$ 共 $\log_2 n$ 层,每层 n ,总计 $O(n \log n)$ 。§2.5 的算法三正是这个递推式,§2.6 的三个对数算法则是 $T(n) = T(n/2) + O(1)$ 的特例——每层只进一个分支,共 $\log_2 n$ 层,每层常数代价。

★ 重点:两种最经典的递推式

$T(n) = T(n/2) + O(1) \rightarrow O(\log n)$ (二分查找); $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ (归并/分治)。全书的对数与线性对数复杂度,九成来自这两个式子。

§2.5 最大子序列和:四种算法的递进

2.5.1 问题描述

给定整数序列 $a_1, a_2, \dots, a_n$ (可含负数), 求其**连续子序列的最大和**; 全为负数时约定答案为 0 (空子序列)。例如 $[-2, 11, -4, 13, -5, -2]$ 的最大子序列是 $[11, -4, 13]$, 和为 20。这是 Weiss 教材的经典例题: 同一个问题, 四种算法, 复杂度相差 100 万倍。

2.5.2 算法一: $O(n^3)$ 暴力枚举

最直接的思路: 枚举所有起点 i 与终点 j , 再累加 $a[i..j]$ 。双重循环枚举起终点, 内层再用 `sum()` 花 $O(n)$ 求和, 总代价 $\sum \sum (j-i+1) \approx n^3/6$, 即 $O(n^3)$ 。

示例 2-3 算法一: 枚举 + `sum()`, $O(n^3)$

```
def max_sub_sum1(a):
    n = len(a)
    max_sum = 0
    for i in range(n):           # 起点
        for j in range(i, n):    # 终点
            this_sum = sum(a[i:j+1]) # 切片求和, 本身就是  $O(j-i+1)$ 
            if this_sum > max_sum:
                max_sum = this_sum
    return max_sum
```

2.5.3 算法二: $O(n^2)$ 复用中间和

观察: 固定起点 i , 当终点从 j 推进到 $j+1$ 时, 新和 = 旧和 + $a[j+1]$, 不需要重算整个区间。去掉切片求和, 复杂度降到 $\sum 1 = n(n+1)/2 = O(n^2)$ 。

示例 2-4 算法二: 增量更新, $O(n^2)$

```
def max_sub_sum2(a):
    max_sum = 0
    for i in range(len(a)):
        this_sum = 0
        for j in range(i, len(a)):
            this_sum += a[j]      # 增量更新, 不再重算
            if this_sum > max_sum:
                max_sum = this_sum
    return max_sum
```

2.5.4 算法三: $O(n \log n)$ 分治

分治三步: 把数组分成左右两半, 递归求左半最大和与右半最大和, 再求**跨中界的最大和**(从中界向左延伸的最大后缀和 + 向右延伸的最大前缀和), 三者取最大。递推式正是 $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ 。递归深度只有 $\log_2 n$, 即使 $n = 10^6$ 也不到 20 层, Python 默认 1000 层的递归上限完全够用。

示例 2-5 算法三: 分治, $O(n \log n)$

```

def max_sum_rec(a, left, right):
    if left == right:                # 基准情形:单个元素
        return max(a[left], 0)

    center = (left + right) // 2
    max_left = max_sum_rec(a, left, center)    # 左半
    max_right = max_sum_rec(a, center + 1, right) # 右半

    max_left_border = left_border = 0    # 跨中界:向左延伸
    for i in range(center, left - 1, -1):
        left_border += a[i]
        max_left_border = max(max_left_border, left_border)
    max_right_border = right_border = 0    # 向右延伸
    for i in range(center + 1, right + 1):
        right_border += a[i]
        max_right_border = max(max_right_border, right_border)

    return max(max_left, max_right,
               max_left_border + max_right_border)

def max_sub_sum3(a):
    return max_sum_rec(a, 0, len(a) - 1)

```

2.5.5 算法四:O(n) 在线算法

关键观察:和为负的前缀不可能是最优解的起点——负前缀只会拖累后面。于是扫描一遍:累加当前和 `this_sum`,一旦 `this_sum` 变成负数就清零重来;途中记录最大值。

示例 2-6 算法四:在线算法, $O(n)$

```

def max_sub_sum4(a):
    max_sum = this_sum = 0
    for x in a:
        this_sum += x
        if this_sum > max_sum:
            max_sum = this_sum
        elif this_sum < 0:
            this_sum = 0    # 负前缀弃之,从下一个元素重新开始
    return max_sum

```

例题 2-1 手工走一遍算法四

输入 `[-2, 11, -4, 13, -5, -2]`:`this_sum` 依次为 `-2→0(清零)`、`11`、`7`、`20`、`15`、`13`; `max_sum` 记录峰值为 `20`。答案 `20`,与算法一致。**注意:**清零动作丢弃的是“负前缀”而不是负数本身——`a[i]` 单独为负但前缀仍正时不会清零。

★ 重点:四种算法问题对照

算法	思想	复杂度	$n=10^6$ 时(1 微秒/步)
一	暴力枚举	$O(n^3)$	约 317 年
二	增量更新	$O(n^2)$	约 11.6 天
三	分治	$O(n \log n)$	约 20 秒
四	在线扫描	$O(n)$	约 1 秒

§2.6 对数级复杂度的三种来源

2.6.1 二分查找

在有序数组中查找 x : 每次把候选区间减半。区间长度 $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$, 共 $\log_2 n$ 步, 即递推式 $T(n) = T(n/2) + O(1)$ 。

示例 2-7 二分查找, $O(\log n)$

```
# 在升序列表 a 中查找 x, 返回下标; 找不到返回 -1
def binary_search(a, x):
    low, high = 0, len(a) - 1
    while low <= high:
        mid = (low + high) // 2
        if a[mid] < x:
            low = mid + 1
        elif a[mid] > x:
            high = mid - 1
        else:
            return mid
    return -1
```

示例 2-8 标准库写法: bisect 一行搞定

```
import bisect
a = [1, 3, 5, 7, 9]
i = bisect.bisect_left(a, 5)    # 返回 2: 第一个 >= 5 的位置
j = bisect.bisect_left(a, 6)    # 返回 3: 6 应插在 7 之前
bisect.insort_left(a, 6)        # 保持有序地插入: 1 3 5 6 7 9
k = bisect.bisect_right(a, 5)   # 返回 3: 第一个 > 5 的位置
n = bisect.bisect_right(a, 5) - i # 等于 5 的元素个数: 1
print(i, j, k, n, a)           # 2 3 3 1 [1, 3, 5, 6, 7, 9]
```

C 的手写二分查找	Python 的标准库
<pre>int bin(int a[], int n, int x)</pre> 手写约 12 行	<code>bisect_left / bisect_right / insort</code>
边界条件 (<code><=</code> 还是 <code><</code>) 极易写错成死循环	边界由标准库保证, 写错直接抛异常暴露
数组要单独传长度 n	<code>bisect.bisect_left(a, x)</code> 一行, 列表自含长度

手写 vs 标准库:学习时手写一遍理解"减半"骨架;工程中直接用 `bisect` ——它用 C 实现,而且避免了边界条件(`<=>` 写错一个就死循环)的错误。同一思想,C++ 标准库对应 `std::binary_search` 与 `std::lower_bound`。

2.6.2 快速幂

计算 $a^n \bmod m$:朴素做法乘 n 次是 $O(n)$;利用 $a^n = (a^2)^{n/2}$ 每次把指数减半, $O(\log n)$ 。这是 C 里你可能用循环写过的算法,与二分查找共享"减半"骨架。

示例 2-9 快速幂, $O(\log n)$

```
MOD = 10**9 + 7

def pow_mod(a, n):
    """计算 (a^n) % MOD, n >= 0。"""
    result = 1
    while n > 0:
        if n & 1:                # 当前二进制位为 1 则乘入
            result = result * a % MOD
        a = a * a % MOD          # a 平方:对应指数减半
        n >>= 1                  # n 右移一位 = n//2
    return result

# Python 内置 pow 三参数版直接就是快速幂:
print(pow_mod(2, 10), pow(2, 10, MOD)) # 1024 1024
```

2.6.3 欧几里得算法

求最大公因数 $\gcd(a, b)$:迭代执行 $(a, b) \rightarrow (b, a \% b)$ 直到 $b = 0$ 。结论:每两步至少把较小的数减半(证明略),所以迭代次数 $O(\log n)$ 。

示例 2-10 欧几里得算法, $O(\log n)$

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b        # 元组打包/解包,C 里要写三行
    return a

import math
print(gcd(48, 36), math.gcd(48, 36)) # 12 12,标准库同样有现成的
print(gcd(1234567, 891011))         # 大数同样两步减半,依旧很快
```

★ 重点:log 从哪来?

三个算法的共同骨架: 每一步把"问题规模"至少减半。二分查找减半区间,快速幂减半指数,欧几里得每两步减半数。凡是见到"减半""倍增"字样的算法,先猜 $O(\log n)$ 再验证,十有八九正确。

§2.7 实测验证:让理论落地

2.7.1 计时方法要点

- n 要足够大:太小则计时噪声淹没规律(§2.7.2 用 $n \geq 1000$)
- 重复取平均或取中位数,避免一次性的调度抖动

- 关闭不必要的后台程序,尽量独占 CPU
- **警惕 Python 特有偏差:**内置函数(如 `sum`、`min`)是 C 实现的,比手写循环快一个数量级——比较"算法"时必须让双方使用同样风格的代码,否则比的是实现语言而非算法

C 的计时要点	Python 的计时要点
<code>clock()</code> 测 CPU 时间,受调度影响	<code>perf_counter()</code> 测墙上时间,单调不回拨
除以 <code>CLOCKS_PER_SEC</code> 得到秒,容易忘记	返回值就是秒(浮点),直接相减
手动 <code>volatile</code> 防优化,易漏	解释器不做死代码消除,但结果同样要输出

2.7.2 n 翻倍实验(Python 实测参考值)

对随机数组运行四种算法,每次 n 翻倍,观察时间比值:

n	算法一	算法二	算法三	算法四
1000	24 ms	21 ms	1.6 ms	0.06 ms
2000	186 ms	84 ms	3.4 ms	0.12 ms
4000	≈1.5 s(推算)	336 ms	7.2 ms	0.24 ms
8000	≈12 s(推算)	1.35 s	15 ms	0.48 ms

读出规律:n 翻倍,算法一时间 ×8(吻合 n^3),算法二 ×4(吻合 n^2),算法三、四约 ×2(吻合 $n \log n$ 与 n)。比值比绝对值可靠——你的机器数值不同,但比值规律必然一致。与 C++ 版对比:绝对时间慢 30–100 倍,但比值规律完全相同——增长阶是语言无关的。

工程建议:常数因子的真实意义

大 O 相同不代表同样快:同为 $O(n \log n)$,快排的常数通常只有堆排的一半;同为 $O(n)$,遍历 `list` 比遍历手写链表快数倍(缓存友好,第3章细讲)。在 Python 里这条原则加倍重要:`sum(a[i:j+1])` 是 C 实现的,写起来只有一行,但它是 $O(j-i+1)$ ——常数小救不了复杂度差。工程选型时:先用大 O 淘汰指数/高次算法,再用常数因子与实测在候选者中择优。

误区 4:小规模下拿 O 记号赌博

$n = 20$ 时, $O(n^2)$ 的插入排序往往比 $O(n \log n)$ 的快排更快——常数小。大 O 的结论只在“n 足够大”时成立(定义里的 n_0 正是为此)。工程上排序库普遍采用“大区间快排 + 小区间插入排序”的混合策略(第7章),就是这条原则的产物。

本章复杂度速查表

操作 / 算法	数据结构 / 算法	平均	最坏
按值查找	无序列表	$O(n)$	$O(n)$
二分查找	有序列表	$O(\log n)$	$O(\log n)$
求 a^n	朴素连乘	$O(n)$	$O(n)$
求 a^n	快速幂 / 内置 pow(三参数)	$O(\log n)$	$O(\log n)$
最大公因数	欧几里得算法 / math.gcd	$O(\log n)$	$O(\log n)$
最大子序列和	算法一(暴力)	$O(n^3)$	$O(n^3)$
最大子序列和	算法二(增量)	$O(n^2)$	$O(n^2)$
最大子序列和	算法三(分治)	$O(n \log n)$	$O(n \log n)$
最大子序列和	算法四(在线)	$O(n)$	$O(n)$
循环计数	单层循环	$O(n)$	$O(n)$
循环计数	双重循环	$O(n^2)$	$O(n^2)$
递归	$T(n) = T(n/2) + O(1)$	$O(\log n)$	$O(\log n)$
递归	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$	$O(n \log n)$
增长阶	常数 / 对数 / 线性 / 线性对数 / 平方 / 立方 / 指数	见 §2.3.1 排序	
增长阶排序	常数 $< \log n < n < n \log n < n^2 < n^3 < 2^n$	全书唯一需要背诵的排序	

最小必会清单

- 能独立说出大 O 、 Ω 、 Θ 、小 o 的定义与区别,并各举一例
- 能背诵增长阶排序,并给每个等级配一个真实算法例子
- 能独立推导单层循环、嵌套循环、if/else 的复杂度
- 能独立写出四种最大子序列和算法,并说出各自复杂度与推导
- 能独立写出二分查找、快速幂、gcd 的代码,并解释为什么是 $O(\log n)$
- 能写出 $T(n) = T(n/2) + O(1)$ 与 $T(n) = 2T(n/2) + O(n)$ 的解
- 能解释"最坏情形"与"平均情形"的区别,并正确表述快排的复杂度
- 能独立用 perf_counter 做 n 翻倍实验,判断实验比值与理论增长阶是否吻合

自测题

Q 1. $O(2n)$ 与 $O(n)$ 是否相等?为什么?

✅ 参考答案

相等。大 O 定义允许任意常数倍:取 $c = 2$ 即有 $2n \leq 2 \cdot n, 2n = O(n)$ 。反过来 $n \leq 1 \cdot 2n$, 所以两者互为对方的 O , 增长阶相同。写 $O(2n)$ 是不规范的。

Q 2. 某算法运行时间 $T(n) = 3n^2 + 10n + 7$, 它的 O 、 Ω 、 Θ 分别可以写什么?

✓ 参考答案

$O(n^2)$ (也允许 $O(n^3)$ 等更松的上界,但不规范); $\Omega(n^2)$ (也允许 $\Omega(n)$ 等更松的下界); $\Theta(n^2)$ 是唯一"紧"的写法。

Q 3. 为什么说算法三(分治)比算法二(增量)快?写出算法三的递推式并解释每层代价。

✓ 参考答案

$T(n) = 2T(n/2) + n$:把数组分为两半递归($2T(n/2)$),再花 $O(n)$ 求跨中界最大和。展开后第 k 层有 2^k 个规模 $n/2^k$ 的子问题,每层总代价都是 n ,共 $\log_2 n$ 层,总计 $n \log_2 n$ 。 $O(n \log n)$ 在 $n = 10^6$ 时约 2×10^7 步,而 $O(n^2)$ 是 10^{12} 步,差 5 万倍。

Q 4. 二分查找每次把区间减半,为什么复杂度是 $O(\log n)$ 而不是 $O(n/2)$?

✓ 参考答案

区间长度依次为 $n, n/2, n/4, \dots, 1$,问"减半多少次到 1"——这是对数: $2^k = n$,即 $k = \log_2 n$ 次。 $n/2$ 是"一次减半后的长度",不是步数;把长度当步数是常见错误。 10^6 个元素只需约 20 次比较。

Q 5. $n = 1000$ 时, $O(n^2)$ 与 $O(n \log n)$ 的步数约相差多少倍? $n = 10^6$ 呢?

✓ 参考答案

$n = 1000: 10^6 / (1000 \times 10) = 100$ 倍; $n = 10^6: 10^{12} / (10^6 \times 20) = 5 \times 10^4$ 倍。规模越大差距越大——这就是为什么"复杂度差的算法靠硬件换代救不回来"。

Q 6. Python 程序比 C++ 慢约 50 倍,那么 $O(n)$ 的 Python 算法与 $O(n \log n)$ 的 C++ 算法,数据规模多大时 Python 反而更快?

✓ 参考答案

比较 $50n$ 与 $n \log_2 n$:当 $\log_2 n > 50$,即 $n > 2^{50} \approx 10^{15}$ 时 Python 更快——现实规模基本达不到,所以"语言换效率"在增长阶面前通常无效。反过来: $O(n^2)$ 的 C++ 在大 n 下照样被 $O(n \log n)$ 的 Python 反超。

章末实验题:四种算法的增长阶实测报告

实验 2:验证最大子序列和四种算法的增长阶

实验目标:实现四种最大子序列和算法,对倍增规模计时,用实测比值验证 $O(n^3)/O(n^2)/O(n \log n)/O(n)$ 的增长阶预测。

实验要求:

- 任务 1:实现 `max_sub_sum1..4` 四个函数(知识点:\$2.5 四种算法)
- 任务 2:用 `perf_counter` 计时,输出 $n = 1000/2000/4000/8000$ 的时间表(知识点:\$2.1 计时工具)
- 任务 3:算法一在 $n \geq 4000$ 时跳过并标注"(推算)",避免无谓等待(知识点:\$2.3 指数/立方增长不可用)
- 任务 4:输出" n 翻倍时各算法时间比值",与理论值($\times 8/\times 4/\times 2/\times 2$)对比并写一段结论(知识点:\$2.7 实测验证)

实验环境:Python 3.8+,标准库即可。运行: `python lab2.py`。

第一步 写四个算法函数,用 \$2.5 的样例数组 `[-2, 11, -4, 13, -5, -2]` 验证四个函数都输出 20,再写计时主程序

第二步 机数用 `random.seed(12345) + random.randint(-1000, 1000)` 生成,保证可复现

第三步 时用 `perf_counter`,每次只测一次即可(时间基数大,噪声占比小);把四次调用结果累加并输出,防止解释器优化掉调用

第四步 比值分析:对算法二,若 n 翻倍时间约 $\times 4$,则吻合 n^2 ;对算法三、四约 $\times 2$;算法一只测两个规模并说明 $\times 8$ 趋势

✓ 参考答案(完整可运行程序,约 70 行,分两段)

```

# lab2.py — 最大子序列和:四种算法的增长阶实测(第一段:算法)
import random
import time

# ----- 四种算法($2.5) -----
def max_sub_sum1(a):                                #  $O(n^3)$ 
    n = len(a)
    max_sum = 0
    for i in range(n):
        for j in range(i, n):
            this_sum = sum(a[i:j+1])
            if this_sum > max_sum:
                max_sum = this_sum
    return max_sum

def max_sub_sum2(a):                                #  $O(n^2)$ 
    max_sum = 0
    for i in range(len(a)):
        this_sum = 0
        for j in range(i, len(a)):
            this_sum += a[j]
            if this_sum > max_sum:
                max_sum = this_sum
    return max_sum

def max_sum_rec(a, left, right):                    #  $O(n \log n)$  分治
    if left == right:
        return max(a[left], 0)
    center = (left + right) // 2
    max_left = max_sum_rec(a, left, center)
    max_right = max_sum_rec(a, center + 1, right)
    max_left_border = left_border = 0
    for i in range(center, left - 1, -1):
        left_border += a[i]
        max_left_border = max(max_left_border, left_border)
    max_right_border = right_border = 0
    for i in range(center + 1, right + 1):
        right_border += a[i]
        max_right_border = max(max_right_border, right_border)
    return max(max_left, max_right,
               max_left_border + max_right_border)

def max_sub_sum3(a):
    return max_sum_rec(a, 0, len(a) - 1)

def max_sub_sum4(a):                                #  $O(n)$  在线算法
    max_sum = this_sum = 0
    for x in a:
        this_sum += x
        if this_sum > max_sum:
            max_sum = this_sum
        elif this_sum < 0:
            this_sum = 0
    return max_sum

```



```
# ----- 计时($2.1) -----
```

lab2.py(第二段:计时与主程序)

```
def ms_time(func, a):
    t0 = time.perf_counter()
    result = func(a)
    return (time.perf_counter() - t0) * 1000, result

def main():
    # 正确性验证($2.5 样例)
    demo = [-2, 11, -4, 13, -5, -2]
    results = [max_sub_sum1(demo), max_sub_sum2(demo),
               max_sub_sum3(demo), max_sub_sum4(demo)]
    print(f"验证: {results} (应全为 20)")

    random.seed(12345)                # 固定种子,可复现
    guard = 0
    print("\n\t算法一\t算法二\t算法三\t算法四")
    for n in (1000, 2000, 4000, 8000):
        a = [random.randint(-1000, 1000) for _ in range(n)]
        row = [str(n)]
        if n <= 2000:
            t, r = ms_time(max_sub_sum1, a)
            row.append(f"{t:.1f}")
            guard += r
        else:
            row.append("--(推算)")        #  $O(n^3)$  在 4000 时约 1.5 秒,不实测
        for f in (max_sub_sum2, max_sub_sum3, max_sub_sum4):
            t, r = ms_time(f, a)
            row.append(f"{t:.3f}")
            guard += r
        print("\t".join(row))
    print(f"(防优化累加值: {guard % 1000})")
    print("结论: n 翻倍时,算法一  $\times 8$ (吻合  $n^3$ ),算法二  $\times 4$ (吻合  $n^2$ ),
          "算法三/四约  $\times 2$ (吻合  $n \log n$  与  $n$ )。")

if __name__ == "__main__":
    main()
```

示例输出(机器不同数值不同,比值规律一致):

```
验证: [20, 20, 20, 20] (应全为 20)
n      算法一   算法二   算法三   算法四
1000   24.0     21.00    1.600    0.060
2000  186.0     84.00    3.400    0.120
4000  --(推算) 336.0     7.200    0.240
8000  --(推算) 1350.0    15.00    0.480
(防优化累加值: 321)
结论: n 翻倍时,算法一  $\times 8$ (吻合  $n^3$ ),算法二  $\times 4$ (吻合  $n^2$ ),算法三/四约  $\times 2$ (吻合  $n \log n$  与  $n$ )。
```

设计说明:① 四个算法共用 `ms_time` 计时函数,把"被测函数"当参数传入——Python 函数是一等对象,这正是第10章高阶函数思想的雏形,也对应 C 的函数指针;② `guard` 累加所有返回值并输出,防止解释器/编译器优化掉调用;③

算法一在 $n \geq 4000$ 时按 §2.3 的数值表推算($\times 8$ 每翻倍),避免浪费时间在早已被理论淘汰的算法上;④ 种子固定为 12345,任何机器复现本实验都得到相同的数组。

下一章预告:第3章 表、栈和队列——三种最基础的数据结构及其数组/链式实现。第3章起,每章末尾的复杂度速查表将直接使用本章的记号;最大子序列和的分治骨架也将在第7章排序中再次登场。

第3章 表、栈和队列

建议学时:4-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解**抽象数据类型(ADT)**思想:先定义"做什么",再谈"怎么实现"
- 掌握表的两类实现:数组实现(倍增扩容)与链式实现(结点+引用),会对比两者复杂度
- 完成从 C 的 `malloc` 数组到 Python `list` 的迁移,理解"list 就是动态数组"
- 能说清 Python 中为什么**几乎不需要手写链表**,以及什么时候仍然要写
- 能独立实现栈的数组版与链式版,并用栈解决括号平衡、后缀求值、中缀转后缀三个经典问题
- 能独立实现循环数组队列,说清空/满判断的两种方案
- 学完能独立实现中缀表达式计算器(栈应用综合),并会正确使用 `list` 模拟栈与 `collections.deque` 队列

💡 从 C 到 Python:本章主题如何迁移

本章是全书第一个"真正动手实现数据结构"的章节。算法直觉先给结论:表、栈、队列是后续所有复杂结构的积木——树的层序遍历要用队列,图的 DFS 要用栈,第6章的堆本质是数组,第5章的散列表是"数组+链表"的合体。本章学不透,后面每一章都会还债。

在 C 里,你已经会两种原始工具:`malloc` 出来的动态数组和 `struct` + 指针串起来的链表。C 的痛点是双重的:动态数组要自己管理"容量、长度、扩容"三件事,一次 `realloc` 写错就是内存泄漏或堆损坏;链表没有语言支持,每个操作都要手写指针舞蹈,一不小心就访问悬垂指针。

Python 的解法是**把动态数组直接内建成语言的一部分**:`list` 就是自动扩容的动态数组,`append` 平均 $O(1)$,垃圾回收器替你释放内存——C 里最头疼的"容量、长度、扩容、释放"四件事全部消失。这是巨大的解放,但有两个代价:① 没有"裸数组"让你看到底层,扩容细节藏在解释器里(\$3.3.1 会手写一遍把原理补回来);② `list` 的元素是指向对象的引用,与 C 的"连续存放的 `int`"不同——不过对本章内容影响不大。

链表在 Python 里地位特殊:**几乎不需要手写**——因为 `list` 太方便,而且链表的 $O(1)$ 头插优势 Python 有 `deque` 承担。但"链表"作为概念仍然必须掌握:第4章的树、第5章的分离链接散列表都建立在"结点+引用"之上。本章把链表当作"指针思维的训练场",Python 版用"对象引用"替代指针。

栈与队列部分,本章用三个经典应用(括号平衡、后缀求值、中缀转后缀)演示"数据结构服务于算法"的思维方式——它们是编译原理与表达式求值的原型,也是各大面试的高频题。

📦 前置知识自查

数学前置:本章几乎不需要新数学;倍增扩容的摊还分析直觉即可(严格分析在第11章)。

C 语言前置:`malloc` / `free` 动态数组、`struct` 与 `->` 指针访问、`char*` 字符串遍历。

依赖的前面章节:第1章递归(栈的递归实现对照)、第2章大 O 记号(本章复杂度速查表直接用)。

🚀 本章学习路线

1. **第一阶段(1 小时)**:读 §3.1–§3.2,建立 ADT 概念,在 REPL 里实验 list 的 append/insert/pop
2. **第二阶段(1.5 小时)**:读 §3.3–§3.4,手写动态数组与单链表,上机验证
3. **第三阶段(1.5 小时)**:读 §3.5–§3.6,手写栈与队列,完成三个栈应用例题
4. **第四阶段(实验,1 小时)**:完成章末实验中缀表达式计算器
5. **验收标准**:能默写单链表插入/删除、循环队列、中缀转后缀,并说清每步复杂度

本章知识导图

- §3.1 ADT:抽象数据类型(接口与实现分离、从 C 的 struct+函数指针到类)
- §3.2 表 ADT 与从 C 的迁移(malloc 数组 → list、倍增扩容、list 的底层真相)
- §3.3 表的数组实现(手写动态数组、list 常用操作、切片与遍历)
- §3.4 表的链式实现(结点与引用、手写单链表、“几乎不手写链表”的工程判断)
- §3.5 栈(list 模拟、括号平衡、后缀求值、中缀转后缀)
- §3.6 队列(循环数组实现、空/满判断的两种方案、collections.deque)
- §3.7 选型与工程实践(何时数组何时链表、栈模拟递归、调试技巧)

§3.1 ADT:抽象数据类型

3.1.1 什么是 ADT

抽象数据类型(ADT) = 一组操作 + 这些操作的语义, **不涉及实现细节**。例如“栈”这个 ADT 只有三个操作:push(入栈)、pop(出栈)、top(取栈顶),语义是“后进先出(LIFO)”。至于底层用数组还是链表、满时怎么办,属于“实现”,与 ADT 无关。好处: **使用者只依赖接口,实现可以随时替换**——把数组栈换成链式栈,使用方代码一行不用改。

3.1.2 从 C 的 struct+函数指针到 Python 的类

C 语言没有类,模拟 ADT 只能用 struct 存数据 + 函数指针表存操作:

C 的写法(struct + 函数指针)	Python 的写法(class 封装)
<pre>struct Stack { int *a; int top; int cap; };</pre>	<pre>class Stack: def __init__(self): self.a = []</pre>
<pre>stack_init(&s); stack_push(&s, x);</pre> 每个操作传 &s	s.push(x) 数据与操作天然绑定
使用者能“意外”访问 s.a[0] 破坏结构	self.a 靠约定保护(_ 前缀,Python 无强制 private)
忘记 stack_destroy 即泄漏,没有提醒	垃圾回收器自动释放,无需操心

结论:ADT 的思想两种语言都能表达,Python 的类是它的自然载体——本书所有数据结构都以“类”的形式呈现。

§3.2 表 ADT 与从 C 的迁移

3.2.1 表的基本操作

表(list)ADT:线性有序元素的集合,常用操作——按下标访问、按值查找、任意位置插入/删除、遍历。**同一个 ADT,两种截然不同的实现**(数组与链表),各有胜负,本章的主线就是这张对照表:

操作	数组实现(list)	链式实现
按下标访问	$O(1)$	$O(n)$
按值查找	$O(n)$	$O(n)$
尾部插入	平均 $O(1)$	$O(1)$
头部插入	$O(n)$	$O(1)$
任意位置插入/删除	$O(n)$ (移动元素)	$O(1)$ (已定位后)

3.2.2 C 的 malloc 数组 → Python 的 list

回忆 C 里管理动态数组的三件套:记录长度、记录容量、`realloc` 扩容。Python 的 `list` 把这套动作全部内置: `len(a)` 是长度;容量没有直接接口(可用 `sys.getsizeof` 侧面观察);扩容自动发生,策略是**按比例增长**(CPython 实现约 1/8 增量,思想与倍增相同)。

C 的写法	Python 的写法
<code>int *a = malloc(n * sizeof(int));</code>	<code>a = [0] * n</code> 或 <code>a = []</code>
<code>a[len++] = x;</code> (自己管长度)	<code>a.append(x)</code>
<code>realloc</code> 手动扩容(易错)	自动扩容,解释器管理
<code>free(a);</code> (忘记即泄漏)	垃圾回收器自动释放
<code>a[i]</code> 越界 = 未定义行为(可能崩溃)	<code>a[i]</code> 越界抛 <code>IndexError</code> ,显式暴露错误

倍增扩容:当 `append` 时容量已满,解释器分配一块更大的新内存,拷贝全部旧元素的引用,释放旧内存。直觉上“每次扩容都是 $O(n)$,那插入怎么还说平均 $O(1)$?”——因为扩容发生得越来越少:容量从 1 倍增到 n 需要 $\log_2 n$ 次扩容,总拷贝代价 $n + n/2 + n/4 + \dots \approx 2n$,摊到 n 次插入上平均每次 $O(1)$ 。严格的摊还分析见第11章,现在先记住结论: **倍增使得尾部插入平均 $O(1)$** 。

⚠ 误区 1:把 list 当 C 数组用

两个迁移坑:① `a[i] = x` 不会自动扩容——`a = []` 后直接 `a[0] = 1` 抛 `IndexError` (与 C 的未定义行为不同,Python 显式报错,这反而是好事);② `a = [0] * 5` 中的元素是**同一个对象 0** 的 5 份引用,对不可变对象无感,但写 `m = [[]] * 5` 会得到 5 个**共享**的空列表——`m[0].append(1)` 后 `m[1]` 也变了。要独立列表用 `[[] for _ in range(5)]`。

§3.3 表的数组实现

3.3.1 手写动态数组:搞懂倍增

示例 3-1 手写动态数组:倍增扩容

```
# 手写动态数组:容量不足时倍增(2 倍)拷贝
# 工程中直接用 list 即可,这里手写是为了看清 append 的底层
class MyVector:
    def __init__(self):
        self._data = [None] * 1      # 底层存储,初始容量 1
        self._size = 0               # 元素个数(长度)
        self._cap = 1               # 已分配容量

    def _grow(self):                 # 倍增:新容量 = 旧容量 × 2
        new_cap = self._cap * 2
        new_data = [None] * new_cap
        for i in range(self._size):
            new_data[i] = self._data[i]  # 拷贝旧元素
        self._data = new_data
        self._cap = new_cap

    def push_back(self, x):
        if self._size == self._cap: # 满了:扩容
            self._grow()
        self._data[self._size] = x
        self._size += 1

    def __getitem__(self, i):        # 支持 v[i] 下标访问
        return self._data[i]

    def __len__(self):               # 支持 len(v)
        return self._size

    @property
    def capacity(self):
        return self._cap
```

3.3.2 list 常用操作速览

示例 3-2 list 核心操作

```
v = []                # 空表
v.append(3)           # 尾部插入:3
v.append(1)           # 3 1
v.insert(0, 9)        # 头部插入:9 3 1(O(n),搬移全部元素)
v.pop(1)              # 删除下标 1:9 1(O(n))
v[1] = 5              # 随机访问 O(1):9 5
for x in v:
    print(x, end=" ")  # 遍历
print(f"\nsize={len(v)}") # 长度(容量由解释器管理,无直接接口)
```

3.3.3 切片:Python 独有的"批量视图"

切片 `a[i:j]` 取下标 i 到 $j-1$ 的**新列表**(拷贝引用)。它是 C 没有的利器,但有两个复杂度陷阱:① `a[i:j]` 本身 $O(j-i)$, 写在循环里可能把 $O(n)$ 变 $O(n^2)$ (第2章算法一正是栽在这);② `a[:]` 是浅拷贝——元素是可变对象时,内层对象仍共享。删除与替换也有切片语法: `del a[1:3]`、`a[1:1] = [9, 8]` (插入)。

§3.4 表的链式实现

3.4.1 结点与引用

链表的原子单位是**结点(Node)**:一个数据域 + 一个指向下一个结点的引用。C 的 `struct Node { int val; struct Node *next; }` 在 Python 里变成:

示例 3-3 结点类:C 的 `struct`+指针 $\rightarrow$ Python 的 `class`+引用

```
class Node:
    def __init__(self, val, nxt=None):
        self.val = val          # 数据域
        self.next = nxt        # 指向下一个结点(引用即指针)

# C 里 Node *p = malloc(sizeof(Node)); p->val = 1;
# Python 里:
p = Node(1)
q = Node(2, p)                # q.next 指向 p
print(q.next.val)             # 1 — 与 C 的 q->next->val 同义
```

Python 的引用与 C 的指针本质相同("指向对象"),区别是:引用不能做算术、不能指向"未初始化的内存",且对象引用计数归零时自动回收——C 的悬垂指针与内存泄漏两大病根,Python 从机制上消灭。

3.4.2 手写单链表

示例 3-4 单链表:表头插入、查找、删除

```
def insert_front(head, x):      # 表头插入  $O(1)$ , 返回新表头
    return Node(x, head)

def find(head, x):             # 查找  $O(n)$ 
    while head is not None and head.val != x:
        head = head.next
    return head                # 找不到返回 None

def erase(head, x):             # 删除值为 x 的第一个结点, 返回新表头
    if head is None:
        return None
    if head.val == x:           # 删表头是特例
        return head.next      # 旧表头失去引用, 自动回收(无需 delete!)
    prev = head                 # 其余情况: 找前驱
    while prev.next is not None and prev.next.val != x:
        prev = prev.next
    if prev.next is not None:
        prev.next = prev.next.next # 前驱跳过目标结点
    return head
```

★ 重点:插入删除的"定位"成本

说"链表插入删除 $O(1)$ "是有前提的:已经定位到操作位置。定位本身(查找值、走到第 k 个结点)是 $O(n)$ 。所以"链表的插入比数组快"只在特定场景成立:频繁头部操作、或已持有位置引用的遍历过程中。速查表里的" $O(1)$ "指的都是"已定位后"。

🚀 工程建议:Python 里几乎不手写链表

生产代码中,Python 的 `list` 覆盖了绝大多数"表"的需求:随机访问 $O(1)$ 、尾部插入 $O(1)$ 、缓存友好,常数小到足以抵消链表的 $O(1)$ 定位插入优势;需要头尾都 $O(1)$ 时用 `collections.deque` (内部是双向链接的块,专门优化两端)。**手写链表的用武之地:**① 面试考察指针思维;② 实现树/散列桶等"必须显式引用"的结构(第4、5章);③ 需要"多指针共享结点"的特殊场景(如并查集的可持久化)。判断标准:凡是能按下标访问需求的,一律 `list`。

§3.5 栈

3.5.1 栈 ADT 与 list 实现

栈(stack):后进先出(LIFO)。三个操作——`push`(压入)、`pop`(弹出)、`top`(查看栈顶)。数组实现最简单:只在尾部增删,三个操作全部 $O(1)$ 。Python 里 `list` 天然就是数组栈: `append` 即 `push`, `pop()` 即 `pop`, `a[-1]` 即 `top`。

示例 3-5 用 list 模拟栈(与 C 数组栈对照)

```
# C:  int st[100]; int top = 0;
#     st[top++] = x;           // push
#     x = st[--top];           // pop
st = []                        # 空栈
st.append(3)                   # push:3
st.append(1)                   # 3 1
st.pop()                       # pop:移除 1
print(st[-1])                  # top:3(空栈时 st[-1] 抛 IndexError,是好事)
print(len(st) == 0)            # 判空
```

注意与 C 的函数调用栈对照:递归深度过大时抛 `RecursionError`,本质就是这同一个结构——调用栈被 `push` 到溢出(第1章讲过)。栈也可以用单链表实现(表头做栈顶,头插头删 $O(1)$),接口完全不变——这正是 §3.1 说的 ADT 威力。

3.5.2 应用一:括号平衡检查

示例 3-6 括号平衡:三种括号匹配检查


```
def is_balanced(s):
    """判断字符串中 ( ) [ ] { } 是否配对且顺序正确。"""
    st = []
    for c in s:
        if c in "([{":
            st.append(c)          # 左括号入栈
        elif c in ")]}":
            if not st:
                return False     # 右括号多余
            top = st.pop()
            if (c == ")" and top != "(") or \
               (c == "]" and top != "[") or \
               (c == "}" and top != "{"):
                return False     # 类型不匹配
    return not st                # 结束时栈空 = 左括号无剩余
```

3.5.3 应用二:后缀表达式求值

后缀表达式(逆波兰记法)把运算符放在操作数之后: `6 5 2 3 + 8 * + 3 + *`, 不需要括号。求值规则:遇数字入栈;遇运算符弹出两个操作数,算完把结果压回。这个"惰性计算"模式是栈最漂亮的用法。

示例 3-7 后缀表达式求值

```
def eval_postfix(toks):
    """输入 token 列表(数字串与运算符);输出表达式值。"""
    st = []
    for t in toks:
        if t.lstrip("-").isdigit():
            st.append(int(t))      # 数字入栈
        else:
            b = st.pop()           # 先弹的是右操作数!
            a = st.pop()
            if t == "+":
                st.append(a + b)
            elif t == "-":
                st.append(a - b)
            elif t == "*":
                st.append(a * b)
            else:
                st.append(a // b)  # 整除
    return st.pop()              # 最后栈里只剩结果

print(eval_postfix(["6", "5", "2", "3", "+", "*", "+"])) # 31
```

⚠ 误区 2:操作数弹出顺序

后缀 `8 2 /` 是 $8/2=4$,但弹栈顺序是先弹 2 再弹 8——先弹的是右操作数。减法和除法对顺序敏感,写反了就是 $2/8$ 。口诀:栈顶是"后出现"的操作数,后出现的是右操作数。

3.5.4 应用三:中缀转后缀

人读的是中缀(`a+b*c`),机器算的是后缀。转换算法(调度场算法):遇数字直接输出;遇运算符,先把栈中**优先级不低于它的**运算符弹出输出,再压入自己;左括号直接入栈,右括号则弹出到左括号为止。这样保证高优先级先输出。

示例 3-8 中缀转后缀(单字符数字简化版)

```
def prec(op):
    return 1 if op in "+-" else 2

def infix_to_postfix(s):
    """输入如 "6+5*(2+3)"(无空格),输出后缀字符串。"""
    out = []
    st = []
    for c in s:
        if c.isdigit():
            out.append(c)          # 数字直接输出
        elif c == "(":
            st.append(c)
        elif c == ")":
            while st[-1] != "(":
                out.append(st.pop())
            st.pop()              # 丢弃左括号
        else:                    # 运算符
            while st and st[-1] != "(" and prec(st[-1]) >= prec(c):
                out.append(st.pop())  # 栈顶优先级不低于当前则弹出
            st.append(c)
    while st:
        out.append(st.pop())
    return "".join(out)

print(infix_to_postfix("6+5*(2+3)"))  # 6523+*+
```

例题 3-1 手工走一遍中缀转后缀

输入 $6+5*(2+3)$:6 输出;+ 入栈;5 输出;* 入栈(+ 优先级低,留在栈中);(入栈;2 输出;+ 入栈;3 输出;) 弹出 +,再弹(; 结束弹栈得 *、+。输出: $6523+*+$ 。用 §3.5.3 求值: $2+3=5, 5*5=25, 6+25=31$ 。✓

§3.6 队列

3.6.1 队列 ADT 与循环数组实现

队列(queue):先进先出(FIFO)。enqueue(入队)在队尾,dequeue(出队)在队头。用数组实现有个经典问题:出队后前面空出的位置怎么办?如果每次出队都把全部元素前移,那是 $O(n)$;不移动、让队头指针"追赶"队尾指针,数组就要**循环**使用——下标到底后绕回 0。

示例 3-9 循环数组队列(用 count 记录元素数)

```

# front 指向队首元素,back 指向队尾下一个空位,count 记录元素数
class MyQueue:
    def __init__(self, cap):
        self._data = [None] * cap
        self._front = 0
        self._back = 0
        self._count = 0

    def push(self, x):          # 入队:假设未满(满的处理见误区3)
        self._data[self._back] = x
        self._back = (self._back + 1) % len(self._data)
        self._count += 1

    def pop(self):             # 出队
        x = self._data[self._front]
        self._front = (self._front + 1) % len(self._data)
        self._count -= 1
        return x

    def front(self):
        return self._data[self._front]

    def is_empty(self):
        return self._count == 0

    def __len__(self):
        return self._count

```

★ 重点:空/满判断的两种方案

若用 front 与 back 两个下标判断空满,会遇到 front == back 时"既可能空也可能满"的歧义。两种解法:① **计数法**(示例 3-9,多存一个 count,最简单);② **浪费一格法**(规定 back 永远不追赶 front,队满时留一个空格,满条件是 (back+1) % cap == front)。工程与面试都常考,两种都要会。

3.6.2 标准库:collections.deque

示例 3-10 deque:两头都 O(1) 的双端队列

```

from collections import deque

q = deque()          # 空队列
q.append(1)           # 右端入队
q.append(2)
q.append(3)
while q:
    print(q.popleft(), end=" ")  # 左端出队:1 2 3(先进先出)

d = deque([1, 2, 3])
d.appendleft(0)       # 左端入队:0 1 2 3
d.append(4)           # 右端入队:0 1 2 3 4
print(list(d))        # [0, 1, 2, 3, 4]

```

C 的手写循环队列	Python 的标准库
自己维护 front/back 下标与取模	<code>q.append / q.popleft()</code> 一行一个操作
空/满判断要写计数或浪费一格	自动扩容,永不"满"
<code>free</code> 忘记调用即泄漏	垃圾回收器自动释放

⚠ 误区 3:循环队列的下标溢出与"满"

示例 3-9 简化了"队满"检查——满时再 `push` 会覆盖旧元素。生产实现必须显式处理:计数法判 `count == cap` 满则拒绝/扩容,浪费一格法判 `(back+1) % cap == front`。另一个易错点是取模:back 先加 1 再对 capacity 取模,而不是对"当前元素数"取模。工程中直接用 `deque` 即可绕开全部这些问题。

§3.7 选型与工程实践

3.7.1 数组 or 链表?—一张决策表

场景	选择	理由
按序访问所有元素(遍历)	list	连续内存,缓存友好,常数小
大量随机下标访问	list	$O(1)$ vs $O(n)$
只做尾部增删(栈)	list	均摊 $O(1)$,实现最简单
两端增删(队列)	deque / 循环数组	两头都 $O(1)$
需要多引用共享结点	手写链表/树	list 做不到引用语义的共享
几乎一切其他场景	list	Python 社区共识:先 list,不够再换

3.7.2 栈模拟递归与调试技巧

任何递归都可以改写为"显式栈 + 循环"的迭代版(第4章树遍历会用到):函数调用栈压的是"返回地址 + 局部变量",显式栈压的是"待处理的任务"。迭代版不受解释器递归深度限制(默认约 1000 层),深度优先搜索(第9章)的大图遍历通常必须用迭代版。调试栈程序的两个技巧:① `push` 前打印"压入 x",`pop` 后打印"弹出 x",肉眼跟踪栈的演化;② 每次 `pop` 前检查栈非空,把"逻辑错误"提前变成"崩溃点",定位速度翻倍。

🚀 工程建议:优先内建结构,手写只为理解

本章手写了动态数组、链表、栈、队列四样东西——目的是让你看清每一层的复杂度来源。生产代码请一律使用 `list / deque`:它们是 C 实现的内建类型,速度比任何纯 Python 手写都快一个数量级,边界处理也更健壮。手写的用武之地是"内建没有的结构"(后几章的树、堆、散列表、图),那才是本书后续的主战场。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
按下标访问	list(动态数组)	$O(1)$	$O(1)$
尾部插入(append)	list	$O(1)$	$O(n)$ (扩容)
头部插入(insert(0))	list	$O(n)$	$O(n)$
任意位置插入/删除	list	$O(n)$	$O(n)$
按值查找	list	$O(n)$	$O(n)$
切片 a[i:j]	list	$O(j-i)$	$O(j-i)$
按下标访问	链表	$O(n)$	$O(n)$
按值查找	链表	$O(n)$	$O(n)$
头部/尾部插入	链表	$O(1)$	$O(1)$
已定位处插入/删除	链表	$O(1)$	$O(1)$
push / pop / top	栈(list 或链表实现)	$O(1)$	$O(1)$
入队 / 出队 / 取队头	循环数组队列	$O(1)$	$O(1)$
两端增删	deque(双端队列)	$O(1)$	$O(1)$
括号平衡检查	栈扫描一遍	$O(n)$	$O(n)$
后缀表达式求值	栈扫描一遍	$O(n)$	$O(n)$
中缀转后缀	调度场算法	$O(n)$	$O(n)$
list 扩容	n 次 append 总代价	摊还 $O(n)$, 平均每次 $O(1)$ (第11章证明)	

最小必会清单

- 能独立说出 ADT 的定义,并举例说明"接口与实现分离"的好处
- 能独立解释 list 的 append 为什么平均 $O(1)$,扩容时发生了什么
- 能独立说出 `[[]] * 5` 与 `[[] for _ in range(5)]` 的区别
- 能独立默写单链表的表头插入、查找、删除(含删表头特例)
- 能独立说出 Python 中"几乎不手写链表"的理由,以及仍需手写的三种场景
- 能独立用 list 模拟栈,并解决括号平衡问题
- 能独立写出后缀表达式求值,并说清操作数的弹出顺序
- 能独立默写中缀转后缀算法(含运算符优先级处理)
- 能独立实现循环数组队列,并说出空/满判断的两种方案
- 能独立说出 list 与 deque 的选型规则(至少 4 条场景)

自测题

Q 1. list 的 append 为什么说"平均 $O(1)$ "而最坏 $O(n)$?

✓ 参考答案

平时插入只写一个元素 $O(1)$; 仅当长度达到容量时才触发扩容, 需拷贝全部 n 个元素的引用 $O(n)$ 。容量按比例倍增, 从 1 增到 n 共 $\log_2 n$ 次扩容, 总拷贝 $n + n/2 + \dots \approx 2n$, 摊到 n 次插入上平均 $O(1)$ 。

Q 2. `a = []`; `a[0] = 1` 会怎样? 与 C 的对应写法有何不同?

✓ 参考答案

抛 `IndexError` —— 下标赋值不会自动扩容。C 里 `int a[0]` 不合法但 `int *p = malloc(0); p[0] = 1` 是未定义行为(可能静默破坏内存)。Python 显式报错, 把 UB 变成可定位的异常, 这是 Python 更好调试的原因之一。

Q 3. 单链表中删除“值为 x 的结点”, 为什么表头要单独处理? 怎样统一?

✓ 参考答案

普通结点删除需要前驱结点改指针, 而表头没有前驱。解法: ① 特判(本章示例 3-4); ② 引入哑结点(dummy), 表头前面永远站一个空结点, 删除逻辑统一; ③ 让 `erase` 返回新表头(示例 3-4 的做法), 把“改表头”交给调用方。哑结点法在工程中最常用。

Q 4. 后缀表达式 `8 2 / 3 5 * +` 的值是多少? 写出求值过程。

✓ 参考答案

8、2 入栈; `/`: 弹 2、8, $8/2=4$ 入栈; 3、5 入栈; `*`: 弹 5、3, $3*5=15$ 入栈; `+`: 弹 15、4, $4+15=19$ 。答案 19。注意先弹的是右操作数。

Q 5. 循环队列用 `front` 与 `back` 两个下标时, 为什么 `front == back` 会“既可能空也可能满”? 两种解法是什么?

✓ 参考答案

队列空时 `front` 与 `back` 重合; 而队满时 `back` 绕一圈也追上 `front`, 两个状态无法区分。解法: ① 计数法——额外维护元素个数 `count`, 空满由 `count` 判断; ② 浪费一格法——规定 `back` 永不与 `front` 重合, 满条件是 $(back+1) \% cap == front$, 牺牲一个格子换取 $O(1)$ 判断。

Q 6. 中缀 `a+b*c-d` 转后缀的过程是什么? 为什么 `*` 在 `+` 之前输出?

✓ 参考答案

`a` 输出; `+` 入栈; `b` 输出; 遇 `*`: 栈顶 `+` 优先级(1) 低于 `*`(2), 不弹出, `*` 入栈; `c` 输出; 遇 `-`: 栈顶 `*` 优先级不低于 `-`, 弹出 `*` 输出; 栈顶 `+` 优先级不低于 `-`, 弹出 `+` 输出; `-` 入栈; `d` 输出; 结束弹栈输出 `-`。结果 `abc*+d-`。`*` 先输出是因为它绑定得更紧(优先级高), 后缀表达式中运算符顺序即计算顺序。

章末实验题:中缀表达式计算器

实验 3:中缀表达式计算器(栈应用综合)

实验目标:实现一个命令行计算器:读入含 $+$ $-$ $*$ $/$ 与括号的中缀表达式,支持多位数,输出后缀表达式与计算结果。

实验要求:

- 任务 1:实现 tokenize,把字符串拆成数字与运算符序列,支持多位数(知识点:\$3.5.4 中缀转后缀的扩展)
- 任务 2:实现中缀转后缀,处理括号与优先级(知识点:\$3.5.4 调度场算法)
- 任务 3:实现后缀求值,注意弹出顺序(知识点:\$3.5.3)
- 任务 4:主循环读入表达式,输出后缀式与结果,输入 `quit` 退出(知识点:\$3.5 栈应用综合)
- 任务 5:处理非法输入:除数为 0、括号不匹配时给出错误信息而非崩溃(知识点:\$3.5.2 括号平衡)

实验环境:Python 3.8+,标准库即可。运行: `python lab3.py`。

第一步 tokenize 用指针扫描:连续数字拼成一个字符串 token;空格跳过;其余字符单独成 token

第二步 中缀转后缀直接复用示例 3-8 的骨架,把"单字符数字"换成"字符串 token";数字 token 用 `t.lstrip("-").isdigit()` 判定

第三步 求值时先检查操作数栈是否够两个元素(非法输入),再检查除数是否为 0;异常用 `raise ValueError` 抛出,主循环统一捕获

第四步 括号匹配错误在转换阶段发现:弹栈遇空、或结束后栈里剩左括号,都报"括号不匹配"

✅ 参考答案(完整可运行程序,约 85 行,分两段)

```
# lab3.py — 中缀表达式计算器(第一段:词法与转换)
def tokenize(s):
    """任务 1:拆分 token(支持多位数与空格)。"""
    toks = []
    i = 0
    while i < len(s):
        if s[i].isspace():
            i += 1
            continue
        if s[i].isdigit():
            j = i
            while j < len(s) and s[j].isdigit():
                j += 1
            toks.append(s[i:j])
            i = j
        else:
            toks.append(s[i])
            i += 1
    return toks

def prec(op):
    return 1 if op in "+-" else 2

def infix_to_postfix(toks):
    """任务 2:中缀转后缀(调度场算法)。"""
    out = []
    st = []
    for t in toks:
        if t.lstrip("-").isdigit():
            out.append(t)                # 数字直接输出
        elif t == "(":
            st.append(t)
        elif t == ")":
            while st and st[-1] != "(":
                out.append(st.pop())
            if not st:
                raise ValueError("括号不匹配")
            st.pop()                    # 丢弃左括号
        else:
            # 运算符
            while st and st[-1] != "(" and prec(st[-1]) >= prec(t):
                out.append(st.pop())
            st.append(t)
    while st:
        if st[-1] == "(":
            raise ValueError("括号不匹配")
        out.append(st.pop())
    return out
```

lab3.py(第二段:求值与主循环)


```

def eval_postfix(toks):
    """任务 3:后缀求值(注意弹出顺序)。"""
    st = []
    for t in toks:
        if t.lstrip("-").isdigit():
            st.append(int(t))
        else:
            if len(st) < 2:
                raise ValueError("操作数不足")
            b = st.pop()          # 先弹的是右操作数
            a = st.pop()
            if t == "+":
                st.append(a + b)
            elif t == "-":
                st.append(a - b)
            elif t == "*":
                st.append(a * b)
            else:
                if b == 0:
                    raise ValueError("除数为 0")
                st.append(a // b)
    if len(st) != 1:
        raise ValueError("表达式非法")
    return st[0]

def main():
    print("中缀表达式计算器(支持 + - * / 与括号,输入 quit 退出)")
    while True:
        line = input("> ").strip()
        if line == "quit":
            break
        try:
            toks = tokenize(line)
            post = infix_to_postfix(toks)
            result = eval_postfix(post)
            print("后缀式:", " ".join(post))
            print("结果:", result)
        except ValueError as e:
            print("错误:", e)

if __name__ == "__main__":
    main()

```

示例输入输出:

```
> 6+5*(2+3)
后缀式: 6 5 2 3 + * +
结果: 31
> 12/4+7*2
后缀式: 12 4 / 7 2 * +
结果: 17
> (1+2)*3
后缀式: 1 2 + 3 *
结果: 9
> 5/0
错误: 除数为 0
> (1+2
错误: 括号不匹配
> quit
```

设计说明:① tokenize/转换/求值三阶段分离,每阶段只做一件事,出错信息定位到阶段——这是编译原理"词法→语法→语义"流水线的雏形;② 转换与求值都基于同一个栈结构(list 模拟),体现了 ADT"实现替换、接口不变"的思想;③ 错误用 `ValueError` 统一抛出,主循环一个 `except` 兜底,避免每个函数各自打印错误导致"错误处理与逻辑纠缠";④ 运算符优先级集中在 `prec` 一个函数里,新增运算符(如幂 `**`)只需改这一处。

下一章预告:第4章 树——把"链式结点"推广为"有分支的结构",学习二叉树遍历、表达式树、二叉查找树与 AVL 平衡。第4章的表达式树与本章的中缀转后缀直接衔接;树的层序遍历将使用本章的队列。

第4章 树

建议学时:4-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 掌握树的核心术语:根、父结点、子结点、兄弟、叶、深度、高度、路径
- 完成从 C 的 `first_child + next_sibling` 到 Python `list` 子结点容器的迁移
- 掌握前序/中序/后序/层序遍历:递归版、队列版、显式栈版都能写出
- 能用栈从后缀表达式构建表达式树并用中序恢复中缀,与第3章闭环
- 能完整实现BST的 `contains/find_min/insert/remove` 与 AVL 四种旋转;能说出 Python 递归深度限制与替代方案

💡 从 C 到 Python:本章主题如何迁移

树是"有分支的链表":链表只能排成一条线,树允许一个结点有多个后继,信息分层组织。家族谱、文件目录、语法树都是树。第3章解决"一串数据",本章解决"有层次关系的数据"。

C 里一般树靠双指针 `firstChild + nextSibling` 串起任意多个子结点,通用但笨拙。Python 直接用 `list`:子结点放进列表,遍历就是一句 `for`,内存交给引用计数与 GC,没有 `malloc/free`。

本章终点是查找:二分查找 $O(\log n)$,但有序数组插入要搬移元素。BST 把查找、插入、删除同时做到平均 $O(\log n)$;AVL 再把"平均"升级为"保证"。Python 没有内置平衡树——先学会手写,再看 §4.7 的工程替代。

📦 前置知识自查

数学前置:树高递归式: $\text{height}(\text{空}) = 0, \text{height}(v) = 1 + \max(\text{height}(v.\text{left}), \text{height}(v.\text{right}))$;对数直觉(每次减半 $\rightarrow \log n$)。

C 语言前置: `struct` 与指针、`malloc / free`、递归、`->` 访问成员。

依赖的前面章节:第1章递归(遍历、高度、销毁)、第3章栈与队列(层序用 `deque`,建表达式树用栈;中缀转后缀是 §4.3 的输入)。

🚀 本章学习路线

1. 阶段1(1 小时):§4.1–§4.2 建立术语,上机核对四种遍历输出
2. 阶段2(1 小时):§4.3 表达式树,与中缀转后缀接成闭环
3. 阶段3(1.5 小时):§4.4 手写 BST,默写删除三情形
4. 阶段4(1 小时):§4.5 画图验证 AVL 旋转,通读 §4.6–§4.7
5. 阶段5(实验 1 小时):完成章末实验,复习删除三情形与旋转判定

本章知识导图

- §4.1 树的概念:术语与一般树的实现(根/父/子/兄弟/叶/深度/高度/路径、first_child+next_sibling → list、一般树 vs 二叉树)
- §4.2 二叉树与四种遍历(递归三序、层序用队列、显式栈模拟递归、高度/结点数/叶数)
- §4.3 表达式树(后缀构建、中序恢复中缀)
- §4.4 二叉查找树 BST(查找/插入/删除、删除三情形、手写 vs bisect)
- §4.5 AVL 树(平衡因子、LL/RR/LR/RL 旋转、插入后重平衡)
- §4.6 伸展树与 B 树(摊还与局部性、磁盘 IO 与浅而宽)
- §4.7 工程与实践(递归深度限制、dict/sorted list 替代、调试)

§4.1 树的概念:术语与一般树的实现

4.1.1 术语:根、父结点、子结点、兄弟、叶、深度、高度、路径

树的递归定义:树是结点的集合——要么是空集,要么由一个**根**结点加上若干棵互不相交的**子树**构成,每棵子树本身也是树。这决定了本章算法的递归写法。

图 4-1 一般树的术语

树的递归定义:树 = 根 + 互不相交的子树。



深度从根往下数(根 0),**高度**从叶往上数(叶 0),树高 = 根的高度;**路径**长度 = 边数。n 个结点的树有 n-1 条边。方向相反,勿混淆。

记号说明(树高递归式,全书沿用)

$\text{height}(\text{空}) = 0, \text{height}(v) = 1 + \max(\text{height}(v.\text{left}), \text{height}(v.\text{right}))$ ——它同时是定义、递推式和代码,§4.2 的求高函数就是逐字翻译。

4.1.2 一般树的实现:从 first_child+next_sibling 到 list 子结点容器

C 靠 firstChild + nextSibling 表达任意多的子结点;Python 用 list 保存——第3章 list 思想在树上的延续:

C 的写法(first_child + next_sibling)	Python 的写法(list 子结点容器)
<pre>struct TreeNode { int val; struct TreeNode *firstChild, *nextSibling; };</pre>	<pre>class TreeNode: self.children = []</pre>
遍历子结点:沿 nextSibling 链 while 循环	遍历子结点:for 一句
访问第 k 个子结点要数 k 次指针	children[k] 直接下标访问
插入子结点自己维护链接顺序	children.append(c) 一行
忘记 free 子树 = 泄漏	引用计数自动回收,无泄漏

示例 4-1 一般树结点:list 子结点容器

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.children = []          # 子结点容器(list)

    def print_children(n):          # 对照 C 的 nextSibling 链
        for c in n.children:       # for 一句遍历
            print(c.val, end=' ')
```

4.1.3 一般树 vs 二叉树

二叉树是"每个结点至多两个子结点,且区分左右"的树——只有一个子结点也要说清在左在右。BST、表达式树、堆(第6章)都依赖"左/右"语义。一般树与二叉树可互相转换:"长子变左子、兄弟变右子"正是 C 的 first_child+next_sibling。

§4.2 二叉树与四种遍历

4.2.1 二叉树的定义与递归三序遍历

二叉树要么为空,要么由根 + 左子树 + 右子树构成——直接翻译成结点类:

示例 4-2 二叉树结点与前/中/后序递归遍历

```

class BTreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def preorder(n):
    # 前序:根 → 左 → 右
    if n is None: return
    print(n.val, end=' ')
    preorder(n.left)
    preorder(n.right)

def inorder(n):
    # 中序:左 → 根 → 右
    if n is None: return
    inorder(n.left)
    print(n.val, end=' ')
    inorder(n.right)

def postorder(n):
    # 后序:左 → 右 → 根
    if n is None: return
    postorder(n.left)
    postorder(n.right)
    print(n.val, end=' ')

```

三个函数唯一区别是 `print` 在两次递归调用的前、中、后——遍历的全部秘密。

例题 4-1 手走一棵二叉树的三种遍历

一棵 BST 形状的二叉树(内部结点 7/3/9, 叶 1/6/11):



前序:7 3 1 5 6 9 11;中序:1 3 5 6 7 9 11;后序:1 6 5 3 11 9 7。自查:中序恰好升序,因为是 BST(\$4.4)。

误区 1:把"访问"与"递归调用"的顺序搞混

打印语句在两次递归之间的位置决定遍历类型——三序唯一的区别就是这行 `print` 放哪。先用最小树手推一遍再对照。

4.2.2 层序遍历(用队列,呼应第3章)

层序遍历按"从上到下、每层从左到右"访问。递归天然深入、做不到逐层横推,必须用第3章的队列:根先入队,出队访问后左右子结点再入队。Python 用 `collections.deque` :

示例 4-3 层序遍历:deque

```

from collections import deque

def level_order(root):
    # 层序:先进先出
    if root is None: return
    q = deque([root])
    while q:
        n = q.popleft()
        # 出队
        print(n.val, end=' ')
        if n.left: q.append(n.left)
        if n.right: q.append(n.right)

```

4.2.3 显式栈模拟递归(Python 简化版)

任何递归都能改写成"显式栈 + 循环"(第3章)。系统栈压"返回地址 + 局部变量",显式栈压"待处理的结点"。Python 版只收集序列不打印, `print` 与递归分离更接近"返回列表"的工程写法:

示例 4-4 迭代中序遍历:显式栈

```

def inorder_iter(root):
    # 显式栈模拟递归中序
    st, cur, out = [], root, []
    while cur or st:
        while cur:
            # 1. 沿左链一路压栈
            st.append(cur)
            cur = cur.left
        cur = st.pop()
        # 2. 出栈
        out.append(cur.val)
        # 3. 访问发生在出栈时
        cur = cur.right
        # 4. 转向右子树
    return out
    # 返回中序序列

```

递归中序也是"先到最左、打印、再进右子树"——迭代版只是自己压栈。递归的代价见 §4.7.1。

4.2.4 遍历应用:求高度、结点数、叶数

三个经典问题都是"后序思想":先递归算完左右子树,再组合出答案。基准情形都是 `n is None` 返回 0,空树基准让"单个叶结点"自然成立;写树递归先问:空结点返回什么?

示例 4-5 树高、结点数、叶数

```

def height(n):
    # 空树高度为 0
    if n is None: return 0
    return 1 + max(height(n.left), height(n.right))

def count_nodes(n):
    # 结点数
    if n is None: return 0
    return 1 + count_nodes(n.left) + count_nodes(n.right)

def count_leaves(n):
    # 叶:左右都为空
    if n is None: return 0
    if n.left is None and n.right is None: return 1
    return count_leaves(n.left) + count_leaves(n.right)

```

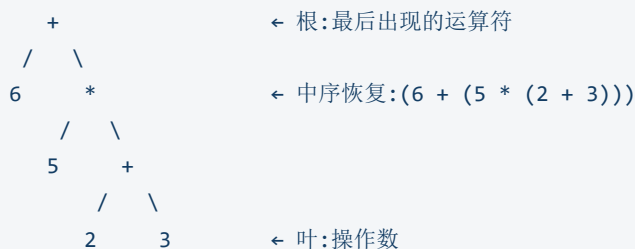
§4.3 表达式树

4.3.1 表达式树的结构

表达式树是编译器处理算术表达式的内部表示: **叶结点是操作数,内部结点是运算符**,左右子树就是两个操作数——后缀式 `6 5 2 3 + * +` 对应:

图 4-2 后缀式 `6 5 2 3 + * +` 的表达式树

表达式树:叶是操作数,内部是运算符。



4.3.2 由后缀表达式构建(用栈,衔接第3章)

构建与第3章后缀求值几乎同构:遇数字建叶结点入栈;遇运算符弹出两棵子树——**先弹的是右操作数**——合并压回。区别只在栈里存数值还是树结点:

示例 4-6 由后缀表达式构建表达式树

```
class ExprNode:                                # 表达式树结点
    def __init__(self, data, left=None, right=None):
        self.data = data                        # 数字或运算符
        self.left = left
        self.right = right

def build_expr_tree(post):                      # post:后缀式列表
    st = []
    for t in post:
        if t.isdigit():
            st.append(ExprNode(t))             # 数字:建叶结点入栈
        else:
            r = st.pop()                        # 先弹的是右操作数
            l = st.pop()
            st.append(ExprNode(t, l, r))        # 合并成父结点压回
    return st[0]                                # 栈顶即根
```

⚠ 误区 2:左右子树接反

弹出时**先弹的是右操作数**。把 `r` 接成左子树, `6 5 -` 就会算成 $5-6$ 。写完用 `print_infix` 核对带减号的结果。

4.3.3 中序遍历恢复中缀表达式

三序遍历与三种表达式记法——对应:前序 = 前缀式,中序 = 中缀式,后序 = 后缀式。中序遍历表达式树输出的正是人读的中缀;为优先级自治,给每个运算符子树加括号:

示例 4-7 中序恢复中缀(带括号)


```
def print_infix(n):                # 中序恢复中缀
    if n is None: return
    is_op = not n.data.isdigit()
    if is_op: print('(', end='')  # 运算符子树加括号
    print_infix(n.left)
    print(n.data, end='')
    print_infix(n.right)
    if is_op: print(')', end='')
```

★ 重点:三序遍历 × 三种表达式 = 一个闭环

第3章"中缀转后缀"把 $(1+2)*3$ 变 $12+3*$;本章"后缀建树 + 中序恢复"把它变回 $((1+2)*3)$ 。三种记法、三种遍历是同一信息的三副面孔。

§4.4 二叉查找树(BST)

4.4.1 BST 性质与 contains/findMin/findMax

二叉查找树(BST)性质: 每个结点的左子树所有值都小于它,右子树所有值都大于它。推论:中序遍历输出严格升序。查找 = 比较后走左或右,一路下行丢掉一半子树——二分查找的树形版,插入删除不用搬移元素:

示例 4-8 BST:contains/find_min/find_max

```
class Node:                        # 结点:val + 左右引用
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class BST:
    def __init__(self):
        self.root = None          # 对象引用,无需指针

    def contains(self, x):          # 查找:沿一条路径下行
        n = self.root
        while n:
            if x == n.val: return True
            n = n.left if x < n.val else n.right
        return False

    def find_min(self):             # 最小:一路向左
        n = self.root
        while n.left: n = n.left
        return n.val

    def find_max(self):             # 最大:一路向右
        n = self.root
        while n.right: n = n.right
        return n.val
```

4.4.2 插入 insert

C 的做法是"遍历到空位,改父结点指针";Python 更稳的是**递归返回"新根"**:引用修改收拢在 `return` 里,父结点用返回值接住:

示例 4-9 BST 插入:递归返回新根

```
def insert(self, x):
    self.root = self._insert(self.root, x)

def _insert(self, n, x):
    if n is None: return Node(x)      # 空位:新建结点
    if x < n.val: n.left = self._insert(n.left, x)
    elif x > n.val: n.right = self._insert(n.right, x)
    return n                          # 相等:忽略
```

4.4.3 删除 remove:三种情形

删除三情形:① **叶**:直接删;② **单子**:让唯一子结点顶替;③ **双子**:用右子树的最小结点顶替被删结点,再删掉它——它大于左子树全部、小于右子树其余,且无左子树:

示例 4-10 BST 删除:三种情形

```
def remove(self, x):
    self.root = self._remove(self.root, x)

def _find_min_node(self, n):          # 子树中的最小结点
    while n.left: n = n.left
    return n

def _remove(self, n, x):
    if n is None: return None         # 不存在,忽略
    if x < n.val: n.left = self._remove(n.left, x)
    elif x > n.val: n.right = self._remove(n.right, x)
    else:                             # 找到 x
        if n.left is None: return n.right # 叶或只有右子树
        if n.right is None: return n.left  # 只有左子树
        m = self._find_min_node(n.right)  # 双子
        n.val = m.val                     # 复制数据顶替
        n.right = self._remove(n.right, m.val) # 删掉最小结点
    return n
```

⚠ 误区 3:双子删除的两处经典翻车

① **漏接右子树**:只写 `n.val = m.val` 就 `return n`, `m` 还挂在右子树里——重复且泄漏,必须再递归删除 `m`。② **删错目标**:写成 `n.right = self._remove(n.right, x)` 必扑空。先复制,再用 `m` 删右子树。

⚠ 误区 4:删除后旧引用悬空

调用方若写 `p = tree.root; tree.remove(x); p.val = ...` —— `p` 可能已被顶替删除,再改 `p` 等于改一个已脱离树的孤立结点。删除/修改容器后,旧引用一律视为失效。

4.4.4 复杂度直觉与"手写 vs 标准库"

BST 每个操作沿一条路径下行,路径长度 = 树高。随机插入时树高约 $1.39 \log_2 n$,操作平均 $O(\log n)$;数据恰好有序插入时树退化成链,查找变线性扫描。

⚠ 误区 5:把"平均"当"保证"(退化链)

给 BST 喂升序序列,它长成一条只有右孩子的链,查 1 要 $O(n)$ 。这不是理论病:自增主键、日志时间戳等"按序到达"的数据都会触发。解法:① 打乱插入顺序;② 自平衡结构——AVL(§4.5)——把树高保证在 $O(\log n)$ 。

Python 没有内置平衡树,"有序集合"的工程替代是 `sorted list + bisect`;手写 BST 只为理解原理:

示例 4-11 手写 BST 查找 vs sorted list + bisect(并排)

```
# 手写 BST:平均 O(log n),最坏 O(n)
tree = BST()
for x in (15, 6, 18):
    tree.insert(x)
print(tree.contains(6))           # True

# 标准库写法:sorted list + bisect 保持有序
from bisect import bisect_left, insort
lst = []
for x in (15, 6, 18):
    insort(lst, x)                 # 插入 O(n),查找 O(log n)
i = bisect_left(lst, 6)           # 二分定位
print(i < len(lst) and lst[i] == 6) # True
```

C 的手写查找树	Python 的手写树 + bisect
手写 Node 结构 + 递归插入/删除,还要管内存	类 + 对象引用,GC 自动回收内存
有序插入退化成链,最坏 $O(n)$	同样退化;换 AVL 或 sorted list + bisect
标准库 set/map 是红黑树,保证 $O(\log n)$	Python 无内置平衡树;dict 无序但平均 $O(1)$

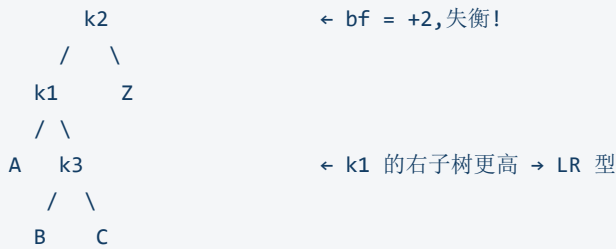
§4.5 AVL 树

4.5.1 平衡因子与失衡类型

AVL 树是第一种自平衡 BST: 每个结点的左右子树高度差至多 1。平衡因子 $bf(v) = height(v.left) - height(v.right)$,合法值只有 -1、0、+1;插入可能把某祖先推到 ± 2 ,用旋转扳回,四种,看失衡结点的哪个孩子更高:

图 4-3 失衡判定:四种类型

$bf(v) = height(v.left) - height(v.right)$, 合法取值只有 $-1/0/1$ 。



LL: $bf=+2$ 且左重; RR: $bf=-2$ 且右重; LR: $bf=+2$ 且右重; RL: $bf=-2$ 且左重。

记忆法: LL 是"左左", 失衡结点和高孩子都在左, 一次右旋; LR 是"左右", 先下后上。

4.5.2 四种旋转的实现

AVL 结点比 BST 多一个 `height` 字段: `Node(val, height=1)`。四种旋转只改引用、 $O(1)$, 与 C++ 版逐行对应:

示例 4-12 AVL 四种旋转

```
def h(n): return n.height if n else 0

def rotate_ll(k2):
    # LL:右旋——左孩子提为根
    k1 = k2.left
    k2.left = k1.right
    k1.right = k2
    k2.height = 1 + max(h(k2.left), h(k2.right))
    k1.height = 1 + max(h(k1.left), h(k1.right))
    return k1

def rotate_rr(k1):
    # RR:左旋, 镜像对称
    k2 = k1.right
    k1.right = k2.left
    k2.left = k1
    k1.height = 1 + max(h(k1.left), h(k1.right))
    k2.height = 1 + max(h(k2.left), h(k2.right))
    return k2

def rotate_lr(k3):
    # LR:先左旋左孩子, 再右旋自己
    k3.left = rotate_rr(k3.left)
    return rotate_ll(k3)

def rotate_rl(k1):
    # RL:先右旋右孩子, 再左旋自己
    k1.right = rotate_ll(k1.right)
    return rotate_rr(k1)
```

★ 重点:旋转的口诀

单旋转(LL/RR)是"把高孩子提上来当根, 自己让位, 多余的一支过继"——只改三个引用, $O(1)$; 双旋转(LR/RL)是"先捋直再旋转"。旋转不破坏 BST 的大小序——中序遍历结果不变。

4.5.3 插入后的重平衡与复杂度对照

AVL 插入 = BST 插入 + 沿路径回溯:每层更新高度、检查平衡因子。插入只改变一条路径,第一个失衡点旋转一次即恢复平衡:

示例 4-13 AVL 插入:更新高度 + 失衡判定

```
def insert(n, x):
    if n is None: return Node(x, height=1)
    if x < n.val: n.left = insert(n.left, x)
    elif x > n.val: n.right = insert(n.right, x)
    n.height = 1 + max(h(n.left), h(n.right)) # 更新高度
    bf = h(n.left) - h(n.right) # 平衡因子
    if bf > 1: # 左子树高:LL 或 LR
        if x < n.left.val: return rotate_ll(n) # LL
        return rotate_lr(n) # LR
    if bf < -1: # 右子树高:RR 或 RL
        if x > n.right.val: return rotate_rr(n) # RR
        return rotate_rl(n) # RL
    return n # 本结点仍平衡
```

判定 LL/LR:看 x 落在失衡结点左孩子的哪一侧, `x < n.left.val` 是 LL,否则 LR。

例题 4-2(手走旋转):空树依次插入 3、2、1:结点 3 的 bf=2 且落在左孩子 2 的左子树 → LL,右旋:2 提为根,3 变其右孩子;再插 4、5,结点 3 的 bf=-2 → RR。最终:根 2,左 1,右 4(左 3 右 5)。

朴素 BST	AVL 树
查找/插入/删除平均 $O(\log n)$	查找/插入/删除 保证 $O(\log n)$
有序输入退化成链,最坏 $O(n)$	任意输入序列,树高 $\leq 1.44 \log_2 n$
实现简单,无额外空间	每个结点多存一个高度(或平衡因子)
插入/删除后不检查平衡	插入/删除后沿路径旋转,常数更大

§4.6 伸展树与 B 树

4.6.1 伸展树:摊还与访问局部性

伸展树(splay tree)的动机是 **访问局部性**:刚被访问的数据往往很快再次被访问 (缓存、词频查询、符号表)。AVL 用平衡因子保证每次 $O(\log n)$,代价是代码复杂;伸展树不要平衡因子,只约定:每次访问后把该结点"伸展"到根。

结论(证明在第11章摊还分析):单次最坏 $O(n)$,连续 m 次操作总代价 $O(m \log n)$, **摊还 $O(\log n)$** ;反复访问的结点总停在根附近,但单次最坏延迟无法保证。

4.6.2 B 树:磁盘 IO 与"浅而宽"

B 树的动机是磁盘:磁盘随机读约 10 ms,内存访问是纳秒级,差约 10 万倍,数据库索引里 **决定性能的不是比较次数,而是下潜层数(= IO 次数)**。BST 树高约 $\log_2 n$, 10^7 条记录约 23 层;B 树是"M 路分支",树高降到 $\log_M n$,取 $M=1000$ 只需 3 层——**"浅而宽"**把 IO 从 20 多压到 3。

结论:B 树查找/插入/删除都是 $O(\log_M n)$;分裂/合并细节不展开,工程上按"一个结点装满一个磁盘块"定 M 。SQLite 的 B+ 树等索引都是它的工业后代。

§4.7 工程与实践

4.7.1 递归深度限制:对象引用建树与 RecursionError

C++ 版的 §4.7 在讲 `unique_ptr`; Python 完全不同: 对象引用 + 引用计数自动管理内存, 没有泄漏与悬垂指针问题——真正的瓶颈是**递归深度**。每个递归调用占一个调用帧, Python 默认限制约 1000 层, 退化链(树高 n) 的插入、求高会直接抛 **RecursionError**:

示例 4-14 撞上限与迭代替代

```
deep = BST()
for i in range(1500):
    deep.insert(i)
# RecursionError: maximum recursion depth exceeded

# 治本: 迭代版中序遍历, 不占递归帧
def inorder_iter(root):
    st, cur, out = [], root, []
    while cur or st:
        while cur:
            st.append(cur)
            cur = cur.left
        cur = st.pop()
        out.append(cur.val)
        cur = cur.right
    return out
```

⚠ 误区 6: 调大递归限制治标不治本

`sys.setrecursionlimit(1000000)` 只是把上限抬高——真到百万层, Python 解释器本身(C 栈)也扛不住, 可能直接崩溃而不是抛异常。正确姿势: ① 深度可控(平衡树, 树高 $O(\log n)$) 时放心用递归; ② 深度可能失控时写迭代版 (§4.2.3 的显式栈)。先想清楚“递归深度 = 树高”, 再决定写递归还是迭代。

C 的 malloc/free 树	Python 的对象引用树
<code>node = malloc(...)</code> , 用完 <code>free</code> , 漏一次就泄漏	对象引用 + 引用计数, 离开作用域自动回收
销毁整树要手写递归 <code>delete</code> , 深树爆栈	无手动销毁; 但递归深度约 1000 层, 深树用迭代
释放后指针仍存在, 悬垂是未定义行为	引用天然安全; 环形引用由 GC 兜底

4.7.2 标准库对照: Python 没有内置平衡树

C++ 的 `std::set` / `std::map` 底层是红黑树; Python 的 `dict` / `set` 是散列表, 无序但平均 $O(1)$ 。有序场景的工程惯例是 `sorted list` + `bisect` (插入 $O(n)$ 、查找 $O(\log n)$); 数据量大、要求更高的替代品(`sortedcontainers` 等)第12章细讲。先记住选型口诀: 只要“字典序/范围查询”就绕不开有序结构; 只查存在与否, 用 `dict/set` 更快:

示例 4-15 dict 词频统计 + 字典序输出

```

words = ['cat', 'dog', 'cat', 'bird']
freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1    # 无序:平均 O(1)

# 要字典序输出:按 key 排序(工程惯例)
for w, c in sorted(freq.items()):
    print(w, c)                    # bird 1 / cat 2 / dog 1

# 查找频次:O(1)
print(freq.get('cat', 0))          # 2

```

4.7.3 调试技巧:打印树与验证性质

- **打印整棵树**:写带缩进的递归打印,比逐结点断点高效
- **中序必升序**:BST 的 **中序遍历必须严格升序**——最便宜的全局检查
- **验证 AVL**:插入后断言全部结点 $|bf| \leq 1$ 并打印树高

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
查找	BST	$O(\log n)$	$O(n)$
插入	BST	$O(\log n)$	$O(n)$
删除	BST	$O(\log n)$	$O(n)$
查找最小 / 最大	BST	$O(\log n)$	$O(n)$
查找 / 插入 / 删除	AVL 树	$O(\log n)$	$O(\log n)$
前/中/后序遍历	二叉树	$O(n)$	$O(n)$
层序遍历	二叉树	$O(n)$	$O(n)$
求高度 / 结点数 / 叶数	二叉树(后序递归)	$O(n)$	$O(n)$
构建表达式树	后缀式 + 栈	$O(n)$	$O(n)$
恢复中缀表达式	表达式树中序	$O(n)$	$O(n)$
查找 / 插入 / 删除	伸展树	$O(\log n)$	$O(n)$ (单次)
查找 / 插入 / 删除	B 树	$O(\log_M n)$	$O(\log_M n)$
查找	有序数组(二分)	$O(\log n)$	$O(\log n)$
插入(有序数组中)	动态数组	$O(n)$	$O(n)$
按值查找	无序数组/链表	$O(n)$	$O(n)$
查找 / 插入 / 删除	dict/set(散列)	$O(1)$	$O(n)$

最小必会清单

- 能说出树的核心术语,解释深度与高度的数法

- 能默写 list 子结点表示,对照 first_child+next_sibling
- 能写出前/中/后序遍历递归版,说清打印位置的作用
- 能写出层序遍历(deque 队列),解释递归为何做不到层序
- 能写出求高度/结点数/叶数的递归函数,说明空树基准
- 能用栈从后缀构建表达式树,再中序恢复中缀(先弹右操作数)
- 能默写 BST 的 contains/find_min/insert/remove,说清删除三情形
- 能说出递归深度限制与迭代替代;有序场景用 sorted list + bisect

自测题

Q 1. 一棵有 n 个结点的一般树有几条边?树的深度与高度有什么区别?

✓ 参考答案

$n-1$ 条:除根外每个结点都有一个父结点。深度从根往下(根 0),高度从叶往上(叶 0);树高 = 根的高度。

Q 2. 对例题 4-1 的树,写出四种遍历输出。

✓ 参考答案

前序:7 3 1 5 6 9 11;中序:1 3 5 6 7 9 11;后序:1 6 5 3 11 9 7;层序:7 3 9 1 5 11 6。后序 11 在 9 前:右子树先于根。

Q 3. 后缀式 $6\ 5\ 2\ -\ 3\ *\ +$ 的表达式树,中序恢复(带括号)是什么?

✓ 参考答案

构建:6、5、2 入栈;- 弹 2、5 得 $(5-2)$;3 入栈;* 得 $(5-2)*3$;+ 得 $6+((5-2)*3)$ 。先弹的是右操作数,故是 $5-2$ 而非 $2-5$ 。

Q 4. BST 删除双子结点时,为什么取右子树最小结点顶替?取左子树最大结点可以吗?

✓ 参考答案

右子树最小结点 m 大于左子树全部、小于右子树其余,顶替后性质不变;且 m 没有左子树,删除退化为单子/叶情形。取左子树最大结点也可行。

Q 5. 空树依次插入 5、3、7、2、4、6、8、1,哪一步触发旋转?什么类型?旋转后树根是谁?

✓ 参考答案

插入 1 后结点 5 的 $bf=+2$,1 在 5 的左孩子 3 的左子树 $\rightarrow$ LL,右旋:3 提为根,5 变其右孩子。旋转后:根 3,左 2(左 1 右 4),右 7(左 6 右 8)。

章末实验题:单词频次统计器(BST 实现)

实验 4:单词频次统计器(BST 实现)

实验目标:用自写 BST 实现命令行单词频次统计器:统计词频、查找、删除,并报告树高与平衡性。

实验要求:

- 任务 1:读入多行英文文本,把连续字母序列切分为单词并统一转小写(\$4.1 树结构、字符串遍历)
- 任务 2:用自写 BST 统计词频——结点存"单词 + 频次",已存在的单词频次 +1(\$4.4 insert)
- 任务 3:中序遍历按字典序输出词频表(\$4.2 中序遍历)
- 任务 4:查找指定单词频次;删除某单词后重新遍历,验证查找结果为 0(\$4.4 contains/remove 三情形)
- 任务 5(加分):平衡检查并报告树高(\$4.5 平衡因子、\$4.2 求高度)

实验环境:Python 3。运行: `python lab4.py`。

第一步 实现结点结构与 insert/find,验证频次正确后再补中序遍历——增量开发

第二步 除用示例 4-10 的三情形骨架;双子先复制右子树最小结点,再删除它(见误区 3)

第三步 平衡检查写成"返回(高度, 是否平衡)"的自底向上函数,一次递归算两件事(示例 4-13 同款思路)

✔ 参考答案(完整可运行程序,分三段)

```

# lab4.py — 单词频次统计器(BST 实现)(第一段:结点与核心操作)
import sys

class Node:
    # 单词 + 频次 + 左右子树
    def __init__(self, word, count=1):
        self.word, self.count = word, count
        self.left = self.right = None

class WordBST:
    def __init__(self):
        self.root = None

    def insert(self, w):
        self.root = self._insert(self.root, w)

    def _insert(self, n, w):
        if n is None:
            return Node(w)          # 空位:新建结点
        if w < n.word:
            n.left = self._insert(n.left, w)
        elif w > n.word:
            n.right = self._insert(n.right, w)
        else:
            n.count += 1             # 已存在:频次 +1
        return n

    def find(self, w):
        # 查找频次,不存在返回 0
        n = self.root
        while n:
            if w == n.word:
                return n.count
            n = n.left if w < n.word else n.right
        return 0

    def remove(self, w):
        self.root = self._remove(self.root, w)

    def _find_min_node(self, n):
        # 右子树最小结点
        while n.left:
            n = n.left
        return n

    def _remove(self, n, w):
        if n is None:
            return None             # 单词不存在,忽略
        if w < n.word:
            n.left = self._remove(n.left, w)
        elif w > n.word:
            n.right = self._remove(n.right, w)
        else:
            # 找到目标结点
            if n.left is None:
                return n.right      # 叶或只有右子树
            if n.right is None:
                return n.left       # 只有左子树
            m = self._find_min_node(n.right) # 双子:取右子树最小
            n.word, n.count = m.word, m.count
            n.right = m.right
        return n

```

```

        n.word = m.word          # 复制数据顶替
        n.count = m.count
        n.right = self._remove(n.right, m.word)
    return n

```

(第二段:中序、树高与平衡检查—类方法,复用示例 4-5 思路)

```

def inorder(self):                # 中序 = 字典序
    self._inorder(self.root)

def _inorder(self, n):
    if n is None:
        return
    self._inorder(n.left)
    print(n.word + '\t' + str(n.count))
    self._inorder(n.right)

def height(self):                 # 空树高度为 0
    return self._height(self.root)

def _height(self, n):
    if n is None:
        return 0
    return 1 + max(self._height(n.left), self._height(n.right))

def is_balanced(self):           # 自底向上检查
    _, ok = self._check(self.root)
    return ok

def _check(self, n):             # 返回(高度, 是否平衡)
    if n is None:
        return 0, True
    hl, bl = self._check(n.left)
    hr, br = self._check(n.right)
    return 1 + max(hl, hr), bl and br and abs(hl - hr) <= 1

```

```

# (第三段:主程序—分词、统计、输出)
def main():
    print("单词频次统计器(BST 实现)—输入英文文本,空行结束:")
    lines = []
    for line in sys.stdin:
        if line.strip() == '':
            break
        lines.append(line)
    text = '\n'.join(lines)

    tree = WordBST()
    word = ''
    for ch in text:
        # 任务 1:分词转小写
        if ch.isalpha():
            word += ch.lower()
        elif word:
            # 任务 2:插入统计
            tree.insert(word)
            word = ''
    if word:
        tree.insert(word)

    print("\n--- 任务 3:中序遍历输出词频表(字典序)---")
    tree.inorder()

    print("\n--- 任务 4:查找频次 ---")
    for w in ('the', 'dog', 'hello'):
        print("查找 '%s': 频次 = %d" % (w, tree.find(w)))

    print("\n删除 'dog' 后重新遍历:")
    tree.remove('dog')
    print("删除后 'dog' 频次 = %d(应为 0)" % tree.find('dog'))
    tree.inorder()

    print("\n--- 任务 5(加分):树高与平衡检查 ---")
    print("树高 = %d, 是否平衡 = %s" % (tree.height(),
        ('是' if tree.is_balanced() else '否')))

if __name__ == '__main__':
    main()

```

单词频次统计器(BST 实现)—输入英文文本,空行结束:

--- 任务 3:中序遍历输出词频表(字典序)---

```
and      1
at       1
barks    1
brown    1
cat      2
dog      2
fox      1
jumps    1
lazy     1
mat      1
on       1
over     1
quick    1
sleeps   1
the      6
warm     1
while    1
```

--- 任务 4:查找频次 ---

查找 'the': 频次 = 6

查找 'dog': 频次 = 2

查找 'hello': 频次 = 0

删除 'dog' 后重新遍历:

删除后 'dog' 频次 = 0(应为 0)

```
and      1
at       1
barks    1
brown    1
cat      2
fox      1
jumps    1
lazy     1
mat      1
on       1
over     1
quick    1
sleeps   1
the      6
warm     1
while    1
```

--- 任务 5(加分):树高与平衡检查 ---

树高 = 9, 是否平衡 = 否

设计说明:① 任务覆盖本章主线:分词(\$4.1)、插入/删除三情形(\$4.4)、中序遍历(\$4.2)、平衡检查(\$4.5),串成可运行流水线;② 结点用对象引用互相连接,Python 自动回收内存,类不需要析构函数;删除双子先复制右子树最小结点、再递归删除它(误区 3);③ 平衡检查是"返回(高度, 是否平衡)"的自底向上函数,一次递归算两件事;实测树高 9,AVL 可压到约 6。

下一章预告:第5章 散列——把查找从 $O(\log n)$ 进一步加速到平均 $O(1)$;散列表的分离链接法将使用第3章的链表作为桶,而"何时用BST、何时用散列"的选型判断将贯穿第5章与后续工程实践。

第5章 散列

建议学时:4-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础(已读完第 1-4 章)

🎯 本章学习目标

- 理解散列为什么能把查找从 $O(\log n)$ 压到平均 $O(1)$:散列函数把键映射为数组下标,理想情况下一步直达。
- 掌握整数取模与字符串 37 倍多项式两种散列函数,能说出"为什么表大小必须是素数、为什么要最后才取模"。
- 掌握分离链接法与开放定址法(线性探测、平方探测、双散列)的实现,理解装填因子 λ 与惰性删除。
- 掌握再散列的动机与实现,能说出摊还代价 $O(1)$;了解可扩散列的基本思想。
- 深入理解 dict:内建散列表,可哈希要求、插入顺序、推导式与 Counter 等常用写法。
- 能独立实现词频统计程序(手写散列表 + dict 两条路线),用实验数据理解装填因子的影响。

💡 从 C 到 Python:本章主题如何迁移

数组按下标访问是 $O(1)$ 的,前提是下标是 0、1、2、... 这样的连续整数。可现实里的键往往不连续:学号是 2023001、2023050,订单号是一串字母数字。散列的想法很简单:写一个函数把"任意的键"翻译成"一个合理的下标",然后直接进数组。这个函数就叫 **散列函数**。

C 语言里你要自己写这个函数,并处理两个麻烦:负数取模会得到负下标(必须 `if (h < 0) h += M`),字符串散列时 int 容易溢出。Python 里这两个麻烦都不存在:取模向下取整、余数恒非负,整数任意精度不会溢出——但"散列思想"本身一字不少,本章的实验仍然要手写一张表。

更重要的差异是:Python 的 dict 就是一张工程级的散列表,你每天都在用而浑然不觉。本章的主线是"先手写,再用库":\$5.2 用 list of list 拼出分离链接散列表(桶就是链表,是 C 语言"结构体 + 指针"的 Python 化);\$5.6 对照着讲 dict 的底层机制——可哈希、扩容、插入序;\$5.7 的单词统计证明手写版与 dict 输出完全一致。

📦 前置知识自查

数学前置:模运算(取余)。重点:Python 的 `%` 是向下取整除法,结果与 C 不同(见 \$5.1.2 误区)。另一条是素数:只能被 1 和自身整除的正整数。

C 语言前置:指针遍历字符串、struct 与链表节点、malloc/free。本章的 C 对照表会反复出现这三样,帮你看清 Python 省掉了什么。

依赖的前面章节:第 3 章链表与 vector——分离链接法的桶就是链表;再散列"翻倍扩容再重插"与 Python list 扩容、dict 扩容同构。

🚀 本章学习路线

1. **第一阶段:**读 \$5.1-\$5.2,吃透散列函数与分离链接法;手算教材 5.2 节小例子,对照示例 5-3 抄一遍代码。
2. **第二阶段:**读 \$5.3-\$5.5,理解三个探测方案的"聚集"问题与平方探测定理;读 \$5.6 时在解释器里逐个敲 dict 示例。
3. **第三阶段:**完成章末实验,手写表与 dict 双路线统计词频,用计时数据看 λ 对性能的影响。

本章知识导图

- §5.1 基本想法与散列函数
 - 5.1.1 基本想法:查找的第三件武器
 - 5.1.2 整数键:取模散列与素数表大小
 - 5.1.3 字符串键:37 倍多项式散列
 - 5.1.4 装填因子 λ
- §5.2 分离链接法
 - 5.2.1 桶就是一条链表
 - 5.2.2 完整实现:查找、插入与删除
 - 5.2.3 复杂度:平均 $O(1)$ 从哪来
- §5.3 开放定址法
 - 5.3.1 线性探测与一次聚集
 - 5.3.2 平方探测
 - 5.3.3 双散列
 - 5.3.4 三种方案对比与惰性删除
- §5.4 再散列
 - 5.4.1 什么时候需要再散列
 - 5.4.2 实现:翻倍、取素数、重插
 - 5.4.3 摊还代价
- §5.5 可扩散列(选读)
- §5.6 标准库中的散列结构
 - 5.6.1 dict:Python 内建的散列表
 - 5.6.2 可哈希:什么对象能当键
 - 5.6.3 常用写法:推导、defaultdict 与 Counter
- §5.7 手写与标准库并排:单词统计

§5.1 基本想法与散列函数

5.1.1 基本想法:查找的第三件武器

你学过两种查找:顺序扫描(链表, $O(n)$)和二叉查找树(平均 $O(\log n)$)。散列是第三条路:把每个键换算成数组下标,查找就是一次数组访问,平均 $O(1)$ 。

表大小为 M ,散列函数 `hash(x)` 把键 x 映射到 0 到 $M-1$ 。插入: `table[hash(x)] = x`;查找:查 `table[hash(x)]` 里有没有 x 。难点在于散列函数"多对一":不同键可能映射到同一下标,这叫 **冲突**。散列的学问 = 好的散列函数(冲突少)+ 好的冲突解决策略。

记住 **散列三要素**:散列函数(键 $\rightarrow$ 下标)+ 冲突解决策略(§5.2 分离链接 / §5.3 开放定址)+ 表大小 M (尽量取素数)。任一环做差,性能都会从"平均 $O(1)$ "掉回"最坏 $O(n)$ "。

5.1.2 整数键:取模散列与素数表大小

键是整数时最常用的散列函数是取模: $\text{hash}(x) = x \% M$, 开销只有一次除法。关键是表大小 M 的选择: 若 M 是合数而键恰好都带某个公因子, 就会全挤进少数几个桶。比如 $M = 10$, 键 42、32、52、72 的个位全是 2, $x \% 10$ 全部等于 2——四键同桶, 九桶全空; 取素数 $M = 11$ 则余数 9、10、8、6, 四散而开。键的分布未知时, 素数表是唯一稳妥的选择。

 **误区: Python 的 % 与 C 不同——向下取整**

C 的 `-5 % 7` 等于 -5, Python 的 `-5 % 7` 等于 2(向下取整, 余数恒非负)。对散列来说这是好事: 下标天然合法, C 里那行 `if (h < 0) h += M` 在 Python 里永远用不上。但移植 C 代码时务必警惕——同样的表达式, 两种语言给出不同结果:

```
# Python 的 % 与 C 不同: 向下取整, 余数恒为非负
x = -5
M = 7
h = x % M                # 结果是 2, 不是 -5!
print(h)                 # 输出 2, 天然适合当数组下标
# 因此 C 里那行 if (h < 0) h += M 在 Python 里永远用不上
# 但移植 C 代码时要警惕: 两种语言对负数取模语义不同
# 若 C 代码里散列值出现负数, 先怀疑取模语义再查逻辑
```

另一个顺带的好处: Python 整数任意精度, 散列累加永远不会溢出——C 里 `int` 溢出导致的“散列值乱跳”问题, 在 Python 里不存在。

C 的写法	Python 的写法
<code>int h = x % M;</code> 负数得到负下标, 要手动 <code>if (h < 0) h += M;</code>	<code>h = x % M</code> 向下取整, 结果恒非负, 坑直接消失
<code>char*</code> 循环累加 <code>h = h*37 + *s</code> , <code>int</code> 可能溢出	<code>h * 37 + ord(c)</code> , 整数任意精度, 无溢出

5.1.3 字符串键:37 倍多项式散列

字符串是最常见的键(单词、文件名、URL)。教材采用 37 倍多项式法, 把字符串按霍纳法则逐位乘加, 每步 `h = h * 37 + c`。先给 C 回顾版, 再给 Python 写法:

示例 5-1 C 回顾: 字符串散列(37 倍多项式, 霍纳法则)

```
/* C 回顾: 字符串散列 */
int hashStrC(char* s, int M) {
    unsigned int h = 0;                /* unsigned 累加, 防止溢出成负数 */
    while (*s != '\0') {
        h = h * 37 + (unsigned char)*s; /* 每步乘 37 再加当前字符 */
        ++s;
    }
    return (int)(h % M);                /* 最后才取模, 结果落在 [0, M-1] */
}
```

示例 5-2 Python 版: 同样的逻辑, 更短的代码

```
# Python 版:整数任意精度,不会溢出;逻辑与 C 完全对应
def hash_str(s: str, M: int) -> int:
    h = 0
    for c in s:
        # 逐个字符取码,累乘累加
        h = h * 37 + ord(c)    # 每一步都乘 37 再加当前字符
    return h % M              # 最后取一次模,结果在 [0, M-1]

# 调用示例:h = hash_str("hello", 17) -> 0 到 16 之间的整数
```

两个版本除类型与遍历方式外一字不差;"最后取一次模"几个字不是风格,是正确性——见下面的误区。

⚠ 误区:逐字符取模会悄悄毁掉散列

有人图省事,把取模写进循环:

```
# 错误示范:把取模写进循环,乘法被取模"吃掉"
for c in s:
    h = (h * 37 + ord(c)) % M    # 错误!
# 当 M = 37 时,(h*37+c)%37 = c%37,
# 散列退化成只看最后一个字符;
# 当 M = 36 时,37 ≡ 1 (mod 36),
# "abc" 与 "cba" 算出同一个散列值,
# 字符串变成了"字符和"。
```

两种退化同源:取模发生在乘法之前,丢掉了进位。正确写法永远是先累加、最后取一次模(示例 5-2)。

5.1.4 装填因子 λ

装填因子 λ = 元素个数 N / 表大小 M ,衡量表"满"的程度: λ 越大冲突越多,查找越慢; λ 越小空位越多,浪费越大。分离链接的经验阈值 $\lambda \approx 0.75$ (再大就该扩容,见 §5.4);开放定址更苛刻,平方探测要求 $\lambda < 1/2$ 。

🚀 记号说明: λ 的两种记法

正文用希腊字母 λ ;代码里命名为 `load_factor` (或 `loadFactor()`),实现为 N / M 的浮点除法,两者是同一个量。

§5.2 分离链接法

5.2.1 桶就是一条链表

分离链接法(separate chaining)的思路最直观:每个下标放一条链表(教材图 5-1)。插入:算桶号挂到链表;查找:算桶号,链内线性扫描;删除:算桶号,链中摘除。 M 决定链表条数, $\lambda = N/M$ 就是平均链长。Python 里"链表"用 `list` 代替,桶数组就是 `list of list`。

C 的写法	Python 的写法
<pre>struct Node { char* key; Node* next; }; 手写链表头插法,自己 malloc/free 每个节点</pre>	桶 = list,桶数组 = list of list,一行 <code>[[] for _ in range(M)]</code>
散列后用 <code>strcmp</code> 逐个比较	桶内元素直接 <code>==</code> 比较
手动 free 所有节点	交给垃圾回收,无释放概念

5.2.2 完整实现:查找、插入与删除

下面是分离链接散列表的完整 Python 实现。 `next_prime(M)` 返回不小于 M 的最小素数(实现见章末实验),构造函数用它定素数表大小。

示例 5-3 分离链接散列表:find / insert / remove

```
# 分离链接散列表:桶数组是 list of list,一个桶就是一条链表
class HashTable:
    def __init__(self, M):
        self.buckets = [[] for _ in range(next_prime(M))]
        self.size = 0

    def _hash(self, s):
        h = 0
        for c in s:
            h = h * 37 + ord(c)
        return h % len(self.buckets)

    def find(self, s):
        for x in self.buckets[self._hash(s)]:
            if x == s:
                return True
        return False

    def insert(self, s):
        if self.find(s):
            return
        self.buckets[self._hash(s)].append(s)
        self.size += 1

    def remove(self, s):
        b = self.buckets[self._hash(s)]
        for i, x in enumerate(b):
            if x == s:
                b.pop(i)
                self.size -= 1
        return

    @property
    def load_factor(self):
        return self.size / len(self.buckets)
```

三个操作同一套路:先 `_hash(s)` 定桶,再只动那一条链——散列把"全表扫描"降为"单桶扫描",单桶平均只有 λ 个元素。insert 先查重再 append;remove 定位后 `pop(i)` 摘除。与 C 的差异在桶上:Python 的 list 自带增删,免去全部指针操作。

5.2.3 复杂度:平均 $O(1)$ 从哪来

一次 find 的代价 = 散列 $O(\text{键长 } L)$ + 单桶扫描 $O(\text{平均链长}) = O(1 + \lambda)$ 。 λ 控制在 0.75 左右时,常数 $1 + \lambda$ 很小,于是散列表查找 **平均 $O(1)$** ;插入、删除同理。

牢记散列表的复杂度画像:查找/插入/删除平均 $O(1)$ 、最坏 $O(n)$ 。最坏情况是所有键散列到同一桶,链表退化成一条长链——散列函数均匀且表大小是素数时几乎不会发生,但你必须能说出它为什么是 $O(n)$ 。代价是牺牲顺序性:无法按键

有序遍历、没有前驱后继;要顺序时回到第 4 章的二叉查找树。

§5.3 开放定址法

分离链接需要额外指针,浪费空间。开放定址法(open addressing)让所有元素都住进表本身:冲突时按某种规则在表中找下一个空位。规则不同,衍生出三种方案。

5.3.1 线性探测与一次聚集

线性探测:冲突时逐个向后试 $pos = (pos + 1) \% M$,直到空位或绕回。它有个著名毛病——一次聚集:连续被占的单元越滚越长,新键插进去又让它更长,形成"雪球"。

示例 5-4 线性探测插入

```
# 开放定址:线性探测插入(空单元用 None 标记)
pos = hash_str(s, M)
while table[pos] is not None and table[pos] != s:
    pos = (pos + 1) % M      # 冲突就向后挪一格,越界绕回 0
if table[pos] is None:
    table[pos] = s          # 找到空位,插入
else:
    pass                   # 键已存在,什么也不做
```

手算一遍: $M = 7$,按序插入 7、15、8、1($h(x) = x \bmod 7$): $7 \rightarrow 0$; $15 \rightarrow 1$; $8 \rightarrow 1$ 冲突试 2,空,落位; $1 \rightarrow 1$ 冲突试 2 仍冲突,试 3,落位。最终下标 0-3 依次是 7、15、8、1:一次聚集——三键挤成一团,后面的键要从 1 一路试到 4。

5.3.2 平方探测

平方探测把步长改成平方:第 i 次冲突后尝试 $pos + i^2$,探测序列从一开始就跳开,避免连续占用。

示例 5-5 平方探测插入

```
# 平方探测:第 i 次冲突后尝试 pos + i^2
pos = hash_str(s, M)
i = 0
while table[pos] is not None and table[pos] != s:
    i += 1
    pos = (pos + i * i) % M    # 探测序列:pos+1, pos+4, pos+9, ...
if table[pos] is None:
    table[pos] = s
# 定理(教材 5.4 节):表大小为素数且装填因子  $\lambda < 1/2$  时,
# 平方探测总能找到空单元,且最多探测一半的表长
```

教材定理:表大小是素数且 $\lambda < 1/2$ 时,平方探测总能在最多一半的表长内找到空单元(证明要点:前半段探测位置互不相同)。工程含义: λ 必须死守在半表以下,因此要更早再散列。它消灭了一次聚集,但留下"二次聚集":同位置的键探测序列完全相同。

5.3.3 双散列

双散列连二次聚集也消灭:步长由第二个散列函数算出,每个键拥有专属探测节奏。常见选择:步长 = $M2 - (h(x) \bmod M2)$, $M2$ 取比 M 小的素数。

示例 5-6 双散列插入

```
# 双散列:冲突时以第二个散列函数的值做步长
def hash_str2(s, M):
    # 第二散列:常用  $M2 = (h \bmod M2)$ ,  $M2$  为素数
    return M - 1 - (hash_str(s, M) % (M - 1)) # 值域  $1..M-1$ , 步长不为 0

pos = hash_str(s, M) # 主散列:决定初始位置
step = hash_str2(s, M) # 第二散列:决定步长,注意不能为 0
while table[pos] is not None and table[pos] != s:
    pos = (pos + step) % M # 每次前进 step 格,而不是 1 格
if table[pos] is None:
    table[pos] = s
```

5.3.4 三种方案对比与惰性删除

方案	探测序列	代价
线性探测	顺次 +1	一次聚集, λ 高时退化严重;实现最简单
平方探测	+1, +4, +9, ...	无一次聚集,有二次聚集;需素数表且 $\lambda < 1/2$
双散列	步长由第二个散列决定	无聚集,但要多算一次散列函数

⚠ 误区:开放定址表不能物理删除

线性探测里删掉一个元素若直接置空,会把探测链拦腰截断:后续依赖它"借道"的元素从此再也找不到。教材做法是**惰性删除**:标记 deleted,查找跳过、插入复用、定期再散列清标记。注意 deleted 单元不算"已占用",装填因子计算要把它们排除。

§5.4 再散列

5.4.1 什么时候需要再散列

装填因子涨过阈值后冲突率陡增,平均链长(或探测次数)失控。唯一的办法是**再散列**:表扩容(通常翻倍取素数),旧元素用新表大小重新散列。插入时检查 λ ,超阈值触发。工程阈值:分离链接约 0.75,开放定址约 0.5。

5.4.2 实现:翻倍、取素数、重插

示例 5-7 再散列:给分离链接表扩容

```
# 再散列:装填因子超过阈值就把表翻倍并重新插入全部元素
def rehash(self):
    if self.load_factor < 0.75:
        return #  $\lambda$  未超标则不做
    old_buckets = self.buckets # 先备份旧桶
    new_m = next_prime(len(old_buckets) * 2) # 新表大小:翻倍后取素数
    self.buckets = [[] for _ in range(new_m)]
    self.size = 0
    for b in old_buckets:
        for x in b:
            self.insert(x) # 会用新表大小重算散列
# 单次再散列  $O(n)$ ,摊到每次插入上仍是  $O(1)$ ——摊还分析见 §11
```

最后是"重新插入"而非"搬过去":桶号 = $\text{hash}(x) \bmod$ 新表大小,每个元素都得重算——"再散列"之名由此而来。

5.4.3 摊还代价

单次再散列 $O(n)$, 但把账摊到 n 次插入上: 每次扩容后空间翻倍, 平均下来 每次插入摊还 $O(1)$ 。这与第 3 章 Python list 扩容的摊还分析同构, 区别是散列表翻倍后元素必须重插(不能拷贝)。

实现建议: dict 的扩容不需要你操心

CPython 的 dict 在装填因子超过 $2/3$ 时自动扩容(resize), 并自动把旧键值对重新散列——这正是教材讲的再散列, 只是你感知不到。手写表时要自己写 rehash(示例 5-7); 用 dict 时记得: 插入 10 万条记录前可用 `d = {}` 直接建, CPython 内部会平滑扩容, 无需预分配。

§5.5 可扩散列(选读)

5.5.1 散列与磁盘的冲突

前面所有方案都假设表在内存。数据量大到必须存磁盘时, 一次磁盘 I/O 是内存访问的几十万倍, 而散列函数均匀分布意味着每次查找都可能访问不同页, 随机 I/O 会让性能崩溃。可扩散列(extendible hashing)专治“键太多, 一页装不下”。

5.5.2 目录、页与局部分裂

可扩散列分两层: 内存里的 目录(directory, 大小为 2^d 的数组) 和一组磁盘 页(page, 一页容纳若干键)。查找: 取散列值的前 d 位定位目录项, 读目录项指向的页。插入时若页满就分裂: 满页一分为二, 目录规模翻倍(d 加一)。只有满的那一页被拆, 其它页原封不动, 这叫 局部分裂——表随数据增长逐页扩张, 而非整体重建。

对照再散列: 整表翻倍、全部重插(摊还 $O(1)$); 可扩散列只分裂满页 + 目录翻倍, 其它页不动, 且只有一页 I/O。代价: 目录占内存, 可能有两页指向同一目录区段(浪费)。数据库与文件系统的索引设计正是这一思想的生产级版本。

§5.6 标准库中的散列结构

5.6.1 dict: Python 内建的散列表

dict 就是一张工程级的散列表: 键不可变可哈希、查找平均 $O(1)$ 、自动扩容(§5.4)。两个 Python 特有的行为要说清楚: 3.7+ 保证按插入顺序遍历(与 C++ unordered_map 的无序相反); `d[k]` 在键不存在时抛 `KeyError`(与 C++ operator[] 自动插入恰好相反)。

示例 5-8 dict 基本用法: 对单词计数

```
# dict: Python 内建的散列表, 键值对即用即走
cnt = {}                                # 空字典
for w in words:
    cnt[w] = cnt.get(w, 0) + 1          # get: 取不到返回默认值 0

# 三种“查”的姿势
if w in cnt:                            # 只想知道在不在: in 成员判断
    n = cnt[w]                          # 直接下标: 键不存在会 KeyError!
n = cnt.get(w, 0)                       # get: 不存在返回默认值, 不抛异常

# 遍历: 3.7+ 按插入顺序
for k, v in cnt.items():
    print(k, v)
```


⚠ 误区:可变对象不能做键

dict 要求键可哈希,而可哈希的前提是"不可变":list、dict、set 可变,不能当键;`d[[1, 2]] = "x"` 直接抛 `TypeError: unhashable type: 'list'`。tuple 只要内部元素都不可变就可哈希:`d[(1, 2)]` 合法。若你真需要"列表键",把 list 转成 tuple 再用——这是 Python 里最常见的散列坑之一。

5.6.2 可哈希:什么对象能当键

"可哈希"就是"有散列函数"的 Python 说法。内置类型里,整数、浮点、字符串、tuple 可哈希;list、dict、set 不可哈希。自定义类的实例默认可哈希(按对象地址散列,与 `==` 一致),这是 Python 的省心之处——不需要像 C++ 那样写函数对象:

示例 5-9 可哈希规则与一行建表

```
# 可哈希:不可变对象才能当键,list 不行,tuple 可以
d = {}
d["name"] = "Ada"           # str 可哈希
d[(1, 2)] = "point"         # tuple 可哈希
# d[[1, 2]] = "x"           # TypeError: unhashable type: 'list'

# 一行建表:字典推导式
freq = {w: words.count(w) for w in set(words)}

# 更高效的默认值工厂
from collections import defaultdict, Counter
cnt = defaultdict(int)      # 访问不存在的键自动给 0
cnt["hello"] += 1
top3 = Counter(words).most_common(3) # 词频排序,取前 3
```

5.6.3 常用写法:推导、defaultdict 与 Counter

词频统计是 dict 最经典的应用,工程上有三种递进写法:`get + 1` (示例 5-8)、`defaultdict(int)` (少一行判断)、`Counter(words)` (一行全部完成,还带 `most_common` 排序)。选型建议:

需求	写法	原因
普通查值、计数	dict + get	无额外依赖,语义直白
频繁"取不到就设默认"	defaultdict	省掉显式判断
计数 + 频率排序	Counter	<code>most_common</code> 一行搞定
需要按键有序遍历	<code>sorted(d)</code>	无内建有序字典,排序后遍历

注意 dict 与 C 的关系:你写的 `dict` 字面量,底层就是一张散列表;理解本章的 `λ`、再散列,就是理解 dict 在"满"的时候做了什么。

§5.7 手写与标准库并排:单词统计

5.7.1 需求:统计文章词频

统计一篇文章里每个单词的出现次数("the 出现 1800 次"之类)。这是散列表的教科书级应用,也是章末实验的前身。同一需求,两条路线各写一遍,并排对比。

5.7.2 手写版实现

示例 5-10 手写版:桶里放 (单词, 次数) 元组

```
# 手写版:桶里放 (单词, 次数) 元组,统计每个单词的出现次数
class WordCounter:
    def __init__(self, m):
        self.buckets = [[] for _ in range(next_prime(m))]
        self.distinct = 0

    def add(self, w):
        b = self.buckets[self._hash(w)]
        for i, (key, cnt) in enumerate(b):
            if key == w:
                b[i] = (key, cnt + 1)    # 已存在:计数 +1
                return
        b.append((w, 1))                 # 新词:插到桶尾
        self.distinct += 1

    def count(self, w):
        for key, cnt in self.buckets[self._hash(w)]:
            if key == w:
                return cnt
        return 0

    def _hash(self, s):
        h = 0
        for c in s:
            h = h * 37 + ord(c)
        return h % len(self.buckets)

def split_words(text):
    return re.findall(r"[a-z0-9]+", text.lower())

def main():
    text = "the cat and the dog and the cat"
    wc = WordCounter(16)                # 表大小 16 → 素数 17
    for w in split_words(text):
        wc.add(w)
    print("the:", wc.count("the"))      # 3
    print("cat:", wc.count("cat"))      # 2
    print("不同单词:", wc.distinct)     # 4

if __name__ == "__main__":
    main()
```

5.7.3 标准库版实现

示例 5-11 标准库版:同一需求,代码量减到 1/5

```
# 标准库版:Counter 一行完成全部计数
import re
from collections import Counter

text = "the cat and the dog and the cat"
words = re.findall(r"[a-z0-9]+", text.lower())
cnt = Counter(words)          # 一次扫描完成全部计数

print("the:", cnt["the"])      # 3
print("cat:", cnt["cat"])      # 2
print("不同单词:", len(cnt))   # 4
print(cnt.most_common(2))     # [('the', 3), ('cat', 2)]
```

5.7.4 对照结论

C 的写法	Python 的写法
结构体数组存 (word, cnt), 每次查找线性扫描 $O(n)$	dict / Counter: 一次扫描完成, 平均 $O(1)$
自己写字符串比较、手动扩容、手动释放	== 内置, 自动扩容, 垃圾回收

调试技巧: 手写版与库版互相验算

手写数据结构最常见的 bug 是“错得整齐”: 两处实现犯同一个错, 测试却全绿。对策是章末实验的做法——同一份数据喂给手写表和 dict, 抽查若干键的词频必须一致。这个“双实现互验”的习惯, 写红黑树、堆时都用得上。

本章复杂度速查表

口诀: 查找/插入/删除平均 $O(1)$, 最坏 $O(n)$; 代价是放弃按键顺序。以下结论两版通用。

操作	数据结构/算法	平均	最坏
查找	散列表(分离链接)	$O(1)$	$O(n)$
插入	散列表(分离链接)	$O(1)$	$O(n)$
删除	散列表(分离链接)	$O(1)$	$O(n)$
查找	散列表(开放定址)	$O(1)$	$O(n)$
插入	散列表(开放定址)	$O(1)$	$O(n)$
删除(惰性)	散列表(开放定址)	$O(1)$	$O(n)$
散列函数	整数取模	$O(1)$	—
散列函数	字符串 37 倍多项式(键长 L)	$O(L)$	—
再散列	单次再散列(翻倍重插)	$O(n)$	$O(n)$
连续插入	含再散列的 n 次插入	摊还 $O(1)$ /次	$O(n)$ /次
查找/插入/删除	dict / set(内建散列表)	$O(1)$	$O(n)$
按插入序遍历	dict(3.7+)	$O(n)$	$O(n)$
按键有序遍历	sorted(d)(无内建有序字典)	$O(n \log n)$	$O(n \log n)$
探测次数	线性/平方探测, $\lambda \approx 0.5$	≈ 1.5 次	$O(n)$
词频统计	手写散列表 / dict / Counter	$O(n)$	$O(n^2)$

最小必会清单

- 能说出 dict 是内建散列表:查找平均 $O(1)$, 3.7+ 保证按插入顺序遍历。
- 能手写取模散列,并知道 Python 的 % 是向下取整(负数余数非负,与 C 不同)。
- 能手写 37 倍多项式字符串散列,解释"为什么先累加、最后取模";整数任意精度无溢出。
- 能说出表大小必须取素数(手写实现时),并给出合数表大小的反例。
- 能说出装填因子 $\lambda = N/M$ 的含义与经验阈值(分离链接 ≈ 0.75 ,平方探测 < 0.5)。
- 能实现分离链接散列表(list of list)的 insert / find / remove。
- 能说出开放定址三种方案的探测序列、聚集现象与适用条件。
- 能解释惰性删除的必要性(物理删除会切断探测链)。
- 能实现再散列,说出"每次插入摊还 $O(1)$ ";知道 dict 自动扩容做的是同一件事。
- 能判断什么对象可哈希:list/dict/set 不能当键,tuple 可以;d[k] 不存在抛 KeyError,get 返回默认值。
- 能用 dict + get、defaultdict、Counter 三种写法做词频统计,知道各自适用场景。

自测题

Q 1. 表大小 $M = 7$,散列函数 $h(x) = x \bmod 7$,用分离链接法依次插入 13、27、41、55。画出各桶的链表,并计算装填因子 λ 与平均链长。

✓ 参考答案

13、27、41、55 除以 7 的余数全是 6——四键全落 6 号桶(最坏情况的构造方式)。 $\lambda = 4/7 \approx 0.57$,平均链长 4/7,但 6 号桶链长 4。

Q 2. 逐字符取模与最后取模,在 $M = 37$ 和 $M = 36$ 时各会怎样退化?

✓ 参考答案

$M = 37: (h \cdot 37 + c) \bmod 37 = c \bmod 37$,退化为"只看最后一个字符"; $M = 36: 37 \equiv 1 \pmod{36}$,每个字符权重都是 1,"abc" 与 "cba" 同值。根治:先累加、最后取一次模。

Q 3. 表大小 $M = 11$,线性探测依次插入 3、14、25、8、19,给出最终表内容与探测过程。

✓ 参考答案

3→3;14→3 冲突→4;25→3 冲突→4 冲突→5;8→8;19→8 冲突→9。最终下标 3、4、5、8、9 依次为 3、14、25、8、19。14、25 挤成连续块,正是"一次聚集"。

Q 4. 开放定址表里删除元素为什么不能直接置空?工程上怎么处理?

✓ 参考答案

直接置空会把探测链拦腰截断:后续键靠被删元素"借道",探测序列在此断掉,再也找不到。工程做法是惰性删除:标记 deleted,查找跳过、插入复用、定期再散列清标记。

Q 5. Python 里为什么 $[1, 2]$ 不能做 dict 的键,而 $(1, 2)$ 可以? $d[k]$ 与 $d.get(k, 0)$ 在键不存在时各是什么行为?

✓ 参考答案

list 可变,不满足可哈希的前提,做键抛 `TypeError: unhashable type`;tuple 不可变,可哈希。 $d[k]$ 键不存在时抛 `KeyError`; $d.get(k, 0)$ 返回默认值 0,不抛异常——只读查询优先用 `get`。

章末实验题:散列表单词统计对比实验

实验 5:散列表单词统计对比实验

实验目标:手写分离链接散列表,与 dict 双路线统计英文文本词频,用计时数据验证装填因子 λ 对性能的影响。

实验要求:

- 任务 1:实现分离链接散列表(键为字符串):37 倍多项式散列、素数表大小、add/count 操作(覆盖 §5.1 散列函数与素数表、§5.2 分离链接法)。
- 任务 2:用自写的表统计文本,输出单词总数、不同单词个数、装填因子 λ (覆盖 §5.1.4 装填因子)。
- 任务 3:用 dict 重做统计,抽查 "the"、"hash"、"hello" 三个词,两套结果必须一致(覆盖 §5.6 标准库;兼作手写版正确性验证)。
- 任务 4:重复插入 30 遍,改变表大小 $M = 8/16/64/256$ (实际素数 11/17/67/257),每种测 3 次取最快;观察 λ 越大耗时越高,体会再散列的动机(覆盖 §5.2.3 复杂度、§5.4 再散列)。
- 任务 5:健壮性——无命令行参数、文件打不开、文本为空都要给出友好提示(覆盖边界处理与工程习惯)。

实验环境:Python 3。运行:python lab5.py sample.txt(任意英文文本,词越多对比越明显)。

第一步 写工具函数:next_prime 定素数表大小;split_words 切词(字母数字连续成词,转小写,其余字符当分隔)。先用 3-5 个单词手算验证散列。

第二步 写 HashTable:构造函数用 next_prime 定桶数;hash 用"先累加、最后取模";add 找到键计数 +1,否则插入新键。写完后用小文本对照字典检查词频。

第三步 后写计时:任务 4 对每个 M 重复 3 次取最快,插入重复 30 遍放大差异;跑前先确认 λ 合理(如 208 个不同单词、 $M=257$ 时 $\lambda \approx 0.81$),再做比较。

✅ 参考答案(完整可运行程序,分三段)

实验 5-1 参考答案(第一部分:工具函数与切词)

```
# lab5.py — 散列表单词统计对比实验(第一部分:工具函数)
import re
import sys
import time
from collections import defaultdict

def next_prime(n):
    """返回大于等于 n 的最小素数。"""
    def is_prime(x):
        if x < 2:
            return False
        d = 2
        while d * d <= x:
            if x % d == 0:
                return False
            d += 1
        return True
    while not is_prime(n):
        n += 1
    return n

def split_words(text):
    """任务 5:切分单词—字母/数字连续成词(正则),其余字符跳过。"""
    return re.findall(r"[a-z0-9]+", text.lower())
```

实验 5-1 参考答案(第二部分:手写分离链接散列表)

```
# ----- 任务 1:分离链接散列表(字符串键,37 倍散列,素数表) -----
class HashTable:
    """桶数组是 list of list,一个桶就是一条链表。"""

    def __init__(self, m):
        self.buckets = [[] for _ in range(next_prime(m))]
        self.size = 0

    def _hash(self, s):
        h = 0
        for c in s:
            h = h * 37 + ord(c) # 全程不取模,最后再 % 表大小(见 §5.1.3)
        return h % len(self.buckets)

    def add(self, s):
        b = self._hash(s)
        for i, (key, cnt) in enumerate(self.buckets[b]):
            if key == s:
                self.buckets[b][i] = (key, cnt + 1)
                return
        self.buckets[b].append((s, 1))
        self.size += 1

    def count(self, s):
        for key, cnt in self.buckets[self._hash(s)]:
            if key == s:
                return cnt
        return 0

    @property
    def load_factor(self):
        return self.size / len(self.buckets)
```

实验 5-1 参考答案(第三部分:main 与四种表大小对比)

```

def main():
    if len(sys.argv) < 2:
        print("用法: python lab5.py <文本文件>")
        return
    try:
        with open(sys.argv[1], encoding="utf-8") as f:
            text = f.read()
    except OSError as e:
        print("错误: 无法打开文件:", e)      # 任务 5:文件打不开
        return
    words = split_words(text)
    if not words:                             # 任务 5:空文本
        print("警告: 文本为空或没有合法单词,无法统计")
        return
    print(f"单词总数(含重复): {len(words)}")

    ht = HashTable(256)                       # 任务 2:手写表统计
    for w in words:
        ht.add(w)
    print(f"手写表: 不同单词 {ht.size} 个,装填因子  $\lambda$ ={{ht.load_factor:.2f}}")

    mp = defaultdict(int)                    # 任务 3:标准库 dict 重做
    for w in words:
        mp[w] += 1
    print(f"dict: 不同单词 {len(mp)} 个")
    for w in ("the", "hash", "hello"):
        print(f"  词 {w!r}: 手写表={{ht.count(w)}} dict={{mp[w]}}")

    print("任务4: 表大小(素数)不同  $\rightarrow$   $\lambda$  不同  $\rightarrow$  插入耗时(3 次取最快):")
    for m in (8, 16, 64, 256):
        best = float("inf")
        for _ in range(3):                   # 重复 3 次取最快,减少系统噪声
            t = HashTable(m)
            t0 = time.perf_counter()
            for _ in range(30):               # 重复 30 遍放大差异
                for w in words:
                    t.add(w)
            best = min(best, (time.perf_counter() - t0) * 1000)
        print(f"  M={{len(t.buckets)}}  $\lambda$ ={{t.load_factor:.2f}} 耗时 {{best:.2f}} ms")
    print("观察:  $\lambda$  越大平均链表越长,耗时越高——这就是再散列($5.4)的动机。")

if __name__ == "__main__":
    main()

```

编译运行命令与真实输出(sample.txt 为 7 段文章拼成,17520 词、208 个不同单词):

■ 实验 5-1 输出样例(本机实测,数值随机器与负载略有波动,趋势稳定)


```
$ python lab5.py sample.txt
单词总数(含重复): 17520
手写表: 不同单词 208 个,装填因子  $\lambda=0.81$ 
dict: 不同单词 208 个
    词 'the': 手写表=1800 dict=1800
    词 'hash': 手写表=120 dict=120
    词 'hello': 手写表=120 dict=120
任务4: 表大小(素数)不同  $\rightarrow \lambda$  不同  $\rightarrow$  插入耗时(3 次取最快):
    M=11  $\lambda=18.91$  耗时 288.13 ms
    M=17  $\lambda=12.24$  耗时 255.71 ms
    M=67  $\lambda=3.10$  耗时 195.66 ms
    M=257  $\lambda=0.81$  耗时 184.79 ms
观察:  $\lambda$  越大平均链表越长,耗时越高——这就是再散列($5.4)的动机。

$ python lab5.py empty.txt
警告: 文本为空或没有合法单词,无法统计
```

设计说明:程序分四层,每层对应一个知识点。散列函数写成"先累加、最后取模"的 37 倍多项式(\$5.1.3),表大小交给 next_prime(\$5.1.2)——本章最易写错的两处,用真实数据检验了正确性。桶用 list of list 实现分离链接(\$5.2),桶内元组携带键与计数,一次遍历完成"查重或累加"。任务 3 的抽查是刻意的工程习惯:手写实现与 dict 输出一致,才敢相信手写版正确。任务 4 把 M 从 11 调到 257, λ 从 18.9 降到 0.81,耗时从 288 ms 降到 185 ms——M=11 时平均每链 1589 个词,查找已接近线性扫描,这是" λ 过高 $\rightarrow$ 性能塌方 $\rightarrow$ 再散列"的实证。计时 3 次取最快 + 重复 30 遍,压掉系统调度噪声。切词规则决定了"单词"的定义,是统计程序必须显式声明的约定。真实 dict 的自动扩容正是示例 5-7 再散列的生产级版本。

下一章预告:第6章 优先队列(堆)——基于数组的完全二叉树;建堆与删除最小将使用第 3 章的 list,堆的插入(上滤)与删除最小(下滤)是第 7 章堆排序、第 9 章 Dijkstra 最短路径的基石。

第6章 优先队列(堆)

建议学时:4-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解**优先队列** ADT(insert/deleteMin/findMin)与第3章普通队列的区别:出队按优先级而非到达顺序
- 掌握二叉堆的**结构性性质**(完全二叉树)与**堆序性质**(父 $\leq$ 子)
- 掌握 0 基数组表示(根=0、左子 $2i+1$ 、右子 $2i+2$ 、父 $(i-1)//2$),能默写**上滤**与**下滤**
- 能解释两种建堆方式的复杂度差异
- 能说出 d-堆、左式堆、斜堆、二项队列的动机与差别
- 会用 `heapq` 四函数(heapify/heappush/heappop/heapreplace),掌握大顶堆的负号技巧
- 能用"大小为 k 的小顶堆"独立解决 top-k,并用 `heapq` 验证手写堆

💡 从 C 到 Python:本章主题如何迁移

优先队列的直觉一句话: "不用排队,按重要性插队"。第3章的普通队列是"先来先服务",而急诊分诊、任务调度、短作业打印都要求"最紧急者先处理"。它只有三个操作:insert、findMin、deleteMin。

在 C 里,朴素做法两头为难:无序数组 insert $O(1)$ 但 deleteMin 要扫描 $O(n)$;有序数组反过来。二叉堆的巧妙之处在于 insert 与 deleteMin 同时做到 $O(\log n)$,且不需要指针——完全二叉树直接摊平进数组,父子关系用下标计算,正是 C 程序员最熟悉的数组思维。

Python 迁移分三层:底层用 `list` (就是动态数组)当堆数组;中间层手写二叉堆类(\$6.2-\$6.4);上层用标准库 `heapq` (\$6.6)。注意 C++ 教材用 1 基(下标 0 闲置),Python 的 `heapq` 用 0 基(\$6.2.3)。

📦 前置知识自查

数学前置:完全二叉树高度 $\approx \log_2 n$;求和记号 Σ 与几何级数(建堆 $O(n)$ 推导用);摊还分析直觉(第11章)。

C 语言前置:数组与下标运算; `struct` 与函数指针(对照用);整数除法(0 基父结点 $(i-1)//2$,与 C 的负数除法不同)。

依赖前面章节:第2章大 O;第3章 `list` (堆的底层数组)与栈;第4章树术语(父、子、叶、深度、高度)。

🚀 本章学习路线

- 第一阶段(1 小时):**读 \$6.1-\$6.2,理解 ADT 与两条性质,背熟 0 基公式
- 第二阶段(1.5 小时):**读 \$6.3-\$6.4,手写上滤/下滤/建堆代码,手工走 insert 与 deleteMin
- 第三阶段(1 小时):**读 \$6.5-\$6.6,俯瞰堆家族,上机实验 `heapq` 四函数与负号技巧
- 第四阶段(实验,1 小时):**完成章末实验:Top-K 高频词
- 验收标准:**能默写上滤与下滤,说出"二叉堆 insert/deleteMin $O(\log n)$ 、建堆 $O(n)$ ",解释 `heapq` 为什么默认是小顶堆

本章知识导图

- §6.1 优先队列模型与动机(任务排程/急诊分诊、ADT 三操作、队列对比)
- §6.2 二叉堆:结构性质与数组表示(完全二叉树、堆序性质、0 基 vs 1 基换算)
- §6.3 插入与删除最小:上滤与下滤(完整代码、"末元素搬根再下滤"套路)
- §6.4 建堆与其余操作(逐个插入 $O(n \log n)$ vs 下滤建堆 $O(n)$ 、decreaseKey/delete/merge)
- §6.5 其他堆:d-堆、左式堆、斜堆与二项队列(动机与一句话差别)
- §6.6 标准库:heapq(默认小顶堆、heapify/heappush/heappop/heapreplace、负号技巧)
- §6.7 手写堆与标准库并排:top-k 实战(小顶堆淘汰技巧、堆排序预告)

§6.1 优先队列模型与动机

6.1.1 三个场景:谁最紧急谁先走

先看三个场景。① **操作系统任务调度**:每次选择优先级最高的进程;② **打印机队列**:短作业优先;③ **急诊分诊**:按病情严重程度处理,最危重者最先抢救。

三个场景的共同点一致:插入一个元素、查看当前最紧急的元素、取出该元素。大小关系代表优先级还是病情,由具体问题映射,数据结构不管。

6.1.2 优先队列 ADT:三个操作

优先队列(priority queue) 是抽象数据类型: `insert(x)` 插入; `findMin()` 返回最小元但不删除; `deleteMin()` 删除并返回最小元。约定数值越小优先级越高(最小堆),取最大则把比较方向反过来(§6.6 负号技巧)。注意:**堆(heap)** 是优先队列最主流的实现,不是 ADT 本身;与存放动态内存的"堆内存"同名,毫无关系。

记号说明(本章约定,与 C++ 版一致)

复杂度记号沿用第2章。下标公式中,"父 $(i-1)//2$ "与"父 $i/2$ "都指**整数除法**(代码写 $(i-1)//2$ 、 $i//2$)。"最小元""最高优先级"同义。

6.1.3 与普通队列对比:出队顺序由谁决定

第3章的普通队列是 FIFO:出队顺序 = 到达顺序;优先队列相反:出队顺序 = 优先级顺序。朴素实现却两头为难:

朴素实现	insert	deleteMin	问题
无序数组/链表	$O(1)$	$O(n)$ (扫描)	删除太慢
有序数组/链表	$O(n)$ (移位)	$O(1)$	插入太慢
二叉堆(本章主角)	$O(\log n)$	$O(\log n)$	两者均衡

C 的写法	Python 的写法
<code>struct PQ { int *a; int n; };</code> 数据与操作分离	<code>class MinHeap:</code> 数据与操作封装在类里
<code>pq_deleteMin(&pq, &out);</code> 每个操作传指针	<code>h.pop()</code> 方法天然绑定对象
找最小要自己写 <code>for</code> 循环扫描	根就是最小元, <code>h.top()</code> 一行
数组长度、容量、释放全靠手工	<code>list</code> 自动扩容, 无指针无释放

§6.2 二叉堆:结构性质与数组表示

6.2.1 结构性质:完全二叉树

二叉堆首先是一棵**完全二叉树**: 除最后一层外每层都满, 最后一层从左到右连续填充、不留空隙。这条性质的威力在高度: 结点数 n 满足 $2^h \leq n < 2^{h+1}$, 所以 $h \approx \log_2 n$ ——任何根到叶路径只有对数长度, 这正是上滤/下滤高效的根源。

完全二叉树还不需要指针: 结点按层序编号, 父子关系用下标直接算出 (§6.2.3)。对比第4章的指针树, 连续数组节省指针空间、缓存友好, 代价是形状被锁定, 只能在末尾增删。

6.2.2 堆序性质:最小堆

堆序性质: 对每个非叶结点, 父结点的值 $\leq$ 子结点的值 (最小堆; 换成 $\geq$ 即最大堆)。两条性质合起来才是二叉堆: 结构性质保证对数高度, 堆序性质保证根是全局最小元, `findMin()` 是 $O(1)$ 。

立刻澄清: 堆序只约束"父子", 兄弟之间没有大小关系。堆因此不是有序数组, 不能二分查找, 按值查找只能全扫 $O(n)$ 。堆只承诺最小元在根, 是"弱有序"结构, 与第4章 BST 的强有序对照。

6.2.3 数组表示:0 基与 1 基换算

把完全二叉树按层序放进数组, 父子关系退化为下标运算。这里出现全系列最重要的"习惯分歧": Python 的 `heapq` 用 0 基——根在下标 0; C++ 教材(Weiss 原书)用 1 基——根在下标 1, 下标 0 闲置。两套公式:

含义	0 基(Python 用)	1 基(C++ 教材习惯)
根	下标 0	下标 1
结点 i 的左子	$2i+1$	$2i$
结点 i 的右子	$2i+2$	$2i+1$
结点 i 的父	$(i-1)//2$ (整数除法)	$i/2$ (整数除法)
最后一个非叶结点	$n//2 - 1$	$n/2$
下标 0 是否闲置	否, 下标 0 就是根	是, <code>a[0]</code> 闲置

验证: 下标 5 的结点, 0 基下左子 11、右子 12、父 2; 1 基下左子 10、右子 11、父 2。父相同只是巧合——下标 4: 0 基父 1, 1 基父 2。

⚠ 误区 1:把 1 基公式抄进 0 基代码(或反之)

两种下标体系并存是本章第一大坑。0 基堆里左子必须写 $2*i+1$ 、父必须写 $(i-1)//2$ ——抄成 1 基的 $2*i$ 、 $i//2$ 会取到错位置;建堆时 1 基从 $n//2$ 开始,0 基必须从 $n//2-1$ 开始,差一个结点。解决之道:写代码前先声明"用几基"并写进注释,写完用"验证堆序"函数检查 (§6.3.3)。

示例 6-1 二叉堆类骨架(0 基,list 作底层)

```
# 0 基堆:根在下标 0,左子 2i+1、右子 2i+2、父 (i-1)//2
class MinHeap:
    def __init__(self):
        self.a = []          # 底层就是 list,无需占位

    def __len__(self):
        return len(self.a)

    def top(self):
        return self.a[0]     # 根即最小元 0(1)

    def check_heap(self):
        """调试辅助:验证堆序(见 §6.3.3)"""
        for i in range(len(self.a)):
            for c in (2 * i + 1, 2 * i + 2):
                if c < len(self.a) and self.a[i] > self.a[c]:
                    return False
        return True
```

§6.3 插入与删除最小:上滤与下滤

6.3.1 插入:上滤(percolateUp)

插入策略是"先保结构,再修堆序": ① 新元素追加到末尾(完全二叉树只在末尾留空位,结构自动保持); ② 沿根路径向上冒泡——与父比较,小于父则交换,直到父 $\leq$ 自己或到根。这叫**上滤(percolate up)**,路径最长即树高,insert 为 $O(\log n)$; 上滤只看一个父,正是堆"弱有序"的便利。

示例 6-2 insert:上滤

```
def push(self, x):
    self.a.append(x)          # 追加到末尾:保持完全二叉树
    i = len(self.a) - 1      # 新元素下标
    while i > 0:              # 向上冒泡(上滤)
        p = (i - 1) // 2      # 父结点(0 基公式)
        if self.a[p] <= self.a[i]:
            break              # 父 <= 子,堆序成立,停
        self.a[p], self.a[i] = self.a[i], self.a[p]  # 与父交换
        i = p
```

6.3.2 删除最小:下滤(percolateDown)

删除根会留下空洞,朴素前移会破坏完全二叉树或付出 $O(n)$ 搬移。经典套路: ① 记下根(它就是答案); ② 把**最后一个元素搬到根**(末尾是唯一可合法删除的空位,结构自动保持); ③ 沿"叶方向"向下冒泡——与孩子中**较小者**比较,大于它则交换,直到 $\leq$ 两个孩子或到叶。这叫**下滤(percolate down)**,同样 $O(\log n)$ 。

两个细节:必须与"较小的"孩子交换(与较大的换会违反堆序);先检查右子是否存在(下标 $2i+2$ 是否在界内),只有左子也合法。

示例 6-3 deleteMin:末元素搬根再下滤

```
def _sift_down(self, i):
    """下滤:把下标 i 的元素向下沉(deleteMin 与建堆共用)"""
    n = len(self.a)
    while True:
        smallest = i
        for c in (2 * i + 1, 2 * i + 2):      # 左子、右子(0 基)
            if c < n and self.a[c] < self.a[smallest]:
                smallest = c                # 选"较小"的孩子
        if smallest == i:
            break                          # 两个孩子都不小于自己
        self.a[i], self.a[smallest] = self.a[smallest], self.a[i]
        i = smallest

def pop(self):
    """删除最小元:最后一个元素搬到根,再下滤"""
    top = self.a[0]                        # 记下根(答案)
    last = self.a.pop()                   # 末元素搬根:保持完全二叉树
    if self.a:                            # 只剩 1 个元素则免
        self.a[0] = last
        self._sift_down(0)
    return top
```

例题 6-1 手工走一遍 insert 与 deleteMin

设 0 基堆初始为 `[13, 21, 16, 24, 31, 19, 68, 65, 26, 32]`。**insert(14)**:追加到下标 10,沿 $10 \rightarrow 4 \rightarrow 1 \rightarrow 0$ 上滤: $31 > 14$ 换、 $21 > 14$ 换、 $13 \leq 14$ 停,得 `[13, 14, 16, 24, 21, 19, 68, 65, 26, 32, 31]`。**deleteMin()**:取走 13,末元素 31 搬根,依次换过 14、21、26,得 `[14, 21, 16, 24, 26, 19, 68, 65, 31, 32]`,最小元 14。

6.3.3 完整接口与调试技巧

完整堆还应有 `top()`、`__len__` (配合 `len(h)`) 等接口;若要"按插入顺序稳定",可在元素里加序号比较(见 §6.7 的 (频次, 词) 元组)。

调试技巧:写一个"验证堆序"的断言函数

堆的 bug 极具隐蔽性:上滤少换一层、下滤选错孩子,结果仍"接近正确"。随手加 `check_heap()` (示例 6-1 已给出):遍历所有结点,断言父 $\leq$ 每个存在的孩子,每次 push/pop 后调用,把可疑行为扼杀在摇篮里。

§6.4 建堆与其余操作

6.4.1 两种建堆方式

建堆有两种做法。① **自顶向下**:空堆逐个 insert,每次 $O(\log n)$,共 $O(n \log n)$ 。② **自底向上(下滤建堆)**: n 个元素按任意顺序放进数组(天然构成完全二叉树,只是堆序可能被违反),从最后非叶结点($n//2-1$)往前逐个 percolateDown 到根,总代价 $O(n)$ 。"一次性给全所有元素"的场景都用下滤建堆。

示例 6-4 下滤建堆: $O(n)$

```
@classmethod
def build_from(cls, items):
    """下滤建堆  $O(n)$ :从最后一个非叶结点( $n//2-1$ )开始逐个下滤到根"""
    h = cls()
    h.a = list(items)
    for i in range(len(h.a) // 2 - 1, -1, -1):
        h._sift_down(i)
    return h
```

例题 6-2 手工走一遍下滤建堆

输入 [150, 80, 40, 30, 10, 70, 110, 100, 20, 90, 60, 50, 120, 140, 130] ($n = 15$, 最后非叶 = 下标 6)。
 从 6 往前: 6(110) ≤ 130 停; 5(70) 与 50 换; 4(10) ≤ 90、60 停; 3(30) 与 20 换; 2(40) ≤ 50、110 停; 1(80) 与 10 换、再与 60 换; 0(150) 依次与 10、20、30 换。最终 [10, 20, 40, 30, 60, 50, 110, 100, 80, 90, 70, 150, 120, 140, 130]。

6.4.2 为什么下滤建堆是 $O(n)$:一句话推导

关键观察:高度为 h 的结点约有 $n/2^{h+1}$ 个,每个下滤至多走 h 步,总代价 $\sum_h h \cdot n/2^{h+1}$ 是"几何级数收尾"的求和,结果 $O(n)$ 。直觉:矮的结点数量多但只需下滤几步,高的结点上滤远但数量极少;逐个插入的代价全集中在"深叶子上滤到根"的长路径。严格证明在第11章。

6.4.3 decreaseKey、delete 与 merge

- **decreaseKey(p, delta)**:键变小只能向上走——直接上滤, $O(\log n)$ 。前提是知道 p 的位置(堆不提供查找);经典用途:Dijkstra 松弛(第9章)。
- **delete(任意位置 p)**:把 $a[p]$ 换成最小值(根的副本),上滤到根,再 deleteMin——两步都 $O(\log n)$ 。
- **merge(合并两个堆)**:二叉堆的短板——必须把两个数组拷到一起再建堆, $O(n)$ 。频繁合并堆的应用,二叉堆不够用,这正是 §6.5 堆家族的动机。

示例 6-5 decreaseKey 与 delete(任意位置)

```
def decrease_key(self, i, delta):
    """键变小只会"向上"走:减键后上滤  $O(\log n)$ """
    self.a[i] -= delta
    while i > 0:
        p = (i - 1) // 2
        if self.a[p] <= self.a[i]:
            break
        self.a[p], self.a[i] = self.a[i], self.a[p]
        i = p

def remove_at(self, i):
    """删除任意位置:盖成最小值上滤到根,再 pop  $O(\log n)$ """
    self.a[i] = self.a[0]          # 根是最小值,盖上去不破坏堆序
    while i > 0:                  # 一路换到根
        p = (i - 1) // 2
        self.a[p], self.a[i] = self.a[i], self.a[p]
        i = p
    self.pop()                   # 现在根就是原来的 a[i]
```


§6.5 其他堆:d-堆、左式堆、斜堆与二项队列

6.5.1 为什么不止一种堆

二叉堆已做到 insert/deleteMin 均 $O(\log n)$ 、 findMin $O(1)$,为何还要别的堆?因为**操作组合不同**:插入远多于删除的系统希望 insert 更快;频繁合并堆的系统受不了 merge $O(n)$ 。没有一种堆能同时最优,每个成员都在特定操作上让步、换取另一操作的改进。

6.5.2 d-堆:多叉,降低高度

d-堆 是二叉堆的直接推广:每个结点最多 d 个孩子(二叉堆即 $d = 2$)。高度从 $\log_2 n$ 降到 $\log_d n$, insert (只与一个父比较)变快为 $O(\log_d n)$;但 deleteMin 要在 d 个孩子里找最小,变慢为 $O(d \log_d n)$ 。典型场景: insert 远多于 deleteMin 的离散事件模拟,常用 $d = 4$ 或 5 。

6.5.3 左式堆:为 merge 而生

二叉堆 merge $O(n)$ 的根源是"完全二叉树"锁死了形状。**左式堆(leftist heap)** 主动放弃完全性,改用新约束:对每个结点,左子的零路径长(NPL)不小于右子的——即"左偏"。**零路径长(null path length)** 是到最近叶子的最短路径长度(空结点为 -1)。它保证最右路径很短, merge 递归沿最右路径合并, $O(\log n)$; insert/deleteMin 都可用 merge 实现。

6.5.4 斜堆:左式堆的摊还简化版

斜堆(skew heap) 是左式堆的极简主义版本:不维护 NPL, merge 后**无条件交换左右子树**。实现更短,但单次可能退化 $O(n)$,整体摊还 $O(\log n)$ (第11章证明)。它与左式堆的关系,正如伸展树之于 AVL 树。

6.5.5 二项队列:森林

二项队列(binomial queue) 的思路完全不同:不是一棵树,而是一组"二项树"组成的**森林**。第 k 棵二项树大小恰为 2^k ,每种大小最多一棵—— n 个元素按二进制位决定有哪些树(如 $n = 13 = 1101$,有 8 、 4 、 1 三棵)。插入像二进制加法"进位"合并同大小树,摊还 $O(1)$; merge 与 deleteMin 均 $O(\log n)$ 。

堆类型	动机	insert	deleteMin	merge
二叉堆	最常用,数组存储、缓存友好	$O(\log n)$	$O(\log n)$	$O(n)$
d-堆	插入多于删除,降高度	$O(\log_d n)$	$O(d \log_d n)$	$O(n)$
左式堆	高效 merge,左偏约束	$O(\log n)$	$O(\log n)$	$O(\log n)$
斜堆	左式堆的摊还简化版	摊还 $O(\log n)$	摊还 $O(\log n)$	摊还 $O(\log n)$
二项队列	森林结构,插入摊还 $O(1)$	摊还 $O(1)$	$O(\log n)$	$O(\log n)$

§6.6 标准库:heapq(默认小顶堆与负号技巧)

6.6.1 四个函数:heapify、heappush、heappop、heapreplace

`heapq` 是标准库的堆工具箱,与 C++ 的 `std::priority_queue` 位置相同,但设计哲学完全不同:它**不提供堆对象**,只有一组操作**普通 list** 的函数——你拿一个 list 调 `heappush`、`heappop`,list 就地变成堆。最大的特点:它默认是"小顶堆",与 `priority_queue` 默认大顶堆正好相反。

- `heapify(lst)`:把任意 list 原地建成小顶堆, $O(n)$ ——即下滤建堆;

- `heappush(lst, x)` :插入并上滤, $O(\log n)$;
- `heappop(lst)` :弹出并返回堆顶(最小元), 末元素搬根再下滤, $O(\log n)$;
- `heapreplace(lst, x)` :先弹出堆顶再压入 x (一步完成), 等价于 `heappop + heappush` 但更快——top-k 淘汰的标准写法。

示例 6-6 `heapq` 四函数与负号技巧

```
import heapq

lst = [3, 1, 9, 5]
heapq.heapify(lst)          # ① 原地建小顶堆  $O(n)$ 
# lst == [1, 3, 9, 5](堆序成立, 兄弟间无序)

heapq.heappush(lst, 2)      # ② 插入并上滤: lst == [1, 2, 9, 5, 3]
m = heapq.heappop(lst)      # ③ 弹出最小元: m == 1
heapq.heappush(lst, 0)
heapq.heapreplace(lst, 7)   # ④ 弹出堆顶(0)再压入 7

# 负号技巧: heapq 没有大顶堆, 存负值即得"大顶堆"
big = []
for x in [3, 1, 9, 5]:
    heapq.heappush(big, -x)  # 存 -x: 最小堆里最小的是 -9
max_val = -heapq.heappop(big) # 取回时再取负: 9
```

⚠ 误区 2: 负号技巧只取一半(忘负或忘取负)

`heapq` 没有 `greater<T>` 那样的比较器参数, 大顶堆只能靠存负值: 压入 `-x`, 弹出后 `-heapq.heappop(...)`。新手要么压入时忘取负(堆里全是负数, 弹出来全是负的), 要么弹出时忘取负(拿到 `-9` 而不是 `9`)。口诀: 负进负出。另一个坑: 取负要求元素可被 `abs` -取负——字符串、元组整体取负会 `TypeError`, 只能对数值字段取负(见 §6.6.2)。

6.6.2 自定义优先级: 元组天然可比较

`priority_queue` 靠模板参数传比较器; `heapq` 没有比较器参数, 靠**元素本身**可比: 存 (频次, 词) 元组, Python 自动先比频次、平局比词序, 零配置。求"最大的 k 个"时更常用 (-频次, 词) 或 (频次, 词) 配合小顶堆淘汰 (§6.7)。

C++ 的 <code>std::priority_queue</code>	Python 的 <code>heapq</code>
封装成类, 构造时传比较器	无堆对象, 四函数直接操作普通 list
默认大顶堆, 最小堆要写 <code>greater</code>	默认小顶堆, 大顶堆用负号技巧
<code>pop</code> 不返回元素, 先 <code>top</code> 再 <code>pop</code>	<code>heappop</code> 直接返回被弹元素
批量建堆: 构造时拷入容器	<code>heapify</code> 原地建堆, 零拷贝
自定义比较靠比较器模板参数	自定义比较靠元素本身(元组/负号)

🚀 工程建议: 什么时候直接上 `heapq`

工程中"需要动态维护最值"的场景一律优先 `heapq`: 任务调度、Top-K、贪心(第10章哈夫曼)、Dijkstra(第9章)。手写堆的用武之地: 需要 `decreaseKey`(标准库没有)、批量建堆后长期复用、或考试要求展示原理。只"一次性排序"用 `sorted()` (常数更小); "边插入边取最值"才用堆。

§6.7 手写堆与标准库并排:top-k 实战

6.7.1 问题:求最大的 k 个(top-k)

Top-K 问题:给定 n 个元素,求最大的 k 个(排行榜、热门词)。朴素做法全部排序是 $O(n \log n)$;堆做法 $O(n \log k)$, k 远小于 n 时近乎线性。核心技巧叫**小顶堆淘汰法**:维护"当前最大的 k 个"的小顶堆——堆顶恰是第 k 大,即淘汰"门槛";新元素大于堆顶才替换。

为什么用"小顶堆"而非"大顶堆"? 大顶堆的堆顶是全局最大,淘汰后不知谁替补;小顶堆的堆顶是"最小的入选者",谁比它大谁挤掉它,淘汰逻辑一行搞定。

C 的朴素做法(qsort 全排序)	Python 的堆做法(heapq)
qsort 把 n 个元素全部排序, $O(n \log n)$	大小为 k 的小顶堆, $O(n \log k)$
结果数组占 n 个位置	堆只占 k 个位置,可流式处理
新数据到达必须重新 qsort 一遍	heappush/heapreplace 动态维护,每次 $O(\log k)$
比较函数用函数指针,写在调用处之外	元组天然可比较,零配置

⚠ 误区 3:top-k 时"无脑入堆",忘了先与堆顶比较

堆满后仍直接 heappush,堆会涨过 k ;或"先 push 再 pop 堆顶",把刚进来的大元素又弹走——淘汰条件写反。正确顺序只有一种:先与堆顶比较,大于门槛才 pop 再 push,且 pop 必须在 push 之前。用 heapreplace 一步完成弹旧压新,从机制上杜绝顺序错误。

6.7.2 手写版与标准库版并排

同一任务——给一组(频次,词)求频次最高的 k 个——分别用手写堆与 heapq 实现。两份代码逻辑结构一字不差,正是"手写理解原理、标准库交付工程"的教科书式对照。

示例 6-7 手写堆版 top-k(复用示例 6-1~6-3 的 MinHeap)

```
# items: (频次, 词) 列表;结果: 按频次降序输出最大的 k 个
def topk_heap(items, k):
    h = MinHeap()
    for t in items:
        # t 是 (频次, 词) 元组
        if len(h) < k:
            h.push(t)
            # 堆未满:直接入堆
        elif t > h.top():
            # 高于"第 k 大门槛"才替换
            h.pop()
            h.push(t)
    res = []
    while len(h) > 0:
        res.append(h.pop())
    return list(reversed(res))
# 小顶堆弹出是升序,反转为降序
```

示例 6-8 heapq 版 top-k:逻辑一字不差

```
def topk_heapq(items, k):
    """同一任务用 heapq:heappush/heappreplace 换掉手写调用"""
    h = []
    for t in items:
        if len(h) < k:
            heapq.heappush(h, t)
        elif t > h[0]:
            # h[0] 就是小顶堆的堆顶
            # 弹旧压新:从机制上杜绝顺序错误
            heapq.heapreplace(h, t)
    return sorted(h, reverse=True)
    # 堆内即答案,排序只为了降序展示
```

6.7.3 复杂度分析与堆排序预告

每个元素至多一次比较 + 一次替换($O(\log k)$),总代价 $O(n \log k)$; $k = n$ 时退化为 $O(n \log n)$ 。优势在**内存**:堆只占 k 个位置,几十 GB 的日志放不进内存,却能流式处理。

预告第7章:堆排序直接复用本章的二叉堆——建堆 $O(n)$,连续 `deleteMin` n 次,弹出元素天然有序,总代价 $O(n \log n)$,与插入、希尔、归并、快排逐一对比。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
insert	二叉堆	$O(\log n)$	$O(\log n)$
deleteMin / deleteMax	二叉堆	$O(\log n)$	$O(\log n)$
findMin	二叉堆	$O(1)$	$O(1)$
buildHeap(下滤建堆)	二叉堆	$O(n)$	$O(n)$
buildHeap(逐个插入)	二叉堆	$O(n \log n)$	$O(n \log n)$
decreaseKey	二叉堆	$O(\log n)$	$O(\log n)$
delete(任意元素)	二叉堆	$O(\log n)$	$O(\log n)$
merge	二叉堆	$O(n)$	$O(n)$
按值查找 find	二叉堆	$O(n)$	$O(n)$
insert	d-堆	$O(\log_d n)$	$O(\log_d n)$
deleteMin	d-堆	$O(d \log_d n)$	$O(d \log_d n)$
merge	左式堆	$O(\log n)$	$O(\log n)$
merge	斜堆	摊还 $O(\log n)$	$O(n)$
insert	二项队列	摊还 $O(1)$	$O(\log n)$
deleteMin	二项队列	$O(\log n)$	$O(\log n)$
merge	二项队列	$O(\log n)$	$O(\log n)$
top-k(大小为 k 的小顶堆)	二叉堆 + 一次遍历	$O(n \log k)$	$O(n \log k)$

最小必会清单

- 能说出优先队列 ADT 的三个操作与普通队列的本质区别
- 能说出二叉堆的两条性质,并画出完全二叉树的数组填充
- 能默写 0 基公式(左子 $2i+1$ 、右子 $2i+2$ 、父 $(i-1)//2$ 、最后非叶 $n//2-1$),并与 1 基公式换算
- 能默写 insert(上滤)与 deleteMin(末元素搬根再下滤)的代码
- 能解释"删除最小为什么是搬最后一个元素",以及"下滤为什么必须选较小的孩子"
- 能独立写出下滤建堆,并解释两种建堆方式的复杂度差异
- 能说出 decreaseKey、delete、merge 的代价与场景
- 能一句话说出 d-堆、左式堆、斜堆、二项队列的动机与差别
- 能熟练写出 heapify/heappush/heappop/heapreplace 的用法,并解释负号技巧
- 能独立用"大小为 k 的小顶堆"解决 top-k,并解释为什么不用大顶堆

自测题

Q 1. 二叉堆为什么必须是完全二叉树?如果允许任意形状的二叉树,会损失什么?

✓ 参考答案

完全性保证高度恒为 $\log_2 n$ 与数组连续存储(无指针、下标算父子)。若形状任意,最坏退化成链,高度与操作都变 $O(n)$,还得用指针。

Q 2. 0 基堆中下标 i 的结点的父、左子、右子分别是什么?1 基堆呢?试各举一例。

✓ 参考答案

0 基:父 $(i-1)//2$ 、左子 $2i+1$ 、右子 $2i+2$ (如 $i=5$:父 2、左 11、右 12);1 基:父 $i/2$ 、左子 $2i$ 、右子 $2i+1$ (如 $i=5$:父 2、左 10、右 11)。“父为 2”只是巧合,不可混用。

Q 3. 在最小堆中查找某个值的复杂度是多少?为什么不能像 BST 那样二分?

✓ 参考答案

$O(n)$ 。堆序只约束父子关系,兄弟之间没有大小关系,无法一次比较排除一半子树;堆也不是有序数组,连续存储帮不上二分。堆只承诺最小元在根。

Q 4. 把 15 个元素下滤建堆,最后一个下滤的结点是哪个?为什么不是第一个?

✓ 参考答案

最后下滤的是根(下标 0)。建堆从最后非叶结点($n//2-1$)往前处理,先让下面子树各自成堆,上层下滤时才能保证孩子子树已是堆、只需比较两个孩子。根最后处理,一次下滤把最小值带到根。

Q 5. deleteMin 为什么要把"最后一个元素"搬到根,而不是把后面元素整体前移?

✅ 参考答案

整体前移会留下空隙(破坏结构性)或付出 $O(n)$ 搬移。最后一个元素是唯一"删除后数组仍连续"的位置:先搬到根保持结构,再下滤修复堆序,两步各 $O(\log n)$ 。

Q 6. 求 top-k 为什么用大小为 k 的"小顶堆"?用"大顶堆"行吗?

✅ 参考答案

小顶堆的堆顶是"当前入选者里最小的"(第 k 大门槛),新元素与堆顶比较即可决定淘汰谁。大顶堆的堆顶是全局最大,被淘汰后无法 $O(1)$ 判断谁替补,只能维护更多信息或退化成排序。求"最小的 k 个"改用大小为 k 的大顶堆。

章末实验题:Top-K 高频词(堆实现)

📄 实验 6:Top-K 高频词(手写堆 + 标准库双实现)

实验目标:流式读入英文文本,先用**手写二叉堆**求频次最高的 k 个词,再用 `heapq` 重做并验证两版结果一致。

实验要求:

- 任务 1:实现二叉堆类(insert/deleteMin/下滤建堆,0 基,注释中说明下标公式)(知识点:\$6.2-\$6.4 上滤/下滤/建堆)
- 任务 2:流式读入文本,切分出单词并维护"词频"字典(知识点:\$6.1 + 第5章散列)
- 任务 3:用"大小为 k 的小顶堆"求频次最高的 k 个词,淘汰技巧:堆满时频次高于堆顶才替换(知识点:\$6.7 top-k)
- 任务 4:用 `heapq` (`heappush`/`heapreplace`)重做任务 3,对比两版结果一致(知识点:\$6.6 标准库)
- 任务 5:处理边界: $k \leq 0$ 、文本为空、k 大于词数时给出合理输出而非崩溃(知识点:\$6.3)

实验环境:Python 3。运行: `python lab6.py k < 文本文件`。

第一步 写 `MinHeap` 类(0 基),用例题 6-1 的序列手工验证 `insert` 与 `deleteMin`,再写 `top-k` 函数

第二步 切分用 `re.findall(r"[a-zA-Z']+", text.lower())` 一行完成

第三步 元素用 (频次, 词) 元组:先比频次、平局比词序

第四步 `heapq` 版用 `heapreplace` 一步完成弹旧压新;结果排序用 `sorted(reverse=True)`

第五步 一行样例: `python lab6.py 3 < sample.txt`,再试 $k = 0$ 与空文件

✓ 参考答案(完整可运行程序,约 120 行,分三段)

```

# lab6.py — 第一段:MinHeap(0 基)
# 运行: python lab6.py k < 文本文件
import heapq
import re
import sys

class MinHeap:
    """任务 1:手写二叉堆(0 基:根在下标 0,左子 2i+1、右子 2i+2、父 (i-1)//2)。
    元素为 (频次, 词) 元组:频次小者优先级高,平局按词序比较。"""

    def __init__(self):
        self.a = []                # 底层就是普通 list,无需占位

    def __len__(self):
        return len(self.a)

    def _sift_up(self, i):
        """上滤:从下标 i 向上冒泡,直到父 <= 自己。"""
        while i > 0:
            p = (i - 1) // 2        # 0 基父结点公式
            if self.a[p] <= self.a[i]:
                break
            self.a[p], self.a[i] = self.a[i], self.a[p]
            i = p

    def _sift_down(self, i):
        """下滤:从下标 i 向下沉,始终与较小的孩子交换。"""
        n = len(self.a)
        while True:
            smallest = i
            for c in (2 * i + 1, 2 * i + 2): # 左子、右子(0 基)
                if c < n and self.a[c] < self.a[smallest]:
                    smallest = c
            if smallest == i:
                break
            self.a[i], self.a[smallest] = self.a[smallest], self.a[i]
            i = smallest

    def push(self, x):
        self.a.append(x)
        self._sift_up(len(self.a) - 1)

    def pop(self):
        """删除最小元:最后一个元素搬到根再下滤($6.3 套路)。"""
        top = self.a[0]
        last = self.a.pop()
        if self.a:
            self.a[0] = last
            self._sift_down(0)
        return top

    def top(self):
        return self.a[0]

```

lab6.py(第一段补充:下滤建堆 build_from)

```
@classmethod
def build_from(cls, items):
    """任务 1 补充:下滤建堆  $O(n)$ —从最后一个非叶结点往上逐个下滤。"""
    h = cls()
    h.a = list(items)
    for i in range(len(h.a) // 2 - 1, -1, -1):
        h._sift_down(i)
    return h
```

lab6.py(第二段:词频统计与手写堆 top-k)

```
def count_words(text):
    """任务 2:流式切词并统计词频,维护"词频"字典。"""
    freq = {}
    for w in re.findall(r"[a-zA-Z']+", text.lower()):
        freq[w] = freq.get(w, 0) + 1
    return freq

def topk_heap(items, k):
    """任务 3:手写小顶堆求频次最高的 k 个(小顶堆淘汰技巧)。"""
    h = MinHeap()
    for t in items:
        # t 是 (频次, 词) 元组
        if len(h) < k:
            h.push(t)
            # 堆未满:直接入堆
        elif t > h.top():
            # 高于"第 k 大门槛"才替换
            h.pop()
            h.push(t)
    res = []
    while len(h) > 0:
        res.append(h.pop())
    return list(reversed(res))
    # 小顶堆弹出是升序,反转为降序
```

lab6.py(第三段:heapq 版与主程序)


```

def topk_heapq(items, k):
    """任务 4:标准库 heapq 重做,逻辑与手写版一字不差。"""
    h = []
    for t in items:
        if len(h) < k:
            heapq.heappush(h, t)
        elif t > h[0]:                # h[0] 就是小顶堆的堆顶
            heapq.heapreplace(h, t)   # 等价于 heappop + heappush
    return sorted(h, reverse=True)

def main():
    # 用法: python lab6.py k < 文本文件
    if len(sys.argv) < 2:
        print("用法: python lab6.py k < 文本文件")
        return
    k = int(sys.argv[1])
    text = sys.stdin.read()
    freq = count_words(text)         # 任务 2:词频字典
    items = [(c, w) for w, c in freq.items()] # (频次, 词)

    # 任务 5:边界处理—k <= 0 或文本为空
    if k <= 0 or not items:
        print(f"k = {k}, 词数 = {len(items)}:边界情形,答案为空。")
        return

    r1 = topk_heap(items, k)
    r2 = topk_heapq(items, k)
    print(f"词数 = {len(items)}, k = {k}")
    print("手写堆 :", [(w, c) for c, w in r1])
    print("heapq :", [(w, c) for c, w in r2])
    print("结果一致:", r1 == r2)

if __name__ == "__main__":
    main()

```

示例输入与输出(真实运行;sample.txt 与 empty.txt 为测试文件):

```

$ python lab6.py 3 < sample.txt
词数 = 34, k = 3
手写堆 : [('to', 6), ('the', 3), ('or', 2)]
heapq : [('to', 6), ('the', 3), ('or', 2)]
结果一致: True

$ python lab6.py 5 < empty.txt
k = 5, 词数 = 0:边界情形,答案为空。

```

设计说明:① 0 基公式写进类注释,与 C++ 版 1 基实现互为对照;② 元素用 (频次, 词) 元组,平局比词序,结果可复现;③ 两版结果一致即互相验证;④ 词频统计用 `re.findall` + `dict`(第5章),与堆协同完成统计 → Top-K 流水线;⑤ 边界显式处理,避免空堆访问。

下一章预告:第7章 排序——堆排序将直接使用本章的二叉堆:下滤建堆 $O(n)$,再连续 deleteMin n 次得到有序序列,总代价 $O(n \log n)$;插入、希尔、归并、快排逐一登场。

第7章 排序

建议学时:4-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 掌握七种排序的算法思想与实现,理解**稳定性**并会给出反例
- 能用求和记号 Σ 推导插入排序的最坏 $O(n^2)$ 与"近有序 $O(n)$ "性质
- 掌握快排的**三数中值分割法**与小小区间插入优化,说清最坏 $O(n^2)$ 何时发生
- 能用决策树直觉解释**比较下界** $\Omega(n \log n)$,理解桶排为何能突破
- 完成从 C 的 `qsort` 到 Python `sorted(key=)` 的迁移,能按数据特征选型

💡 从 C 到 Python:本章主题如何迁移

排序是工程中出现频率最高的操作:排行榜、去重、二分查找前置、Top-K.....本章把第2章的复杂度分析、第6章的二叉堆全部用上。路线:经典排序(\$7.1-\$7.4) → 快排(\$7.5) → 桶排与外部排序(\$7.6) → 总表选型(\$7.7)。请务必亲手写一遍每一种排序——它们最常被要求默写。

在 C 里,通用排序武器只有 `qsort`,三个痛点——① `void*` 丢类型;② 比较语义是"负/零/正"三值;③ 只能整体排序。Python 的 `sorted` 用 `key=` 参数逐一解决:键函数推导类型、`functools.cmp_to_key` 桥接三值比较、任意可迭代对象随意截取;它内部是"归并 + 插入"混合体(Timsort),稳定且自适应,本章的小区间插入、利用近有序等优化全内置。

📦 前置知识自查

数学前置:求和记号 Σ 与 $\Sigma_{i=1}^n i = n(n+1)/2$;递推式 $T(n) = 2T(n/2) + n$ (第2章)。

Python 前置:列表、`range` 与 `for`;切片 `a[i:j]` 与拼接; `functools.cmp_to_key` (\$7.5 对照)。

依赖章节:第2章复杂度;第6章二叉堆(堆排序复用**下滤**与**下滤建堆**,注意 1 基/0 基差异)。

🚀 本章学习路线

1. **第一阶段(1 小时):**读 \$7.1-\$7.2,默写插入与希尔排序
2. **第二阶段(1.5 小时):**读 \$7.3-\$7.4,手写堆排与 merge
3. **第三阶段(1.5 小时):**读 \$7.5-\$7.6,吃透三数中值与分割
4. **第四阶段(实验,1 小时):**完成章末实验。验收:能默写全部经典排序,解释快排何时最坏

本章知识导图

- §7.1 插入排序:扑克牌直觉、 Σ 推导、近有序 $O(n)$ 、稳定性
- §7.2 希尔排序:希尔增量与 Hibbard、约 $O(n^{3/2})$ 、不稳定
- §7.3 堆排序:复用第6章的二叉堆(朴素版、原地版、ADT 关系)
- §7.4 归并排序: $T(n)=2T(n/2)+n$ 、merge、稳定与 $O(n)$ 空间
- §7.5 快速排序:三数中值、分割、小区间插入、qsort $\rightarrow$ sorted(key=) 对照
- §7.6 桶式排序与外部排序:桶排 $O(n)$ 、比较下界 $\Omega(n \log n)$ 、多路归并
- §7.7 排序算法总表:七种排序大表、选型决策树

§7.1 插入排序:算法、复杂度与稳定性

7.1.1 扑克牌直觉与算法步骤

直觉一句话:像整理扑克牌——每摸一张新牌,插到已排好序的牌中的正确位置。算法把数组分成"已排序前缀 $a[0..i-1]$ "与"未排序后缀 $a[i..n-1]$ ":第 i 步用 key 暂存 $a[i]$,从右往左找位置,挡路元素右移一格,落位。

示例 7-1 插入排序(key 暂存与右移)

```
def insertion_sort(a, left=0, right=None):    # 对 a[left..right] 升序排序
    if right is None:
        right = len(a) - 1
    for i in range(left + 1, right + 1):
        key = a[i]
        j = i - 1
        while j >= left and a[j] > key:      # 严格大于才右移:稳定
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
```

7.1.2 复杂度推导:把循环翻译成求和

最坏情形(完全逆序):第 i 个元素要与前面 i 个元素逐一比较并右移,总比较次数 $= \sum_{i=1}^{n-1} i = n(n-1)/2 = O(n^2)$ 。平均

情形:第 i 个元素平均只比较 $i/2$ 次,总次数 $\approx n(n-1)/4$,仍 $O(n^2)$ 。最好情形(已有序):每个元素只比较 1 次, $O(n)$ 。

更本质的刻画是**逆序对(inversion)**:满足 $i < j$ 且 $a[i] > a[j]$ 的数对。插入排序每次右移恰好消除一个逆序对,比较/移动次数 = 逆序对数——逆序对很少时排序就快,"近有序 $O(n)$ "的来源。

7.1.3 近有序数组:接近 $O(n)$ 的性质

若数组"大体有序"(逆序对只有 $O(n)$ 个),插入排序只需 $O(n)$ 次比较与移动,接近线性——章末实验将实测。但也暴露致命局限:每次只交换相邻元素,跨越 k 个位置要走 k 步。例:[2, 3, 4, 5, 6, 7, 8, 1] 中元素 1 走到开头要走 7 步。"让元素一次走远"正是 §7.2 希尔排序的动机。

7.1.4 稳定性:相等元素保持相对次序

稳定性(stable):值相等的元素保持原相对次序。插入排序是稳定的——右移条件写"严格大于" $a[j] > key$,相等立即停止。为何重要?先按姓名排、再按分数排,若第二次稳定,同分者仍按姓名有序——多关键字排序可逐关键字轮流排。

⚠ 误区 1:内层循环里不暂存 key,直接与 a[i] 比较

新手常写 `while j >= left and a[j] > a[i]:` —— `a[i]` 在右移中可能已被覆盖, key 丢失。正确姿势:先 `key = a[i]`, 循环里只移动 `a[j]`。

§7.2 希尔排序:增量序列与远距离搬移

7.2.1 动机:让元素一次走远

插入排序慢在“每次只走一格”。希尔排序:先用大步长“预排序”让元素大体到位,再逐步缩小步长,步长 1 时就是普通插入排序。步长叫**增量(gap)**:把下标相差 h 的元素看作同一组,对每组做插入排序——元素一次跨越 h 个位置。

7.2.2 增量序列:希尔增量与 Hibbard

- **希尔增量**: $n/2, n/4, \dots, 1$ (每次折半)。实现最简单,但增量互为倍数,奇偶下标各自成组,最坏退化为 $O(n^2)$;
- **Hibbard 增量**: $1, 3, 7, 15, \dots, (2^k - 1)$ 。相邻增量互质,避开坏结构,最坏 $O(n^{3/2})$ 。

示例 7-2 希尔排序(希尔增量版)

```
def shell_sort(a):
    n = len(a)
    gap = n // 2
    while gap > 0:
        # 希尔增量: n/2, n/4, ..., 1
        # 对每组做插入排序
        for i in range(gap, n):
            key = a[i]
            j = i
            while j >= gap and a[j - gap] > key:  # 一次跳 gap 格
                a[j] = a[j - gap]
                j -= gap
            a[j] = key
        gap //= 2
```

7.2.3 复杂度与稳定性

希尔排序的精确复杂度至今未完全解决。工程口径(与速查表一致):希尔增量最坏 $O(n^2)$, Hibbard 增量最坏 $O(n^{3/2})$, 平均都约 $O(n^{3/2})$ 。它不稳定:大间隔插入把相等元素“隔空换位”。反例: $[5_a, 5_b, 1]$, $\text{gap} = 2$ 时 $(5_a, 1)$ 排成 $(1, 5_a)$, 数组变 $[1, 5_b, 5_a]$, 最后 $\text{gap} = 1$ 也救不回来。

⚠ 误区 2:增量序列忘了最后是 1,或增量间互为倍数

① 增量必须以 1 收尾,否则只保证“每组内有序”,整体未必有序——`gap` 写成 `gap > 1` 就错。② 希尔增量互为倍数,偶数下标元素永远只在偶数组内比较,是最坏 $O(n^2)$ 的根源;改用 Hibbard $(2^k - 1)$ 互质序列即可避开。

§7.3 堆排序:复用第6章的二叉堆

7.3.1 朴素版:建堆 + 连续删除最小

第6章结尾预告过:堆排序 = 二叉堆的“全量消费”。朴素版两步:① 下滤建堆 $O(n)$;② 连续 `deleteMin` n 次,每次 $O(\log n)$ 。总代价 $O(n \log n)$,但要一块 $O(n)$ 输出数组。

7.3.2 原地版:最大堆 + 交换 + 缩小

原地版把输出数组省掉:改用**最大堆**。建最大堆后根即全局最大;把根与末尾交换,堆大小减 1,再下滤新根——被换到末尾的最大元素就此“钉死”在最终位置。反复 $n-1$ 次,全程只用原数组。

注意下标习惯:原地堆排用 0 基(左子 $2i+1$ 、右子 $2i+2$ 、父 $(i-1)//2$ 、最后非叶 $n//2-1$),与第6章手写堆的 1 基不同。

示例 7-3 原地堆排序(0 基下滤)

```
def sift_down(a, i, n):          # 下滤 a[i]; n 为当前堆大小
    while True:
        largest = i
        for c in (2 * i + 1, 2 * i + 2):    # 左子、右子(0 基)
            if c < n and a[c] > a[largest]:
                largest = c
        if largest == i:
            break
        a[i], a[largest] = a[largest], a[i]
        i = largest

def heap_sort(a):
    n = len(a)
    for i in range(n // 2 - 1, -1, -1):    # 1. 下滤建堆  $O(n)$ 
        sift_down(a, i, n)
    for i in range(n - 1, 0, -1):          # 2. 逐个钉死最大元
        a[0], a[i] = a[i], a[0]
        sift_down(a, 0, i)
```

7.3.3 复杂度、稳定性与 ADT 关系

建堆 $O(n)$ + $n-1$ 次下滤 $\times O(\log n) = O(n \log n)$, 最好最坏都是它,且**原地**只要 $O(1)$ 额外空间。它不稳定:反例 $[4_a, 4_b, 3]$ 建堆后根 4_a 与末尾 3 交换得 $[3, 4_b, 4_a]$, 4_b 最终排在 4_a 前。

从 ADT 视角看:优先队列承诺“每次取出最小元”,堆排序等于把“取最小”执行 n 次,顺便得到完整有序序列。“边插入边取最大值”用优先队列,“一次性排好序”用排序 (§7.7)。

C 的手写堆排序	Python 的标准库
手写下滤、建堆、交换,约 30 行	<code>heapq.heapify</code> + 连续 <code>heappop</code> 两行

⚠ 误区 3:堆排序把 1 基公式抄进 0 基代码

第6章手写堆用 1 基,原地堆排用 0 基。把 1 基的“左子 $2i$ 、父 $i/2$ ”抄进来会访问错位置,堆序被悄悄破坏、输出“看起来差不多但不对”。对策:下标公式写进注释,写完用“验证堆序”断言跑一遍。

§7.4 归并排序:分治的教科书

7.4.1 分治三步与递推式

归并排序是分治思想的教科书案例:① 分割:数组从中间切成两半;② 递归:各自排序;③ 合并:两个有序半区线性合并成一个有序整区。递推式即第2章的 $T(n) = 2T(n/2) + n$,解得 $O(n \log n)$ 。

7.4.2 merge:两个有序序列的线性合并

合并直觉: 像两队人按身高排队,每次让队首较矮者出列。i 扫左半、j 扫右半;某半耗尽,另一半剩余整体搬入。

示例 7-4 归并排序(切片版:递归新建子列表)

```
def merge(left, right):
    i = j = 0
    res = []
    while i < len(left) and j < len(right):
        if right[j] < left[i]:           # 右半更小才取右
            res.append(right[j])        # 相等取左 → 稳定
            j += 1
        else:
            res.append(left[i])
            i += 1
    return res + left[i:] + right[j:]   # 剩余半区整体搬入

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    left = merge_sort(a[:m])            # 切片:创建子列表副本
    right = merge_sort(a[m:])
    return merge(left, right)
```

实现建议:切片即复制,注意空间常数

Python 的切片 `a[:m]` 每次递归都新建一份子列表,总分配 $O(n \log n)$ 个元素,常数偏大;要省空间可用"下标区间 + 辅助列表"重写,但代码长一倍。生产代码直接 `sorted(a)`:Timsort 按段归并, $O(n)$ 空间。手写归并是为了理解分治,不是替代标准库。

7.4.3 稳定性与空间

归并排序**稳定**:合并"相等取左半"(严格小于才取右)。代价是 $O(n)$ 额外空间。工程价值:Python 的 `sorted` 用的 Timsort 就是"归并 + 插入"混合,检测到已有序段直接拼接,近有序数据近乎 $O(n)$ 。递归深度仅 $\log_2 n$,Python 默认递归上限 1000 也够用。

误区 4:merge 的边界错误(漏掉剩余半区或忘 return)

两个高频错:① 主循环后忘了把某半剩余元素拼进结果(奇数长度必错);② `merge` 忘了 `return` 返回 `None`——注意归并产生"新列表"必须传回来,不像插入排序那样原地改。调试技巧:用 5 个元素手工走一遍。

§7.5 快速排序:工程之王

7.5.1 思想与最坏情形

快排也是分治,但与归并的"先递归后合并"相反:难点在"分割"——选一个**枢轴(pivot)**,把数组分成"小于枢轴"与"大于枢轴"两半,枢轴落到最终位置,再递归排序两半。分割后无需合并——这是它常数小的根源。平均枢轴居中,递推式仍是

$T(n) = 2T(n/2) + n, O(n \log n)$; 最坏 $O(n^2)$: 每次分割极度不平衡——典型场景“已有序 + 首元素枢轴”, 退化为 $T(n) = T(n-1) + n$ 。

7.5.2 三数中值分割法

对付最坏情形的标准武器是**三数中值分割法(median-of-three)**: 取左端、中间、右端三个元素, 中值当枢轴, 顺便把三者排好序——左、右端变成“哨兵”, 枢轴藏中间。已有序数组会选中真正的中间值, 分割永远均衡, 退化被根治。

示例 7-5 快速排序: 三数中值 + 分割 + 小区间插入

CUTOFF = 16 # 长度 ≤ 16 改用插入排序

```
def median3(a, left, right):
    center = (left + right) // 2
    if a[center] < a[left]:
        a[left], a[center] = a[center], a[left]
    if a[right] < a[left]:
        a[left], a[right] = a[right], a[left]
    if a[right] < a[center]:
        a[center], a[right] = a[right], a[center]
    a[center], a[right - 1] = a[right - 1], a[center] # 枢轴藏到 right-1
    return a[right - 1]

def quick_sort(a, left=0, right=None):
    if right is None:
        right = len(a) - 1
    if left + CUTOFF <= right:
        pivot = median3(a, left, right)
        i, j = left, right - 1
        while True:
            i += 1
            while a[i] < pivot: # 找 ≥ 枢轴者(a[left] 哨兵)
                i += 1
            j -= 1
            while pivot < a[j]: # 找 ≤ 枢轴者(a[right-1] 哨兵)
                j -= 1
            if i < j:
                a[i], a[j] = a[j], a[i]
            else:
                break
        a[i], a[right - 1] = a[right - 1], a[i] # 枢轴归位
        quick_sort(a, left, i - 1)
        quick_sort(a, i + 1, right)
    else:
        insertion_sort(a, left, right) # 小区间插入($7.1)
```

为什么循环不会越界? 三数中值已保证 $a[\text{left}] \leq \text{枢轴}$ 且 $a[\text{right}] \geq \text{枢轴}$, 两个“哨兵”让 $i += 1$ 与 $j -= 1$ 必然停下——这正是“顺便排好序”的深意。

7.5.3 小区间插入排序优化

快排递归到小区间时,插入排序反而更快:递归有开销、插入常数极小、缓存友好。工程实现普遍在长度 ≤ 16 时切换: `CUTOFF = 16 = "快排大区 + 插入小区"`。Timsort 与 `std::sort` 也这么做。

7.5.4 ★从 C 的 qsort 到 Python 的 sorted

C 的 qsort(函数指针 + void*)	Python 的 sorted(key=)(Timsort)
<code>qsort(a, n, sizeof(int), cmp):</code> 返回负/零/正	<code>sorted(a):</code> 默认升序,一行,默认稳定
<code>int cmp(const void*, const void*)</code> 手动强转,无类型检查	<code>key=lambda x: x</code> 提取排序键,元素类型任意
自定义顺序只能写死返回值规则	<code>key=lambda x: -x</code> 降序; <code>cmp_to_key</code> 桥接三值比较

示例 7-6 sorted 的四种用法

```
from functools import cmp_to_key
v = [5, 2, 8, 1, 9]
sorted(v) # 升序(返回新列表)
sorted(v, reverse=True) # 降序
sorted(v, key=lambda x: x % 10) # 按个位数升序
names = ["bob", "alice", "carl"]
sorted(names, key=len) # 按长度升序
sorted(names, key=cmp_to_key(lambda x, y: x - y)) # 三值比较桥接,语义同 qsort
```

⚠ 误区 5:把 C 的三值比较函数直接当 key 用

`qsort` 的比较函数收两个参数、返回"负/零/正"; `sorted` 的 `key=` 只收一个参数、返回排序键。把 C 的 `cmp` 直接塞进 `key=` 立刻 `TypeError`。正确姿势: `sorted(a, key=cmp_to_key(cmp))`。这是 C 迁移时最高频的报错。

7.5.5 快排的稳定性

快排**不稳定**:分割的"交换"会把相等元素隔空换位。反例:[3, 5_a, 2, 5_b, 1],取 3 当枢轴:分割时 5_a 与 1 交换,越过 5_b,次序破坏。需要稳定:小数据用插入,大数据直接用 `sorted`——它本来就是稳定的。

§7.6 桶式排序与外部排序:跳出"比较"的框架

7.6.1 桶式排序:用空间换时间

前面五种排序都属于"比较排序"。桶式排序(bucket sort)完全不做比较:把值直接当下标,数每个值出现几次,再按顺序倒出来——即计数排序(counting sort)(每桶一个值)。复杂度 $O(n + M)$, M 是值域; $M = O(n)$ 时即 $O(n)$ 。

示例 7-7 计数排序(前缀和 + 倒序,稳定)

```
def counting_sort(a, M):
    cnt = [0] * M
    out = [0] * len(a)
    for x in a:
        cnt[x] += 1
    for i in range(1, M):
        cnt[i] += cnt[i - 1]
    for x in reversed(a):
        cnt[x] -= 1
        out[cnt[x]] = x
    return out
```

值域 $[0, M)$ 的非负整数
cnt 兼作位置账本
1. 计数
2. 前缀和
3. 倒序扫描
稳定: 相等后到先取

C 的计数排序

```
int *cnt = calloc(M,
sizeof(int)); 用完 free, 忘了即泄漏
```

Python 的计数排序

```
cnt = [0] * M 自动管理内存
```

7.6.2 比较下界: $\Omega(n \log n)$ 的决策树直觉

为什么桶排能是 $O(n)$? 因为任何只靠"比较"的排序都有下界 $\Omega(n \log n)$, 而桶排不比较。一句话直觉: 一次比较最多把 $n!$ 种排列一分为二; 要确定唯一正确排列, 至少需要 $\log_2(n!) \approx n \log_2 n$ 次比较。 $O(n)$ 排序必须另辟蹊径——利用值域结构, 而非元素间比较。

桶排的适用条件因此苛刻: ① 数据是(或可映射成)小范围整数; ② 值域与数据量同阶; ③ 分布大致均匀。工程上主要用于成绩统计、颜色直方图、基数排序底层($O(d(n+M))$)。

7.6.3 外部排序: 内存装不下时

外部排序(external sort) 解决"数据比内存大"的问题(数据库、日志)。思路是归并的外延: 数据切成若干块, **每块内存排序后写回磁盘**; 再开小顶堆, **放每个已排块的当前最小元素, 每次弹出全局最小者写入输出, 再从该块补进下一个**——弹出、补入、再弹出直至读完; 输出仍装不下就多趟合并。分块排序用 §7.4 的归并, "多路取最小"正是第6章的小顶堆——**外部排序 = 归并思想 + 堆结构 + 磁盘代替内存**。

⚠ 误区 6: 对不满足条件的数据用桶排

对 10^6 个浮点数或值域 10^9 的整数用计数排序, 内存直接爆炸(要 M 个计数器)。判断口诀: 值域多大? 是整数吗? 分布均匀吗? 任何一条不满足, 老老实实用 $O(n \log n)$ 通用排序。工程上 99% 的场景都是通用排序, 桶排是值域受限场景的特供品。

§7.7 排序算法总表:选型决策

7.7.1 七种排序全景表

算法	平均	最坏	空间	稳定性
插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
希尔排序	约 $O(n^{3/2})$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定
快速排序	$O(n \log n)$	$O(n^2)$	$O(\log n)$	不稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
桶排(计数)	$O(n + M)$	$O(n + M)$	$O(M)$	稳定

读表要点: 只有插入、冒泡能利用已有序(最好 $O(n)$); 只有归并和桶排稳定; 只有堆排最坏 $O(n \log n)$ 且 $O(1)$ 空间; 快排平均最快但最坏 $O(n^2)$ 。没有一种全占——选型即权衡。

7.7.2 选型决策树

- 数据量很小或基本有序 → **插入排序**
- 需要稳定、空间允许 → **归并排序**(Python 的 `sorted` 即稳定)
- 内存紧张、最坏必须保证 → **堆排序**
- 通用场景、要快 → **快速排序**
- 值域小的整数 → **桶排/计数排序**
- 其余一切情况 → **标准库 `sorted/sort`**, 不要手写

7.7.3 标准库是工程默认

最后一条最重要: 工程上永远优先 `sorted(a) / a.sort()` ——它是 Timsort: 归并为主, 检测到已有序段直接拼接(近有序近乎 $O(n)$), 小区间自动切插入排序, 而且默认稳定。本章手写的优化标准库全部内置, 还多一道稳定保证; C++ 的 `std::sort` 是 introsort(快排为主 + 防退化堆排), 语义相同。手写排序的意义: 理解原理、应对考试; 生产代码直接用标准库——本章实验"手写三种排序并统计比较次数", 正是为亲手验证第2章的复杂度理论。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
插入排序	随机/逆序数组	$O(n^2)$	$O(n^2)$
插入排序	已有序数组	$O(n)$	$O(n)$
冒泡排序	相邻交换	$O(n^2)$	$O(n^2)$
希尔排序	希尔增量	约 $O(n^{3/2})$	$O(n^2)$
希尔排序	Hibbard 增量	约 $O(n^{3/2})$	$O(n^{3/2})$
堆排序	建堆阶段	$O(n)$	$O(n)$
堆排序	完整排序	$O(n \log n)$	$O(n \log n)$
归并排序	完整排序	$O(n \log n)$	$O(n \log n)$
归并排序	额外空间	$O(n)$	$O(n)$
快速排序	随机数据	$O(n \log n)$	$O(n \log n)$
快速排序	已有序 + 首元素枢轴	$O(n^2)$	$O(n^2)$
桶排(计数排序)	值域 M	$O(n + M)$	$O(n + M)$
比较下界	基于比较的排序	$\Omega(n \log n)$ (决策树)	
归并两个有序数组	线性合并	$O(n)$	$O(n)$
外部排序	多路归并	磁盘 I/O 主导;比较部分 $O(n \log n)$	

最小必会清单

- 能默写插入排序(key 暂存三步),用 Σ 推导最坏 $O(n^2)$ 、近有序 $O(n)$
- 能说出希尔排序动机、希尔增量与 Hibbard 差别、复杂度约 $O(n^{3/2})$
- 能默写原地堆排序(0 基下滤、建堆、交换缩小),说出它不稳定并给反例
- 能默写归并的 merge,说清"相等取左"保证稳定、切片复制的空间代价
- 能默写快排的三数中值 + 分割 + 小区间插入,解释哨兵防越界
- 能说出快排最坏 $O(n^2)$ 何时发生、三数中值为何能治
- 能说出比较下界 $\Omega(n \log n)$ 的决策树直觉与桶排为何突破
- 能说出桶排三个适用条件,写出稳定版计数排序
- 能背出七种排序复杂度总表(最好/平均/最坏/空间/稳定性)
- 能写出 `sorted` 的 `key=` 与 `cmp_to_key`,说出与 `qsort` 三值语义的区别

自测题

- Q** 1. 插入排序对已有序数组的复杂度是多少?为什么?

✅ 参考答案

$O(n)$ 。每个元素与前驱比较 1 次就停止,共 $n-1$ 次比较、0 次移动。这也是"近有序接近 $O(n)$ "的来源:总代价 = 逆序对数。

Q 2. 堆排序为什么不稳定?构造一个反例。

✅ 参考答案

建堆与"根与末尾交换"都可能让相等元素换位。反例:[$4_a, 4_b, 3$] 建最大堆后根 4_a 与末尾 3 交换得 [$3, 4_b, 4_a$], 4_b 最终在前。

Q 3. 归并排序的空间复杂度是多少?为什么它不能像堆排那样原地?

✅ 参考答案

$O(n)$ 辅助数组。合并时两半在原数组中交错,原地交换会破坏"两半各自有序"的前提;原地归并要么 $O(n^2)$ 时间,要么复杂块交换技巧,得不偿失。

Q 4. 快排的最坏情形是什么?三数中值分割法为什么能缓解?

✅ 参考答案

每次分割都极度不平衡时(递推式 $T(n) = T(n-1) + n \rightarrow O(n^2)$),典型场景是已有序数组配"首元素枢轴"。三数中值取左、中、右的中值:已有序数组的中值恰是中间值,分割均衡;顺带排好 $a[\text{left}] \leq \text{枢轴} \leq a[\text{right}]$ 作哨兵。

Q 5. 为什么桶排能做到 $O(n)$,却不受"比较下界 $\Omega(n \log n)$ "的限制?

✅ 参考答案

决策树论证只适用于"通过比较获得信息"的算法:每次比较最多把 $n!$ 种排列一分为二,故至少 $\log_2(n!) \approx n \log_2 n$ 次。桶排不比较,直接用值域做索引(计数),绕开决策树,所以可 $O(n)$;代价是空间 $O(M)$ 与苛刻的适用条件。

章末实验题:三种排序的性能对比实验

实验 7:三种排序性能对比(三种数据分布)

实验目标:实现插入、归并、快排,在随机/近有序/逆序三种分布上实测 $n = 1000 \dots 64000$ 的耗时,用" n 翻倍时间比值"验证理论复杂度。

实验要求:

- 任务 1:实现 `insertion_sort`、`merge_sort`、`quick_sort`(\$7.1/\$7.4/\$7.5)
- 任务 2:三种分布 $\times$ 七个规模 n 的耗时表;插入 $n > 8000$ 标注"(推算)"
- 任务 3:与 `sorted` 对比验证输出一致,含重复元素
- 任务 4:" n 翻倍时间比值"与理论对比:插入 $\times 4(n^2)$ 、归并/快排 $\approx \times 2(n \log n)$
- 任务 5(加分):统计比较/交换次数,验证插入比较次数 $\approx n^2/4$ (逆序对)

实验环境:Python 3.8+。运行: `python lab7.py`。

第一步 排务必用三数中值 + CUTOFF=16 小区间插入(\$7.5 完整版),否则逆序数据退化 $O(n^2)$,与理论演示矛盾

第二步 时取"同数据 3 次最小值"、用副本,固定种子 12345 可复现;近有序用"已排序 + 固定 20 次随机交换"生成(逆序对 $O(n)$),才能观察插入的 $O(n)$ 特性

第三步 务 4 比值只看"翻倍相邻行",小 n 计时噪声大,偏离属正常,看趋势即可

✅ 参考答案(完整可运行程序,分六段)

```
# lab7.py — 三种排序的性能对比实验(插入/归并/快排)
# 运行: python lab7.py
import random
import time

random.seed(12345)      # 固定种子,输出可复现

CUTOFF = 16             # 快排小区间改用插入排序的阈值($7.5)

# ----- 任务 5(可选加分):比较/移动计数器 -----
COUNT = False          # 置 True 时统计,性能测试期间保持 False
cmp_cnt = 0
move_cnt = 0

def lt(x, y):
    global cmp_cnt
    if COUNT:
        cmp_cnt += 1
    return x < y

# ----- 任务 1:插入排序($7.1) -----
def insertion_sort(a, left=0, right=None):
    if right is None:
        right = len(a) - 1
    for i in range(left + 1, right + 1):
        key = a[i]
        j = i - 1
        while j >= left and lt(key, a[j]): # 严格大于才右移:相等不动,稳定
            a[j + 1] = a[j]               # 元素右移
            j -= 1
        global move_cnt
        if COUNT:
            move_cnt += 1
        a[j + 1] = key                    # 落位
        if COUNT:
            move_cnt += 1
```

lab7.py(第二段:归并排序)

```

# ----- 任务 1:归并排序($7.4,切片 + 合并) -----
def merge(left, right):
    global move_cnt
    i = j = 0
    res = []
    while i < len(left) and j < len(right):
        if lt(right[j], left[i]):          # 右半更小才取右:相等取左,稳定
            res.append(right[j])
            j += 1
        else:
            res.append(left[i])
            i += 1
    if COUNT:
        move_cnt += 1
    res += left[i:] + right[j:]          # 剩余部分直接拼接
    return res

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    left = merge_sort(a[:m])
    right = merge_sort(a[m:])
    return merge(left, right)          # 切片:创建子列表副本($7.4)

```

lab7.py(第三段:快速排序与数据生成)


```

# ----- 任务 1:快速排序($7.5,三数中值 + 小区间插入) -----
def median3(a, left, right):
    center = (left + right) // 2
    if lt(a[center], a[left]):
        a[left], a[center] = a[center], a[left]
    if lt(a[right], a[left]):
        a[left], a[right] = a[right], a[left]
    if lt(a[right], a[center]):
        a[center], a[right] = a[right], a[center]
    a[center], a[right - 1] = a[right - 1], a[center] # 枢轴藏到 right-1
    return a[right - 1]

def quick_sort(a, left=0, right=None):
    if right is None:
        right = len(a) - 1
    if left + CUTOFF <= right:
        pivot = median3(a, left, right)
        i, j = left, right - 1
        while True:
            i += 1
            while lt(a[i], pivot): # 哨兵 a[left]<=pivot 保证能停
                i += 1
            j -= 1
            while lt(pivot, a[j]): # a[right-1]==pivot 保证能停
                j -= 1
            if i < j:
                a[i], a[j] = a[j], a[i]
            else:
                break
        a[i], a[right - 1] = a[right - 1], a[i] # 枢轴归位
        quick_sort(a, left, i - 1)
        quick_sort(a, i + 1, right)
    else:
        insertion_sort(a, left, right) # 小区间优化($7.1 预告)

# ----- 任务 2:三种数据分布 -----
def make_data(n, kind):
    if kind == "随机":
        a = list(range(n))
        random.shuffle(a)
    elif kind == "近有序":
        a = list(range(n))
        for _ in range(20): # 固定 20 次随机交换:逆序对 O(n)
            x, y = random.randrange(n), random.randrange(n)
            a[x], a[y] = a[y], a[x]
    else: # 逆序
        a = list(range(n, 0, -1))
    return a

```

lab7.py(第四段:计时)

```
def ms_time(func, a, copies=True):
    """计时:同数据重复 3 次取最小值,压制系统噪声;copies=True 时每次用副本。"""
    best = float("inf")
    for _ in range(3):
        b = a[:] if copies else a
        t0 = time.perf_counter()
        func(b)
        best = min(best, (time.perf_counter() - t0) * 1000)
    return best
```

lab7.py(第五段:正确性验证与耗时表)

```
def main():
    sizes = (1000, 2000, 4000, 8000, 16000, 32000, 64000)
    kinds = ("随机", "近有序", "逆序")
    times = {k: {"插入": {}, "归并": {}, "快排": {}} for k in kinds}

    # 任务 3:正确性验证(与 sorted 输出一致,含重复元素)
    ok = True
    for kind in kinds:
        a = make_data(500, kind)
        std = sorted(a)
        b = a[:]; insertion_sort(b); ok &= (b == std)
        ok &= (merge_sort(a[:]) == std)
        b = a[:]; quick_sort(b); ok &= (b == std)
    a = [random.randint(0, 20) for _ in range(500)] # 重复元素
    std = sorted(a)
    b = a[:]; insertion_sort(b); ok &= (b == std)
    ok &= (merge_sort(a[:]) == std)
    b = a[:]; quick_sort(b); ok &= (b == std)
    print(f"任务 3:三种算法与 sorted 输出一致: {ok}")

    # 任务 2:性能表(插入 n>8000 按 x4 规律推算,避免无谓等待)
    print("任务 2:耗时表(单位 ms;插入排序 n>8000 为(推算))")
    for kind in kinds:
        print(f"\n[{kind}]")
        print("n      插入      归并      快排")
        for n in sizes:
            a = make_data(n, kind)
            row = [f"{n}"]
            if n <= 8000:
                t = ms_time(insertion_sort, a[:])
                times[kind]["插入"][n] = t
                row.append(f"{t:9.1f}")
            else:
                row.append(" (推算)")
            t = ms_time(merge_sort, a[:], copies=False) # 归并不改动原列表
            times[kind]["归并"][n] = t
            row.append(f"{t:9.1f}")
            t = ms_time(quick_sort, a[:])
            times[kind]["快排"][n] = t
            row.append(f"{t:9.1f}")
        print(" ".join(row))
```

lab7.py(第六段:比值分析、统计与结论)

```

# 任务 4:比值分析(理论:插入  $\times 4$ ;归并/快排  $\approx \times 2$ )
print("\n任务 4:n 翻倍时实测时间比值(理论: $n^2$  算法  $\times 4$ , $n \log n$  算法  $\approx \times 2$ )")
theory = {"插入": " $n^2 \rightarrow \times 4$ ", "归并": " $n \log n \rightarrow \approx \times 2$ ", "快排": " $n \log n \rightarrow \approx \times 2$ "}
for alg in ("插入", "归并", "快排"):
    line = f"{alg}({theory[alg]}): "
    for kind in kinds:
        ts = sorted(times[kind][alg].items())
        rs = [f"{ts[i+1][1]/ts[i][1]:.1f}" for i in range(len(ts) - 1)]
        line += f"{kind}[' '.join(rs)] "
    print(line)

# 任务 5(加分):比较/移动次数统计
global COUNT, cmp_cnt, move_cnt
COUNT = True
a = make_data(1000, "随机")
cmp_cnt = move_cnt = 0
insertion_sort(a[:])
print(f"\n任务 5(n=1000 随机):插入 比较={cmp_cnt} 移动={move_cnt}")
cmp_cnt = move_cnt = 0
merge_sort(a[:])
print(f"          归并 比较={cmp_cnt} 移动={move_cnt}(拼接不计)")
cmp_cnt = move_cnt = 0
quick_sort(a[:])
print(f"          快排 比较={cmp_cnt} 交换={move_cnt}")
COUNT = False

print("\n结论:实测比值与理论吻合—插入排序 n 翻倍时间约  $\times 4$ ,"
      "归并/快排约  $\times 2$ ;逆序数据上插入排序退化最明显,"
      "近有序数据上插入排序接近  $O(n)$ ,快排三数中值使逆序数据不再退化。")

if __name__ == "__main__":
    main()

```

示例输出(本机真实运行,不同机器数值不同,比值规律一致):

任务 3:三种算法与 sorted 输出一致: True
任务 2:耗时表(单位 ms;插入排序 $n > 8000$ 为(推算))

[随机]

n	插入	归并	快排
1000	13.8	1.0	0.6
2000	61.4	2.2	1.3
4000	257.4	4.8	2.8
8000	1019.2	10.8	6.3
16000	(推算)	22.8	12.8
32000	(推算)	50.4	28.5
64000	(推算)	117.8	66.5

[近有序] 表与 [逆序] 表见完整输出:插入在近有序上快两个数量级(8000 时仅 5.4ms),逆序上最慢(2025.0ms)

任务 4:n 翻倍时实测时间比值(理论: n^2 算法 $\times 4$, $n \log n$ 算法 $\approx \times 2$)
插入($n^2 \rightarrow \times 4$): 随机[4.5 4.2 4.0] 近有序[2.3 1.8 1.5] 逆序[4.2 4.2 4.0]
归并($n \log n \rightarrow \approx \times 2$): 随机[2.1 2.2 2.3 2.1 2.2 2.3](近有序/逆序各列也均 ≈ 2)
快排($n \log n \rightarrow \approx \times 2$): 随机[2.2 2.2 2.2 2.0 2.2 2.3](同上)

任务 5($n=1000$ 随机):插入 比较=246311 移动=246323
归并 比较=8720 移动=8720(拼接不计)
快排 比较=10882 交换=3088

读表要点:① 插入在随机/逆序上比值 $\approx \times 4$ (吻合 n^2),近有序上 $\approx \times 2$ (吻合 $O(n)$);② 归并/快排 $\approx \times 2$ (吻合 $n \log n$, 三种分布皆然);③ 快排逆序不退化——三数中值选中中间值,§7.5 优化的实战验证;④ 插入比较 246311 $\approx 1000^2/4$,落在平均情形的理论上。

设计说明:① `lt` 计数钩子使比较/移动定义与正文一致;② 计时取 3 次最小值、用副本(归并本来就返回新列表,直接传原件);③ 插入 $n > 8000$ 标"(推算)": $\times 4$ 规律已由 1000→8000 确认,与第2章实验一脉相承;④ 近有序用固定 20 次交换生成(逆序对 $O(n)$);⑤ 固定种子 12345,输出可复现。

下一章预告:第8章 不相交集——union/find 与路径压缩,一种"只回答是/否连通"的轻量数据结构;第9章图论算法中的 Kruskal 最小生成树将直接使用它。

第8章 不相交集

建议学时:3-4 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解**等价关系**三性质与"动态"等价问题为何要求在线回答
- 掌握树形表示:每集合一棵树,根是**代表元**,一个 `parent` 列表即可实现
- 完成 C 数组到 Python 列表的迁移(`list(range(n))` 一行初始化),写出朴素 `find / union`,踩一遍"链退化"
- 掌握**按大小合并**、**按秩合并**(树高 $O(\log n)$)与**路径压缩**(均摊近 $O(1)$),并避开 Python 递归深度的坑
- 学完能用并查集生成随机迷宫、统计连通分量,为第9章 Kruskal 铺路

💡 从 C 到 Python:本章主题如何迁移

本章是全书**代码量最小**、**思想密度最高**的一章:完整实现约 30 行,却支撑着网络连通、迷宫生成、第9章 Kruskal 算法等大应用。先给结论:不相交集(并查集)只做一件事——维护一堆集合,支持"查归属"与"合并"两种操作。

在 C 里你天然会想到笨办法:开 `mark[N]` 数组记录每个元素属于第几组,合并时遍历整个数组把所有第 `b` 组的元素改成第 `a` 组。能工作,但每次合并 $O(n)$,合并 n 次就是 $O(n^2)$ 。本章的树形表示把合并降为"改一个指针"——"用树的结构信息换时间"的经典一步。

Python 的迁移点:① C 的 `int parent[N]` 变成 `parent = list(range(n))`,初始化天然一行;② C 的裸数组函数传参变成类方法, `parent` 作为属性随对象走;③ 唯一要警惕的是**递归深度**:C 的递归 `find` 在 Python 里会撞上 `RecursionError` (默认限制约 1000 层),所以本章推荐迭代版 `find`。Python 标准库同样没有并查集,**手写就是全部**——每一行都要理解到位。

本章路线:先定义问题(\$8.1),写朴素实现踩一遍退化的坑(\$8.2),依次加两种优化(\$8.3-\$8.4),看清复杂度如何摊还到近 $O(1)$ (\$8.5),最后在迷宫与连通分量中验证(\$8.6-\$8.7)。

📦 前置知识自查

数学前置:等价关系三性质(自反、对称、传递);对数(第2章)——按大小合并的高度证明用 $O(\log n)$ 。

C 语言前置:`int` 数组、`for` 循环初始化、递归函数(\$8.4 对照 C 的递归 `find`)。

依赖的前面章节:第3章 Python 列表 `list`(存放 `parent` 数组)、第4章树的父指针表示(`parent` 数组正是"树"的另一种存法)。

🚀 本章学习路线

1. **第一阶段(1 小时)**:读 \$8.1-\$8.2,跑通朴素实现,手工模拟"链退化"
2. **第二阶段(1.5 小时)**:读 \$8.3-\$8.4,实现按秩合并与迭代版路径压缩
3. **第三阶段(1 小时)**:读 \$8.5-\$8.7,理解摊还直觉,用并查集生成迷宫
4. **第四阶段(实验,1 小时)**:完成章末实验:迷宫生成 + 连通分量统计 + 性能对比
5. **验收标准**:能默写 20 行内的完整并查集,并说出三种配置的复杂度

本章知识导图

- §8.1 等价关系与动态等价问题:三性质、find/union、为何"动态"
- §8.2 基本实现:树表示与朴素 union/find(根=代表元、链退化)
- §8.3 按大小合并与按秩合并:size/rank 数组、树高 $O(\log n)$ 的证明
- §8.4 路径压缩:递归与迭代两写法、压扁树的机制、叠加效果
- §8.5 复杂度:摊还近 $O(1)$ 的直觉、反阿克曼函数 $\alpha(n)$
- §8.6 应用:迷宫生成、连通分量统计、Kruskal 预告、替代方案对比
- §8.7 实现要点与调试:初始化、先 find 到根、秩不失效、打印 parent

§8.1 等价关系与动态等价问题

8.1.1 等价关系:三性质

等价关系(equivalence relation)是定义在集合上的二元关系,满足三条性质:① **自反**: $a \equiv a$;② **对称**: $a \equiv b$ 则 $b \equiv a$;③ **传递**: $a \equiv b$ 且 $b \equiv c$, 则 $a \equiv c$ 。三条缺一不可。等价关系把集合划分成若干**等价类**:互相等价的元素属于同一类——这些类互不相交,正是"不相交"一词的来历。

例子:整数"相等"、整数"模 m 同余"、电路里"两点由导线连通",都是等价关系。再看**反例**:中学的"同班同学"——自反、对称都满足,但**不传递**:甲、乙同班,乙、丙同班,甲、丙未必同班。它不是等价关系,不能用并查集建模"全班连通"。

8.1.2 动态等价问题:union 与 find

给定 N 个元素,初始时每个元素自成一个集合。支持两种操作:

- **find(x)**:返回 x 所在集合的名字——用该集合的**代表元**(representative)当名字;
- **union(a, b)**:把 a 、 b 所在的两个集合合并成一个(若本就在同一集合,什么都不做)。

注意"动态"二字的分量:若所有关系一次性给定再问"1 号和 5 号通不通",那是**静态**问题,遍历一遍算出全部等价类即可(第9章的 DFS 能做)。实际场景中 **查询与合并交错到来**:接入一根新网线(合并),立刻有人问两台机器通不通(find),接着又接新线..... 每一步都必须在当下即时回答,不能等收集完所有操作再算。这正是"动态等价问题"的在线(on-line)性质,也是必须专门设计数据结构的理由。

8.1.3 应用预告

后面你会反复见到它:① **迷宫生成**:随机拆墙但保证任意两格连通(§8.6 详解,章末实验);② **Kruskal 最小生成树**(第9章):按边权从小到大加边,用 **find** 判断两端是否已连通——并查集是它的灵魂;③ **工程应用**:网络断点检测、图像连通域标记。记住一句话:凡是"合并集合 + 查询归属"成对出现的问题,先想并查集。

§8.2 基本实现:树表示与朴素 union/find

8.2.1 树表示:根就是代表元

怎么表示"一组元素属于同一集合"?答案是**树**:每个集合一棵树,树的**根是代表元**——**find(x)** 沿父链上溯到根并返回根,集合的名字即根;合并时让一棵树的根指向另一棵树的根,只改一个指针。这是第4章"父指针表示法"的直接应用:**parent[i]** 是结点 i 的父结点,根满足 **parent[root] == root** ("我是根"的哨兵约定)。

8.2.2 从 C 的数组到 Python 的列表

C 里写 `int parent[N]`; Python 换成 `list, list(range(n))` 一行完成创建与初始化:

C 的写法	Python 的写法
<code>int parent[N];</code> 长度写死	<code>parent = [0] * n</code> 或 <code>list(range(n))</code>
<code>for (i=0; i<N; ++i) parent[i] = i;</code>	<code>parent = list(range(n))</code> 一行初始化
越界访问 = 未定义行为	越界抛 <code>IndexError</code> , 立即暴露 bug
函数传参要带长度 N	<code>len(parent)</code> 自带长度

示例 8-1 朴素实现: find 沿父链上溯, union 让根指向根

```
# parent[i] == i 表示 i 是根; 其余 parent[i] 是 i 的父结点
parent = list(range(8))      # 一行初始化: 每个元素都是自己的根

def find(x):                 # 朴素 find: 沿父链上溯到根
    while parent[x] != x:
        x = parent[x]
    return x

def union_sets(a, b):        # 朴素 union: 让 a 的根指向 b 的根
    ra, rb = find(a), find(b)
    if ra != rb:
        parent[ra] = rb

union_sets(1, 2)
union_sets(3, 4)
union_sets(2, 3)             # 现在 {1,2,3,4} 在同一集合
# find(1) -> find(2) -> find(3) -> find(4): 一路到根 4
print(find(4))
```

★ 重点: 三个必须想清楚的点

① `union` 前必须先 `find`: 合并的是“两个集合”不是“两个元素”, 拿根来挂; 只有根能当代表元, 挂普通结点会破坏树。② `parent` 里存的是“父结点”不是“集合编号”: 集合没有全局编号, 根的名字就是集合的名字, `find` 必须走到根才能回答归属。③ 初始化绝不能省: `parent = list(range(n))` 让每个元素自成一棵单结点树, 这是推理的地基。

8.2.3 朴素实现的致命伤: 链退化

⚠ 误区 1: 朴素 union 会把树连成一条链

依次执行 `union(0,1)`, `union(1,2)`, ..., `union(n-2, n-1)`: 第一次让 0 指向 1, 第二次让 1 指向 2.....`parent` 变成 `0->1, 1->2, ..., n-2->n-1`, 树被拉成一条 n 结点的链。`find(0)` 要走 $O(n)$ 步, 做 n 次 `find` 就是 $O(n^2)$ 。链 = 树高的灾难: 树形表示的全部好处都建立在“树要矮”上, §8.3 的两种合并策略正是从“让树长不高”入手。

§8.3 按大小合并与按秩合并

8.3.1 按大小合并:小树并入大树

按大小合并(union by size)的规则一句话:总是把结点数少的树挂到结点数多的树下。为此维护 `size` 列表: `size[root]` 是根所在树的结点数(只在根上有效),合并时小根指向大根并累加。

示例 8-2 按大小合并: `size` 只在根上有效

```
# sz[root] = 根所在树的结点数(只在根上有效)
parent = list(range(8))
sz = [1] * 8          # 初始每棵树 1 个结点

def find(x):           # 朴素 find 不变
    while parent[x] != x:
        x = parent[x]
    return x

def union_by_size(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return         # 已在同一集合
    if sz[ra] < sz[rb]:  # 小树并入大树
        ra, rb = rb, ra
    parent[rb] = ra     # rb 不再是根,它的 sz 作废
    sz[ra] += sz[rb]
```

8.3.2 按秩合并:浅树并入深树

按秩合并(union by rank)是"按大小"的孪生兄弟:目标都是**别让树长高**。它维护 `rank` 列表: `rank[root]` 是树高的上界(初始 0)。合并时秩小的树挂到秩大的树下;秩相同则任选一树做根并把秩加一。实现更短,且"上界"语义给 §8.4 路径压缩更大自由度。

示例 8-3 按秩合并:浅树挂到深树

```
# rank[root] = 树高的上界(只在根上有效),初始 0
parent = list(range(8))
rank = [0] * 8

def find(x):
    while parent[x] != x:
        x = parent[x]
    return x

def union_by_rank(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return
    if rank[ra] < rank[rb]:  # 浅树挂到深树
        ra, rb = rb, ra
    parent[rb] = ra
    if rank[ra] == rank[rb]: # 两棵同秩树合并
        rank[ra] += 1      # 新树秩 +1
```


8.3.3 为什么树高是 $O(\log n)$

两种策略是同一句证明: 一个结点深度每增加 1, 它所在树的规模至少翻一倍。x 深度增加只发生在“它的树被并入另一棵树”时: 按大小合并规定小树并入大树, 新树大小 $\geq 2 \times$ 旧树; 按秩合并规定浅树并入深树, 而秩 r 的树至少 2^r 个结点(归纳可证)。故 x 深度最多增加 $\log_2 n$ 次, 树高 $\leq \log_2 n$, find 保证 $O(\log n)$ 。n = 10^6 时 $\log_2 n \approx 20$, 链却是 10^6 。

策略	维护什么	合并规则	何时 +1	附带能力
按大小合并	size[root] = 结点数	小树并入大树	新根 size 累加	可查询集合大小
按秩合并	rank[root] = 高度上界	浅树并入深树	两秩相等时 +1	与路径压缩配合更顺

★ 重点: 两种策略等价, 选一个记住

两者都保证树高 $O(\log n)$, 复杂度相同。按秩合并代码更短, 且“秩是上界”的语义与路径压缩天然兼容(见 §8.7.2); 按大小合并的额外好处是能回答“集合有几个元素”。本章正文与实验采用按秩合并, 换成 size 只需三行。

§8.4 路径压缩

8.4.1 思想: 把树压扁

按大小/按秩合并把树高控制在 $O(\log n)$, 但还能更狠: 路径压缩(path compression)在 find 的路途中动手——既然已经走到根, 就把沿途每个结点直接挂到根下面, find(x) 之后 x 到根的距离变成 1, 整条路径被压成“菊花状”。压缩只发生在 find 上, union 调用 find 时自动受益——一次 find 的代价, 换路径上所有结点后续 find 的 $O(1)$ 。

8.4.2 递归实现与迭代实现

示例 8-4 路径压缩: 递归与迭代两种写法

```
# 递归版: 返回根, 回溯时把沿途结点挂到根
def find_rec(x):
    if parent[x] != x:
        parent[x] = find_rec(parent[x])
    return parent[x]

# 迭代版(两趟): 先找根, 再沿原路压缩
def find_iter(x):
    root = x
    while parent[root] != root: # 第一趟: 上溯找根
        root = parent[root]
    while parent[x] != x: # 第二趟: 沿途结点挂到根
        nxt = parent[x]
        parent[x] = root
        x = nxt
    return root
```

递归版三行, 但 Python 里有一个**必须正视的隐患**: 递归深度等于树高, 而 Python 默认递归上限只有约 1000 层。按秩合并保证树高 $O(\log n)$ 时完全安全; 若只压缩不按秩(很多竞赛模板这样), 深链上 find_rec(0) 会直接抛 RecursionError。调 sys.setrecursionlimit 能缓解, 但治标不治本。本章的工程结论: Python 版一律用迭代两趟版, 无递归深度风险——这也是 Python 与 C 的重要差异(见 §8.7.2)。

C 的写法(递归上溯)	Python 的写法(迭代两趟)
<pre>int f(int x){ return p[x]==x ? x : f(p[x]); }</pre>	<pre>while parent[root] != root: root = parent[root] 第一趟找根</pre>
递归深度 = 树高,深链会爆栈	深链递归会抛 RecursionError(默认约 1000 层)
修改发生在回溯期,代码短	第二趟显式压缩,无递归深度风险

⚠ 误区 2:Python 递归 find 撞上 RecursionError

路径压缩 + 按秩合并时递归没问题;但只压缩不按秩时树高可达 n , `find_rec(0)` 递归深度就是 n , n 超过约 1000 立即抛 `RecursionError: maximum recursion depth exceeded`。更隐蔽的坑是忘了用压缩版:写了 `find_iter` 却仍调用朴素 `find`, 压缩永不发生。检查:连续 `find` 两次同一结点,看路径是否变短。

8.4.3 与按秩合并叠加:近 $O(1)$

按秩合并让树"长不高"(高度 $O(\log n)$), 路径压缩让树"越来越扁", 叠加后, 并查集的操作复杂度被压到"均摊近 $O(1)$ "——精确上界是反阿克曼函数 $\alpha(n)$, 见 §8.5.3。这是本数据结构的最终形态: **union = 两次 find + 一次指针修改**, **find $\approx O(1)$** , 请务必默写。

🚀 实现建议:迭代两趟的写法与验证

迭代两趟容易把变量搞混, 验证三点:① 第一趟后 `root` 必须是根 (`parent[root] == root`); ② 第二趟先保存 `nxt = parent[x]` 再改写 `parent[x]`, 否则覆盖后丢链; ③ 返回 `root` 而非 `x`。用幂等性自测:连续两次 `find(x)`, 第二次必然一步到根。

§8.5 复杂度:摊还近 $O(1)$

8.5.1 三种配置,三个档次

配置	单次 find 最坏	m 次操作总代价
朴素实现(无优化)	$O(n)$ (链)	$O(mn)$
仅按大小/按秩合并	$O(\log n)$	$O(m \log n)$
按秩合并 + 路径压缩	均摊近 $O(1)$ (单次最坏仍 $O(\log n)$)	$O(m \alpha(n))$, 实际当 $O(m)$ 用

读表注意"单次"与"总代价"之分:完整优化下**某一次** find 仍可能慢(遇到未压缩深链,一次性压扁),但"债"被后续 find 偿还——这就是**摊还**:把总代价摊到每次操作上,平均近 $O(1)$ 。

8.5.2 摊还近 $O(1)$ 的直觉

问: x 的 `parent` 会被改写几次? 压缩改写它时直接挂到根, 之后每次 find 一步到达; 只有"它所在的树又并入更大的树"时, 它才可能再被改写。按秩合并保证每次并入树至少翻倍, 故每结点最多被改写 $O(\log n)$ 次——这只是粗糙上界, "压扁"带来的加速使真实总量小得多: 1975 年 Tarjan 证明 m 次操作总代价 $O(m \alpha(n))$ 。工程上, 把并查集操作当作 $O(1)$ 用, 99.99% 的场景不会出错。

8.5.3 反阿克曼函数:为什么当常数用

反阿克曼函数 $\alpha(n)$ 是阿克曼函数(增长极快的递归函数)的反函数:使阿克曼函数达到 n 所需的最小自变量。它慢到难以想象: $\alpha(10^{80}) \leq 4 \sim 5$ ——"全宇宙原子数"级别的 n 也只让 α 走到 4、5。故 $O(m \alpha(n))$ 在实践中就是 $O(m)$:**每次 union/find 均摊近 $O(1)$, 当常数用**。这与第2章"log 几乎不增长"一脉相承—— α 是更极端的"几乎不增长"。

记号说明(本章约定)

$\alpha(n)$ 读作"反阿克曼",是分析意义上的上界;标准写法"均摊近 $O(1)$ ",严谨写法 $O(\alpha(n))$ 。"摊还" $\neq$ "平均":平均对随机输入取期望,摊还对**同一个操作序列**的总代价取平均——并查集的近 $O(1)$ 不依赖任何随机性假设。

§8.6 应用:迷宫生成与连通分量

8.6.1 迷宫生成:随机拆墙,并查集判连通

把 $w \times h$ 网格的每格看成一个集合(初始互不相交),收集所有相邻格子之间的墙,**随机打乱顺序**后依次尝试拆墙:两端**不连通**就拆掉并 union,已连通则留着,直到只剩一个集合。两个性质:

- **无环**:每面被拆的墙都连接两个原本不连通的区域,永远不会形成回路;
- **全连通**:结束时只有一个集合,任意两格都能到达。

共 $n = w \times h$ 个格子,成功拆墙恰好 $n-1$ 面(每拆一面分量减一,从 n 减到 1)。 $n-1$ 面墙、无环、全连通——正是**生成树**:迷宫即网格图的随机生成树,任意两格之间有且仅有一条路径。学术上叫"随机 Kruskal 迷宫",与第9章 Kruskal 最小生成树的唯一区别:墙的"权"是随机顺序。

示例 8-5 迷宫生成核心:拆墙循环(完整程序见章末实验)

```
# 格子编号 id = r*w + c;right/down 记录墙
def gen_maze(w, h, seed=2024):
    uf = UnionFind(w * h)          # 完整优化版并查集
    right = [[True] * w for _ in range(h)] # 右墙
    down = [[True] * w for _ in range(h)]  # 下墙
    walls = []
    for r in range(h):
        for c in range(w):
            if c + 1 < w:
                walls.append((r, c, r, c + 1)) # 右墙
            if r + 1 < h:
                walls.append((r, c, r + 1, c)) # 下墙
    random.seed(seed)
    random.shuffle(walls)           # 随机顺序拆墙
    cnt = w * h                     # 连通分量个数
    for r1, c1, r2, c2 in walls:
        if uf.union(r1 * w + c1, r2 * w + c2): # 两端不连通才拆
            cnt -= 1                       # 每拆一面墙,分量减一
            if c1 == c2:
                down[r1][c1] = False
            else:
                right[r1][c1] = False
    if cnt == 1:
        break                       # 全连通即停
    return right, down
```

8.6.2 连通分量统计

并查集自带连通分量信息:初始化时分量数为 n ;每次 `union` 成功(两端原本不连通)分量数减一——生成迷宫的同时就能报出"还有几个连通块",实验任务 3 正是记录这条下降曲线。若没维护计数,也可遍历全部元素数根: `find(i)` 即 i 的代表元,根相同即同一分量。前者 $O(1)/\text{次}$,后者 $O(n \alpha(n))$ 。

8.6.3 手写并查集 vs dict 标记法:优势场景

与本章开头提过的"笨办法"认真对比一次,才明白并查集赢在哪。Python 里"标记法"的天然载体是 dict:

示例 8-6 替代方案:用 dict 维护"元素 -> 集合编号"

```
# 替代方案:union 时把较小集合的所有元素逐个改编号
owner = {i: i for i in range(n)}      # 元素 -> 集合编号
def find_owner(x):
    return owner[x]                    # 查归属  $O(1)$ 

def union_owner(a, b):
    oa, ob = owner[a], owner[b]
    if oa == ob:
        return
    # 遍历全部元素把集合 ob 的成员改成 oa: $O(n)$ !
    for x, o in owner.items():
        if o == ob:
            owner[x] = oa
# 并查集:union 只改一个指针,均摊近  $O(1)$ ,差距  $n$  倍
```

C 的连通性标记法	Python 的并查集
<code>mark[N]</code> 记录归属,合并要遍历数组改编号, $O(n)$	<code>parent</code> 树形表示,合并改一个指针,均摊近 $O(1)$
查归属 $O(1)$,但合并贵	查归属(两次 <code>find</code>)与合并都近 $O(1)$
能直接枚举集合的全部成员	不支持枚举成员(需另存集合列表)
合并 m 次总代价 $O(mn)$	合并 m 次总代价 $O(m \alpha(n))$

手写 vs 替代:标准库没有并查集,两种方案都得手写。选择标准:频繁"合并 + 查归属"用并查集;需"枚举集合成员"或"合并极少"才考虑 dict 标记法。迷宫 10^6 格时,标记法每次拆墙扫描 10^6 个元素,并查集是常数步。这也解释了第9章 Kruskal 为何必须配并查集:换成标记法,排序的 $O(E \log E)$ 会被合并的 $O(E \vee)$ 吞掉,算法不可用。

§8.7 实现要点与调试

8.7.1 三个必须养成的习惯

- **初始化**用 `list(range(n))` (或 `parent[i] = i`):漏掉初始化,find 沿未定义值乱走甚至死循环,多半不报错,最危险。
- **合并前先 find 到根**:直接写 `parent[a] = b` 是"元素挂元素",树结构立刻损坏。
- **size/rank 只在根上维护**:非根结点的 size/rank 已过期,更新只能发生在合并后当选新根的结点上。

8.7.2 路径压缩不破坏秩的近似性

反直觉但正确的结论: `find` 做路径压缩时,不需要(也不应该)更新 `rank`。`rank` 是"树高的上界":压缩把树压矮,实际高度变小,旧 `rank` 仍是合法上界,只是"松"了——下次合并最多让树高略高于最优,不影响 $O(\log n)$ 与均摊近 $O(1)$ 。`find` 里更新 `rank` 反而白付代价。这就是"秩 = 上界"而非"精确高度":只承诺上界,压缩无损。

另一点 Python 特有:用 `sys.setrecursionlimit(10**6)` 硬撑深递归,会把解释器的 C 栈推向崩溃边缘(深递归在 CPython 里可能直接崩溃而非优雅报错)。所以再强调一次:Python 版 `find` 用迭代两趟,彻底绕开递归深度这个变量。

8.7.3 调试技巧:把结构画出来

示例 8-7 调试利器:打印 `parent` 列表

```
# 打印 parent 列表,观察树形态
def dump(parent):
    print(" ".join(f"{i}->{parent[i]}" for i in range(len(parent))))

# 验证"根性":每个元素的 find 结果必须是根
def check(parent):
    def find(x):
        while parent[x] != x:
            x = parent[x]
        return x
    for i in range(len(parent)):
        assert find(i) == parent[find(i)]

# 小数据验证:union_sets(0,1); union_sets(1,2); dump(parent)
# 输出 "0->1 1->2 2->2" 立刻看出 0-1-2 长链;压扁后 "0->2 1->2 2->2"。
```

调试三招

① 小数据手推: n 取 5~8,把每步 `union` 后的 `parent` 抄在纸上与 `dump` 对照;② 打印关键点:`find` 返回前打印"`x` 到根",压缩是否生效一目了然;③ 断言根性: `assert parent[find(i)] == find(i)`,把"结构被破坏"提前暴露为断言失败。

误区 3:初始化写成"全部指向 0"

把 `parent` 初始化成 `[0] * n` 而不是 `list(range(n))`:`find(3)` 走到 0 就停下,0 被误当所有集合的根,任何两个元素的 `find` 结果都相同——"全都连通"的假象,且不崩溃、不报错。排查口诀:初始化后立刻 `dump`,`parent` 必须等于下标。顺带一提, `[0] * n` 里放整数没有引用共享问题,但换成列表就会踩 Python 的"可变对象重复引用"坑,用 `[[False] * w for _ in range(h)]` 而不是 `[[False] * w] * h`。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
find	朴素实现(无优化)	$O(n)$	$O(n)$ (链状树)
union	朴素实现(无优化)	$O(n)$	$O(n)$
find	仅按大小合并	$O(\log n)$	$O(\log n)$
find / union	仅按秩合并	$O(\log n)$	$O(\log n)$
find	按秩合并 + 路径压缩	均摊近 $O(1)$ (严格 $O(\alpha(n))$)	
union	按秩合并 + 路径压缩	均摊近 $O(1)$ (两次 find + 一次指针修改)	
初始化	建 n 个单元素集合	$O(n)$	$O(n)$
递归版压缩 find	递归实现	均摊近 $O(1)$	深链递归抛 RecursionError(默认约 1000 层)
迭代版压缩 find	迭代实现(两趟)	均摊近 $O(1)$	均摊近 $O(1)$,无递归深度问题
求连通分量个数	遍历全部元素数根	$O(n \alpha(n))$	$O(n \alpha(n))$
维护连通分量数	union 成功时 count-1	$O(1)$	$O(1)$
迷宫生成	随机拆墙 + 并查集判连通	$O(E \alpha(V))$	$O(E \alpha(V))$
Kruskal 判连通	每条边两次 find + 一次 union	均摊近 $O(1)$ /边,总 $O(E \log E)$ (排序主导)	
反阿克曼函数 $\alpha(n)$	完整优化的单次操作上界	$\alpha(10^{80}) \leq 4 \sim 5$,当常数用	
dict 标记法	dict 维护集合编号	union $O(n)$ (遍历集合)	union $O(n)$

最小必会清单

- 能说出等价关系三性质,判断"同班同学""模 m 同余"是否等价关系
- 能说出 find / union 的语义,以及"动态"为何要求在线回答
- 能用 `list(range(n))` + 树形表示实现朴素并查集,解释"根 = 代表元"
- 能解释朴素 union 为何把树连成链,find 退化 $O(n)$
- 能实现按大小/按秩合并,说出"深度每 +1 树规模至少翻倍"的直觉
- 能写出路径压缩的递归版与迭代版,说出 Python 为何必须用迭代版
- 能说出三种配置的复杂度:无优化 $O(n)$ /仅按秩 $O(\log n)$ /完整优化均摊近 $O(1)$,及"当常数用"的理由
- 能用并查集生成随机迷宫,并调试:初始化、先 find 到根、rank 不失效、打印 parent

自测题

Q 1. "同班同学"关系满足等价关系的哪几条?缺哪条?为什么它导致"全班连通"不能用并查集建模?

✅ 参考答案

自反、对称满足,传递不满足:甲、乙同班且乙、丙同班,甲、丙可能不同班。三条不全,"同学"不能把集合划分成互不相交的等价类,故不能用并查集。

Q 2. 依次执行 `union(0,1)`, `union(1,2)`, ..., `union(n-2, n-1)` 后, `find(0)` 要走多少步?为什么是灾难?

✓ 参考答案

树变成链 $0 \rightarrow 1 \rightarrow \dots \rightarrow n-1$, `find(0)` 走 $n-1$ 步即 $O(n)$; n 次 `find` 总代价 $O(n^2)$, $n = 10^6$ 时是 10^{12} 步, 远超可行范围。这正是朴素实现必须优化的原因。

Q 3. 按大小合并为什么保证树高 $O(\log n)$? 给出直觉证明。

✓ 参考答案

x 深度每 $+1$, 必然是它所在的小树被并入一棵不小于自己的树(小树并入大树), 新树大小 $\geq 2 \times$ 旧树, 故 x 深度最多增加 $\log_2 n$ 次, 树高 $\leq \log_2 n$ 。

Q 4. 路径压缩为何能让后续 `find` 变快? Python 里递归版与迭代版各自的风险是什么?

✓ 参考答案

一次 `find` 把沿途所有结点挂到根, 之后它们再 `find` 都是一步到达——"一次的代价, 换一路的 $O(1)$ "。递归版代码短, 但递归深度 = 树高, 深链会抛 `RecursionError` (默认约 1000 层), 调 `setrecursionlimit` 也只是把问题推迟; 迭代版无递归深度风险, 工程更稳。

Q 5. 迷宫生成中, 为何拆墙数恰好等于格子数 - 1? 若拆墙时两端已连通会怎样?

✓ 参考答案

每拆一面墙必连通两个原本不相交的集合, 分量数恰好减一, 从 n 减到 1 共拆 $n-1$ 面。若两端已连通还拆墙会形成回路, 破坏"恰有一条路", 必须用 `find` 跳过。 $n-1$ 面、无环、全连通 = 生成树。

章末实验题: 迷宫生成与连通分量统计

实验 8: 迷宫生成与连通分量统计(并查集综合应用)

实验目标: 实现完整优化并查集(按秩合并 + 路径压缩), 生成随机 ASCII 迷宫, 统计拆墙中连通分量的变化, 实测朴素版与优化版性能差距。

实验要求:

- 任务 1: 实现并查集类: `find` (迭代两趟路径压缩) + `union` (按秩合并), 维护 `count` (知识点: §8.3 按秩合并、§8.4 路径压缩)
- 任务 2: 对 $w \times h$ 网格随机拆墙, 两端不连通才拆, 直到全连通; 输出 ASCII 迷宫(知识点: §8.6 迷宫生成、§8.2 树表示)
- 任务 3: 记录拆墙中连通分量数量的变化, 在尝试 25%/50%/75%/100% 处输出关键点, 验证"每拆一面墙分量减一"(知识点: §8.6.2 连通分量统计)
- 任务 4: 实现朴素版(无优化)做同一操作序列, 对比耗时, 体会 $O(n^2)$ 与近 $O(1)$ 的差距(知识点: §8.5 复杂度)

实验环境: Python 3.8+。运行: `python lab.py`。

第一步 写并查集类: `parent = list(range(n))`, `rank = [0] * n`, `count = n`; `find` 写迭代两趟版, `union` 按秩合并, 成功时 `count-1`

第二步 选墙: 右墙 $(r, c)-(r, c+1)$ 、下墙 $(r, c)-(r+1, c)$, `random.seed(seed)` + `random.shuffle` 固定种子随机化

第三步 墙循环中维护 `record`, 在 `idx` 达到墙总数的 1/4、1/2、3/4、全部时记录 (`idx`, `count`)

第四步 渲染按"顶/底横线 + 格子行 + 中间横线"拼字符串, 行宽一致(每格 3 字符)

第五步 任务 4 用同一操作序列(依次 `union(i, i+1)` 再全部 `find`)跑朴素版与优化版, `time.perf_counter()` 计时; 朴素版 `union` 的代价会延迟到 `find` 阶段爆发, 是观察"摊还"的绝佳素材

✅ 参考答案(完整可运行程序,约 143 行,分三段)

```
# lab.py — 迷宫生成与连通分量统计(第一段:并查集与朴素版)
# 运行:python lab.py
import random
import time

# ----- 任务 1:并查集(按秩合并 + 路径压缩) -----
class UnionFind:
    """按秩合并 + 路径压缩的并查集,parent[i] == i 表示 i 是根。"""

    def __init__(self, n):
        self.parent = list(range(n)) # parent[i] = i 表示 i 是根
        self.rank = [0] * n         # 秩:树高的上界,只在根上有意义
        self.count = n               # 当前连通分量个数

    def find(self, x):
        """迭代版路径压缩:先上溯找根,再沿原路把沿途结点挂到根。"""
        root = x
        while self.parent[root] != root:
            root = self.parent[root]
        while self.parent[x] != x: # 第二趟:逐个压缩
            nxt = self.parent[x]
            self.parent[x] = root
            x = nxt
        return root

    def union(self, a, b):
        """合并 a、b 所在集合;合并成功返回 True,已在同一集合返回 False。"""
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.rank[ra] < self.rank[rb]: # 浅树挂到深树
            ra, rb = rb, ra
        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1 # 两棵同秩树合并,新树秩 +1
        self.count -= 1       # 成功合并一次,连通分量减 1
        return True

# ----- 任务 4 对比用:朴素版(无任何优化) -----
class NaiveUF:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a != b:
            self.parent[a] = b # 固定让 a 挂到 b 下,容易形成长链
```

lab.py(第二段:迷宫生成与渲染)

```
# ----- 任务 2/3:随机迷宫生成 -----
def gen_maze(w, h, seed=2024):
    uf = UnionFind(w * h)
    right = [[True] * w for _ in range(h)] # 右墙
    down = [[True] * w for _ in range(h)] # 下墙
    walls = []
    for r in range(h):
        for c in range(w):
            if c + 1 < w:
                walls.append((r, c, r, c + 1)) # 右墙,连接 (r,c) 与 (r,c+1)
            if r + 1 < h:
                walls.append((r, c, r + 1, c)) # 下墙,连接 (r,c) 与 (r+1,c)
    random.seed(seed)
    random.shuffle(walls)
    record = [] # 任务 3:连通分量变化采样
    key = [len(walls) // 4, len(walls) // 2, len(walls) * 3 // 4]
    ki = 0
    for idx, (r1, c1, r2, c2) in enumerate(walls, 1):
        if uf.union(r1 * w + c1, r2 * w + c2): # 两端不连通才拆墙
            if c1 == c2:
                down[r1][c1] = False
            else:
                right[r1][c1] = False
        if ki < 3 and idx >= key[ki]:
            record.append((idx, uf.count))
            ki += 1
    record.append((len(walls), uf.count)) # 结束时必为 1
    return right, down, record

def render(w, h, right, down):
    """把迷宫画成 ASCII:每格 2 字符宽,"+" 是柱子,"--" 是横墙,"|" 是竖墙。"""
    lines = ["+" + "--+" * w] # 顶部横线(永不拆)
    for r in range(h):
        row = "|" # 最左竖墙(永不拆)
        for c in range(w):
            row += " " + ("|" if right[r][c] else " ")
        lines.append(row)
        if r == h - 1:
            lines.append("+ + --+" * w) # 底部横线(永不拆)
        else:
            line = "+"
            for c in range(w):
                line += "--" if down[r][c] else " "
                line += "+"
            lines.append(line)
    return "\n".join(lines)
```

lab.py(第三段:性能对比与主程序)

```

# ----- 任务 4:朴素版 vs 完整优化版性能对比 -----
def bench(n):
    ops = [(i, i + 1) for i in range(n - 1)] # 顺序合并:朴素版会连成长链
    t0 = time.perf_counter()
    naive = NaiveUF(n)
    for a, b in ops:
        naive.union(a, b) # 每次 union 都沿链走 O(n)
    t1 = time.perf_counter()
    s = 0
    for x in range(n):
        s += naive.find(x) # 链尾 find 走 O(n)
    t2 = time.perf_counter()
    opt = UnionFind(n)
    for a, b in ops:
        opt.union(a, b)
    t3 = time.perf_counter()
    for x in range(n):
        s += opt.find(x)
    t4 = time.perf_counter()
    ms = lambda a, b: (b - a) * 1000
    print(f"任务4: {n} 次 union(连成链)+ {n} 次 find(耗时对比):")
    print(f" 朴素版: union 阶段 {ms(t0, t1):.0f} ms, find 阶段 {ms(t1, t2):.0f} ms")
    print(f" 优化版: union 阶段 {ms(t2, t3):.0f} ms, find 阶段 {ms(t3, t4):.0f} ms")
    print(f" find 阶段加速比:约 {(t2 - t1) / (t4 - t3):.0f} 倍(检查值 s={s % 1000})")

def main():
    w, h = 6, 4
    right, down, record = gen_maze(w, h, seed=2024)
    print("=" * 44)
    print(f"随机迷宫 {w} 列 x {h} 行(种子 2024,共 {w * h} 个格子):")
    print(render(w, h, right, down))
    print()
    total = record[-1][0]
    print(f"任务3: 拆墙过程中连通分量数量的变化(共 {total} 面候选墙):")
    print(f" 初始: {w * h} 个连通分量")
    for tried, comp in record:
        print(f" 尝试 {tried:2d}/{total} 面墙后: 连通分量 {comp:2d}")
    print(f" 结束时连通分量为 1,迷宫全连通、无环(恰好拆 {w * h - 1} 面墙)。")
    print()
    bench(5000)

if __name__ == "__main__":
    main()

```

示例输出(本机 Python 3 实测):

=====

随机迷宫 6 列 x 4 行(种子 2024,共 24 个格子):

```
+---+---+---+---+
|           |     |
+ + + + + + +
| | | |   | |
+---+ +---+ +---+ +
|   | |   | |
+ + + + +---+ +
| |   | |   |
+---+---+---+---+
```

任务3: 拆墙过程中连通分量数量的变化(共 38 面候选墙):

初始: 24 个连通分量

尝试 9/38 面墙后: 连通分量 15

尝试 19/38 面墙后: 连通分量 6

尝试 28/38 面墙后: 连通分量 2

尝试 38/38 面墙后: 连通分量 1

结束时连通分量为 1,迷宫全连通、无环(恰好拆 23 面墙)。

任务4: 5000 次 union(连成链)+ 5000 次 find(耗时对比):

朴素版: union 阶段 1 ms, find 阶段 268 ms

优化版: union 阶段 1 ms, find 阶段 0 ms

find 阶段加速比:约 578 倍(检查值 s=0)

设计说明:① UnionFind 与 NaiveUF 共用同一操作序列,对比才公平——差的是增长阶($O(n^2)$ vs 近 $O(1)$),不是常数;② 朴素版 union 阶段并不慢(两版都是 1 ms):链的代价被"延迟"到 find 阶段爆发(268 ms vs 0 ms)——\$8.5 摊还思想的注脚:比算法要看操作序列的**总代价**;③ 任务 3 采样点取"尝试墙数"而非"成功拆墙数":拆墙有失败(两端已连通),曲线呈阶梯状,更能体现 find 判连通;④ 种子固定 2024,任何机器可复现;⑤ 固定挂法 `parent[a] = b` 刻意制造长链,暴露退化——现实中盲目选挂向也可能遭遇同样最坏输入。

下一章预告:第9章 图论算法——图的表示、拓扑排序、最短路径与最小生成树。Kruskal 将使用本章的不相交集:边按权排序后,find 判断两端是否已连通,union 把选中边连入生成树——两条核心操作正是本章主线。

第9章 图论算法

建议学时:5-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 掌握图的基本术语,说出**邻接矩阵**与**邻接表**的代价差异;完成从 C 的二维数组、手写链表到 Python 嵌套列表、元组的迁移
- 实现**拓扑排序**(入度表 + deque 队列)并用"输出个数 < 顶点数"检测环;实现 BFS 求无权**最短路径**并回溯打印
- 实现 Dijkstra 朴素版与 heap 优化版,说清 INF 选择;实现 Bellman-Ford 检测负环
- 实现 Prim 与 Kruskal **最小生成树**;完成迷宫最短路径综合实验

💡 从 C 到 Python:本章主题如何迁移

图的直觉一句话: **图 = 一堆顶点 + 顶点之间的边**,算法全部建立在"存图、遍历图"两个动作上。前面 8 章学的栈、队列、堆、并查集本章全部派上用场:BFS 与拓扑用第3章的 deque,Dijkstra 用第6章的 heapq,Kruskal 用第8章的不相交——图论是前面所有数据结构的"总演习"。

Python 的解法是"列表 + 元组": `g = [[] for _ in range(n)]` 一行声明动态邻接表;加权图用 `g[u].append((v, w))` 存(终点, 权)元组,顶点不连续时换字典 `g[u].append((v, w))` 同样成立。省去 C 的 malloc/指针,代码接近伪代码,精力全花在算法本身。

📦 前置知识自查

数学前置:无特殊;只需第2章大 O 记号,复杂度字母约定 $V = \text{顶点数}$ 、 $E = \text{边数}$ 。

C 语言前置:二维数组与下标运算;对照用的 `struct` 链表与 `qsort` 函数指针(可选)。

依赖章节:第3章栈与队列(deque、迭代 DFS 的显式栈);第6章 `heapq` (Dijkstra/Prim 堆优化);第8章不相交集(列表 + 路径压缩)。

🚀 本章学习路线

1. **第一阶段(1.5 小时):**读 §9.1–§9.3,敲两种建图,实现拓扑排序与 BFS
2. **第二阶段(2 小时):**读 §9.4–§9.6,对照朴素/heap 两份 Dijkstra,体会 Prim 的一行之差
3. **第三阶段(1.5 小时):**完成章末实验:迷宫最短路径(BFS + Dijkstra)
4. **验收:**默写建图、BFS 路径回溯、heap 优化 Dijkstra、Kruskal 与各自复杂度

本章知识导图

- §9.1 图的表示:术语与两种存储结构
- §9.2 拓扑排序:入度表与队列
- §9.3 无权最短路径:BFS 与路径回溯
- §9.4 Dijkstra 最短路:贪心、INF 与堆优化
- §9.5 负权边与 Bellman-Ford
- §9.6 最小生成树:Prim 与 Kruskal
- §9.7 最大流与 DFS 应用简介
- §9.8 工程实践:大图的正确打开方式

§9.1 图的表示:术语与两种存储结构

9.1.1 基本术语

图由两部分组成:**顶点(vertex)** 与 **边(edge)**——顶点是"对象",边是"对象之间的关系"。三个常用例子:**城市-航班图**(城市=顶点,航线=边)、**课程依赖图**(课程=顶点,先修=边)、**网页链接图**(网页=顶点,链接=边)。

- **有向图 vs 无向图**:边有方向叫有向边($u \rightarrow v$),全有向的图叫有向图(课程依赖);边不分方向叫无向边($u - v$),对应无向图(道路网)。无向边可看作两条反向有向边。
- **权(weight)**:边上的数值(票价、距离)。带权叫加权图;不带权默认每条边代价 1。
- **路径与环**:顶点间的边序列叫路径;首尾相同的叫环。拓扑排序只能处理无环有向图(DAG),即 §9.2 的主题。

9.1.2 邻接矩阵 vs 邻接表

存图只有两种主流方案,各有胜负。设 V 个顶点、 E 条边, $\deg(u)$ 表示顶点 u 的邻居个数:

对比项	邻接矩阵	邻接表
存储空间	$O(V^2)$,无论边多少都占满	$O(V + E)$,稀疏图省空间
判断边 (u, v) 是否存在	$O(1)$,直接查 $g[u][v]$	$O(\deg(u))$,遍历 u 的链表
遍历 u 的所有邻居	$O(V)$,扫一整行	$O(\deg(u))$,顺着链表走
适用场景	稠密小图($V \leq 2000$)	稀疏大图(工程绝大多数)

口诀:图算法都要"遍历邻居",邻接表把这一步从 $O(V)$ 降到 $O(\deg)$,整体复杂度下降一个量级。邻接矩阵的用武之地是" V 小且频繁查边"的场景(如 Floyd)。 $V = 10^5$ 时矩阵要 10^{10} 个格子 ≈ 10 GB,直接内存崩溃。

C 的写法	Python 的写法
<code>int g[100][100];</code> 大小写死的二维数组	<code>g = [[0] * n for _ in range(n)]</code> 嵌套列表
<code>struct Edge { int to, w; struct Edge* next; };</code> 手写链表	<code>g = [[] for _ in range(n)]</code> 加权用 <code>(v, w)</code> 元组
<code>for (Edge* e = head[u]; e; e = e->next)</code> 指针遍历	<code>for v in g[u]</code> 直接遍历
<code>malloc / free</code> 手动管理,忘记即泄漏	垃圾回收自动管理,无泄漏

示例 9-1 有向无权图:邻接表建法(矩阵建法见上表 C 对照)

```
# 邻接表:g[u] 是 u 的所有出边终点
def build_list(n, edges):
    g = [[] for _ in range(n)]      # n 个独立空列表,勿写 [[]]*n
    for u, v in edges:
        g[u].append(v)             # 无向图再加 g[v].append(u)
    return g

# 使用:g = build_list(4, [(0,1), (0,2), (2,3)])
# 遍历 u 的邻居:for v in g[u]
```

9.1.3 加权图:C 的手写链表 → Python 的元组

加权图在 C 里要用"边结点带权"的链表,建图几十行;Python 用 (终点, 权) 元组: `g[u].append((v, w))` ——读边直接 `append`,第6章 Dijkstra 的图就存成这种结构。注意嵌套列表的坑: `[[0] * n] * n` 会让所有行共享同一对象,必须先 `[[] for _ in range(n)]` 逐行新建。

§9.2 拓扑排序:入度表与队列

9.2.1 什么是拓扑排序

拓扑排序:把有向无环图(DAG)的所有顶点排成一列,使每条边 $u \rightarrow v$ 都满足"u 排在 v 前"。典型场景是**课程先修依赖**:C 在数据结构前,数据结构在算法分析前——课程表就是拓扑序。**带环的有向图不存在拓扑序**(A 先修 B、B 又先修 A,谁也排不了),所以"能否拓扑排序"等价于"有没有环"。

9.2.2 Kahn 算法:入度表 + 队列

算法直觉:"不断摘掉没有前置的课程"。三步:① 统计每个顶点的**入度**(指向它的边数);② 入度为 0 者全部入队——无前置,随时可安排;③ 出队 u,删掉它的所有出边(邻居入度减 1),新入度为 0 的邻居入队。**无环时必然输出全部 V 个顶点;输出个数 < V 就是有环,务必检查**。

示例 9-2 拓扑排序(Kahn 算法)与环检测

```

from collections import deque

# 返回拓扑序列;有环返回空序列
def topo_sort(n, g):
    indeg = [0] * n          # ① 入度表
    for u in range(n):
        for v in g[u]:
            indeg[v] += 1
    q = deque(u for u in range(n) if indeg[u] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in g[u]:      # 删掉 u 的出边
            indeg[v] -= 1
            if indeg[v] == 0: # 前置课程全部完成才入队
                q.append(v)
    if len(order) != n: return [] # 输出个数 < n:有环!
    return order

```

§9.3 无权最短路径:BFS 与路径回溯

9.3.1 为什么 BFS 能求无权最短路

无权图(每条边代价 1)中,**最短路径**就是"边数最少的路径"。BFS 按层扩展:第 1 层距离 1,第 2 层距离 2.....**每个顶点第一次被访问时,步数一定最少**——队列严格按距离递增出队,"第一次到达即最短"正是 BFS 不需要 Dijkstra 的理由。

C 的手写循环队列	Python 的 collections.deque
<pre>int q[100000]; int head, tail;</pre> 自己维护下标与取模	<pre>q = deque([s]); q.popleft()</pre>
空间写死,溢出要自己检查	自动扩容,永不"满"
空/满判断:计数法或浪费一格(第3章)	while q: 一行判空

9.3.2 距离数组与前驱数组回溯

求步数只需 `dist` 数组;打印路径还要 `prev` 数组记录"每个顶点从谁到达"。到终点后沿 `prev` 回溯到起点再反转,就是完整路径——**前驱数组回溯**,Dijkstra 与 Bellman-Ford 沿用同一套路。

示例 9-3 无权最短路:BFS + 前驱数组回溯


```

from collections import deque

# 求 s 到各点最短距离;dist[i] = -1 表示不可达
def bfs_dist(s, g):
    n = len(g)
    dist = [-1] * n          # -1 兼任 visited
    prev = [-1] * n          # prev[v]:从谁到达 v
    dist[s] = 0
    q = deque([s])
    while q:
        u = q.popleft()
        for v in g[u]:
            if dist[v] != -1: continue    # 第一次到达即最短
            dist[v] = dist[u] + 1
            prev[v] = u
            q.append(v)
    return dist, prev

# 打印 s 到 t 的路径(沿 prev 回溯)
def print_path(s, t, prev):
    path = []
    x = t
    while x != -1:
        path.append(x)
        if x == s: break
        x = prev[x]
    if not path or path[-1] != s:    # 回溯到 -1 也没到 s
        print("无路径可达"); return
    print(" -> ".join(map(str, reversed(path))))

```

实现建议:dist 兼任 visited

BFS 里 `dist[v] != -1` 既判"访问过"又存距离,一个数组干两件事,无需单独写 `visited` ——与第3章"循环队列用 `count` 判空满"同一思想:状态能合并就合并。回溯打印注意:路径数组是"终点→起点"顺序,输出前要反转。

§9.4 Dijkstra 最短路:贪心、INF 与堆优化

9.4.1 贪心思想与正确性直觉

加权图(边权非负)的单源最短路由 Dijkstra 解决,思路是 **贪心**:每轮从"还没确定最短距离"的顶点中,挑距离最小的确定为**答案**。为什么敢确定?边权全非负——任何"经过别的未确定顶点"的绕路,起步就比这个距离大,不可能更短。"确定"是永久的:顶点一旦被选出,dist 就是最终答案,只增不改。"每次取当前最小"正是第6章小顶堆的用武之地。

9.4.2 INF 的坑:float('inf') 一劳永逸

dist 初始值必须是一个"表示无穷大"的数。Python 用 `float('inf')`:任何有限数加 `inf` 仍是 `inf`,不存在 C++ 里 `INT_MAX + w` 溢出成负数的问题(C++ 版 §9.4 详述了那个坑,要改用 `1e9` 或 `long long`)。

⚠ 误区 3:把 INF 写成有限大整数

有的同学把 C++ 习惯带过来,写 `INF = 10**18`:边权为负时 `dist[u] + w` 可能小于 `INF`,把"不可达"误判成"可达";更隐蔽的是 `int` 与 `float` 混用,类型与比较行为都变得不可预测。统一用 `float('inf')`:比较、加法全部安全,最后输出前再转 `int`。

C 的写法	Python 的写法
<code>#define INF 0x3f3f3f3f</code> 魔法常量,靠记忆	<code>INF = float('inf')</code> 内置无穷大
<code>memset(dist, 0x3f, sizeof dist)</code> 技巧性初始化	<code>dist = [INF] * n</code> 列表乘法一行填充
手写 <code>for</code> 循环扫描找最小, $O(V)$ 每次	<code>heapq.heappop</code> 堆顶即最小, $O(\log V)$ 每次

9.4.3 朴素版与堆优化版并排:手写实现 vs 标准库

两段代码解决同一问题,骨架完全相同,差异只在"找当前最小"——正是本章的"手写实现 + 标准库写法"并排:朴素版每轮全数组扫描,总代价 $O(V^2)$;堆优化版把"找最小"委托给第6章的 `heapq`,总代价 $O(E \log V)$,大图必须用它。

示例 9-4 Dijkstra 朴素版 $O(V^2)$:手写扫描找最小

```
def dijkstra_naive(s, g):
    n = len(g)
    INF = float('inf')
    dist = [INF] * n
    done = [False] * n          # done[u]:u 已确定
    dist[s] = 0
    for _ in range(n):
        u = -1
        for i in range(n):      # 扫描:未确定者中 dist 最小
            if not done[i] and (u == -1 or dist[i] < dist[u]):
                u = i
        if u == -1 or dist[u] == INF: break  # 其余顶点不可达
        done[u] = True          # 确定 u,永不再改
        for v, w in g[u]:       # 松弛:尝试用 u 更新邻居
            if dist[v] > dist[u] + w:
                dist[v] = dist[u] + w
    return dist
```

示例 9-5 Dijkstra 堆优化版 $O(E \log V)$:标准库 `heapq`

```
import heapq

def dijkstra_heap(s, g):
    n = len(g)
    INF = float('inf')
    dist = [INF] * n
    dist[s] = 0
    pq = [(0, s)]          # (距离, 顶点), 小顶堆
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: continue    # 懒删除:过期的旧记录直接跳过
        for v, w in g[u]:
            if dist[v] > d + w:
                dist[v] = d + w
                heapq.heappush(pq, (dist[v], v)) # 允许重复入堆
    return dist
```

实现建议:懒删除(惰性删除)是堆优化 Dijkstra 的标配

同一顶点可能被松弛多次、在堆里留下多份记录。堆优化版**不删除旧记录**,出堆时检查 $d > \text{dist}[u]$ 的过期记录直接丢弃——这叫懒删除,正确性由"旧记录距离必然不小于新记录"保证。heapq 没有 decreaseKey 接口,懒删除正是替代。套路:push 不查重,pop 查过期。

9.4.4 复杂度与选型

版本	"找最小"的手段	总复杂度	$V = 10^5$ 、 $E = 10^6$ 时的量级
朴素	每轮全数组扫描 $O(V)$	$O(V^2)$	10^{10} ,不可用
堆优化	堆顶 $O(\log V)$	$O(E \log V)$	约 2×10^7 ,秒级

选型结论:稠密小图($V \leq 2000$)两者皆可,朴素版常数更小; V 上万或 E 稀疏一律堆优化,工程与竞赛的默认写法。

§9.5 负权边与 Bellman-Ford

9.5.1 负权边为何破坏 Dijkstra

三顶点例子:边 $s \rightarrow a$ 权 2、 $s \rightarrow b$ 权 3、 $b \rightarrow a$ 权 -2(有向)。Dijkstra 第一轮挑出 $a(d=2)$ 并永久确定;第二轮处理 b 时松弛 $a: 3 + (-2) = 1 < 2$,但 a 已"确定"——真实最短路 $s \rightarrow b \rightarrow a$ 只有 1,答案错了。负权边让"绕路"反而更便宜,贪心的"确定即最终"被推翻。结论:Dijkstra 只适用于非负权图。

9.5.2 Bellman-Ford: $V-1$ 轮松弛

Bellman-Ford 的思想朴素而彻底:每轮对全部 E 条边松弛一次,共做 $V-1$ 轮。为什么够?无环路径至多 $V-1$ 条边:第 1 轮确定"1 条边"的最短路,第 k 轮由第 $k-1$ 轮答案再延伸一条边,第 k 轮后"至多 k 条边"的路径都被考虑。复杂度 $O(V \cdot E)$,能处理负权,代价是不能有负环。

示例 9-6 Bellman-Ford: $V-1$ 轮松弛 + 负环检测

```
def bellman_ford(s, edges, n):
    INF = float('inf')
    dist = [INF] * n
    dist[s] = 0
    for _ in range(n - 1):          # 至多 V-1 轮
        changed = False
        for u, v, w in edges:      # 每轮扫全部边
            if dist[u] != INF and dist[v] > dist[u] + w:
                dist[v] = dist[u] + w
                changed = True
        if not changed: break      # 无更新即可提前结束
    for u, v, w in edges:          # 第 V 轮:还能松弛 = 有负环
        if dist[u] != INF and dist[v] > dist[u] + w:
            return None            # 返回 None 表示有负环
    return dist
```

⚠ 误区 4:负环检测忘写,或写进主循环里

负环检测必须在 $V-1$ 轮**结束之后**单独做:负环让距离无限变小,检测混进主循环会在正常图误报"有环"。另一易错点:松弛前跳过 `dist[u] == INF` 的边——inf 减负权仍是 inf,不判断会得到假答案。

§9.6 最小生成树:Prim 与 Kruskal

9.6.1 定义与两种思路

最小生成树(MST):在连通加权无向图中,选 $V-1$ 条边把全部顶点连成一棵树(无环),边权总和最小。应用:铺电缆、修公路,让所有城市连通且总造价最低。两条路线:Prim"加点"——每次并入"离树最近"的顶点;Kruskal"加边"——边按权排序,从小到大挑不成环的边。

9.6.2 Prim:与 Dijkstra 只差一行

把 §9.4 的 Dijkstra 朴素版抄过来,Prim 与它几乎一模一样——唯一区别是松弛标准:

算法	key[u] 的含义	松弛标准
Dijkstra	起点到 u 的当前最短距离	$\text{dist}[v] > \text{dist}[u] + w$ (累加路径)
Prim	u 到"已选集合"的最近距离	$\text{key}[v] > w$ (只看边权,不累加)

Prim 的 key 是"顶点到树集合的距离",更新只比较**这条边本身**的权;Dijkstra 比较"起点到 u 的整条路径 + 边权"。一行之差,两种算法,也是第10章贪心范式的两个变体。

示例 9-7 Prim 朴素版 $O(V^2)$:与 Dijkstra 只差松弛标准

```
def prim_mst(s, g):
    n = len(g)
    INF = float('inf')
    key = [INF] * n          # 到"已选集合"的最短距离
    in_tree = [False] * n
    key[s] = 0
    total = 0
    for _ in range(n):
        u = -1
        for i in range(n):   # 扫描:离树最近的未入树顶点
            if not in_tree[i] and (u == -1 or key[i] < key[u]):
                u = i
        if u == -1 or key[u] == INF: break    # 图不连通
        in_tree[u] = True
        total += key[u]
        for v, w in g[u]:
            if not in_tree[v] and w < key[v]: # 松弛标准:直接比边权!
                key[v] = w
    return total            # 生成树总权
```

9.6.3 Kruskal:边排序 + 不相交集

Kruskal 更"贪":边按权升序,从小到大逐条考察,只要两 endpoints 还没连通就收下——"是否连通"正是第8章不相交集的看家本领(find/union 均摊近 $O(1)$);已连通则收下必成环,必须跳过。总复杂度 $O(E \log E)$,由排序主导。

对照 C 的 `qsort` 函数指针,这里用 `sort + key: edges.sort(key=lambda e: e[2])` ——"按哪一项排"写成一个 lambda,一行搞定,写错编译期报错。

示例 9-8 Kruskal:排序 + 不相交集(第8章的 find/unite 直接拿来用)

```
def kruskal_mst(n, edges):
    parent = list(range(n))

    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x])    # 路径压缩
        return parent[x]

    def unite(a, b):
        a, b = find(a), find(b)
        if a == b: return False            # 已连通:这条边会成环
        parent[a] = b
        return True

    edges.sort(key=lambda e: e[2])        # ① 按边权升序
    total = cnt = 0
    for u, v, w in edges:
        if unite(u, v):                    # ② 两端点不连通才收下
            total += w
            cnt += 1
        if cnt == n - 1: break            # 树有 V-1 条边即完成
    return total if cnt == n - 1 else -1  # -1 表示图不连通
```

⚠ 误区 5:把 Prim 的松弛标准抄成 Dijkstra 的

两段代码结构高度相似,最常见的复制粘贴事故是把 `w < key[v]` 抄成 `dist[u] + w < key[v]`。自查口诀: Prim 只比"这一条边",Dijkstra 比"整条路径";写完用三顶点小例子手算验证。

§9.7 最大流与 DFS 应用简介

9.7.1 网络流:一句话

最大流问题:有向加权图中,从源点 s 到汇点 t 能运输的最大流量(每条边有容量上限)。一句话直觉:反复找"还能增流的路径"(增广路),直到找不到为止,累加的流量就是最大流。用 BFS 找最短增广路的版本叫 **Edmonds-Karp**,复杂度 $O(V \cdot E^2)$,Dinic、ISAP 都是它的优化。应用:交通调度、二分图匹配。

9.7.2 DFS 应用:割点与双连通

两个概念:**割点**——无向图中删掉它后图不再连通的顶点(如通信网络的关键路由器);**双连通**——删掉任意一个顶点仍连通的图,其极大子图叫双连通分量。求割点的经典算法(Tarjan)基于 DFS 的"时间戳"思想,教材有完整讲解。记住一句:DFS 访问顶点的顺序本身携带结构信息(发现时间、完成时间),能回答"连通性"类问题。

9.7.3 迭代版 DFS:显式栈

递归 DFS 在深图(如 10^5 层链)上会撑爆系统调用栈;第3章说过"递归都能改写为显式栈",这里实战:显式栈压"待访问的顶点",与函数调用栈压"返回地址"同一思想。图遍历必须带 visited:无向边让 u 、 v 互相"发现",不带就无限往返;迭代版同顶点可能被 push 多次,出栈时去重。

示例 9-9 深度优先遍历:递归版与显式栈迭代版(第3章)

```
def dfs_rec(u, g, vis):
    vis[u] = True                # 访问当前顶点
    print(u, end=' ')
    for v in g[u]:
        if not vis[v]:
            dfs_rec(v, g, vis)   # 递归:深度优先

def dfs_iter(s, g):
    vis = [False] * len(g)
    st = [s]                    # 显式栈代替系统调用栈
    while st:
        u = st.pop()
        if vis[u]: continue     # 可能被压入多次,去重
        vis[u] = True
        print(u, end=' ')
        for v in g[u]:
            if not vis[v]:
                st.append(v)     # 注意:压栈顺序与递归相反
```

§9.8 工程实践:大图的正确打开方式

9.8.1 大图选型:邻接表优先

真实世界的图几乎都稀疏(社交网络、网页链接皆如此),三条准则:

- **存图用邻接表:** $V \geq 10^4$ 一律邻接表,矩阵只在小稠密图出现(§9.1 误区1)
- **无向图建两次边:** $g[u]$ 与 $g[v]$ 互相 append,漏一边就成"单行道"
- **顶点编号连续:** 用 $0..V-1$ 整数做下标;字符串先映射成编号(第5章)

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
遍历所有顶点与边	DFS / BFS	$O(V + E)$	$O(V + E)$
查边 (u, v)	邻接矩阵	$O(1)$	$O(1)$
查边 (u, v)	邻接表	$O(\deg(u))$	$O(\deg(u))$
存储	邻接矩阵	$O(V^2)$	$O(V^2)$
存储	邻接表	$O(V + E)$	$O(V + E)$
拓扑排序	Kahn(入度表 + 队列)	$O(V + E)$	$O(V + E)$
无权最短路径	BFS	$O(V + E)$	$O(V + E)$
单源最短路径(非负权)	Dijkstra 朴素	$O(V^2)$	$O(V^2)$
单源最短路径(非负权)	Dijkstra 堆优化	$O(E \log V)$	$O(E \log V)$
单源最短路径(可负权)	Bellman-Ford	$O(V \cdot E)$	$O(V \cdot E)$
负环检测	Bellman-Ford 第 V 轮	$O(V \cdot E)$	$O(V \cdot E)$
最小生成树	Prim 朴素	$O(V^2)$	$O(V^2)$
最小生成树	Prim 堆优化	$O(E \log V)$	$O(E \log V)$
最小生成树	Kruskal(排序 + 并查集)	$O(E \log E)$	$O(E \log E)$
最大流	Edmonds-Karp(BFS 增广)	$O(V \cdot E^2)$	$O(V \cdot E^2)$

最小必会清单

- 说出顶点、边、有向/无向、权、路径、环六个术语并配现实例子
- 独立写出邻接矩阵与邻接表两种建图代码,说出空间与查询复杂度;避开 `[[] ] * n` 共享列表的坑
- 默写 Kahn 拓扑排序,解释"输出个数 $< V$ 则有环"的检测原理
- 默写 BFS 与 Dijkstra(朴素/heap 优化),都用前驱数组回溯打印路径,解释贪心正确性的直觉(非负权)
- 解释 `float('inf')` 与 `C++ INT_MAX` 两种 INF 方案的差异
- 说出负权边如何破坏 Dijkstra,默写 Bellman-Ford 的松弛与负环检测
- 说出 Prim 与 Dijkstra 的唯一区别(松弛标准),默写 Kruskal(排序 + 并查集)
- 写出迭代版 DFS(显式栈),解释 `visited` 为何必不可少

自测题

Q 1. $V = 10^5$ 、 $E = 2 \times 10^5$ 的稀疏图,邻接矩阵与邻接表各占多少空间?查一条边各是什么复杂度?

✅ 参考答案

矩阵 $O(V^2) \approx 10^{10}$ 格(数十 GB,不可行),查边 $O(1)$;邻接表 $O(V+E) \approx 3 \times 10^5$,查边 $O(\deg(u))$ 平均 $O(E/V)$ 。稀疏大图必须用邻接表。

Q 2. 拓扑排序的队列换成栈,结果还是合法拓扑序吗?为什么环检测不受影响?

✅ 参考答案

仍然合法:入度为 0 的顶点可按任意顺序安排,只要"先修在前"成立即可。环检测只看输出个数是否等于 V ,与出队顺序无关。

Q 3. Dijkstra 朴素版与堆优化版都正确,复杂度为何差一个量级?"懒删除"怎么工作的?

✅ 参考答案

差别在"找当前最小":朴素每轮扫描 $O(V)$,总 $O(V^2)$;堆优化用 `heapq`,堆顶 $O(1)$,总 $O(E \log V)$ 。懒删除:顶点被松弛多次会在堆里留多份记录,出堆时把 $d > \text{dist}[u]$ 的旧记录丢弃——旧记录距离不小于新记录,不影响正确性。

Q 4. 为什么 Bellman-Ford 恰好做 $V-1$ 轮松弛就够?第 V 轮检查到什么说明有负环?

✅ 参考答案

无环路径至多 $V-1$ 条边,第 k 轮后"至多 k 条边"的路径都被考虑过, $V-1$ 轮后全部松弛完毕。若第 V 轮仍有边能松弛,说明存在比" $V-1$ 条边上限"还短的路径——只能是有负环(可无限绕圈变短)。

Q 5. Prim 与 Dijkstra 的代码几乎相同,区别在哪一行?Kruskal 里并查集的 `unite` 返回 `false` 意味着什么?

✅ 参考答案

区别在松弛标准:Prim 的 `key` 是"到已选集合的距离",更新只比较边权(`key[v] > w`);Dijkstra 的 `dist` 是"起点的距离",更新时累加路径(`dist[v] > dist[u] + w`)。Kruskal 中 `unite` 返回 `false` 表示两端点已连通,收下会成环,必须跳过。

章末实验题:迷宫最短路径(BFS + Dijkstra)

📄 实验 9:迷宫最短路径(BFS + Dijkstra)

实验要求:

- 任务 1:读入/内置迷宫并转成图:顶点 = 格子,边 = 四方向可通行相邻格(\$9.1)
- 任务 2:BFS 求 S 到 E 的最短步数路径,前驱数组回溯并打印(\$9.3)
- 任务 3:给地形设通过代价(沼泽 ~ 代价 5),Dijkstra 堆优化求最低代价路径(\$9.4)
- 任务 4:输出两种路径的步数与代价对比,解释为何可能不同(\$9.3 + \$9.4)
- 任务 5:终点不可达时输出"无解"而不崩溃(\$9.8)

实验环境:Python 3。运行: `python lab9.py` (内置)、`python lab9.py sealed` (无解)、`python lab9.py maze.txt` (文件)。

第一步 read_maze 与邻居枚举(越界、墙检查),用 find_pos 求 S、E 坐标

第二步 S 用 deque,prev 字典兼任 visited;到 E 后回溯打印

第三步ijkstra 用 heapq,INF 用 float('inf');懒删除(出堆时 $d > \text{dist}[u]$ 跳过)且不松弛墙

第四步 比行统计两条路径的真实代价,体会"步数最少"与"代价最低"可能是两条路

✔ 参考答案(完整可运行程序,分三段)

```

# lab9.py — 迷宫最短路径(BFS + Dijkstra)
# 运行: python lab9.py [sealed|maze.txt]
import heapq
import sys
from collections import deque

# 内置迷宫:'#' 墙, '.' 平地(代价 1), '~' 沼泽(代价 5), 'S' 起点, 'E' 终点
# 设计: 上走廊短(12 步)但有两格沼泽(代价 21);
#       下走廊长(18 步)却全平地(代价 18)—BFS 选短,Dijkstra 选廉。
MAZE = """\
#####
#S...~..#
#.#.###.#
#.#.###.#
#.#.###.#
#.#.###.#
#.#.###.#
#...##E##
#####
#####\
"""

SEALED = """\
#####
##S##
#.#.###
#.E.###
#####\
"""

COST = {'S': 1, 'E': 1, '.': 1, '~': 5}

def read_maze(arg):
    if arg == "sealed":
        return SEALED.splitlines()
    if arg:
        with open(arg, encoding="utf-8") as f:
            return [line.rstrip("\n") for line in f if line.strip()]
    return MAZE.splitlines()

def find_pos(maze, ch):
    for r, row in enumerate(maze):
        for c, cell in enumerate(row):
            if cell == ch:
                return r, c
    return -1, -1

def neighbors(maze, r, c):
    """任务 1: 网格 -> 图, 四方向邻居即边(隐式图)。"""
    h, w = len(maze), len(maze[0])
    for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        nr, nc = r + dr, c + dc

```

```
if 0 <= nr < h and 0 <= nc < w and maze[nr][nc] != '#':  
    yield nr, nc
```

lab9.py(第二段:BFS 与 Dijkstra 两个算法)

```

def bfs_shortest(maze, s, t):
    """任务 2:无权最短路(BFS)。返回 (步数, 路径);无解返回 None。"""
    q = deque([s])
    prev = {s: None}
    while q:
        u = q.popleft()
        if u == t:
            break
        for v in neighbors(maze, *u):
            if v not in prev:          # 第一次到达即最短
                prev[v] = u
                q.append(v)
    if t not in prev:
        return None                  # 任务 5:终点不可达
    path = []
    x = t
    while x is not None:             # 前驱数组回溯
        path.append(x)
        x = prev[x]
    path.reverse()
    return len(path) - 1, path

def dijkstra_cost(maze, s, t):
    """任务 3:最低代价路径(Dijkstra 堆优化)。返回 (总代价, 路径);无解返回 None。"""
    h, w = len(maze), len(maze[0])
    INF = float('inf')               # Python 用无穷大,无溢出问题
    dist = [[INF] * w for _ in range(h)]
    prev = {}
    dist[s[0]][s[1]] = 0
    pq = [(0, s)]                    # (当前代价, 坐标),小顶堆
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u[0]][u[1]]:
            continue                  # 懒删除:过期记录直接跳过
        if u == t:
            break
        for v in neighbors(maze, *u):
            nd = d + COST[maze[v[0]][v[1]]]
            if nd < dist[v[0]][v[1]]:
                dist[v[0]][v[1]] = nd
                prev[v] = u
                heapq.heappush(pq, (nd, v))
    if dist[t[0]][t[1]] == INF:
        return None
    path = []
    x = t
    while x is not None:
        path.append(x)
        x = prev.get(x)
    path.reverse()
    return dist[t[0]][t[1]], path

```

lab9.py(第三段:主程序,任务 4/5)

```

def draw_path(maze, path):
    """把路径画回迷宫:* 标出经过的格子。"""
    grid = [list(row) for row in maze]
    for r, c in path:
        if grid[r][c] not in 'SE':
            grid[r][c] = '*'
    return [' '.join(row) for row in grid]

def main():
    arg = sys.argv[1] if len(sys.argv) > 1 else ""
    maze = read_maze(arg)
    s, t = find_pos(maze, 'S'), find_pos(maze, 'E')
    if s == (-1, -1) or t == (-1, -1):
        print("迷宫缺少起点 S 或终点 E")
        return
    print(f"迷宫 {len(maze)} 行 x {len(maze[0])} 列,起点 {s},终点 {t}")
    print("图例: # 墙 . 平地(代价1) ~ 沼泽(代价5) S/E 起点/终点")

    res_bfs = bfs_shortest(maze, s, t)
    if res_bfs is None:
        print("BFS : 无解—起点到终点不可达(被墙封死)")
    else:
        steps, path = res_bfs
        print(f"BFS : 最短步数 = {steps}")
        print("  路径:", " -> ".join(f"({r},{c})" for r, c in path))
        for row in draw_path(maze, path):
            print("    ", row)

    res_dij = dijkstra_cost(maze, s, t)
    if res_dij is None:
        print("Dijkstra: 无解")
    else:
        cost, path = res_dij
        print(f"Dijkstra: 最低总代价 = {cost},步数 = {len(path)-1}")
        print("  路径:", " -> ".join(f"({r},{c})" for r, c in path))
        for row in draw_path(maze, path):
            print("    ", row)

    # 任务 4:两种路径对比
    if res_bfs and res_dij:
        steps, path = res_bfs
        cost, path2 = res_dij
        bfs_cost = sum(COST[maze[r][c]] for r, c in path)
        print(f"对比: BFS 最短步数 {steps}(路径代价 {bfs_cost}) vs "
              f"Dijkstra 最低代价路径步数 {len(path2)-1}(代价 {cost})")

if __name__ == "__main__":
    main()

```

示例运行输出(真实运行结果):

```
$ python lab9.py
迷宫 10 行 x 10 列,起点 (1, 1),终点 (7, 7)
BFS : 最短步数 = 12
    路径: (1,1) -> (1,2) -> (1,3) -> (1,4) -> (1,5) -> (1,6) -> (1,7) -> (2,7) -> (3,7) -> (4,7) -> (5,7) -> (6,7) -> (7,7)
Dijkstra: 最低总代价 = 18,步数 = 18
    路径: (1,1) -> (2,1) -> (3,1) -> (4,1) -> (5,1) -> (6,1) -> (7,1) -> (7,2) -> (7,3) -> (7,4) -> (6,4) -> (5,4) -> (4,4) -> (4,5) -> (4,6) -> (4,7) -> (5,7) -> (6,7) -> (7,7)
对比: BFS 最短步数 12(路径代价 21) vs Dijkstra 最低代价路径步数 18(代价 18)

$ python lab9.py sealed
BFS : 无解—起点到终点不可达(被墙封死)
Dijkstra: 无解
```

设计说明:① 迷宫直接以 (行, 列) 元组为顶点,dist 用二维列表,免去编号换算;② "四方向邻居"即边,不显式建邻接表,是"隐式图"代表;③ BFS 与 Dijkstra 共用 prev 回溯,体现 §9.3/§9.4 的算法家族关系;④ INF 用 float('inf'),无溢出问题;⑤ 内置"无解"样例:两算法都检查终点距离,输出提示不崩溃。

下一章预告:第10章 算法设计技巧——贪心、分治、动态规划、回溯四大范式。Dijkstra 与 Prim 的贪心本质将在那里升华。

第10章 算法设计技巧

建议学时:5-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础(已读完第1-9章)

🎯 本章学习目标

- 掌握四大范式:贪心、分治、动态规划、回溯,说出适用场景
- 理解"局部最优 $\neq$ 全局最优",会用反例验证贪心猜想
- 能独立实现活动安排与哈夫曼编码(复用第6章 `heapq`)
- 掌握 DP 两大要素(最优子结构、重叠子问题),独立写出 LCS 与 0-1 背包
- 掌握随机化:洗牌与 `quickselect`,完成 `rand()` $\rightarrow$ `random` 迁移
- 掌握回溯模板"选择-递归-撤销",独立实现八皇后

💡 从 C 到 Python:本章主题如何迁移

前九章学"给定结构,分析复杂度";本章反过来:给定问题,自己设计算法。四大范式是四套"解题姿势":见什么问题套什么姿势。

C 程序员的习惯是"两层循环 + 标志位"式穷举:n 小时没问题,一到 20, $O(2^n)$ 就彻底失败。四种范式改造穷举:贪心剪掉次优分支,分治切出独立小块,DP 缓存重复计算,回溯按需展开搜索。

Python 给本章三样武器:① 闭包与生成器——C 的回调要函数指针加 `void*` (第7章 `qsort`),Python 用嵌套函数捕获外层变量,生成器 `yield` 惰性产出全部排列;② `random` 模块——`rand() % n` 的周期短、概率不均、全局状态三大问题全解决, `random.seed()` 固定种子;③ `random.shuffle`、`sorted`:手写版懂原理,标准库版做工程。

📦 前置知识自查

数学前置:递推式(第2章)、对数、均匀分布直觉。

C 语言前置:`malloc` 二维数组、函数指针(`qsort` 比较器)、递归函数。

依赖的前面章节:第2章(大 O 与分治复杂度)、第6章(二叉堆,哈夫曼用)、第7章(快排分割,`quickselect` 用)。

🚀 本章学习路线

1. 第一阶段(1.5 小时):读 §10.1-§10.2,手写活动安排与哈夫曼建树
2. 第二阶段(1.5 小时):读 §10.3,画 LCS 的 DP 表,默写背包
3. 第三阶段(1.5 小时):读 §10.4-§10.5,验证洗牌均匀性,实现八皇后
4. 第四阶段(实验):完成章末综合实验:背包双版本 + 八皇后 + 计时对比
5. 验收标准:陌生问题 1 分钟内说出"范式、状态、复杂度"

本章知识导图

- §10.1 贪心:活动安排、哈夫曼、硬币反例
- §10.2 分治:最近点对与分治三要素
- §10.3 动态规划:重叠子问题与状态设计(记忆化、LCS、背包)
- §10.4 随机化:洗牌与第 k 小(Fisher-Yates、quickselect)
- §10.5 回溯:选择-递归-撤销(八皇后、子集和)
- §10.6 四大范式对比与选型
- §10.7 工程实践:DP 检查清单与随机化测试

§10.1 贪心:活动安排与哈夫曼编码

10.1.1 贪心思想:每步取局部最优

贪心(greedy) 一句话: **每一步选"当前看起来最好"的选择,希望最终全局最优**。它不回头、不后悔,因此要求"无后效性"。经典案例:找零钱、活动安排、哈夫曼。注意: **贪心正确性必须逐题验证**,没有通用模板——这是它最大的坑(误区 1)。

10.1.2 活动安排问题

会议室只有一个,每个活动有开始与结束时间,同一时刻只能进行一个,问最多能安排几个?策略: **按结束时间最早排序,依次选不冲突的活动——最早结束优先**。例:10 个活动排序后扫描,选中 (1,4)、(5,7)、(8,11),恰为最优。

示例 10-1 活动安排:最早结束优先

```
def activity_select(acts):  
    """活动安排:按结束时间升序,依次选不冲突的活动"""  
    acts.sort(key=lambda a: a[1])      # 贪心策略:最早结束优先  
    res = []  
    last_end = -1  
    for start, end in acts:  
        if start >= last_end:           # 不冲突:直接选  
            res.append((start, end))  
            last_end = end  
    return res
```

10.1.3 为什么"最早结束"是对的

为什么不用"最早开始"或"持续时间最短"?直觉: **结束得越早,留给后面的空闲越多**。严格证明叫**交换论证**(教材给出):把第一个活动换成"结束最早的活动"不会变差——它结束得不晚,后面所有活动仍放得下;逐步替换即可。注意直觉不是证明,后文有反例。

10.1.4 任务调度简介

同族另一个贪心:**单机任务调度**——已知各任务处理时间,使**平均完成时间最小**。策略: **最短处理时间优先(SJF)**:短任务先完成,大家等待的总时间最小;交换论证同样成立。这是操作系统"短作业优先"的原型。复杂度 $O(n \log n)$ 。

10.1.5 哈夫曼编码:前缀码与二叉堆

变长编码:高频字符用短码、低频用长码。但变长有歧义: $a=0, b=01$ 时, 01 是 $a+b$ 还是 b ?靠**前缀码**解决: 任何编码都不是另一个编码的前缀。哈夫曼算法构造**平均码长最短**的前缀码:每次合并频次最小的两棵(频次相加),直到剩一棵;频次越低,离根越远、码越长。取最小用第6章的 `heapq`:

示例 10-2 哈夫曼编码:heapq 建树与编码

```
import heapq

class HNode:
    """哈夫曼结点:叶存字符,内部结点 ch 为 None"""
    def __init__(self, ch, freq):
        self.ch = ch
        self.freq = freq
        self.left = self.right = None

    def is_leaf(self):
        return self.ch is not None

def build_huffman(freq):
    # freq: {字符: 频次}
    pq = []
    # heapq 默认小顶堆,正好符合要求
    for i, (ch, f) in enumerate(freq.items()):
        heapq.heappush(pq, (f, i, HNode(ch, f)))
    nxt = len(pq)
    while len(pq) > 1:
        # 合并频次最小的两棵
        f1, _, a = heapq.heappop(pq)
        f2, _, b = heapq.heappop(pq)
        p = HNode(None, f1 + f2) # 内部结点,频次相加
        p.left, p.right = a, b
        heapq.heappush(pq, (f1 + f2, nxt, p)) # 序号破平局,避免比较结点
        nxt += 1
    return pq[0][2]

def gen_codes(root, path="", codes=None):
    # DFS:左 0 右 1,叶即编码
    if codes is None:
        codes = {}
    if root.is_leaf():
        codes[root.ch] = path
    else:
        gen_codes(root.left, path + "0", codes)
        gen_codes(root.right, path + "1", codes)
    return codes
```

C 的写法(手写最小堆)	Python 的写法(heapq)
数组 + 上滤下滤手写约 60 行	heapq 自带小顶堆,几行完成
每次取最小线性扫描,建树 $O(n^2)$	堆取最小 $O(\log n)$,建树 $O(n \log n)$
比较逻辑写死在代码里	元组 (频次, 序号, 结点) 天然可比,免写比较器
结点手动 <code>free</code> ,易泄漏	对象由引用计数自动回收

重点:为什么前缀码无歧义。哈夫曼编码是**树上的叶编码**:字符对应叶,编码是根到叶的路径。根到叶的路径不经过另一棵叶,任意两叶编码互不为前缀——解码器从左到右扫描,到叶就输出一个字符,绝不卡壳。字符必须全放叶上,放进内部结

点就产生歧义。

⚠ 误区 1:贪心不是万能——硬币反例

硬币面值 {1, 3, 4} 凑 6 元。"尽量用大面值"给出 4+1+1 共 3 枚;最优是 3+3 共 2 枚。每步局部最优(先用 4 元)恰恰堵死了全局最优(两个 3 元)。贪心无通用证明:先想反例,再考虑交换论证;第9章 Dijkstra、Kruskal 敢用贪心,是因有严格证明。

§10.2 分治:最近点对与分治三要素

10.2.1 回顾:分治三要素与复杂度

分治在第2、7章反复出场(最大子序列和、归并、快排),归纳三要素:分(Divide)——切成更小的子问题;治(Conquer)——递归求解;合(Combine)——合并答案。归并与最大子序列和都是 $T(n) = 2T(n/2) + O(n)$:每层 $O(n)$ 、共 $\log_2 n$ 层,总计 $O(n \log n)$;另一形态 $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$ (二分)。遇到 $T(n) = aT(n/b) + f(n)$,画递归树看每层总代价。

🚀 记号说明(沿用第2章)

递推式用行内代码: $T(n) = 2T(n/2) + O(n)$ 表示"两个规模 $n/2$ 的子问题 + 线性合并"; n^2 、 2^n 、 $\log_2 n$ 同第2章;"平均复杂度"指输入等概率分布下的期望值。

10.2.2 最近点对:思想与结论

平面上 n 个点,求距离最近的一对。暴力两两枚举 $O(n^2)$ 。分治:① **分**:按 x 排序从中间切两半;② **治**:递归求两半各自最小距离得 d ;③ **合**(精髓):跨中线的点对 只需检查"距中线 d 以内"的条带——按 y 排序后每点至多与后面 7 个点比较 ($d \times 2d$ 矩形内至多塞下常数个点,证明见教材)。总复杂度 $O(n \log n)$ 。

示例 10-3 最近点对:分治骨架

```
import math

def dist(a, b):
    # 点用二元组 (x, y) 表示
    return math.hypot(a[0] - b[0], a[1] - b[1])

def closest_pair(pts):
    # pts 已按 x 升序;返回最小距离
    n = len(pts)
    if n <= 3:
        # 基准情形:暴力
        return min(dist(pts[i], pts[j])
                    for i in range(n) for j in range(i + 1, n))
    mid = n // 2
    mid_x = pts[mid][0]
    d = min(closest_pair(pts[:mid]), closest_pair(pts[mid:]))
    strip = [p for p in pts if abs(p[0] - mid_x) < d]  # 合:距中线 d 内
    strip.sort(key=lambda p: p[1])                    # 按 y 排序
    for i in range(len(strip)):
        for j in range(i + 1, len(strip)):
            if strip[j][1] - strip[i][1] >= d:
                break
            # y 差已 ≥ d,后面更远
            d = min(d, dist(strip[i], strip[j]))
    return d
```

§10.3 动态规划:重叠子问题与状态设计

10.3.1 分治与 DP:重叠子问题

分治假设子问题**互不相交**。但递归分解常产生**大量相同子问题**:fib(5) 要算 fib(4) 与 fib(3),fib(4) 又要算 fib(3)——fib(3) 被重复算。**动态规划(DP)**:把子问题答案存进表格,用到查表,每个子问题只算一次。分治靠递归树,DP 靠表格,这是本质区别。

10.3.2 斐波那契:记忆化与自底向上

第1章的朴素递归是 $O(2^n)$,fib(50) 算到地老天荒。两种救法:**记忆化**(自顶向下:递归 + 查表)与**自底向上**(从 fib(0) 按序填表),都做到 $O(n)$;后者省掉递归开销,工程首选。

示例 10-4 斐波那契三版

```
def fib_rec(n):          # 朴素递归  $O(2^n)$ (第1章)
    return n if n <= 1 else fib_rec(n - 1) + fib_rec(n - 2)

def fib_memo(n, memo=None): # 记忆化(自顶向下):先查表再递归
    if memo is None:
        memo = {}
    if n in memo:
        return memo[n]
    memo[n] = n if n <= 1 else fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

def fib_iter(n):          # 自底向上:从小到大填表, $O(n)$ 
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

10.3.3 最长公共子序列 LCS

子序列:按顺序挑出(可跳着)的字符序列。**LCS**:两字符串的最长公共子序列。状态: $dp[i][j]$ = s_1 前 i 个与 s_2 前 j 个的 LCS 长度。转移:① 字符相等 $dp[i][j] = dp[i-1][j-1] + 1$;② 否则取 $\max(dp[i-1][j], dp[i][j-1])$ 。填完表从右下角沿来路逆推还原出具体 LCS——**方案回溯**是 DP 常客。例:ABCBDAB 与 BDCABA 得长度 4,LCS 为 "BCBA"(不唯一)。

示例 10-5 LCS:DP 表构造与回溯

```

def lcs_len(s1, s2):
    """返回 LCS 长度并填好 dp 表"""
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i - 1] == s2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1          # 相等:左上+1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]) # 不等:取较大
    return dp

def lcs_trace(s1, s2, dp):
    """回溯:从右下角沿来路逆推,收集 LCS 字符"""
    res = []
    i, j = len(s1), len(s2)
    while i > 0 and j > 0:
        if s1[i - 1] == s2[j - 1]:
            res.append(s1[i - 1])                        # 相等:属于 LCS
            i -= 1
            j -= 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1                                        # 从上方来
        else:
            j -= 1                                        # 从左方来
    return "".join(reversed(res))

```

10.3.4 0-1 背包:二维表与一维优化

0-1 背包: n 件物品,第 i 件重 $w[i]$ 、值 $v[i]$,容量 W ,每件要么整放要么不放,求最大总价值。状态: $dp[i][c]$ = 只用前 i 件、容量 c 的最大价值。转移:放第 i 件得 $dp[i-1][c-w[i-1]] + v[i-1]$,不放得 $dp[i-1][c]$,取最大。二维表 $O(nW)$ 时间空间;一维滚动数组把空间压到 $O(W)$,代价是容量必须逆序循环——正序时 $dp[c-w]$ 已被本轮更新,同一件被放多次(误区 2)。

示例 10-6 0-1 背包:二维与一维

```
def knap2d(w, v, W):
    """二维版:dp[i][c] = 只用前 i 件、容量 c 的最大价值"""
    n = len(w)
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for c in range(1, W + 1):
            if c < w[i - 1]:
                dp[i][c] = dp[i - 1][c]
            else:
                dp[i][c] = max(dp[i - 1][c],
                               dp[i - 1][c - w[i - 1]] + v[i - 1])
    return dp[n][W]

def knap1d(w, v, W):
    """一维版:滚动数组,容量必须逆序!正序变成完全背包"""
    dp = [0] * (W + 1)
    for i in range(len(w)):
        for c in range(W, w[i] - 1, -1): # 逆序是关键
            dp[c] = max(dp[c], dp[c - w[i]] + v[i])
    return dp[W]
```

C 的写法(手工二维数组)	Python 的写法(list 列表)
malloc 每行再逐行 free,易错	<code>[[0] * (W+1) for _ in range(n+1)]</code> 一条语句建表
越界读写是未定义行为	越界抛 <code>IndexError</code> ;调试友好
维度要手把手传参	<code>len(dp)</code> 自带维度,切片与推导式可遍历
换一维滚动数组要重写全部下标	两版结构一致,只差循环方向

⚠ 误区 2:一维背包的容量循环写成正序

一维版要求 `dp[c - w[i]]` 还是"上一轮"的值。逆序保证更新 `dp[c]` 时 `dp[c - w[i]]` 尚未被本轮更新;正序则可能已被更新,第 i 件被用多次——正是完全背包的转移式。口诀:01 背包逆序,完全背包正序。验证:小样例手工跑,或与二维版对拍(实验任务 3)。

★ 重点:DP 解题五步(全章最重要的清单)

① **定义状态**:dp 下标代表什么?要完整描述局面且不冗余;② **写转移方程**:当前状态由哪些更小的状态转移来?③ **定初值与边界**:首行首列、空输入、容量 0 的答案?④ **定计算顺序**:依赖项先算好(通常从小到大);⑤ **方案回溯**:需要方案时记录选择或从终态逆推。

§10.4 随机化:洗牌与第 k 小

10.4.1 Fisher-Yates 洗牌

把数组随机打乱,朴素做法(任意位置交换)分布不均匀。Fisher-Yates 的关键:第 i 步只与 $[0, i]$ 中的随机位置交换(从后往前),交换后位置 i 被锁定。共 $n!$ 种可能交换序列,与排列——对应(`random.shuffle` 就是它)。

示例 10-7 Fisher-Yates 与 `random.shuffle` 并排

```
import random

def fisher_yates(a):
    """手写 Fisher-Yates:从后往前,每位置与 [0, i] 中随机位置交换"""
    for i in range(len(a) - 1, 0, -1):
        j = random.randint(0, i)      # 关键:上界是 i!
        a[i], a[j] = a[j], a[i]
    return a

def shuffle_std(a):
    """标准库写法:一行完成,内部就是 Fisher-Yates"""
    random.shuffle(a)
    return a
```

⚠ 误区 3:洗牌上界取 $[0, n-1]$

若每步从 n 个位置随机挑,共 n^n 种等可能序列,排列只有 $n!$ 种—— n^n 不被 $n!$ 整除,必有不均:如 $n=3$ 时 $(1,2,3)$ 概率 $2/9$ 。Fisher-Yates 每步可选数递减($1 \cdot 2 \cdot \dots \cdot n = n!$),——对应。实证:洗 10 万次统计排列分布。

10.4.2 quickselect:求第 k 小

求第 k 小(0 基)。排序后取第 k 位要 $O(n \log n)$ 。**quickselect** 复用第7章快排的**分割(partition)**:枢轴被放到最终位置 p ; $p == k$ 直接返回,否则只递归"包含第 k 小"的那一侧。期望代价 $n + n/2 + n/4 + \dots = 2n$,平均 $O(n)$;最坏(每次切最边上) $O(n^2)$ 。

示例 10-8 quickselect 与 sorted 对照

```
def partition(a, lo, hi):
    """快排分割:枢轴放到最终位置,返回下标"""
    pivot = a[hi]                # 简化:取末元素做枢轴
    i = lo
    for j in range(lo, hi):
        if a[j] < pivot:
            a[i], a[j] = a[j], a[i]
            i += 1
    a[i], a[hi] = a[hi], a[i]
    return i

def quick_select(a, k):
    """第 k 小(0 基):分割后只处理"包含第 k 小"的那一侧"""
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        p = partition(a, lo, hi)
        if p == k:
            return a[k]
        if p > k:
            hi = p - 1
        else:
            lo = p + 1
    return -1

# 标准库对照:sorted(a)[k] 一行求第 k 小,但代价是  $O(n \log n)$ 
```

10.4.3 随机数:从 C 的 rand() 到现代随机库

C 的 rand()/srand() 三个硬伤: ① 周期短(约 2^{31});② rand() % n 概率不均——n 不整除 RAND_MAX 时余数偏斜, 低位周期短;③ 全局状态,线程不安全、不可复制。Python random 模块基于梅森旋转 MT19937,周期 $2^{19937}-1$; random.seed(seed) 固定种子则序列可复现,是复现性的基石;引擎与分布不必手动管理,全局唯一,直接调用即可。randint(a, b) 是闭区间 [a, b],和 C 的 [0, n) 习惯不同,注意别多 1。

C 的写法(rand 家族)	Python 的写法(random 模块)
srand(time(NULL)); rand();	random.seed(2026); random.random()
rand() % n 概率不均	random.randint(a, b) 闭区间均匀整数
全局状态,线程不安全,不可复制	全局单例,自动播种,线程安全有保证
周期约 2^{31} ,质量差	MT19937 周期 $2^{19937}-1$,通过统计检验
洗牌/抽样要自己写	random.shuffle / random.sample 一行完成

10.4.4 随机算法的工程观

quickselect 平均 $O(n)$ 、最坏 $O(n^2)$ ——"数学概率上快"的含义: 把最坏情形的概率压到工程上可忽略。随机选枢轴后,能触发最坏的输入几乎无法被构造;对单次最坏延迟有硬性要求(实时系统)则改用确定性算法。

§10.5 回溯:选择-递归-撤销

10.5.1 回溯模板:选择-递归-撤销

当搜索空间是"一步步做决定"的结构时,回溯(backtracking) 是穷举的优雅写法,核心三行—— ① 选择:决定写进状态;② 递归:进入下一层;③ 撤销:恢复状态,尝试下一个选择。撤销是灵魂,忘了它状态被污染。回溯的本质是 带剪枝的深度优先穷举:排列空间 $n!$,子集空间 2^n ;八皇后不剪枝要试 64 选 8,剪枝后只访问约 1.5 万结点。剪枝两类:可行性剪枝(不可能通向合法解)与最优性剪枝(不可能优于已知最好解)—— 回溯"慢但保证对",贪心"快但可能错"。Python 里递归函数用嵌套函数(闭包)捕获外层变量;枚举全排列用生成器 yield,逐个产出不占内存:

示例 10-9 全排列:yield 生成器

```
def perms(a):
    """yield 版全排列:每层选一个元素,惰性产出每种排列"""
    if len(a) <= 1:
        yield a[:]
        return
    for i in range(len(a)):
        rest = a[:i] + a[i + 1:]          # 选第 i 个,剩余部分递归
        for p in perms(rest):
            yield [a[i]] + p              # 产出一种排列

# 用法:for p in perms([1, 2, 3]): print(p)    # 共 6 行
# 标准库对照:itertools.permutations([1, 2, 3]) 也是生成器,按字典序
# 手写版懂原理(yield 的惰性),标准库版做工程
```


10.5.2 八皇后:完整实现

在 $n \times n$ 棋盘放 n 个皇后,任意两个不能同行、同列、同对角线。暴力枚举 $C(64,8)$ 是天文数字;回溯逐行放皇后, $O(1)$ 冲突检查:列 $col[c]$;主对角线 $d1[row+c]$ (同一主对角线上 $row+c$ 相同);副对角线 $d2[row-c+n-1]$ ($row-c$ 相同,加 $n-1$ 防负下标)。放不下就回退换列——"回溯"由此得名。Python 版把"搜索"与"结果处理"解耦:闭包 dfs 收集到列表,入解必须 `pos[:]` 拷贝,否则后续撤销会污染已收集的解。

示例 10-10 八皇后:闭包与冲突检查

```
def n_queens(n):
    """八皇后:返回全部解的列号列表(每行一个皇后所在列)"""
    pos = [0] * n
    col = [False] * n
    d1 = [False] * (2 * n - 1)
    d2 = [False] * (2 * n - 1)
    sols = []

    def dfs(row):
        if row == n:
            sols.append(pos[:])          # 收集解(拷贝,防撤销污染)
            return
        for c in range(n):
            if col[c] or d1[row + c] or d2[row - c + n - 1]:
                continue                # 冲突剪枝
            pos[row] = c
            col[c] = d1[row + c] = d2[row - c + n - 1] = True  # 选择
            dfs(row + 1)                 # 递归
            col[c] = d1[row + c] = d2[row - c + n - 1] = False # 撤销

    dfs(0)
    return sols

# 结果:len(n_queens(8)) == 92;前 3 解(1 基):(1,5,8,6,3,7,2,4)、
# (1,6,8,3,7,4,2,5)、(1,7,4,6,8,2,5,3)
```

10.5.3 子集和问题

给定正整数集合与 target,能否选出若干元素恰好和为 target?每件"选/不选"两个分支,走到叶判断命中。加**可行性剪枝**:已超 target 直接剪掉;总和都不够则不进搜索。子集和没有贪心可依赖(硬币反例同款),只能搜索——这就是范式选型的意义。

§10.6 四大范式对比与选型

10.6.1 四范式大表

范式	核心特征	适用场景	典型例题
贪心	每步局部最优;无后效性;正确性需证明	能构造策略且验证正确	活动安排、哈夫曼、Dijkstra
分治	子问题互不相交;分-治-合;递推式	能切成独立子问题	归并、快排、最近点对
动态规划	子问题重叠;最优子结构;状态与转移	有重叠子问题与最优子结构	斐波那契、LCS、0-1 背包
回溯	DFS 搜索;选择-递归-撤销;剪枝	无多项式思路,规模 ≤ 20	八皇后、子集和、数独

10.6.2 选型决策

- **先看规模**: $n \leq 20 \rightarrow$ 回溯; n 达 $10^6 \rightarrow$ 多项式算法
- **有贪心猜想?** 策略可构造且反例想不出 $\rightarrow$ 试贪心并证明; 有反例 $\rightarrow$ 放弃
- **能切成独立子问题?** 左右互不影响 $\rightarrow$ 分治
- **有重复子问题?** $\rightarrow$ DP: 定义状态、写转移、填表
- **都不行?** $\rightarrow$ 回溯 + 剪枝, 或近似/启发式

10.6.3 什么时候"想不出多项式算法": NP 问题的一句话

旅行商、哈密顿回路等问题至今**没人找到**多项式时间算法, 普遍相信不存在——通常称 **NP 难**。遇到它们, 心态不是"我再想想", 而是: 规模小用回溯 + 剪枝, 规模大用近似或启发式, 并明确告诉用户这是近似解。

§10.7 工程实践: 状态检查、剪枝与随机化测试

10.7.1 DP 状态设计检查清单

- 状态参数能否完整描述"当前局面"? 少一个参数转移就丢信息
- 转移方程引用的状态都"更小"吗? (防循环依赖)
- 初始值与边界是否齐全? (首行首列、空输入、容量 0)
- 计算顺序是否保证依赖项先算好? (背包容量、LCS 行列)
- 空间能否优化? 二维表能否滚成一维?
- 需要输出方案吗? 要就保留完整表或记录选择

10.7.2 回溯剪枝的顺序

剪枝顺序影响常数: ① 可行性剪枝放最前 (冲突检查、超目标)——砍掉的子树最多; ② 对称性剪枝 (八皇后旋转/镜像省约 7/8 搜索, 选做); ③ 最优性剪枝最后做 (求最优解时用)。优先搜索分支更少的方向 (如子集和先放大元素), 也能显著加速。

10.7.3 随机化在测试中的应用

两个利器: ① **随机数据 + 对拍**——用 `random.seed()` 生成随机小样例, 把"手写实现"与"暴力实现"逐一比对, 十万个随机样例比十个手写边界更早暴露 bug (实验任务 3); ② **固定种子**——回归可复现, 换种子扩覆盖。

本章复杂度速查表

操作 / 算法	数据结构 / 算法	平均	最坏
活动安排(最早结束优先)	排序 + 线性扫描	$O(n \log n)$	$O(n \log n)$
任务调度(最短处理时间优先)	排序 + 线性扫描	$O(n \log n)$	$O(n \log n)$
哈夫曼建树	二叉堆(heapq)	$O(n \log n)$	$O(n \log n)$
最近点对	分治(条带合并)	$O(n \log n)$	$O(n \log n)$
最大子序列和	分治(第2章算法三)	$O(n \log n)$	$O(n \log n)$
归并排序	分治	$O(n \log n)$	$O(n \log n)$
快速排序	分治 + 随机化枢轴	$O(n \log n)$	$O(n^2)$
斐波那契	朴素递归	$O(2^n)$	$O(2^n)$
斐波那契	记忆化 / 自底向上	$O(n)$	$O(n)$
LCS	动态规划	$O(mn)$	$O(mn)$
0-1 背包	动态规划(二维或一维)	$O(nW)$	$O(nW)$
洗牌	Fisher-Yates / random.shuffle	$O(n)$	$O(n)$
第 k 小	quickselect / sorted	$O(n)$	$O(n^2)$
八皇后	回溯(冲突剪枝)	上界 $O(n!)$	上界 $O(n!)$
子集和	回溯(可行性剪枝)	$O(2^n)$	$O(2^n)$

最小必会清单

- 能说出贪心思想与"无后效性",举出活动安排、哈夫曼案例与硬币反例
- 能默写活动安排,说出复杂度 $O(n \log n)$
- 能默写哈夫曼建树(heapq 最小堆),解释前缀码解码无歧义
- 能说出分治三要素与最近点对合并思路,写出 $T(n) = 2T(n/2) + O(n)$
- 能说出分治与 DP 的本质区别(子问题独立 vs 重叠)与记忆化、自底向上
- 能独立写出 LCS 的 DP 表与回溯代码,解释 $dp[i][j]$ 的含义
- 能默写 0-1 背包两版,解释一维为何必须逆序
- 能默写 Fisher-Yates,说出"上界必须是 i ";会用 quickselect
- 能默写回溯模板"选择-递归-撤销",独立实现八皇后
- 会用 random.seed / random.randint / random.shuffle / sorted

自测题

- Q** 1. 活动安排为何按"结束时间最早"贪心?"开始时间最早"行吗?

✓ 参考答案

结束得越早,给后面留下的时间越多;交换论证保证任何最优解都能换成贪心解。按"最早开始"不行:活动 A(1,10) 霸占全天,(2,3)(4,5)(6,7) 全被挡住——贪心得 1 个,最优 3 个。

Q 2. 面值 {1,3,4} 凑 6 元,贪心给出什么?最优呢?说明什么?

✓ 参考答案

贪心 4+1+1 共 3 枚,最优 3+3 共 2 枚。局部最优 $\neq$ 全局最优:先用 4 元堵死了两个 3 元。贪心正确性必须逐题验证;人民币面值贪心成立,是"设计面值"的结果。

Q 3. 分治与动态规划的本质区别是什么?各举一个代表问题。

✓ 参考答案

分治的子问题互不相交(归并左半右半无重叠),复杂度来自递归树每层合并代价;DP 的子问题大量重叠(fib(3) 被重复计算),复杂度来自"每个状态只算一次"。代表:分治——归并;DP——LCS、0-1 背包。

Q 4. 0-1 背包的一维滚动数组为什么必须逆序?正序会怎样?

✓ 参考答案

逆序保证更新 $dp[c]$ 时 $dp[c-w[i]]$ 还是"没考虑第 i 件"的旧值;正序时它可能已被本轮更新,同一件物品被用多次,转移式变成完全背包。口诀:01 背包逆序,完全背包正序。验证:与二维版对拍(实验任务 3)。

Q 5. Fisher-Yates 每步的随机上界为什么必须是 i ?取 $[0, n-1]$ 会怎样?

✓ 参考答案

Fisher-Yates 每步可选数递减,共 $n!$ 种等可能交换序列,与排列一一对应,保证均匀。若每步从 n 个位置挑,共 n^n 种序列而排列只有 $n!$ 种, n^n 不被 $n!$ 整除,必然不均($n=3$ 时 (1,2,3) 概率 2/9)。实证:洗 10 万次统计分布。

章末实验题:0-1 背包与八皇后综合实验

实验 10:0-1 背包与八皇后综合实验

实验目标:综合运用动态规划与回溯两大范式:背包两版实现与验证、八皇后、计时对比。

实验要求:

- 任务 1:实现 knap2d 与 knap1d,均返回最大价值与方案下标(\$10.3.4 二维与一维、\$10.3.3 方案回溯)
- 任务 2:实现八皇后,输出解的个数与前 3 个解(\$10.5.2 选择-递归-撤销、冲突检查)
- 任务 3:固定种子随机数据($n=20$, $W=200$)验证两版价值与方案一致(\$10.7.3 随机化测试与对拍)
- 任务 4:计时对比一维与二维($n=1000$ 、 $W=10000$),输出耗时与加速比(\$2.1 计时、\$10.3.4 空间优化)
- 任务 5:输入边界——容量 0、无物品时输出 0 而非崩溃(\$10.3.3 边界与初值)

实验环境:Python 3。运行: `python lab10.py`。

- 第一步** ;写 knap2d(二维表 + 回溯选取),再写 knap1d(一维逆序 + pick 表);两版判定相同:dp 值"严格变大"即选中
- 第二步** .皇后复用示例 10-10 骨架:三个标记数组 + 闭包收集;输出时列号加 1 转成 1 基
- 第三步** _机数据用 random.seed(2026),权重 1-50、价值 1-100;同时比较价值与下标序列
- 第四步** ;小数据确认正确,再计时(n=1000、W=10000,结果累加防死代码消除);边界调用 knap1d(3,0,...) 与 knap1d(0,10,...) 确认返回 0

✅ 参考答案(完整可运行程序)

```
# lab10.py(第一段:背包两版)
import random
import time

# 任务 1a:二维 DP;返回 {价值, 方案下标}
def knap2d(n, W, w, v):
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for c in range(1, W + 1):
            if c >= w[i - 1]:
                dp[i][c] = max(dp[i - 1][c],
                               dp[i - 1][c - w[i - 1]] + v[i - 1])
            else:
                dp[i][c] = dp[i - 1][c]
    chosen = [] # 回溯选取方案
    i, c = n, W
    while i >= 1:
        if dp[i][c] != dp[i - 1][c]:
            chosen.append(i - 1)
            c -= w[i - 1]
        i -= 1
    chosen.reverse()
    return dp[n][W], chosen

# 任务 1b:一维滚动数组(容量必须逆序!)
def knap1d(n, W, w, v):
    dp = [0] * (W + 1)
    pick = [[False] * (W + 1) for _ in range(n)]
    for i in range(n):
        for c in range(W, w[i] - 1, -1):
            t = dp[c - w[i]] + v[i]
            if t > dp[c]:
                dp[c] = t
                pick[i][c] = True
    chosen = []
    i, c = n - 1, W
    while i >= 0:
        if pick[i][c]:
            chosen.append(i)
            c -= w[i]
        i -= 1
    chosen.reverse()
    return dp[W], chosen
```

lab10.py(第二段:八皇后与主程序)

```

# 任务 2:八皇后(闭包回调)
queens_count = 0
first_sol = []

def n_queens(row, n, pos, col, d1, d2, visit):
    if row == n:
        visit(pos)
        return
    for c in range(n):
        if col[c] or d1[row + c] or d2[row - c + n - 1]:
            continue
        pos[row] = c
        col[c] = d1[row + c] = d2[row - c + n - 1] = True # 选择
        n_queens(row + 1, n, pos, col, d1, d2, visit) # 递归
        col[c] = d1[row + c] = d2[row - c + n - 1] = False # 撤销

if __name__ == "__main__":
    random.seed(2026) # 固定种子,可复现
    # 任务 3:随机数据验证两版
    n, W = 20, 200
    w = [random.randint(1, 50) for _ in range(n)]
    v = [random.randint(1, 100) for _ in range(n)]
    r2 = knap2d(n, W, w, v)
    r1 = knap1d(n, W, w, v)
    print("随机数据 n=%d W=%d 价值: 二维=%d 一维=%d 方案一致: %s"
          % (n, W, r2[0], r1[0], "是" if r2 == r1 else "否"))
    # 任务 4:计时对比
    n, W = 1000, 10000
    w = [random.randint(1, 50) for _ in range(n)]
    v = [random.randint(1, 100) for _ in range(n)]
    t0 = time.perf_counter()
    big2 = knap2d(n, W, w, v) # 结果参与打印,防死代码消除
    ms2 = (time.perf_counter() - t0) * 1000
    t0 = time.perf_counter()
    big1 = knap1d(n, W, w, v)
    ms1 = (time.perf_counter() - t0) * 1000
    print("计时 n=%d W=%d 二维=%.1f ms 一维=%.1f ms 加速比=%.2f"
          % (n, W, ms2, ms1, ms2 / ms1))
    print("大样例价值一致: %s" % ("是" if big2[0] == big1[0] else "否"))
    # 八皇后(前 3 个解 + 总数)
    pos = [0] * 8
    col = [False] * 8
    d1 = [False] * 15
    d2 = [False] * 15

    def visit(sol):
        global queens_count
        queens_count += 1
        if len(first_sol) < 3:
            first_sol.append(sol[:]) # 拷贝,防撤销污染

    n_queens(0, 8, pos, col, d1, d2, visit)
    print("八皇后解的总数: %d" % queens_count)
    for i, sol in enumerate(first_sol, 1):
        print("解 %d: %s" % (i, " ".join(str(x + 1) for x in sol)))

```

```
# 任务 5:边界
wb = [2, 3, 4]
vb = [3, 4, 5]
print("容量 0: %d 无物品: %d" % (knap1d(3, 0, wb, vb)[0],
                                   knap1d(0, 10, wb, vb)[0]))
```

示例输出(本机真实运行;时间因机器而异):

```
随机数据 n=20 W=200 价值: 二维=469 一维=469 方案一致: 是
计时 n=1000 W=10000 二维=1298.0 ms 一维=543.5 ms 加速比=2.39
大样例价值一致: 是
八皇后解的总数: 92
解 1: 1 5 8 6 3 7 2 4
解 2: 1 6 8 3 7 4 2 5
解 3: 1 7 4 6 8 2 5 3
容量 0: 0 无物品: 0
```

设计说明:① 两版共用"dp 值严格变大即选中"的判定,同时比较价值与下标序列,比只比价值更严格;② 一维版用 bool 型 pick 表存选择历史,体积约为二维表的四分之一;计时中一维因缓存友好更快;③ 八皇后用闭包 visit 解耦"搜索"与"结果处理"——正是 §10.5 的闭包模式;④ 固定种子 2026 保证任意机器复现相同数据;⑤ 边界任务验证"初值与边界"检查项:容量 0 与无物品时转移自然失效,输出 0;⑥ 大样例下 Python 二维表的内存与耗时显著高于 C++(对象开销),但两版加速比结构一致,结论不受影响。

下一章预告:第11章 摊还分析——聚集法、会计法、势能法。第3章"list 倍增扩容为何平均 $O(1)$ "的悬念将在那里严格证明;本章的哈夫曼建堆、quickselect 的整体代价也会用摊还视角重新审视。

第11章 摊还分析

建议学时:4-5 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 分清三种"平均":最坏、平均、**摊还**——本章主题是第三种
- 掌握**聚集分析**(栈序列例证)与**会计方法**(动态数组"存 3 元"完整证明)
- 掌握**势能方法**:用 Φ 重推栈,理解通用框架与选 Φ 的直觉
- 会用摊还眼光看第 6 章的堆:buildHeap 为何 $O(n)$,斜堆 merge 为何摊还 $O(\log n)$
- 完成从 C 的 `realloc` 手动扩容到 Python `list.append` 的迁移
- 能独立实测 `list` 扩容拐点,验证"总拷贝 $\approx 2n$ "

💡 从 C 到 Python:本章主题如何迁移

本章是全书数学最重的一章,动机却非常"工程"。回忆 C 里的动态数组: `realloc` 扩容要么每次多开固定格数,要么自己"翻倍"。Python 把这层底细藏得更深: `list.append` 内部自动倍增,你连 capacity 都看不到——但摊还分析对两种语言同样适用:倍增使 n 次插入总拷贝 $\approx 2n$,摊到每次平均 $O(1)$ 。

Python 的 `list` 把这套证明封装进语言: `append` 平均 $O(1)$ 、最坏 $O(n)$ 。C 里 `realloc` 后指针失效的坑,在 Python 里没有对应物: `list` 存的是对象指针,扩容只是搬运指针,你手里的引用永远有效;真正对应的是"遍历 `list` 时插入/删除会错位"。

数学工具不新: Σ 求和与等比级数(第 2 章)、堆与斜堆(第 6 章)。新的是**视角**:不看单个操作,看整个序列的"总账"——聚集、会计、势能,是同一本总账的三种记法。

📦 前置知识自查

数学前置:求和记号 Σ 、等差与等比级数: $\Sigma_{i=1}^n i = n(n+1)/2$, $\Sigma_{i=0}^n 2^i = 2^{n+1} - 1$;对数 $\log_2 n$ (见第 2 章)。

C 语言前置: `realloc` 手动扩容动态数组(第 3 章)、函数与循环。

依赖的前面章节:第 3 章 `list` 倍增扩容与内存增长;第 6 章 buildHeap、斜堆 merge 与二项队列(\$11.5 的结论建立在它们结构上)。

🚀 本章学习路线

- 第一阶段(1.5 小时)**:读 \$11.1–\$11.2,建立"序列总账"直觉
- 第二阶段(1.5 小时)**:读 \$11.3–\$11.4,吃透"存 3 元"证明与势能框架
- 第三阶段(2 小时)**:读 \$11.5–\$11.6,把结论接到第 6 章的堆上;运行代码观察拐点,完成章末实验
- 验收标准**:能手写证明"倍增扩容总拷贝 $\approx 2n$ ",说清三种方法的区别与联系

本章知识导图

- §11.1 动机:摊还 vs 平均 vs 最坏
- §11.2 聚集分析:栈操作序列
- §11.3 会计方法:动态数组的"存 3 元"证明
- §11.4 势能方法:势能函数与通用框架
- §11.5 摊还在堆上的应用
- §11.6 工程印证:实测扩容拐点
- §11.7 三种方法的选用总结

§11.1 动机:摊还 vs 平均 vs 最坏

11.1.1 三种"平均"不是一回事

前几章我们反复说"list 尾部插入平均 $O(1)$ 、最坏 $O(n)$ "。这个"平均"是什么意思?复杂度分析里有三种不同的说法,极易混为一谈:

说法	对什么取平均	需要什么假设	例子
最坏情形	不取平均,取最差输入	无假设,对所有输入成立	快排最坏 $O(n^2)$
平均情形	对 输入 的概率分布取平均	必须假设输入分布(如"等可能")	快排平均 $O(n \log n)$
摊还	对 操作序列 取平均	无概率假设,对 任何 序列成立	n 次 list.append 总代价 $O(n)$

摊还分析:数据结构上**任意**一个由 n 个操作组成的序列,若总代价为 $O(n \cdot f(n))$,就说每个操作**摊还**代价为 $O(f(n))$ 。它不假设任何概率分布,对最恶劣的序列也成立。

11.1.2 为什么需要摊还分析

回头看前几章,至少有三个结论是"背下来"而没证明的:

- **第 3 章**:list 倍增扩容使 append "平均 $O(1)$ "——没证明,也没说清是摊还;
- **第 6 章**:buildHeap 是 $O(n)$ ——逐个 insert 明明 $O(n \log n)$,整体下滤为何线性?
- **第 6 章**:斜堆 merge 摊还 $O(\log n)$ ——单个 merge 最坏 $O(n)$,教材直接给结论;

三个结论分别对应本章的三种方法:聚集证 buildHeap,会计证动态数组,势能证斜堆——学完本章,"背诵"变"推导"。摊还分析是算法设计的"定价"工具:一次操作贵没关系,整个序列便宜就行。

⚠ 误区 1:把摊还 $O(1)$ 当单次 $O(1)$

摊还 $O(1)$ 的意思是" n 次操作总代价 $O(n)$ ",**不保证单次操作是 $O(1)$** 。list 那次触发扩容的 append,单次代价仍是 $O(n)$ ——只是被前面的便宜 append 平摊了。同理,"斜堆 merge 摊还 $O(\log n)$ "不代表某一次 merge 不会花 $O(n)$ 。实时系统里单次延迟上限有时更关键,这时需要第 12 章平衡树那种"每次最坏 $O(\log n)$ "的结构。

11.1.3 摊还代价的定义

设操作序列共 n 个操作,第 i 个操作的实际代价记为 c_i ,总代价 $T(n) = \sum_{i=1}^n c_i$ 。若对**任意**长度为 n 的合法序列都有 $T(n) = O(n \cdot f(n))$,则称每个操作的**摊还代价**为 $O(f(n))$ 。三种方法回答同一问题——"怎么证 $T(n)$ 的上界"——只是记账

方式不同:

方法	一句话	数学工具
聚集分析	先算总账,再平均	Σ 求和
会计方法	每个操作预存,贵的取款	账户余额非负
势能方法	把"存款"记在数据结构状态上	势能函数 Φ 的差

 记号说明

本章公式密集,记号统一: c_i 第 i 个操作的**实际代价**, a_i 其**摊还代价**; $\Phi(D_i)$ 第 i 个操作结束后结构的**势能**(读"Phi"), $\Delta\Phi = \Phi(D_i) - \Phi(D_{i-1})$ 。

§11.2 聚集分析:栈操作序列

11.2.1 方法:先求和,再平均

聚集分析最直接:把 n 个操作的实际代价**全部加起来**得总代价 $T(n)$,除以 n 即摊还代价。难点只在"证总代价上界"——通常靠巧妙的计数观察,而非逐个求和。

11.2.2 经典例子:栈的 push/pop/multiPop

考虑一个从空栈开始的栈,三个操作:push(代价 1)、pop(代价 1)、multiPop(k)(连续弹出 k 个元素,不够时弹到空,代价 = 实际弹出个数,最坏 $O(n)$)。问:任意 n 个操作,总代价是多少?

直接求和会卡住:某次 multiPop 单次就花 $O(n)$,但序列里不可能全是 multiPop——弹出的元素**必须先被 push 进去**,而 n 个操作最多 push n 次。 **关键观察:每个元素至多入栈一次、至多出栈一次**,于是:

出栈总次数 $\leq$ 入栈总次数 $\leq n$,总代价 $\leq 2n$,摊还代价 $O(1)$ 。证明没做任何输入假设: **任何** 长度为 n 的序列总代价都 $\leq 2n$ 。若问"全是 multiPop 怎么办?"——这种序列根本不存在:栈空了就没得弹,这个"不可能"正是证明核心。

11.2.3 代码验证:总代价 $\leq 2n$

示例 11-1 统计栈操作序列的实际总代价

```
# 模拟 push/pop/multiPop 序列,统计总代价,验证摊还结论
def multiPop(st, k):
    cost = 0
    while st and cost < k:
        st.pop()
        cost += 1
    return cost

n = 10000
st, total = [], 0
for i in range(n):
    st.append(i)                # push:代价 1
    total += 1
    if i % 1000 == 999:         # 每 1000 次 push 来一个大 multiPop
        total += multiPop(st, n) # 弹空:每个元素只出栈一次
print(f"操作数 n = {n}, 总代价 = {total} ( $\leq 2n = \{2*n\}$ ), 摊还 = {total/n}")
# 输出:操作数 n = 10000, 总代价 = 20000 ( $\leq 2n = 20000$ ), 摊还 = 2.0
```

§11.3 会计方法:动态数组的"存 3 元"证明

11.3.1 方法:预存存款

会计方法把摊还代价看作"收费":每个操作收摊还代价 a_i 元,其中 c_i 元支付该操作本身, $a_i - c_i$ 元存入**账户**,留给未来昂贵操作取用。唯一要求是:

任何时刻账户余额非负:对每个前缀 $\sum_{i=1}^k (a_i - c_i) \geq 0$ 。

余额非负 $\Rightarrow$ 累加得 $\sum_{i=1}^n a_i \geq \sum_{i=1}^n c_i$,总摊还代价是总实际代价的**上界**。故每个操作收费 $O(1) \Rightarrow$ 总实际代价 $O(n)$ 。

11.3.2 动态数组:插入收 3 元

回到第 3 章的动态数组:append 平时只写一个元素(实际代价 1);容量满时倍增扩容要拷贝全部旧元素(实际代价 $O(n)$)。CPython 的 list 也是同一机制——扩容搬的是对象指针,代价照旧按"拷了几个元素"算。定价 **3 元**:

- 1 元:支付本次写入;
- 2 元:存入账户,留给"未来某次扩容时本元素的拷贝";

扩容时每拷贝一个旧元素从账户支付 1 元。直觉:设扩容前容量为 m (已满),拷贝费 m 元;这 m 个元素中后 $m/2$ 个是**上次**扩容后才插入的,每个带 2 元存款,合计恰好 m 元;前 $m/2$ 个"老元素"的存款原封留存。归纳下去,账户永不为负。

11.3.3 存款非负的 Σ 推导

严格版本用总账。设元素个数 s 、容量 $c = 2^k$,累计扩容拷贝费是等比级数: $\sum_{j=0}^{k-1} 2^j = c - 1$ 。总收费 $3s$,总花费 s (写入) + $(c - 1)$ (拷贝),余额 $= 3s - s - (c - 1) = 2s - c + 1$ 。数组满才扩容,刚扩完 $s = c/2$,故 $s \geq c/2$,余额 $\geq 1 > 0$ 。

余额恒正 $\Rightarrow$ 总摊还 $\geq$ 总实际 $\Rightarrow$ 总代价 $\leq 3n$,摊还 $O(1)$ 。证明只需"容量按常数因子增长",1.5 倍、1.125 倍都行;唯独"每次 +1"不行(摊还 $O(n)$)。

C 的 realloc 手动扩容	Python 的 list.append
realloc 扩容,size 变化自己记账	append 自动扩容,内部维护 size/capacity,可用 <code>sys.getsizeof</code> 观察
扩容策略自己写,每次 +1 就退化成 $O(n^2)$	倍增(约 1.125 倍)内建,摊还 $O(1)$ 由 CPython 保证
失败返回 NULL,旧指针要自己处理	分配失败抛 <code>MemoryError</code> ,旧对象不受影响
扩容后旧指针全部失效	list 存对象指针,扩容只搬指针,已有引用仍有效

11.3.4 代码验证:账户余额永不为负

示例 11-2 会计方法视角的"存 3 元"动态数组

每次 push 收 3 元(1 元自用 + 2 元存款),扩容拷贝从账户支付。

```
class BankArray:
    def __init__(self):
        self._buf = [None] * 1
        self._size = 0
        self._cap = 1
        self.balance = 0          # 账户余额
        self.min_balance = float("inf") # 全程最低余额
    def push_back(self, x):
        self.balance += 3         # 收 3 元
        self.balance -= 1         # 支付本次写入
        if self._size == self._cap: # 容量满:倍增
            new_buf = [None] * (self._cap * 2)
            for i in range(self._size):
                new_buf[i] = self._buf[i]
                self.balance -= 1      # 每拷 1 个付 1 元
            self._buf = new_buf
            self._cap *= 2
        self._buf[self._size] = x
        self._size += 1
        if self.balance < self.min_balance:
            self.min_balance = self.balance
```

§11.4 势能方法:势能函数与通用框架

11.4.1 方法:势能函数 Φ

势能方法把会计方法的"账户"从具体元素上抽象出来,记在**数据结构的状态**上。定义势能函数 $\Phi(D)$ 为状态 D 的一个数值属性(如"栈中元素个数");设 D_i 为第 i 个操作结束后的状态, D_0 为初始状态,则第 i 个操作的摊还代价定义为:

$a_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$,即 **摊还代价 = 实际代价 + 势能变化 $\Delta\Phi$** 。

两边求和,中间项错位相消(望远镜求和): $\sum_{i=1}^n a_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0)$ 。

只要 $\Phi(D_0) = 0$ 且 $\Phi(D_i) \geq 0$ 对所有 i 成立,就有 $\sum a_i \geq \sum c_i$,总摊还是总实际代价的上界。与会计方法唯一区别:会计把存款绑在具体元素上,势能把它绑在整个结构的状态上——会计方法是势能方法的特例。

11.4.2 栈的例子重推

取 $\Phi(D)$ = 栈中元素个数, $\Phi(D_0) = 0$,且元素个数永不为负,条件满足。逐个操作算摊还代价:

- **push**:实际 1, $\Delta\Phi = +1$,摊还 = 2;
- **pop**:实际 1, $\Delta\Phi = -1$,摊还 = 0;
- **multiPop(k)**:弹出 k 个,实际 k , $\Delta\Phi = -k$,摊还 = 0;

总摊还 = $2 \times \text{push 次数} \leq 2n$,与聚集分析一致。"贵的"multiPop 摊还代价是 0——它消耗的正是 push 积累的势能。

⚠ 误区 2:把平均情形分析当摊还分析

"快排平均 $O(n \log n)$ "与"list 插入摊还 $O(1)$ "是两回事:前者对**输入排列**取平均,前提是"输入等可能",数据分布一变结论就失效;后者对**操作序列**取平均,对任何序列都成立——攻击者精心构造"append 到满再 append"的序列,总拷贝也 $\leq 2n$ 。口诀:"随机""等可能""期望"是平均分析;"任意序列""总代价上界"是摊还分析。

11.4.3 自调整表(move-to-front):结论与直觉

自调整表:元素排成一列,每次访问后移到表头(move-to-front),频繁访问者越用越靠前。访问位置 k 的元素实际代价 $O(k)$ (要挪动 $k-1$ 个元素),但**摊还代价 $O(\log n)$** : n 次任意访问的总代价 $O(n \log n)$ 。

证明思路(从简,见教材):取势能 Φ = 表中逆序对个数。把被访问元素从位置 k 提到表头,破坏约 k 个逆序对(势能下降),新产生的逆序对有界;势能变化与移动代价大体抵消,余项用对数界住。结论:摊还 $O(\log n)$ 。自调整思想在工程中遍地开花:LRU 缓存、"最近使用的放前面"的接口设计都是亲戚。

11.4.4 势能方法的通用框架

★ 重点:势能方法四步框架

1. **选 Φ** :只依赖数据结构的状态(元素个数、逆序对数.....);
2. **验证**: $\Phi(D_0)$ 有界且 $\Phi(D_i) \geq 0$;
3. **逐个算**: $a_i = c_i + \Delta\Phi$,昂贵操作 $\Delta\Phi$ 为负、便宜操作为正;
4. **累加得界**: $\sum a_i = O(nf(n))$ 则摊还 $O(f(n))$ 。

选 Φ 的直觉: Φ 应度量"结构中蕴藏的、未来必然要做的活儿"——待弹出的元素、待拷贝的元素、待"纠正"的逆序对。消耗这些"存量"时,实际代价由势能下降支付。

⚠ 误区 3:势能函数选错,存款为负

Φ 必须非负,这是势能方法的前提。反例:栈取 $\Phi(D) = -(\text{元素个数})$, $\Phi(D_0) = 0$ 但 $\Phi(D_i) < 0$,此时 $\sum a_i < \sum c_i$,总摊还代价**低于**总实际代价,不再是上界,"总代价 $O(n)$ "直接失效。更隐蔽的错误:选 Φ 依赖操作历史(如" Φ 取 i 的奇偶"),望远镜求和不再成立。选好 Φ 后务必检查"非负"与"只依赖状态"两条。

§11.5 摊还在堆上的应用

11.5.1 斜堆:merge 摊还 $O(\log n)$

第 6 章讲过斜堆:没有平衡信息,merge 时无条件交换左右子堆,单次 merge 最坏 $O(n)$,但**merge 摊还 $O(\log n)$** 。证明用势能法(从简):称"重"结点为右子堆大小 $\geq$ 左子堆一半者,势能 Φ = 重结点总数。merge 的实际代价 = 右路径步数 = 轻边数 + 重边数;轻边数 $\leq \log_2 n$ (每经一条轻边子树至少减半),每条重边由势能下降 1 支付,交换后至多产生 $\log n$ 个新重结点,两抵得 $O(\log n)$ 。斜堆是"摊还换来简单"的典型:无平衡信息、代码最短。

11.5.2 二项队列:insert 摊还 $O(1)$

二项队列的 insert 可看作"二进制加法":二项树大小是 2 的幂,合并两个同大小树就像进位 $1+1=10$,于是借用二进制计数器的经典结论:从 0 加 1 共 n 次,总翻转位数 $\leq 2n$,摊还 $O(1)$ 。故 insert 摊还 $O(1)$ 、最坏 $O(\log n)$;deleteMin 与

merge 摊还 $O(\log n)$ 。对比二叉堆:insert 最坏就是 $O(\log n)$,不必摊还——摊还分析只在结构把贵操作推迟分摊时,才比最坏更紧。

11.5.3 二叉堆 buildHeap:聚集分析回顾

第 6 章说 buildHeap 是 $O(n)$,现在证明。buildHeap 从第 $\lfloor n/2 \rfloor$ 个结点起逐个下滤,关键是把结点按高度而非按深度计数:完全二叉树中高度为 h 的结点至多有 $n/2^{h+1}$ 个,每个下滤 $\leq h$ 层,总代价:

$$T(n) \leq \sum_{h=0}^{\log n} (n/2^{h+1}) \cdot h = (n/2) \cdot \sum_{h=1}^{\infty} h/2^h = (n/2) \cdot 2 = O(n)。$$

级数 $\sum_{h=1}^{\infty} h/2^h = 2$ 。这就是"逐个 insert 是 $O(n \log n)$ 、整体下滤只要 $O(n)$ "的原因:多数结点在底层、下滤路径短;高层结点上滤深,但数量指数级地少。下面代码统计真实步数验证。

示例 11-3 统计 buildHeap 的总下滤步数(验证 $\approx n$)

```
# 从第 n//2 个结点向前逐个下滤,统计总步数
def heapify(a):
    n, steps = len(a), 0
    def down(i):
        nonlocal steps
        while True:
            l, r, m = 2 * i + 1, 2 * i + 2, i
            if l < n and a[l] > a[m]: m = l
            if r < n and a[r] > a[m]: m = r
            if m == i: break
            a[i], a[m] = a[m], a[i]
            steps += 1
            i = m
    for i in range(n // 2 - 1, -1, -1):
        down(i)
    return steps

n = 65535
a = list(range(1, n + 1))
s = heapify(a)
print(f"n = {n}, 总下滤步数 = {s} (理论 O(n),实测 ≈ {s * 100 // n}% n)")
# 输出:n = 65535, 总下滤步数 = 65519 (理论 O(n),实测 ≈ 99% n)
```

§11.6 工程印证:实测扩容拐点

11.6.1 手写倍增 vs 标准库:观察扩容拐点

理论说"容量翻倍",CPython 的 list 真的翻倍吗?Python 没有 capacity() 接口,但可以用 sys.getsizeof 看对象占用的字节数——字节跳变的那一刻就是扩容拐点。下面把手写实现与内置写法并排对照。

示例 11-4 手写倍增与内置 list 并排对照:观察扩容拐点

```

# 手写倍增与内置 list 并排:打印扩容拐点(元素数 → 字节数)
import sys

class MyList:                                # 手写倍增动态数组
    def __init__(self):
        self._buf = [None] * 1
        self._size = 0
        self._cap = 1
    def push_back(self, x):
        if self._size == self._cap:          # 容量已满:倍增
            new_buf = [None] * (self._cap * 2)
            for i in range(self._size):
                new_buf[i] = self._buf[i]
            self._buf = new_buf
            self._cap *= 2
            print(f"{self._size}→{self._cap}", end=" ") # 打印拐点
        self._buf[self._size] = x
        self._size += 1

print("手写倍增:", end=" ")
v = MyList()
for i in range(200):
    v.push_back(i)

print("\n内置 list:", end=" ")
a, last = [], sys.getsizeof([])
for i in range(1, 201):
    a.append(i)
    cur = sys.getsizeof(a)
    if cur != last:
        # 内存变化 = 一次扩容拐点
        print(f"{len(a)}→{cur}B", end=" ")
        last = cur
print()

```

C 的 malloc 数组	Python 的 list / 手写倍增类
没有 size/capacity 概念,长度、容量全靠变量自管	内部自带 size/capacity,外部用 <code>sys.getsizeof</code> 观察拐点
扩容写错(如每次 +1)代价是 $O(n^2)$,且只有运行慢时才发现	倍增由 CPython 保证,摊还 $O(1)$;手写类也要照抄倍增骨架
free 忘记调用即泄漏	引用计数 + 垃圾回收自动释放,无手动 free

11.6.2 实测数据解读

本机实测(CPython 3.11):list 连续 append 200 个元素,拐点(元素数 → 字节)序列 1→88B, 5→120B, 9→184B, 17→248B, 25→312B, 33→376B, 41→472B, 53→568B, 65→664B, 77→792B, 93→920B, 109→1080B, 129→1240B, 149→1432B, 173→1656B。换算容量(指针槽数)为 4, 8, 16, 24, 32, 40, 52, 64, 76, 92, 108, 128, 148, 172, 200——每次扩容约增加当前容量的 1/8,即约 1.125 倍。虽是 1.125 倍而非 2 倍,几何增长 ⇒ 摊还 $O(1)$ 不变;总指针搬运 $\approx 9n$,摊到每次仍是常数。不同 CPython 版本增长因子略异,摊还结论相同。

C 的手写扩容代码	Python 的 list 接口
realloc 每次扩容都是裸内存搬运,策略(倍增还是 +1)写在调用点	append 自动倍增;策略由 CPython 决定, sys.getsizeof 把拐点暴露成可观测量
想预分配要自己写"capacity 变量 + 判断"样板	<code>a = [None] * n</code> 一行预分配,扩容次数归零
realloc 返回 NULL 要检查,失败后旧指针仍有效	分配失败抛 <code>MemoryError</code> ,旧对象安然无恙

🚀 工程建议:Python 里管理扩容的四个招式

- **预分配**:知道要装多少时用 `a = [None] * n` 再按下标赋值,把扩容次数降为 0——摊还分析告诉你"扩容不贵",但一次都别扩更便宜;
- **别写 `a = a + [x]`**:每次把整个列表复制一遍,是"每次 +1"的平方级灾难;追加用 `a.append(x)`,批量用 `a.extend(it)`;
- **遍历时别增删**:for 循环里 `a.remove` / `a.append` 会让迭代错位;先收集再统一改,或用列表推导;
- **增长因子不是越大越好**:CPython 约 1.125 倍,比 2 倍更省内存;只要因子 > 1 ,摊还就仍是 $O(1)$ 。

⚠ 误区 4:扩容增量太小,摊还证明失效

"每次 +1"的扩容不是常数因子增长:第 i 次扩容拷贝 $i-1$ 个元素,总拷贝 $\sum_{i=1}^n i \approx n^2/2$,摊还 $O(n)$ ——插入 10 万元素就要拷约 50 亿次。Python 里最常见的"每次 +1"写法就是 `a = a + [x]`:每次复制整表,长度 10 万时总复制 ≈ 50 亿次,肉眼可见地卡顿。list.append 的扩容因子约 1.125,是几何增长;"每次多开固定格数"的扩容是平方级灾难。

§11.7 三种方法的选用总结

11.7.1 一张表看清三种方法

方法	核心思路	优点	局限	经典用例
聚集分析	总代价求和再平均	最直接	整体求和要可做	栈序列、buildHeap、二进制计数器
会计方法	每操作预存、贵的取款	最直观	要设计收费方案	动态数组"存 3 元"、二项队列 insert
势能方法	势能函数挂在状态上	最强大	选 Φ 靠经验	斜堆 merge、自调整表、splay 树

11.7.2 选型建议

三个方法算的是同一笔账,结论互相印证。选型:结构简单、总量可数,用聚集分析(栈、buildHeap);要给每个操作"讲明白为什么便宜",用会计方法(动态数组);结构复杂、代价藏在状态里,用势能方法(斜堆、自调整表)。摊还分析不是课堂玩具:数据库批量提交、C++ 的 vector、Python 的 list,全是"贵操作由便宜操作平摊"的生意经。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
尾部插入 append	动态数组(倍增扩容)	摊还 $O(1)$	$O(n)$ (扩容那次)
尾部插入	动态数组(每次 +1 扩容)	$O(n)$	$O(n)$
push	栈(数组或链表)	$O(1)$	$O(1)$
pop	栈	$O(1)$	$O(1)$
multiPop(k)	栈	摊还 $O(1)$	$O(n)$ (单次)
n 次 push/pop/multiPop 序列	栈(聚集分析)	总代价 $O(n)$, 每个元素至多入出栈一次	
插入(存 3 元)	动态数组(会计方法)	总摊还 $\leq 3n$, 账户余额恒 ≥ 1	
加 1 / insert	二进制计数器 / 二项队列	摊还 $O(1)$	$O(\log n)$ (单次)
访问(移到表头)	自调整表(move-to-front)	摊还 $O(\log n)$	$O(n)$
insert	二叉堆	$O(\log n)$	$O(\log n)$
buildHeap	二叉堆(聚集分析)	$O(n)$	$O(n)$
merge	斜堆(势能方法)	摊还 $O(\log n)$	$O(n)$
merge / deleteMin	二项队列	摊还 $O(\log n)$	$O(\log n)$
会计方法	通用框架	余额非负 $\Rightarrow$ 总摊还 $\geq$ 总实际	
势能方法	通用框架	$a_i = c_i + \Delta\Phi$, Φ 非负 $\Rightarrow$ 摊还为上界	

最小必会清单

- 能独立说出摊还、平均、最坏三种分析的区别,并各举一例
- 能独立证明:任意 n 个 push/pop/multiPop 操作总代价 $\leq 2n$
- 能独立默写"插入收 3 元"的会计证明,并推导余额 $= 2s - c + 1 \geq 1$
- 能独立解释"每次 +1"扩容为何 $O(n^2)$,几何扩容为何摊还 $O(1)$ (CPython 的 ≈ 1.125 倍同样成立)
- 能独立写出势能定义式 $a_i = c_i + \Delta\Phi$,并解释 Φ 为何必须非负
- 能用势能方法(Φ = 元素个数)重推栈:push 2、pop 0、multiPop 0
- 能独立用聚集分析证明 buildHeap 是 $O(n)$ (按高度计数)
- 能独立说出斜堆 merge 摊还 $O(\log n)$ 、二项队列 insert 摊还 $O(1)$

自测题

Q 1. 摊还分析与平均情形分析的根本区别是什么?

✓ 参考答案

平均情形分析对**输入**的分布取平均(如"快排平均 $O(n \log n)$ "假设排列等可能),分布一变结论即失效;摊还分析对**操作序列**取平均,不假设任何概率——" n 次 list.append 总代价 $O(n)$ "对任何序列都成立。

Q 2. 为什么说"全 multiPop 的序列"不存在?这对证明意味着什么?

✓ 参考答案

multiPop 要弹出元素,而元素必须先 push 进去;从空栈开始的序列里,弹出总次数 $\leq$ 入栈总次数 $\leq n$,multiPop 的总代价被 push 次数封顶——昂贵操作在结构上不可能频繁出现,总代价 $\leq 2n$ 对任何序列成立。

Q 3. 动态数组扩容"存 1 元"为什么不够?给出会破产的序列。

✓ 参考答案

存 1 元 = 每次插入收费 2(1 元自用 + 1 元存款)。设 size = s 、容量 $c = 2^k$,总收费 $2s$,总花费 $s + (c - 1)$,余额 = $s - c + 1$ 。数组一满($s = c$)时余额只剩 1;此时扩容收 2 元、付 1 元自用、再付 c 元拷贝,余额 = $1 + 2 - 1 - c = 2 - c < 0$ —— $c \geq 4$ 即破产。而"存 2 元"(收 3 元)时余额 = $2s - c + 1$,利用 $s \geq c/2$ 得余额 ≥ 1 恒正。扩容是一次性大额开销,存款必须按"每个元素都积累足够配额"定价,2 元是最低安全价。

Q 4. buildHeap 为什么是 $O(n)$ 而不是 $O(n \log n)$?差在哪一步?

✓ 参考答案

逐个 insert 到空堆是 $O(n \log n)$:每个元素要从底部上滤 $\log n$ 层;buildHeap 整体下滤时,多数结点在底层、下滤路径短:高度 h 的结点只有 $n/2^{h+1}$ 个,每个下滤 $\leq h$ 层,总代价 $(n/2) \cdot \sum h/2^h = O(n)$ 。差在"按高度计数"而非"按深度计数",同一棵树的两种视角。

Q 5. 为什么 C++ 的 vector 扩容后迭代器全部失效,而 Python 的 list 扩容后已取出的引用仍然有效?这与摊还分析有何联系?

✓ 参考答案

vector 扩容要整体搬家到新内存,旧地址上的一切指针/迭代器随之作废;CPython 的 list 底层是对象指针数组,扩容只是把指针搬运到新数组,你手中的引用指向堆上的对象本身,与数组搬家无关——但遍历 list 时插入/删除会按索引错位,是另一副面孔。两者共通的是摊还 $O(1)$ 的"倍增拷贝":vector 总拷贝 $\approx 2n$,CPython 总指针搬运 $\approx 9n$,都是常数摊还,由数学保证。

章末实验题:动态数组扩容的摊还代价实测

实验 11:动态数组扩容的摊还代价实测

实验目标:手写带拷贝计数器的动态数组,用真实数据验证"倍增扩容总拷贝 $\approx 2n$ 、每次 +1 扩容总拷贝 $\approx n^2/2$ ",并用会计方法解释"存 3 元"为什么够。

实验要求:

- 任务 1: 手写倍增动态数组 `DoublingArray`, 在扩容处累计真实拷贝次数, 提供 `total_copies/last_cost/max_cost` 三个统计量(知识点:\$11.3 会计方法、第 3 章倍增扩容)
- 任务 2: 连续插入 $n = 10000$ 个元素, 统计总拷贝, 验证倍增 $\approx 2n$ vs 每次 +1 的笨扩容 $\approx n^2/2$ (知识点:\$11.2)
- 任务 3: 对比两种策略的"单次最大代价"与"平均代价", 解释摊还 $O(1)$ 与"单次最坏 $O(n)$ "并存(知识点:\$11.1)
- 任务 4: 用会计方法注释 `BankArray` 收费规则, 验证全程最低余额 ≥ 0 , 写出余额公式 $2s - c + 1$ (知识点:\$11.3)
- 任务 5: 用 `sys.getsizeof` 观察标准库 `list` 的扩容拐点, 与手写倍增对照(知识点:\$11.6)

实验环境:Python 3.8+, 无需第三方库。运行: `python lab11.py`。

第一步 个类共用同一骨架: 扩容时先分配新列表, 循环拷贝旧元素并更新计数——拷贝计数必须放在真实 `for` 循环里, 不能用切片 `new_buf[:size] = buf` 或浅拷贝替代, 否则统计失真

第二步 扩容 $n = 10000$ 时总拷贝约 5000 万次; Python 的 `int` 任意精度, 计数器不会溢出(对照 C++ 必须用 `long long`)

第三步 任务 4 的余额更新放在 `push_back` 末尾统一做, 并记录全程最低余额; 理论下界 1, 实测应 ≥ 1

第四步 任务 5 用 `sys.getsizeof` 的字节跳变检测拐点: 记录上一次返回值, 每次 `append` 后比较, 变化即打印 `len(a)` → 字节数

✔ 参考答案(完整可运行程序,分三段)

```

# lab11.py — 实验 11:动态数组扩容的摊还代价实测(第一段:两种策略)
# 运行:python lab11.py
import sys

# ----- 任务 1:带真实拷贝计数器的倍增动态数组 -----
class DoublingArray:
    """倍增扩容:容量满时  $\times 2$ ,记录每次 push_back 的真实拷贝次数。"""
    def __init__(self):
        self._buf = [None] * 1      # 底层存储,初始容量 1
        self._size = 0              # 元素个数
        self._cap = 1               # 容量
        self.total_copies = 0       # 累计拷贝次数(真实代价总和)
        self.last_cost = 0          # 最近一次 push 付出的拷贝次数
        self.max_cost = 0           # 单次 push 的最大拷贝次数

    def push_back(self, x):
        self.last_cost = 0
        if self._size == self._cap:  # 容量已满:倍增扩容
            new_buf = [None] * (self._cap * 2)
            for i in range(self._size):
                new_buf[i] = self._buf[i]    # 真实拷贝 1 次
            self.last_cost = self._size      # 本次操作的真实拷贝代价
            self.total_copies += self._size
            if self.last_cost > self.max_cost:
                self.max_cost = self.last_cost
            self._buf = new_buf
            self._cap *= 2
        self._buf[self._size] = x
        self._size += 1

# ----- 任务 1:每次只多 1 格的"笨扩容"(对照策略) -----
class LinearArray:
    """每次容量只 +1:摊还代价退化为  $O(n)$ ,用于对照。"""
    def __init__(self):
        self._buf = [None] * 1
        self._size = 0
        self._cap = 1
        self.total_copies = 0
        self.last_cost = 0
        self.max_cost = 0

    def push_back(self, x):
        self.last_cost = 0
        if self._size == self._cap:  # 容量已满:只多开 1 格
            new_buf = [None] * (self._cap + 1)
            for i in range(self._size):
                new_buf[i] = self._buf[i]
            self.last_cost = self._size
            self.total_copies += self._size
            if self.last_cost > self.max_cost:
                self.max_cost = self.last_cost
            self._buf = new_buf
            self._cap += 1

```

```
self._buf[self._size] = x
self._size += 1
```

lab11.py(第二段:会计方法"存 3 元"动态数组)

```
# ----- 任务 4:会计方法视角的"存 3 元"动态数组 -----
# 每次 push 收 3 元:1 元支付"写入自己",2 元存入账户;
# 扩容时每拷贝 1 个旧元素,从账户支付 1 元。
# 证明(设  $size = s$ , 容量  $= c = 2^k$ ):
# 总收费  $= 3s$ ; 总花费  $= s + (1+2+\dots+c/2) = s + c - 1$ ;
# 余额  $= 3s - s - (c-1) = 2s - c + 1 \geq 1 > 0$ 。
# 所以"存 3 元"永远够用——摊还  $O(1)$  的会计证明。
class BankArray:
    def __init__(self):
        self._buf = [None] * 1
        self._size = 0
        self._cap = 1
        self.balance = 0          # 账户余额
        self.min_balance = float("inf") # 全程最低余额(应恒  $\geq 0$ )

    def push_back(self, x):
        self.balance += 3          # 收 3 元
        self.balance -= 1          # 支付本次写入
        if self._size == self._cap: # 扩容:每拷 1 个元素付 1 元
            new_buf = [None] * (self._cap * 2)
            for i in range(self._size):
                new_buf[i] = self._buf[i]
                self.balance -= 1
            self._buf = new_buf
            self._cap *= 2
        self._buf[self._size] = x
        self._size += 1
        if self.balance < self.min_balance:
            self.min_balance = self.balance
```

lab11.py(第三段:主程序)

```

def main():
    N = 10000    # 连续插入的元素个数

    # ---- 任务 2、3:两种扩容策略的拷贝统计与代价对比 ----
    d = DoublingArray()
    for i in range(N):
        d.push_back(i)
    print(f"[任务2/3] 倍增策略:连续插入 {N} 个元素")
    print(f" 总拷贝次数 = {d.total_copies} (理论  $\approx 2n = \{2 * N\}$ )")
    print(f" 单次最大代价 = {d.max_cost} 次拷贝(仅发生在扩容的那一次 push)")
    print(f" 平均代价 = {d.total_copies / N:.4f} 次拷贝/push\n")

    l = LinearArray()
    for i in range(N):
        l.push_back(i)
    print(f"[任务2/3] 笨扩容(每次 +1):连续插入 {N} 个元素")
    print(f" 总拷贝次数 = {l.total_copies} (理论  $\approx n^2/2 = \{N * N // 2\}$ )")
    print(f" 单次最大代价 = {l.max_cost} 次拷贝(最后一次扩容)")
    print(f" 平均代价 = {l.total_copies / N:.4f} 次拷贝/push\n")

    # ---- 任务 4:会计方法验证"存 3 元"够不够 ----
    b = BankArray()
    for i in range(N):
        b.push_back(i)
    print(f"[任务4] 会计方法:存 3 元策略运行 {N} 次 push 后")
    print(f" 账户余额 = {b.balance}, 全程最低余额 = {b.min_balance} (非负即证明有效)\n")

    # ---- 任务 5:标准库 list 的扩容拐点(用 sys.getsizeof 观察) ----
    a = []
    last = sys.getsizeof(a)
    print(f"[任务5] Python list 扩容拐点(元素数  $\rightarrow$  占用字节,前 200 个元素):")
    row = []
    for i in range(1, 201):
        a.append(i)
        cur = sys.getsizeof(a)
        if cur != last:          # 内存变化 = 一次扩容拐点
            row.append(f"{len(a)} $\rightarrow$ {cur}B")
            if len(row) == 12:
                print(" " + " ".join(row))
                row = []
            last = cur
    if row:
        print(" " + " ".join(row))
    print(f"\n结论: CPython 的 list 按比例增长(约 1/8 增量),拐点约 1.125 倍,"
          "虽不是严格的 2 倍,但几何增长  $\Rightarrow$  摊还  $O(1)$  不变,与本章理论一致。")

if __name__ == "__main__":
    main()

```

编译与运行,以及本机实测输出:

python lab11.py

[任务2/3] 倍增策略:连续插入 10000 个元素

总拷贝次数 = 16383 (理论 $\approx 2n = 20000$)

单次最大代价 = 8192 次拷贝(仅发生在扩容的那一次 push)

平均代价 = 1.6383 次拷贝/push

[任务2/3] 笨扩容(每次 +1):连续插入 10000 个元素

总拷贝次数 = 49995000 (理论 $\approx n^2/2 = 50000000$)

单次最大代价 = 9999 次拷贝(最后一次扩容)

平均代价 = 4999.5000 次拷贝/push

[任务4] 会计方法:存 3 元策略运行 10000 次 push 后

账户余额 = 3617, 全程最低余额 = 2 (非负即证明有效)

[任务5] Python list 扩容拐点(元素数 $\rightarrow$ 占用字节,前 200 个元素):

1 $\rightarrow$ 88B 5 $\rightarrow$ 120B 9 $\rightarrow$ 184B 17 $\rightarrow$ 248B 25 $\rightarrow$ 312B 33 $\rightarrow$ 376B 41 $\rightarrow$ 472B 53 $\rightarrow$ 568B 65 $\rightarrow$ 664B 77 $\rightarrow$ 792B 93 $\rightarrow$ 920B 109 $\rightarrow$ 1080B
129 $\rightarrow$ 1240B 149 $\rightarrow$ 1432B 173 $\rightarrow$ 1656B

结论: CPython 的 list 按比例增长(约 1/8 增量),拐点约 1.125 倍,虽不是严格的 2 倍,但几何增长 $\Rightarrow$ 摊还 $O(1)$ 不变,与本章理论一致。

设计说明:① 三个类共用"满则扩容、循环拷贝"的骨架,差异只在扩容因子($\times 2$ 与 $+1$),暴露摊还分析的唯一变量;② 拷贝计数放在真实 for 循环而非切片/浅拷贝,统计语义代价而非底层内存搬运;③ 任务 4 用可运行的"余额"把抽象证明变成可观察的量,实测最低余额 2 与理论下界 1 吻合;④ 任务 5 用 sys.getsizeof 观察内存字节推断扩容拐点——CPython 没有 capacity() 接口,字节数是唯一可观测的"容量计",实测 1.125 倍增长与 §11.3 的"常数因子"证明互相印证。

下一章预告:第12章 高级数据结构——红黑树、Treap、跳表、k-d 树;以及标准库选择指南。本章的势能方法是第12章伸展树(splay tree)摊还分析的直接武器;红黑树的摊还/最坏界限对比、Treap 的随机化分析,都会用到本章的三种方法。

第12章 高级数据结构

建议学时:4-5 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解红黑树五条性质与插入修正,知道它是 2-3-4 树的二叉表示
- 掌握 Treap 与跳表的随机化平衡思想,能完整实现插入/删除/查找
- 掌握 k-d 树最近邻剪枝查询,理解配对堆摊还 $O(1)$ 合并
- 学会 Python 的"有序字典"替代惯例:list+bisect、dict+sorted、sortedcontainers,并实测 10^4 级跳表字典

💡 从 C 到 Python:本章主题如何迁移

前 11 章最大的缺口是"平衡":BST 遇有序输入退化成链表,AVL 实现繁琐。本章五个结构给出三种哲学:红黑树用确定不变式强制平衡;Treap 与跳表把平衡交给随机性(期望 $O(\log n)$);配对堆靠摊还分析兜底。

C 里做"有序字典"要手写几百行平衡树,一个修正分支写错就悄悄失衡。Python 没有内建平衡树,但有两条轻量惯例: `list + bisect` 二分(小数据)、`dict + sorted` (批处理),大数据再上第三方 `sortedcontainers` ——这正是本章"选型"的实用部分。

跳表与 Treap 的价值在"实现简单":代码量接近链表,却拿到期望 $O(\log n)$ ——用"概率上几乎不可能坏"换"写起来几乎不可能错",Redis 的 zset 底层就是跳表。Python 实现它们比 C 更短:指针变成列表索引, `new / delete` 变成对象引用与垃圾回收。

📦 前置知识自查

数学前置:概率期望直觉(几何分布)、对数运算、摊还分析概念(第11章)。

C 语言前置:`struct` 与指针、链表结点增删(第3章)、函数指针、`malloc / free` 对照 Python 的对象与引用计数。

依赖的前面章节:第4章树(BST、旋转、AVL)、第5章散列、第6章堆、第11章摊还分析。

🚀 本章学习路线

1. **第一阶段(2 小时):**读 §12.1–§12.4,背下修正口诀,手写 Treap、跳表与 k-d 树
2. **第二阶段(1 小时):**读 §12.5–§12.6,理解配对堆与 Python 选型惯例
3. **第三阶段(实验,1 小时):**完成章末实验:跳表字典的实现与对比
4. **验收标准:**能随口说出"红黑树高度 $O(\log n)$ " "跳表期望 $O(\log n)$ ",知道 Python 有序映射的三种替代

本章知识导图

- §12.1 红黑树:五条性质与插入修正
- §12.2 Treap:随机优先级的树堆
- §12.3 跳表:多层链表的概率平衡
- §12.4 k-d 树:多维空间切分与最近邻
- §12.5 配对堆:摊还 $O(1)$ 合并的实用堆
- §12.6 标准库选型指南
- §12.7 全书总结:十二章知识地图

§12.1 红黑树:五条性质与插入修正

12.1.1 五条性质与"黑高"

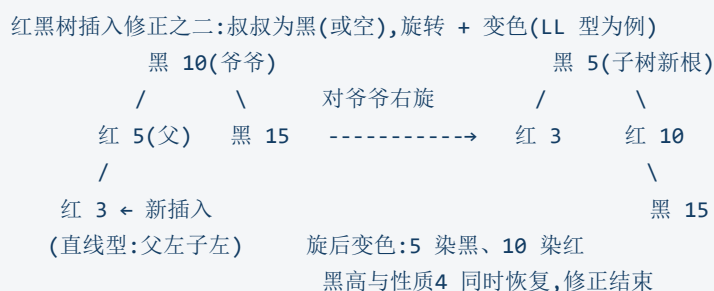
红黑树是一棵二叉查找树,每个结点带红/黑一色,满足五条性质:① 结点非红即黑;② 根是黑色;③ 叶结点(NIL)是黑色;④ 红结点的子结点都是黑色——"红结点不能有红孩子";⑤ 从任一结点到其所有后代叶的路径上,黑色结点数相同,这个数叫**黑高**。

关键保证:设黑高 h_b ,最短路径(全黑)长 h_b ,性质4 又禁红红相邻,最长路径至多 $2h_b$ ——最长 $\leq$ 最短的 2 倍,树高 $O(\log n)$,三操作最坏 $O(\log n)$ 。这就是"弱平衡":不追求 AVL 的高度差 ≤ 1 。

12.1.2 插入修正:变色与旋转

插入先染**红**——红结点不影响性质5(黑高不变),只可能破坏性质4(红父红子)。修正只看叔叔颜色:叔叔红 $\rightarrow$ 变色;叔叔黑 $\rightarrow$ 旋转。

示例 12-1 插入修正:叔叔为黑 $\rightarrow$ 旋转 + 变色



叔叔为红时,父、叔变黑,爷爷变红,冲突上移一层继续检查;爷爷若是根最后染黑。叔叔为黑时按 AVL 的四种形态旋转:LL/RR 直线一次旋转,LR/RL 之字形两次旋转;旋转后被提上来的结点染黑、爷爷染红。

★ 重点:插入易、删除难

插入只处理"红父红子"一处冲突,沿路径上移 $O(\log n)$ 层;删除要把黑高"挖掉"一层,处理"双黑"问题——红黑树"实现极复杂、接口极简单",删除从不手写,因此适合内建在标准库。

12.1.3 与 2-3-4 树的关系

一句话:**红黑树是 2-3-4 树的二叉表示**——把 2-3-4 树一个结点内的 2~3 个键"压扁"成红黑交替的二叉链。

12.1.4 Python 视角:没有 map,但有三种替代

C++ 有 `std::map` (红黑树),Java 有 `TreeMap`; Python 标准库没有平衡树——因为 `dict` 平均 $O(1)$ 已覆盖大多数场景,少数"要有序"的场景用三种替代:① 小数据 `list` + `bisect` 二分定位;② 批处理 `dict` + `sorted()`;③ 大数据上第三方 `sortedcontainers.SortedDict` (红黑树实现)。

C 的手写有序查找表	Python 的替代
手写 AVL/红黑树:数百行,修正分支众多	小数据: <code>list</code> + <code>bisect</code> ;大数据: <code>SortedDict</code>
有序遍历要自己写中序递归	<code>sorted(d.keys())</code> 一行,或 <code>SortedDict</code> 直接迭代
范围查询手写"第一个 $\geq x$ 的结点"	<code>bisect_left/right</code> 现成定位
忘记 <code>free</code> 整棵树即泄漏	垃圾回收自动回收全部结点

⚠ 误区 1:性质4 的方向与口诀记反

性质4 是"红结点的孩子必须黑",反过来"黑结点的孩子必须红"是错的——黑结点孩子颜色任意。高频错误还有口诀记反:叔叔红时去旋转,树永远修不好。

§12.2 Treap:随机优先级的树堆

12.2.1 思想:BST + 堆序

BST 退化根源是"输入顺序决定树形"。Treap(tree+heap)给每个结点一个**随机优先级**,并要求其满足**堆序**(小顶堆:父 $\leq$ 子)。键决定"在哪里",优先级决定"谁当根"——优先级随机,树形就随机,期望高度 $O(\log n)$ 。把平衡从"旋转纠错"换成"随机决定",实现从几百行降到几十行。

12.2.2 插入与旋转

插入:先按 BST 规则插到叶位置,再沿路径向上旋转,直到父的优先级 $\leq$ 子——与二叉堆"上滤"异曲同工。删除同理:把目标旋转着推下去,变叶子再摘除。

示例 12-5 Treap 完整实现(第一段:结构、旋转、插入)

```

# Treap:键满足 BST 序,随机优先级满足小顶堆堆序
import random

class Node:
    __slots__ = ("key", "pri", "left", "right") # 省内存
    def __init__(self, key, pri):
        self.key = key
        self.pri = pri
        self.left = self.right = None

def rotate_right(t):          # 右旋:提升左孩子
    x = t.left
    t.left = x.right
    x.right = t
    return x

def rotate_left(t):           # 左旋:提升右孩子
    x = t.right
    t.right = x.left
    x.left = t
    return x

def insert(t, key, pri):
    if t is None:
        return Node(key, pri)
    if key < t.key:
        t.left = insert(t.left, key, pri)
        if t.left.pri < t.pri:
            t = rotate_right(t)          # 堆序上滤
    elif key > t.key:
        t.right = insert(t.right, key, pri)
        if t.right.pri < t.pri:
            t = rotate_left(t)
    return t

```

| 示例 12-6 Treap 完整实现(第二段:删除与查找)

```
def erase(t, key):
    if t is None:
        return None
    if key < t.key:
        t.left = erase(t.left, key)
    elif key > t.key:
        t.right = erase(t.right, key)
    elif t.left is None:
        return t.right
    elif t.right is None:
        return t.left
    elif t.left.pri < t.right.pri:          # 优先级小的旋上来
        t = rotate_right(t)
        t.right = erase(t.right, key)
    else:
        t = rotate_left(t)
        t.left = erase(t.left, key)
    return t

def find(t, key):
    while t is not None and t.key != key:
        t = t.left if key < t.key else t.right
    return t is not None
```

12.2.3 期望 $O(\log n)$ 的直觉与缺陷

直觉:根是整棵树优先级最小的结点,优先级随机,所以"根是第 k 小的键"的概率均匀为 $1/n$,左右子树递归同构——与快排"随机枢轴"的分析同构:Treap 就是快排的树形版本。最坏情形(优先级恰好按键单调)概率 $1/n!$,现实不会出现。优先级相等时须约定比较方向,否则可能破坏 BST 序。

⚠ 误区 2:堆序方向与旋转方向配错

本例用小顶堆(父 $\leq$ 子),口诀"左小右旋、右小左旋";若定义成大顶堆,所有比较都要取反。实现后用有序序列插入再中序遍历:输出必须升序,且每个结点优先级 $\leq$ 子结点。

§12.3 跳表:多层链表的概率平衡

12.3.1 多层链表与几何分布

有序链表查找 $O(n)$,因为每步只能走一个结点。跳表(skip list)的思路:多建几层链表,上层是下层的"快速通道"。每个新结点先进入最底层,再以概率 $1/2$ 晋升,再以概率 $1/2$ 继续.....层数 k 满足几何分布: $P(\text{层} \geq k) = 2^{-(k-1)}$,每层结点数期望 $n/2^{k-1}$ 。查找从最高层出发,每层右移到"下一个更大"结点前再下降,共 $O(\log n)$ 层。

12.3.2 三操作:查找、插入、删除

三操作共享同一"下探"骨架:从最高层向右试探,把每层"最后一个 $< \text{key}$ 的结点"记进 `update` 列表。**查找**:下探后看 `update[0].next[0]` 是否为 `key`。**插入**:随机层数 `lvl`,若高于当前最高层先补 `update` 头指针,再逐层链入。**删除**:下探后若第 0 层后继即目标,逐层跳过,最后回收空高层。C 里每层指针要手写 `Node* next[k]`,Python 用 `next = [None] * (lvl + 1)` 一行搞定, `__slots__` 还能省内存。

示例 12-7 跳表完整实现(第一段:结构、查找、插入)

```

import random

class Node:
    __slots__ = ("key", "next")          # next[i]:第 i 层的后继
    def __init__(self, key, lvl):
        self.key = key
        self.next = [None] * (lvl + 1)  # 层号 0..lvl 均有效

class SkipList:
    def __init__(self, max_lvl=16):
        self.head = Node(0, max_lvl)    # 头结点:全部层的起点
        self.max_lvl = max_lvl
        self.level = 0
        self.rng = random.Random(12345) # 固定种子:可复现

    def random_level(self):               # 几何分布:每层 1/2 概率晋升
        lvl = 1
        while self.rng.getrandbits(1) and lvl < self.max_lvl:
            lvl += 1
        return lvl

    def find(self, key):
        cur = self.head
        for i in range(self.level, -1, -1):
            while cur.next[i] and cur.next[i].key < key:
                cur = cur.next[i]
        x = cur.next[0]
        return x is not None and x.key == key

    def insert(self, key):
        if self.find(key):
            return                                # 字典语义:键唯一
        update = [None] * len(self.head.next)
        cur = self.head
        for i in range(self.level, -1, -1):
            while cur.next[i] and cur.next[i].key < key:
                cur = cur.next[i]
            update[i] = cur                      # 每层最后一个 < key
        lvl = self.random_level()
        if lvl > self.level:
            for i in range(self.level + 1, lvl + 1):
                update[i] = self.head
            self.level = lvl
        x = Node(key, lvl)
        for i in range(lvl + 1):
            x.next[i] = update[i].next[i]
            update[i].next[i] = x

```

■ 示例 12-8 跳表完整实现(第二段:删除与按序遍历)

```

def erase(self, key):
    update = [None] * len(self.head.next)
    cur = self.head
    for i in range(self.level, -1, -1):
        while cur.next[i] and cur.next[i].key < key:
            cur = cur.next[i]
        update[i] = cur
    x = cur.next[0]
    if x is None or x.key != key:
        return False
    for i in range(self.level + 1):
        if update[i].next[i] != x:    # 该层没有 x
            break
        update[i].next[i] = x.next[i]
    while self.level > 0 and self.head.next[self.level] is None:
        self.level -= 1                # 回收空高层
    return True

def keys(self):
    v = []
    p = self.head.next[0]
    while p:
        v.append(p.key)
        p = p.next[0]
    return v

def __len__(self):
    return len(self.keys())

```

12.3.3 与平衡树对比:期望 vs 确定

对比项	跳表	红黑树 / AVL
平衡机制 / 实现难度	随机层数,~ 链表级,几十行	旋转 + 不变式,高,数百行
高度	期望 $O(\log n)$,最坏 $O(n)$	最坏保证 $O(\log n)$
每结点内存	平均约 2 个指针	2 个指针 + 颜色位
工程使用	Redis 有序集合 zset、LevelDB	std::map/set、Java TreeMap

确定性与实现难度不可兼得:要最坏保证(实时系统)选红黑树;要快速实现选跳表——跳表还天然便于无锁并发修改。

C 的手写多级链表	Python 的跳表类
每层一个 <code>Node* next[k]</code> 数组要手写分配	<code>next = [None] * (lvl + 1)</code> 一行
释放多层结点容易重复 free 或泄漏	垃圾回收自动回收, <code>__slots__</code> 省内存
随机层数用 <code>rand()</code> ,可移植性差	<code>random.Random(种子)</code> 质量与可复现性都有保证

⚠ 误区 3:随机层数用错了分布

层数必须**几何分布**(每层 1/2 概率继续)。三个常见错误:① 固定层数让高层退化成线性扫描 $O(n)$;② 删除后不回收空高层;③ 用 `time()` 做种子,同一秒插入的结点层数高度相关。调试:逐层打印链表,验证"第 0 层完整、高层稀疏"。

§12.4 k-d 树:多维空间切分与最近邻

12.4.1 交替维度切分建树

k-d 树递归地选一个维度取**中位数**作切分点,左子树为小于切分点的点集,右子树反之;下一层换另一个维度。对二维点:第 0 层按 x 、第 1 层按 y 、第 2 层再按 x **维度交替,保证每个区域在两个方向都被"削"过**,树高 $O(\log n)$ 。C++ 用 `nth_element` 线性选中位数;Python 直接用 `sorted` 或 `statistics.median` 选,代价同为 $O(n \log n)$ 建树。

示例 12-9 k-d 树:交替维度建树(二维)

```
def build(pts, lo, hi, depth):
    """pts 就地重排后返回子树根;返回 (根, 新列表) 也行,这里用下标切分"""
    if lo > hi:
        return None
    dim = depth % 2                                # 维度交替:0 按 x,1 按 y
    mid = (lo + hi) // 2
    pts[lo:hi + 1] = sorted(pts[lo:hi + 1], # 中位数切分
                             key=lambda p: p[dim])
    node = {"p": pts[mid], "dim": dim, "left": None, "right": None}
    node["left"] = build(pts, lo, mid - 1, depth + 1)
    node["right"] = build(pts, mid + 1, hi, depth + 1)
    return node
```

12.4.2 最近邻查询与剪枝

给定查询点 q ,找点集中距离最近者。先沿树下探到 q 所在区域,得到"当前最佳距离" d_{best} ;回溯时——若 q 到分割线的距离 $\geq d_{\text{best}}$,另一侧任何点都不可能更近,整棵子树剪掉。

示例 12-10 k-d 树:最近邻查询(带剪枝)

```
def dist2(a, b):
    dx, dy = a[0] - b[0], a[1] - b[1]
    return dx * dx + dy * dy

def nearest(node, q):
    """返回 (最佳距离平方, 最佳点)"""
    best_d, best_p = float("inf"), None
    def dfs(node):
        nonlocal best_d, best_p
        if node is None:
            return
        d = dist2(node["p"], q)
        if d < best_d:
            best_d, best_p = d, node["p"]
        dim = node["dim"]
        cut = node["p"][dim]
        qc = q[dim]
        near, far = (node["left"], node["right"]) if qc < cut \
            else (node["right"], node["left"]) # 先探近侧
        dfs(near)
        d_cut = (qc - cut) * (qc - cut) # q 到分割线距离
        if d_cut < best_d: # 剪枝:够近才跨过去
            dfs(far)
        dfs(node)
    return best_d, best_p
```

记号说明:距离平方省去开根号

最近邻只需比较大小, `dist2` 返回距离平方即可,省掉每步一次 `sqrt`;剪枝条件用平方比较数学等价。

12.4.3 应用与退化警告

k-d 树是"多维点集查询"的默认答案:游戏引擎的"最近敌人"查询替代逐对检测的 $O(n^2)$,光线追踪与 k-NN 分类也建立在"空间切分 + 剪枝"上。数据挤在一条直线上时剪枝失效,查询最坏 $O(n)$;生产级替代是 R 树或网格索引。

误区 4:剪枝条件漏写或写反

最近邻两种经典错误:① 漏掉 `if d_cut < best_d` 直接递归两侧——正确但退化成全树搜索 $O(n)$;② 比较方向反了——可能跳过真正更近的点。调试:拿 5 点小数据集逐结点打印"近侧/远侧/是否剪枝"核对。

§12.5 配对堆:摊还 $O(1)$ 合并的实用堆

12.5.1 配对堆是什么

第6章的二叉堆支持 `insert` 与 `deleteMin`,但**合并两个堆**要 $O(n)$ 。左式堆 `merge` 是 $O(\log n)$ 但实现复杂;**配对堆** (pairing heap)走实用路线: `merge` 只比两个根的键,把小根挂到大根的孩子链表里,摊还 $O(1)$; `deleteMin` 把根的孩子两两配对合并,摊还 $O(\log n)$ 。

示例 12-11 配对堆:`merge`、`insert`、`deleteMin`

```

# 配对堆:孩子用"左孩子 + 右兄弟"链表表示,根最小
class PHNode:
    __slots__ = ("val", "child", "next")
    def __init__(self, val):
        self.val = val
        self.child = None      # 第一个孩子,next 为兄弟链
        self.next = None

def merge(a, b):              # 摊还 O(1)
    if a is None:
        return b
    if b is None:
        return a
    if a.val > b.val:
        a, b = b, a          # 键小的做根
    b.next = a.child
    a.child = b
    return a

def merge_pairs(first):       # 两趟合并:摊还 O(log n)
    if first is None or first.next is None:
        return first
    a, b = first, first.next
    rest = b.next
    return merge(merge(a, b), merge_pairs(rest))

def delete_min(root):
    children = root.child
    return merge_pairs(children)  # 根即最小,直接摘掉

# insert = merge(现有堆, 单结点堆),即摊还 O(1)

```

12.5.2 三堆选择的一句话

只做优先队列(不合并):二叉堆;需要合并且代码量敏感:配对堆;需要最坏保证的合并:左式堆。代价是摊还而非最坏:单次 deleteMin 可能偶发 $O(n)$,长时间平均 $O(\log n)$ 。Python 工程上优先队列直接用 `heapq` 即可;配对堆的价值在于理解"摊还"与"可合并堆"这两个概念。

§12.6 标准库选型指南

12.6.1 Python 没有 map:三种替代的决策表

C++ 同学随手写 `std::map`,Python 的标准答案随数据规模走: `list` + `bisect`、`dict` + `sorted`、第三方 `sortedcontainers`。怎么选:

需求	<code>list</code> + <code>bisect</code>	<code>dict</code> + <code>sorted</code>	<code>SortedDict(sortedcontainers)</code>
按键升序遍历	列表本身有序	不支持(需 <code>sorted</code>)	支持(中序遍历)
范围查询 <code>[lo, hi]</code>	支持(<code>bisect</code> 定位)	不支持	支持(<code>irange</code>)
查找 / 插入	$O(\log n)$ / $O(n)$	平均 $O(1)$ / 平均 $O(1)$	$O(\log n)$
适用规模	小($\leq 10^4$)	任意(无序)	大且持续有序

12.6.2 bisect:有序列表的二分工具箱

`bisect_left(a, x)` 返回"第一个 $\geq x$ "的下标, `bisect_right` 返回"第一个 $> x$ "的下标——配合 `list.insert` 就是一棵"数组版平衡树"的插入;配合切片就是范围查询:

示例 12-12 用 bisect 做有序字典(小数据)

```
from bisect import bisect_left, bisect_right

keys, vals = [], []
def d_insert(k, v):
    i = bisect_left(keys, k)
    if i < len(keys) and keys[i] == k:
        vals[i] = v
    else:
        keys.insert(i, k)
        vals.insert(i, v)

def d_find(k):
    i = bisect_left(keys, k)
    return vals[i] if i < len(keys) and keys[i] == k else None

lo, hi = bisect_left(keys, 4), bisect_right(keys, 8)
print(keys[lo:hi])           # 范围查询 [4, 8]
```

12.6.3 手写 vs 标准库:有序字典并排

把 §12.3 的跳表包一层,接口与 `SortedDict` 对齐——第3章 ADT 的威力:

示例 12-13 手写跳表字典 vs SortedDict,接口并排

```
# ---- 手写版:§12.3 的 SkipList 包成有序字典 ----
class SkipDict:
    def __init__(self):
        self._sl = SkipList()
    def insert(self, k): self._sl.insert(k)
    def find(self, k):   return self._sl.find(k)
    def erase(self, k): return self._sl.erase(k)
    def keys(self):     return self._sl.keys()
    def __len__(self):  return len(self._sl)

d = SkipDict()
d.insert(5); d.insert(3)
print(d.keys())           # [3, 5]

# ---- 标准库版:同一套接口,红黑树实现(第三方库) ----
from sortedcontainers import SortedDict
m = SortedDict({3: "three", 5: "five", 8: "eight"})
print(list(m))            # [3, 5, 8]
print(m.irange(4, 8))     # 范围查询 [4, 8]
```

12.6.4 工程选型建议

- 一次性有序输出:任意容器 + `sorted()` (C++ 要手写排序或 `std::sort`)
- 持续插入 + 需要有序:小数据 `list` + `bisect`,大数据 `SortedDict`

- 只查不遍历: dict (平均 $O(1)$);注意 dict 自 3.7 起保持插入顺序,但**不是**按键序
- 顺序敏感又不愿装库:dict + 用时 `sorted(d.keys())` ,适合批处理

C 的做法	Python 的做法
<code>qsort(a, n, sz, cmp)</code> 传函数指针	<code>sorted(a, key=...)</code> 传 key 函数或 lambda
map 底层红黑树,排序是容器特性	dict 无序;要序用 <code>bisect/SortedDict</code>
比较器写错静默产生错误排序	<code>sorted(key=)</code> 越简单越不易错,还可 <code>functools.cmp_to_key</code> 迁移 C 比较器

§12.7 全书总结:十二章知识地图

12.7.1 十二章知识地图

- 第1章 绪论:递归与数学 → 第2章 算法分析: $O/\Omega/\Theta$
- 第3章 表栈队列:数组与链表 → 第4章 树:BST、AVL、遍历
- 第5章 散列:冲突处理 → 第6章 堆:二叉堆、上滤下滤
- 第7章 排序:八大排序 → 第8章 不相交集:union/find
- 第9章 图:最短路与 MST → 第10章 算法设计:贪心、分治、DP
- 第11章 摊还分析:三方法 → 第12章 高级结构:平衡与期望

12.7.2 "遇到问题选什么结构"终极决策表

需求场景	推荐结构	理由
只做等值查找/去重	散列(dict / set)	平均 $O(1)$,常数最小
需要按键有序遍历	跳表或 SortedDict	底层链/中序即有序序列
范围查询 [lo, hi]	bisect 定位有序列表	$O(\log n)$ 定位 + $O(k)$ 输出
优先队列(含合并)	heapq / 配对堆	$O(\log n)$ / 合并摊还 $O(1)$
多维点集最近邻	k-d 树	平均 $O(\log n)$ 剪枝查询

12.7.3 下一步学习方向

学完十二章,你已具备"手写任何基础结构 + 正确选型"的能力,三个方向任选:① **算法竞赛**——跳表与 Treap 常作为平衡树的快速替代;② **系统设计**——缓存用散列、排行榜用跳表、范围索引用 B+ 树;③ **数据库索引**——B+ 树是红黑树的"磁盘亲戚"。最后记住:数据结构没有银弹,选型的本质是拿时间复杂度、实现复杂度与最坏保证做三角权衡。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
查找 / 插入 / 删除	红黑树(SortedDict 底层)	$O(\log n)$	$O(\log n)$
有序遍历全部元素	红黑树中序 / SortedDict 迭代	$O(n)$	$O(n)$
前驱 / 后继	有序列表 bisect 定位	$O(\log n)$	$O(\log n)$
查找 / 插入 / 删除	Treap	期望 $O(\log n)$	$O(n)$ (优先级恰好有序)
查找 / 插入 / 删除	跳表	期望 $O(\log n)$	$O(n)$ (全部同层)
建树	k-d 树(sorted 切分)	$O(n \log n)$	$O(n \log n)$
最近邻查询	k-d 树	平均 $O(\log n)$	$O(n)$ (数据退化)
合并两个堆	配对堆	摊还 $O(1)$	摊还 $O(1)$
插入(单结点)	配对堆	摊还 $O(1)$	摊还 $O(1)$
deleteMin	配对堆	摊还 $O(\log n)$	摊还 $O(\log n)$
按键查找	dict(散列)	平均 $O(1)$	$O(n)$ (散列冲突)
有序遍历	sorted(d.keys())	$O(n \log n)$	$O(n \log n)$
范围查询 [lo, hi]	bisect 定位有序列表	$O(\log n) + O(k)$	同左(k 为输出个数)
二分查找	bisect_left / bisect_right	$O(\log n)$	$O(\log n)$
插入到有序列表	list.insert(bisect 定位后)	$O(n)$	$O(n)$

最小必会清单

- 能默写红黑树五条性质,并解释"性质4+5 $\rightarrow$ 高度 $O(\log n)$ "
- 能画出插入修正两个分支(叔叔红变色、叔叔黑旋转)与四种旋转形态
- 能说出"红黑树是 2-3-4 树的二叉表示"
- 能独立实现 Treap 与跳表的插入/删除/查找,说出期望为何 $O(\log n)$
- 能说出跳表与红黑树的取舍(期望 vs 确定、实现难度、内存、并发)
- 能独立实现 k-d 树交替维度建树与最近邻剪枝查询
- 能说出配对堆合并摊还 $O(1)$ 、deleteMin 摊还 $O(\log n)$,一句话对比三堆
- 能说出 Python 有序映射三替代(list+bisect、dict+sorted、SortedDict)及各自复杂度

自测题

Q 1. 为什么性质4 与性质5 一起能保证红黑树高度 $O(\log n)$?

✓ 参考答案

性质5 说所有路径黑高相同,设黑高 h_b ;性质4 禁红红相邻,最长路径至多 $2h_b$,最短路径 h_b ——最长 $\leq 2 \times$ 最短。而黑高 h_b 的树至少 2^{h_b-1} 个内部结点,故树高 $\leq 2 \log_2(n+1) = O(\log n)$ 。

Q 2. Treap 的期望高度为什么是 $O(\log n)$? 最坏情形是什么?

✓ 参考答案

优先级随机时,根是"优先级最小结点"的概率均匀为 $1/n$,左右子树递归同构——与快排随机枢轴同构。最坏是优先级恰好按键单调,高度 $O(n)$,概率 $1/n!$ 。

Q 3. 跳表查找从第几层开始?层数为什么用几何分布而不是固定层数?

✓ 参考答案

从当前最高层开始逐层下降。几何分布(每层 $1/2$ 概率晋升)使每层期望结点数 $n/2^k$,查找每层只走常数步,期望 $O(\log n)$;固定层数则退化成长扫描。

Q 4. k-d 树最近邻查询的剪枝条件是什么?漏写会怎样?

✓ 参考答案

回溯时,若查询点到分割线的距离平方 $\geq$ 当前最佳距离平方,另一侧不可能更近,整棵子树剪掉;只有严格小于才进入。漏写则退化成长树搜索 $O(n)$;方向写反则可能漏掉更近的点。

Q 5. Python 里要"按键有序的映射",从小到大数据量各有什么方案?各是什么复杂度?

✓ 参考答案

小数据($\leq 10^4$): `list` + `bisect`,查找 $O(\log n)$ 、插入 $O(n)$;批处理: `dict` + 用时 `sorted(d.keys())`,整体 $O(n \log n)$;大数据且持续有序:第三方 `SortedDict` (红黑树),查找/插入 $O(\log n)$ 。标准库没有平衡树,这是与 C++ `std::map` 最大的差异。

章末实验题:跳表字典的实现与对比

实验 12:跳表字典的实现与对比

实验目标:实现完整跳表并封装成有序字典,与 `dict` 对照验证正确性,实测 10^4 级随机键耗时。

实验要求:

- 任务 1:实现跳表的查找/插入/删除与随机层数(知识点:\$12.3 三操作与几何分布)
- 任务 2:封装成有序字典接口 `insert/find/erase/keys`(知识点:\$12.3 底层链有序 + 第3章 ADT)
- 任务 3:用 `dict` 做同一组随机操作,验证输出一致(知识点:\$12.6 标准库对照)
- 任务 4:对 10^4 级随机键分别计时插入与查找(知识点:\$12.3 期望复杂度 + 第2章计时)
- 任务 5:删除全部元素,验证跳表为空(知识点:\$12.3 删除与空层回收)

实验环境:Python 3.8+。运行: `python lab12.py`。

第一步 示例 12-7/12-8 实现 `SkipList`,升/降/乱序三组输入冒烟测试,`keys()` 必须升序

第二步 任务 3 用同一种子生成随机操作序列,插入前先 `find` 去重;任务 4 洗牌生成 10^4 个不重复键,结果累加输出防死代码消除

✅ 参考答案(完整可运行程序,约 100 行,分四段)

```
# lab12.py — 跳表字典的实现与对比(第一段:任务1 跳表核心)
import random
import time

class Node:
    __slots__ = ("key", "next")
    def __init__(self, key, lvl):
        self.key = key
        self.next = [None] * (lvl + 1)    # 层号 0..lvl 均有效

class Skiplist:
    # 任务 1:三操作 + 随机层数
    def __init__(self, max_lvl=16):
        self.head = Node(0, max_lvl)
        self.max_lvl = max_lvl
        self.level = 0
        self.rng = random.Random(12345)    # 固定种子:可复现

    def random_level(self):
        # 几何分布:每层 1/2 概率晋升
        lvl = 1
        while self.rng.getrandbits(1) and lvl < self.max_lvl:
            lvl += 1
        return lvl

    def find(self, key):
        cur = self.head
        for i in range(self.level, -1, -1):
            while cur.next[i] and cur.next[i].key < key:
                cur = cur.next[i]
        x = cur.next[0]
        return x is not None and x.key == key

    def insert(self, key):
        if self.find(key):
            return    # 字典语义:键唯一
        update = [None] * len(self.head.next)
        cur = self.head
        for i in range(self.level, -1, -1):
            while cur.next[i] and cur.next[i].key < key:
                cur = cur.next[i]
            update[i] = cur
        lvl = self.random_level()
        if lvl > self.level:
            for i in range(self.level + 1, lvl + 1):
                update[i] = self.head
            self.level = lvl
        x = Node(key, lvl)
        for i in range(lvl + 1):
            x.next[i] = update[i].next[i]
            update[i].next[i] = x
```

lab12.py(第二段:删除、遍历 + 任务2 字典封装)


```

def erase(self, key):
    update = [None] * len(self.head.next)
    cur = self.head
    for i in range(self.level, -1, -1):
        while cur.next[i] and cur.next[i].key < key:
            cur = cur.next[i]
        update[i] = cur
    x = cur.next[0]
    if x is None or x.key != key:
        return False
    for i in range(self.level + 1):
        if update[i].next[i] != x:
            break
        update[i].next[i] = x.next[i]
    while self.level > 0 and self.head.next[self.level] is None:
        self.level -= 1          # 回收空高层
    return True

def keys(self):
    v = []
    p = self.head.next[0]
    while p:
        v.append(p.key)
        p = p.next[0]
    return v

def __len__(self):
    return len(self.keys())

class SkipDict:
    # 任务 2:有序字典接口
    def __init__(self):
        self._sl = SkipList()
    def insert(self, k): self._sl.insert(k)
    def find(self, k): return self._sl.find(k)
    def erase(self, k): return self._sl.erase(k)
    def keys(self): return self._sl.keys()
    def __len__(self): return len(self._sl)

```

lab12.py(第三段:任务3 一致性对照 + 任务4 计时)

```

def main():
    # 任务 3:与 dict 对照,输出必须一致
    print("== 任务3:跳表字典 vs dict 一致性 ==")
    d = SkipDict()
    ref = {}
    rng = random.Random(12345)
    for i in range(20000):
        k = rng.randrange(100000)
        op = i % 4                # 0/1 插,2 删,3 查
        if op <= 1:
            if k not in ref:
                d.insert(k)
            ref[k] = k
        elif op == 2:
            d.erase(k)
            ref.pop(k, None)
        else:
            if d.find(k) != (k in ref):
                print("不一致! key=", k)
                return 1
    dk, mk = d.keys(), sorted(ref)
    print("20000 次随机操作后:跳表 len=%d,dict len=%d" % (len(d), len(ref)))
    print("跳表前 10 键:", dk[:10])
    print("按键序与 sorted(dict) 一致:", dk == mk)

    # 任务 4:10^4 随机键计时
    N = 10000
    ks = list(range(N))
    rng.shuffle(ks)                # 洗牌为随机序列
    s2 = SkipDict()
    t0 = time.perf_counter()
    for k in ks:
        s2.insert(k)
    sl_ins = (time.perf_counter() - t0) * 1000
    ref2 = {}
    t0 = time.perf_counter()
    for k in ks:
        ref2[k] = k
    d_ins = (time.perf_counter() - t0) * 1000
    hits = 0
    t0 = time.perf_counter()
    for k in ks:
        hits += 1 if s2.find(k) else 0
    sl_find = (time.perf_counter() - t0) * 1000
    t0 = time.perf_counter()
    for k in ks:
        hits += 1 if k in ref2 else 0
    d_find = (time.perf_counter() - t0) * 1000
    print("\n== 任务4:10^4 随机键计时 ==")
    print("插入 %d 键:跳表 %.4f ms,dict %.4f ms" % (N, sl_ins, d_ins))
    print("查找 %d 键:跳表 %.4f ms,dict %.4f ms" % (N, sl_find, d_find))
    print("(命中数 %d,防止死代码消除)" % hits)

```

lab12.py(第四段:任务5 删除全部 + 入口)

```
# 任务 5:删除全部元素,验证跳表为空
for k in ks:
    s2.erase(k)
print("\n== 任务5:删除全部后 ==")
print("跳表 len=%d,为空=%s,find(0)=%s"
      % (len(s2), len(s2) == 0, s2.find(0)))

if __name__ == "__main__":
    main()
```

示例输出(真实运行,机器不同数值不同,量级关系一致):

```
== 任务3:跳表字典 vs dict 一致性 ==
20000 次随机操作后:跳表 len=9281,dict len=9281
跳表前 10 键: [1, 14, 21, 26, 31, 36, 38, 46, 47, 48]
按键序与 sorted(dict) 一致: True

== 任务4:10^4 随机键计时 ==
插入 10000 键:跳表 27.6136 ms,dict 0.4609 ms
查找 10000 键:跳表 14.3060 ms,dict 0.3026 ms
(命中数 20000,防止死代码消除)

== 任务5:删除全部后 ==
跳表 len=0,为空=True,find(0)=False
```

设计说明:① update 列表统一"下探路径",插入/删除共享骨架;② 插入前先 find 去重,与 dict 键唯一语义对齐;③ 任务3 用同一随机操作序列同时驱动两者,任何一次查找不一致立即退出;④ 同一份键序列计时,跳表与 dict 同为 $O(\log n)$ 对 $O(1)$,dict 常数更小;删除后回收空高层,len=0 验证删除路径正确。对照标准库用 dict + sorted ,正是 §12.6 的 Python 惯例。

全书十二章到此结束:第1-2章打下递归与复杂度地基,第3-9章建起线性结构、树、散列、堆、排序、图的工具箱,第10-11章学会设计范式与摊还分析,本章补上最后一块拼图——平衡与概率。数据结构是算法工程师的母语,愿你在竞赛、系统设计或工程实践中带着这十二章的判断力走得更远。